J2EE

15 August 2022 04:15 PM

Session 1:

J2EE Overview

The Application Server implements Java 2 Enterprise Edition (J2EE) 1.4 technology. The J2EE platform is a set of standard specifications that describe application components, APIs, and the runtime containers and services of an application server.

J2EE Applications

J2EE applications are made up of components such as JavaServer Pages (JSP), Java servlets, and Enterprise JavaBeans (EJB) modules. These components enable software developers to build large-scale, distributed applications. Developers package J2EE applications in Java Archive (JAR) files (similar to zip files), which can be distributed to production sites. Administrators install J2EE applications onto the Application Server by deploying J2EE JAR files onto one or more server instances (or clusters of instances).
The following figure illustrates the components of the J2EE platform discussed in the following sections.

Containers

Each server instance includes two containers: web and EJB. A container is a runtime environment that provides services such as security and transaction management to J2EE components. Web components, such as Java Server Pages and servlets, run within the web container. Enterprise JavaBeans run within the EJB container.

J2EE Services

The J2EE platform provides services for applications, including:

- **Naming -** A naming and directory service binds objects to names. A J2EE application can locate an object by looking up its Java Naming and Directory Interface (JNDI) name.
- **Security -** The Java Authorization Contract for Containers (JACC) is a set of security contracts defined for the J2EE containers. Based on the client's identity, containers can restrict access to the container's resources and services.
- **Transaction management -** A transaction is an indivisible unit of work. For example, transferring funds between bank accounts is a transaction. A transaction management service ensures that a transaction is either completed, or is rolled back.
- Message Service - Applications hosted on separate systems can communicate with each other by exchanging messages using the Java™ Message Service (JMS). JMS is an integral part of the J2EE platform and simplifies the task of integrating heterogeneous enterprise applications.

Web Services

Clients can access a J2EE 1.4 application as a remote web service in addition to accessing it through HTTP, RMI/IIOP, and JMS. Web services are implemented using the Java API for XML-based RPC (JAX-RPC). A J2EE application can also act as a client to web services, which would be typical in network applications.
Web Services Description Language (WSDL) is an XML format that describes web service interfaces. Web service consumers can dynamically parse a WSDL document to determine the operations a web service provides and how to execute them. The Application Server distributes web services interface descriptions using a registry that other applications can access through the Java API for XML Registries (JAXR).

Client Access

Clients can access J2EE applications in several ways. Browser clients access web applications using hypertext transfer protocol (HTTP). For secure communication, browsers use the HTTP secure (HTTPS) protocol that uses secure sockets layer (SSL).

Rich client applications running in the Application Client Container can directly lookup and access Enterprise JavaBeans using an Object Request Broker (ORB), Remote Method Invocation (RMI) and the internet inter-ORB protocol (IIOP), or IIOP/SSL (secure IIOP). They can access applications and web services using HTTP/HTTPS, JMS, and JAX-RPC. They can use JMS to send messages to and receive messages from applications and message-driven beans.
Clients that conform to the Web Services-Interoperability (WS-I) Basic Profile can access J2EE web services. WS-I is an integral part of the J2EE standard and defines interoperable web services. It enables clients written in any supporting language to access web services deployed to the Application Server.
The best access mechanism depends on the specific application and the anticipated volume of traffic. The Application Server supports separately configurable listeners for HTTP, HTTPS, JMS, IIOP, and IIOP/SSL. You can set up multiple listeners for each protocol for increased scalability and reliability.
J2EE applications can also act as clients of J2EE components such as Enterprise JavaBeans modules deployed on other servers, and can use any of these access mechanisms.

External Systems and Resources

On the J2EE platform, an external system is called a **resource**. For example, a database management system is a JDBC resource. Each resource is uniquely identified and by its Java Naming and Directory Interface (JNDI) name. Applications access external systems through the following APIs and components:

- **Java Database Connectivity (JDBC) -** A database management system (DBMS) provides facilities for storing, organizing, and retrieving data. Most business applications store data in relational databases, which applications access via JDBC. The Application Server includes the PointBase DBMS for use sample applications and application development and prototyping, though it is not suitable for deployment. The Application Server provides certified JDBC drivers for connecting to major relational databases. These drivers are suitable for deployment.
- **Java Message** Service - Messaging is a method of communication between software components or applications. A messaging client sends messages to, and receives messages from, any other client via a messaging provider that implements the Java Messaging Service (JMS) API. The Application Server includes a high-performance JMS broker, the Sun Java System Message Queue. The Platform Edition of Application Server includes the free Platform Edition of Message Queue. Application Server Enterprise Edition includes Message Queue Enterprise Edition, that supports clustering and failover.
- **J2EE Connectors** - The J2EE Connector architecture enables integrating J2EE applications and existing Enterprise Information Systems (EIS). An application accesses an EIS through a portable J2EE component called a **connector** or **resource adapter**, analogous to using JDBC driver to access an RDBMS. Resource adapters are distributed as standalone Resource Adapter Archive (RAR) modules or included in J2EE application archives. As RARs, they are deployed like other J2EE components. The Application Server includes evaluation resource adapters that integrate with popular EIS.
- **JavaMail -** Through the JavaMail API, applications can connect to a Simple Mail Transport Protocol (SMTP) server to send and receive email.

From <https://docs.oracle.com/cd/E19159-01/819-3680/abfar/index.html>

J2EE Container

Normally, thin-client multitiered applications are hard to write because they involve many lines of intricate code to handle transaction and state management, multithreading, resource pooling, and other complex low-level details. The component-based and platform-independent J2EE architecture makes J2EE applications easy to write because business logic is organized into reusable components. In addition, the J2EE server provides underlying services in the form of a container for every component type. Because you do not have to develop these services yourself, you are free to concentrate on solving the business problem at hand.

Container Services

Containers are the interface between a component and the low-level platform-specific functionality that supports the component. Before a web component, enterprise bean, or application client component can be executed, it must be assembled into a J2EE module and deployed into its container.

The assembly process involves specifying container settings for each component in the J2EE application and for the J2EE application itself. Container settings customize the underlying support provided by the J2EE server, including services such as security, transaction management, Java Naming and Directory Interface
™

Container Types

The deployment process installs J2EE application components in the J2EE containers illustrated in Figure 1-5.

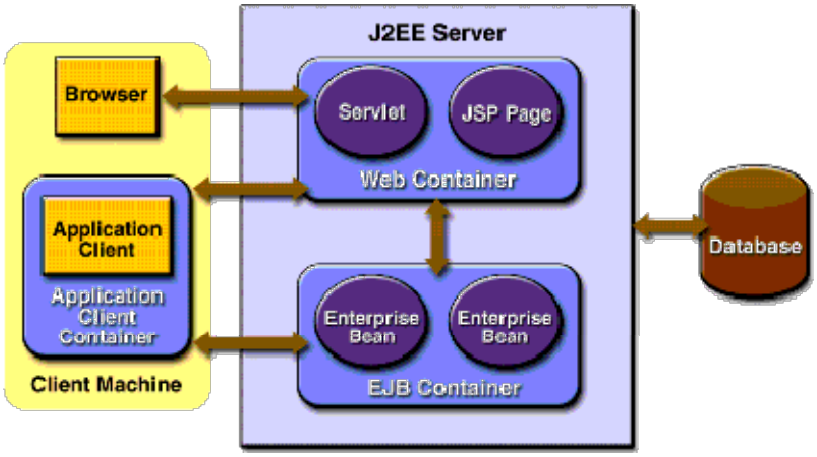


Figure 1-5 J2EE Server and Containers

J2EE server
The runtime portion of a J2EE product. A J2EE server provides EJB and web containers.
Enterprise JavaBeans (EJB) container
Manages the execution of enterprise beans for J2EE applications. Enterprise beans and their container run on the J2EE server.
Web container
Manages the execution of JSP page and servlet components for J2EE applications. web components and their container run on the J2EE server.
Application client container
Manages the execution of application client components. Application clients and their container run on the client.
Applet container
Manages the execution of applets. Consists of a web browser and Java Plug-in running on the client together.

From <https://docs.oracle.com/cd/E17802_01/j2ee/j2ee/1.4/docs/tutorial-update6/doc/Overview3.html>

? Packaging Web applications

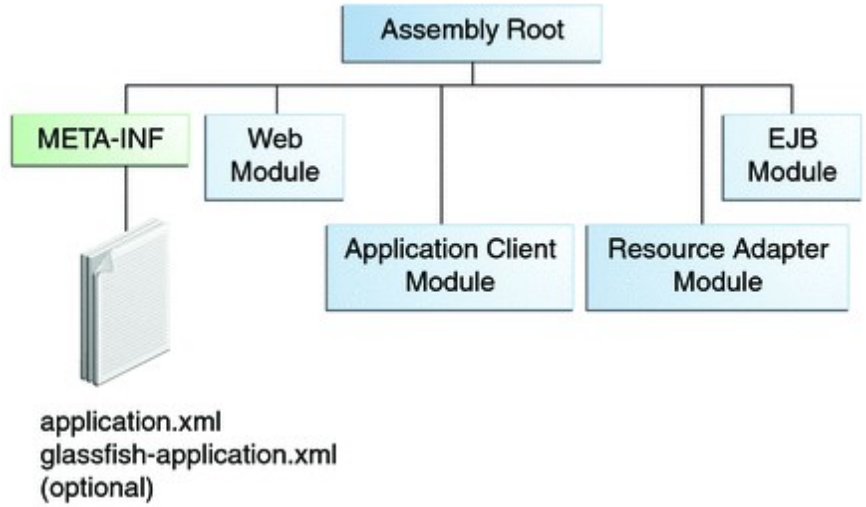
A complete web application may consist of several different resources:

- JSP pages,
- servlets,
- applets,
- static HTML pages,
- custom tag libraries and
- other Java class files.

Packaging Applications

*A Java EE application is delivered in a Java Archive (JAR) file, a Web Archive (WAR) file, or an Enterprise Archive (EAR) file.
*A WAR or EAR file is a standard JAR (.jar) file with a .war or .ear extension.
*Using JAR, WAR, and EAR files and modules makes it possible to assemble a number of different Java EE applications using some of the same components.
*No extra coding is needed; it is only a matter of assembling (or packaging) various Java EE modules into Java EE JAR, WAR, or EAR files.
*An EAR file (see Figure 1-6) contains Java EE modules and, optionally, deployment descriptors.
* A **deployment descriptor**, an XML document with an .xml extension, describes the deployment settings of an application, a module, or a component. Because deployment descriptor information is declarative, it can be changed without the need to modify the source code. At runtime, the Java EE server reads the deployment descriptor and acts upon the application, module, or component accordingly.

Figure 1-6 EAR File Structure



*The two types of deployment descriptors are Java EE and runtime.
*A **Java EE deployment descriptor** is defined by a Java EE specification and can be used to configure deployment settings on any Java EE-compliant implementation.
*A **runtime deployment descriptor** is used to configure Java EE implementation-specific parameters.
*A **Java EE module** consists of one or more Java EE components for the same container type and, optionally, one component deployment descriptor of that type. An enterprise bean module deployment descriptor, for example, declares transaction attributes and security authorizations for an enterprise bean. A Java EE module can be deployed as a stand-alone module.

Java EE modules are of the following types:

- EJB modules, which contain class files for enterprise beans and, optionally, an EJB deployment descriptor. EJB modules are packaged as JAR files with a .jar extension.
- Web modules, which contain servlet class files, web files, supporting class files, image and HTML files, and, optionally, a web application deployment descriptor. Web modules are packaged as JAR files with a .war (web archive) extension.
- Application client modules, which contain class files and, optionally, an application client deployment descriptor. Application client modules are packaged as JAR files with a .jar extension.
- Resource adapter modules, which contain all Java interfaces, classes, native libraries, and, optionally, a resource adapter deployment descriptor. Together, these implement the Connector architecture (see Java EE Connector Architecture) for a particular EIS. Resource adapter modules are packaged as JAR files with an .rar (resource adapter archive) extension.

From <https://docs.oracle.com/javaee/6/tutorial/doc/bnaby.html>

? J2EE compliant web application

Java EE compliant means that it passes the Java EE Technology Compatibility Kit. Oracle created the specification for Java EE (previously called **J2EE**) along with a large test suite checking that the application **server** behaves as the specification requires.

The Web Applications Container

A Web application contains an application's resources, such as servlets, JavaServer Pages (JSPs), JSP tag libraries, and any static resources such as HTML pages and image files. A Web application adds service-refs (Web services) and message-destination-refs (JMS destinations/queues) to an application. It can also define links to outside resources such as Enterprise JavaBeans (EJBs).

Web applications deployed on WebLogic Server use a standard J2EE deployment descriptor file and a WebLogic-specific deployment descriptor file to define their resources and operating attributes.

JSPs and HTTP servlets can access all services and APIs available in WebLogic Server. These services include EJBs, database connections by way of Java Database Connectivity (JDBC), Java Messaging Service (JMS), XML, and more.

A Web archive (WAR file) contains the files that make up a Web application. A WAR file is deployed as a unit on one or more WebLogic Server instances. A WAR file deployed to WebLogic Server always includes the following files:

- One servlet or Java Server Page (JSP), along with any helper classes.
- A web.xml deployment descriptor, which is a J2EE standard XML document that describes the contents of a WAR file.
- A weblogic.xml deployment descriptor, which is an XML document containing WebLogic Server-specific elements for Web applications.
- A WAR file can also include HTML or XML pages and supporting files such as image and multimedia files.

The WAR file can be deployed alone or packaged in an Enterprise application archive (EAR file) with other application components. If deployed alone, the archive must end with a .war extension. If deployed in an EAR file, the archive must end with an .ear extension.

BEA recommends that you package and deploy your stand-alone Web applications as part of an Enterprise application. This is a BEA best practice, which allows for easier application migration, additions, and changes. Also, packaging your applications as part of an Enterprise application allows you to take advantage of the split development directory structure, which provides a number of benefits over the traditional single directory structure.

Servlets

A servlet is a Java class that runs in a Java-enabled server. An HTTP servlet is a special type of servlet that handles an HTTP request and provides an HTTP response, usually in the form of an HTML page. The most common use of WebLogic HTTP Servlets is to create interactive applications using standard Web browsers for the client-side presentation while WebLogic Server handles the business logic as a server-side process. WebLogic HTTP servlets can access databases, Enterprise JavaBeans, messaging APIs, HTTP sessions, and other facilities of WebLogic Server.

What You Can Do with Servlets

- Create dynamic Web pages that use HTML forms to get end-user input and provide HTML pages that respond to that input. Examples of this utilization include online shopping

carts, financial services, and personalized content.
- Create collaborative systems such as online conferencing.
- Have access to a variety of APIs and features by using servlets running in WebLogic Server. For example:
    - Session tracking—Allows a Web site to track a user's progress across multiple Web pages. This functionality supports Web sites such as e-commerce sites that use shopping carts. WebLogic Server supports session persistence to a database, providing fail-over between server down time and session sharing between clustered servers.
    - JDBC drivers (including BEA)—JDBC drivers provide basic database access. With WebLogic Server's multi-tier JDBC implementations, you can take advantage of connection pools, server-side data caching, and transactions.
    - Enterprise JavaBeans—Servlets can use Enterprise JavaBeans (EJB) to encapsulate sessions, data from databases, and other functionality.
    - Java Messaging Service (JMS)—JMS allows your servlets to exchange messages with other servlets and Java programs.
    - Java JDK APIs—Servlets can use the standard Java JDK APIs.
    - Forwarding requests—Servlets can forward a request to another servlet or other resource.
- Easily deploy servlets written for any J2EE-compliant servlet engine to WebLogic Server.

Servlet Development Key Points

The Servlet 2.4 specification, part of the Java 2 Platform, Enterprise Edition, defines the implementation of the servlet API and the method by which servlets are deployed in enterprise applications. Deploying servlets on a J2EE-compliant server, such as WebLogic Server, is accomplished by packaging the servlets and other resources that make up an enterprise application into a single unit, the Web application. A Web application utilizes a specific directory structure to contain its resources and a deployment descriptor that defines how these resources interact and how the application is accessed by a client.

Java Server Pages

Java Server Pages (JSPs) are Web pages coded with an extended HTML that makes it possible to embed Java code in a Web page. JSPs can call custom Java classes, called *taglibs*, using HTML-like tags. The WebLogic appc compiler weblogic.appc generates JSPs and validates descriptors. You can also precompile JSPs into the WEB-INF/classes/ directory or as a JAR file under WEB-INF/lib/ and package the servlet class in the Web archive to avoid compiling in the server. Servlets and JSPs may require additional helper classes to be deployed with the Web application.

JSPs are a Sun Microsystems specification for combining Java with HTML to provide dynamic content for Web pages. When you create dynamic content, JSPs are more convenient to write than HTTP servlets because they allow you to embed Java code directly into your HTML pages, in contrast with HTTP servlets, in which you embed HTML inside Java code. JSP is part of the Java 2 Enterprise Edition (J2EE).

JSPs enable you to separate the dynamic content of a Web page from its presentation. It caters to two different types of developers: HTML developers, who are responsible for the graphical design of the page, and Java developers, who handle the development of software to create the dynamic content.

Because JSPs are part of the J2EE standard, you can deploy JSPs on a variety of platforms, including WebLogic Server. In addition, third-party vendors and application developers can provide JavaBean components and define custom JSP tags that can be referenced from a JSP page to provide dynamic content.

What You Can Do with JSPs

- Combine Java with HTML to provide dynamic content for Web pages.
- Call custom Java classes, called *taglibs*, using HTML-like tags.
- Embed Java code directly into your HTML pages, in contrast with HTTP servlets, in which you embed HTML inside Java code.
- Separate the dynamic content of a Web page from its presentation.

WebLogic Server handles JSP requests in the following sequence:

1. A browser requests a page with a `.jsp` file extension from WebLogic Server.
2. WebLogic Server reads the request.
3. Using the JSP compiler, WebLogic Server converts the JSP into a servlet class that implements the `javax.servlet.jsp.JspPage` interface. The JSP file is compiled only when the page is first requested, or when the JSP file has been changed. Otherwise, the previously compiled JSP servlet class is re-used, making subsequent responses much quicker.
4. The generated `JspPage` servlet class is invoked to handle the browser request.

Web Application Security

You can secure a Web application by restricting access to certain URL patterns in the Web application or programmatically using security calls in your servlet code.

At runtime, your username and password are authenticated using the applicable security realm for the Web application.

Authorization is verified according to the security constraints configured in web.xml or the external policies that might have been created using Administration Console for the Web application.

At runtime, the WebLogic Server active security realm applies the Web application security constraints to the specified Web application resources. Note that a security realm is shared across multiple virtual hosts.

P3P Privacy Protocol

The Platform for Privacy Preferences (P3P) provides a way for Web sites to publish their privacy policies in a machine-readable syntax. The WebLogic Server Web application container can support P3P.

There are three ways to tell the browser about the location of the p3p.xml file:

- Place the a policy reference file in the "well-known location" (at the location /w3c/p3p.xml on the site).
- Add an extra HTTP header to each response from the Web site giving the location of the policy reference file.
- Place a link to the policy reference file in each HTML page on the site.

From <https://docs.oracle.com/cd/E13222_01/wls/docs92/webapp/basics.html>

? Deployment tools.

Tools for Deployment

This section discusses the various tools that can be used to deploy modules and applications. The

deployment tools include:

- Apache Ant
- The deploytool
- JSR 88
- The FastJavac Compiler
- The asadmin Command
- The Administration Console

Apache Ant

Ant can help you assemble and deploy modules and applications.

The deploytool

You can use the deploytool, provided with Sun Java System Application Server, to assemble J2EE applications and modules, configure deployment parameters, perform simple static checks, and deploy the final result.

JSR 88

You can write your own JSR 88 client to deploy applications to the Sun Java System Application Server.

The FastJavac Compiler

By default, Sun Java System Application Server uses the built-in JDK compiler to compile applications during deployment. You can also use Sun ONE Studio's FastJavac compiler, which has a faster compilation rate, during deployment.

To use the FastJavac compiler, you must configure the administration server's `domain.xml` file with the path to the compiler. Edit the `java-config` element as follows:

```
<java-config java-home="/install_dir/jdk" server-
classpath="classpath" ... >
  ...
  <jvm-options>
    -
Dcom.sun.aas.deployment.java.compiler=/install_dir/studio5/bin/fastjav
ac/fastjavac.sun
  </jvm-options>
  ...
  <property name="com.sun.aas.deployment.java.compiler.options"
    value="-jdk /install_dir/jdk" />
  ...
</java-config>
```

The asadmin Command

You can use the `asadmin` command to deploy or undeploy applications and individually deployed modules on local servers. This section describes the `asadmin` command only briefly. For full details, see the *Sun Java System Application Server Administration Guide*.

The Administration Console

You can use the Administration Console to deploy modules and applications to both local and remote Sun Java System Application Server sites.

? Web application life cycle

A web application consists of web components; static resource files, such as images; and helper classes and libraries. The web container provides many supporting services that enhance the capabilities of web components and make them easier to develop. However, because a web application must take these services into account, the process for creating and running a web application is different from that of traditional stand-alone

Java classes.

The process for creating, deploying, and executing a web application can be summarized as
follows:

1. Develop the web component code.
2. Develop the web application deployment descriptor, if necessary.
3. Compile the web application components and helper classes referenced by the components.
4. Optionally, package the application into a deployable unit.
5. Deploy the application into a web container.
6. Access a URL that references the web application.

From <https://docs.oracle.com/javaee/6/tutorial/doc/bnadu.html>

Deploying web applications.

Steps to Deploy a Web Application

To deploy a Web Application:

1. Arrange the resources (servlets, JSPs, static files, and deployment descriptors) in the prescribed directory format.

2. Write the Web Application deployment descriptor (`web.xml`). In this step you register servlets, define servlet initialization parameters, register JSP tag libraries, define security constraints, and define other Web Application parameters. (Information on the various components of Web Applications is included throughout this document.)
3. Create the WebLogic-Specific Deployment Descriptor (`weblogic.xml`). In this step you define JSP properties, JNDI mappings, security role mappings, and HTTP session parameters. If you do not need to define any of the attributes defined in this file, you do not need to create the file.
4. Archive the files in the above directory structure into a `.war` file. Only use archiving when the Web Application is ready for deployment in a production environment. (During development you may find it more convenient to update individual components of your Web Application by developing your application in exploded directory format.
   To create a `.war` archive, use this command line from the root directory containing your Web Application:
   `jar cv0f myWebApp.war .`
   This command creates a Web Application archive file called `myWebApp.war`.
5. Deploy the Web Application on Weblogic Server in one of two ways:

- using the Administration Console or
- by copying the Web Application into the applications directory of your domain.

To deploy a Web Application in archived `war` format using the Administration Console (you cannot deploy a Web Application in exploded directory format using this procedure):

a. Select the Web Applications node in the left pane.
b. Click Install a New Web Application.
c. Browse to the location in your file system of the `.war` file.
d. Click Upload.
   This procedure creates a new entry in the `config.xml` file containing the configuration for your Web Application and copies your Web Application to an internal location.
   To deploy a Web Application (in either archived or exploded format) by copying:

e. Copy a `.war` file or the top-level directory containing a Web Application in exploded directory format into the `mydomain/config/applications` directory of your WebLogic Server distribution. (Where `mydomain` is the name of your domain.) As soon as the copy is complete, WebLogic Server automatically deploys the Web Application.
f. (optional) Use the Administration Console to configure the Web Application. Once you change any attributes (see step 6., below) for the Web Application, the configuration is written to the `config.xml` file and the Web Application will be statically deployed the next time you restart WebLogic Server. If you do not use the Administration Console, your Web Application is still deployed automatically every time you start WebLogic Server, even though configuration information has not been saved to the `config.xml` file.
Note: If you deploy your Web Application in expanded form, read Modifying Components of a Web Application.
Note: If you modify any component of a `.war` file in its original location in your file system, you must redeploy your `.war` file by uploading it again from the Administration Console.

6. Assign deployment attributes for your Web Application:
   a. Open the Administration Console
   b. Select the Web Applications node.
   c. Select your Web Application.
   d. Assign your Web Application to a WebLogic Server, cluster, or Virtual Host.
   e. Select the File tab and define the appropriate attributes.

From <https://docs.oracle.com/cd/E13222_01/wls/docs60/adminguide/config_web_app.html>

? Web Services Support

Web services are web-based enterprise applications that use open, XML-based standards and transport protocols to exchange data with calling clients. The Java EE platform provides the XML APIs and tools you need to quickly design, develop, test, and deploy web services and clients that fully interoperate with other web services and clients running on Java-based or non-Java-based platforms.
To write web services and clients with the Java EE XML APIs, all you do is pass parameter data to the method calls and process the data returned; or for document-oriented web services, you send documents containing the service data back and forth. No low-level programming is needed because the XML API implementations do the work of translating the application data to and from an XML-based data stream that is sent over the standardized XML-based transport protocols. These XML-based standards and protocols are introduced in the following sections.
The translation of data to a standardized XML-based data stream is what makes web services and clients written with the Java EE XML APIs fully interoperable. This does not necessarily mean that the data being transported includes XML tags because the transported data can itself be plain text, XML data, or any kind of binary data such as audio, video, maps, program files, computer-aided design (CAD) documents and the like. The next section introduces XML and explains how parties doing business can use XML tags and schemas to exchange data in a meaningful way.

XML

XML is a cross-platform, extensible, text-based standard for representing data. When XML data is exchanged between parties, the parties are free to create their own tags to describe the data, set up schemas to specify which tags can be used in a particular kind

of XML document, and use XML stylesheets to manage the display and handling of the data.

For example, a web service can use XML and a schema to produce price lists, and companies that receive the price lists and schema can have their own stylesheets to handle the data in a way that best suits their needs. Here are examples:

- One company might put XML pricing information through a program to translate the XML to HTML so that it can post the price lists to its intranet.
- A partner company might put the XML pricing information through a tool to create a marketing presentation.
- Another company might read the XML pricing information into an application for processing.

SOAP Transport Protocol

Client requests and web service responses are transmitted as Simple Object Access Protocol (SOAP) messages over HTTP to enable a completely interoperable exchange between clients and web services, all running on different platforms and at various locations on the Internet. HTTP is a familiar request-and response standard for sending messages over the Internet, and SOAP is an XML-based protocol that follows the HTTP request-and-response model.

The SOAP portion of a transported message handles the following:

- Defines an XML-based envelope to describe what is in the message and how to process the message
- Includes XML-based encoding rules to express instances of application-defined data types within the message
- Defines an XML-based convention for representing the request to the remote service and the resulting response

WSDL Standard Format

The Web Services Description Language (WSDL) is a standardized XML format for describing network services. The description includes the name of the service, the location of the service, and ways to communicate with the service. WSDL service descriptions can be stored in UDDI registries or published on the web (or both). The Sun Java System Application Server provides a tool for generating the WSDL specification of a web service that uses remote procedure calls to communicate with clients.

UDDI and ebXML Standard Formats

Other XML-based standards, such as Universal Description, Discovery and Integration (UDDI) and ebXML, make it possible for businesses to publish information on the Internet about their products and web services, where the information can be readily and globally accessed by clients who want to do business.

From <https://docs.oracle.com/javaee/5/tutorial/doc/bnabs.html>

Sessions 2, 3 & 4:

? Servlets: Dynamic Content Generation

You create dynamic content by accessing Java programming language object properties.

Using Objects within JSP Pages

You can access a variety of objects, including enterprise beans and JavaBeans components, within a JSP page. JSP technology automatically makes some objects available, and you can also create and access application-specific objects.

Using Implicit Objects

Implicit objects are created by the web container and contain information related to a particular request, page, session, or application.

Using Application-Specific Objects

When possible, application behavior should be encapsulated in objects so that page designers can focus on presentation issues. Objects can be created by developers who are proficient in the Java programming language and in accessing databases and other services. The main way to create and use application-specific objects within a JSP page is to use JSP standard tags to create JavaBeans components and set their properties, and EL expressions to access their properties. You can also access JavaBeans components and other objects in scripting elements.

Using Shared Objects

The conditions affecting concurrent access to shared objects apply to objects accessed from JSP pages that run as multithreaded servlets.

From <https://docs.oracle.com/cd/E19159-01/819-3669/bnahl/index.html>

? Advantages of Servlets over CGI

Servlets are server side components that provides a powerful mechanism for developing server web applications for server side. Earlier CGI was developed to provide server side capabilities to the web applications. Although CGI played a major role in the explosion of the Internet, its performance, scalability and reusability issues make it less than optimal solutions. Java Servlets changes all that. Built from ground up using Sun's write once run anywhere technology java servlets provide excellent framework for server side processing.

Using servlets web developers can create fast and efficient server side applications and can run it on any servlet enabled web server. Servlet runs entirely inside the Java Virtual Machine. Since the servlet runs on server side so it does not depend on browser compatibility.

Servlets have a number of advantages over CGI and other API's. They are:

1. Platform Independence

Servlets are written entirely in java so these are platform independent. Servlets can run on any Servlet enabled web server. For example if you develop an web application in windows machine running Java web server, you can easily run the same on apache web server (if Apache Serve is installed) without modification or compilation of code. Platform independency of servlets provide a great advantages over alternatives of servlets.

2. Performance

Due to interpreted nature of java, programs written in java are slow. But the java servlets runs very fast. These are due to the way servlets run on web server. For any program initialization takes significant amount of time. But in case of servlets initialization takes place first time it receives a request and remains in memory till times out or server shut downs. After servlet is loaded, to handle a new request it simply creates a new thread and runs service method of servlet. In comparison to traditional CGI scripts which creates a new process to serve the request.

3. Extensibility

Java Servlets are developed in java which is robust, well-designed and object oriented language which can be extended or polymorphed into new objects. So the java servlets take all these advantages and can be extended from existing class to provide the ideal solutions.

4. Safety

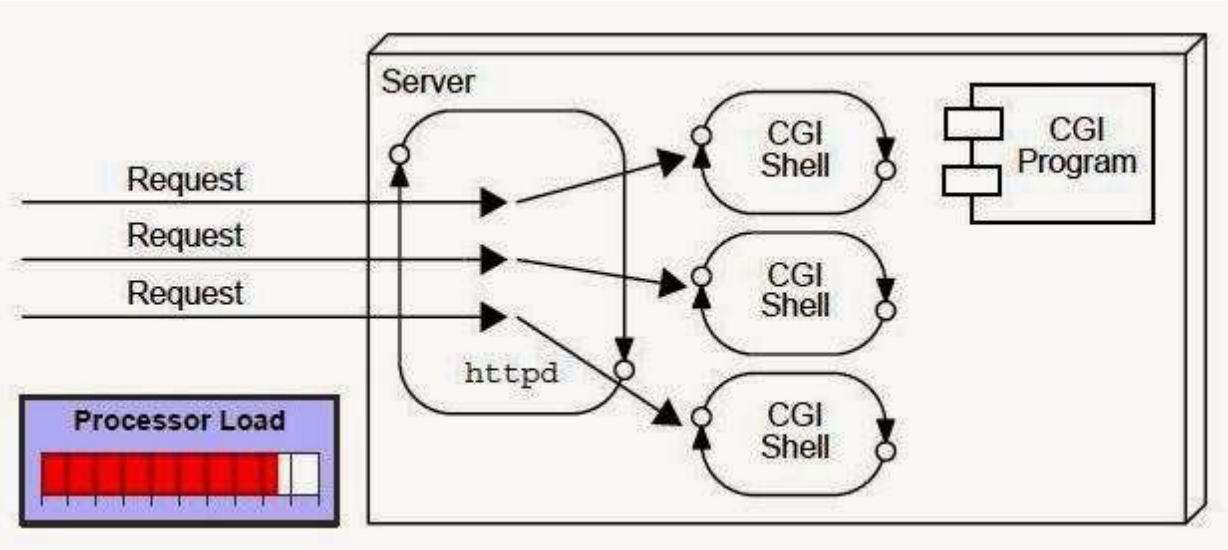
Java provides very good safety features like memory management, exception handling etc. Servlets inherits all these features and emerged as a very powerful web server extension.

5. Secure

Servlets are server side components, so it inherits the security provided by the web server. Servlets are also benefited with Java Security Manager.

CGI(Common Gateway Interface)

CGI technology enables the web server to call an external program and pass HTTP request information to the external program to process the request. For each request, it starts a new process.



Disadvantages of CGI

There are many problems in CGI technology:

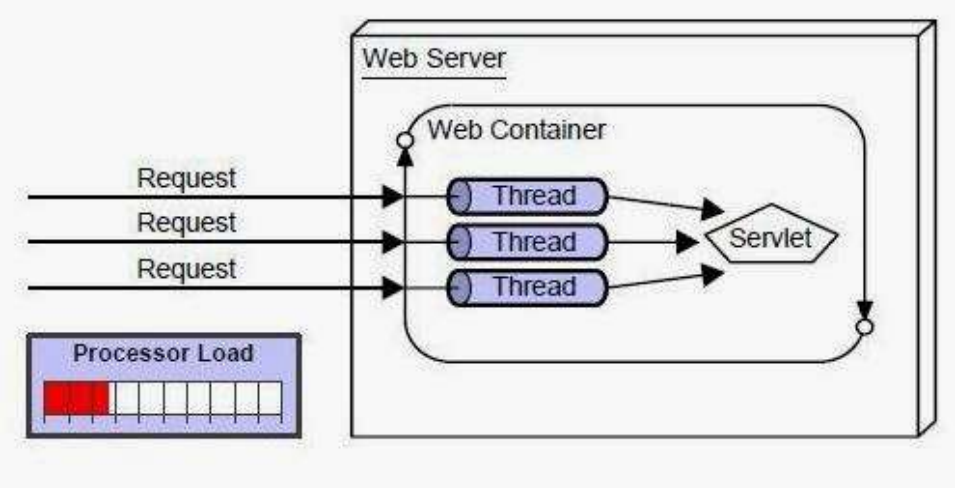
- If number of clients increases, it takes more time for sending response.
- For each request, it starts a process and Web server is limited to start processes.
- It uses platform dependent language e.g. C, C++, perl.

Advantage of Servlet

There are many advantages of Servlet over CGI. The web container creates threads for handling the multiple requests to the servlet. Threads have a lot of benefits over the Processes such as they share a common memory area, lighweight, cost of communication between the threads are low. The basic benefits of servlet are as follows:

- **better performance:** because it creates a thread for each request not process.
- **Portability:** because it uses java language.

- **Robust:** Servlets are managed by JVM so no need to worry about momory leak, garbage collection etc.
- **Secure:** because it uses java language.



From <https://www.dineshonjava.com/advantages-of-servlets-over-cgi/>

? Servlet Life cycle

The entire life cycle of a Servlet is managed by the **Servlet container** which uses the **javax.servlet.Servlet** interface to understand the Servlet object and manage it. So, before creating a Servlet object, let's first understand the life cycle of the Servlet object which is actually understanding how the Servlet container manages the Servlet object.

Stages of the Servlet Life Cycle: The Servlet life cycle mainly goes through four stages,

- Loading a Servlet.
- Initializing the Servlet.
- Request handling.
- Destroying the Servlet.

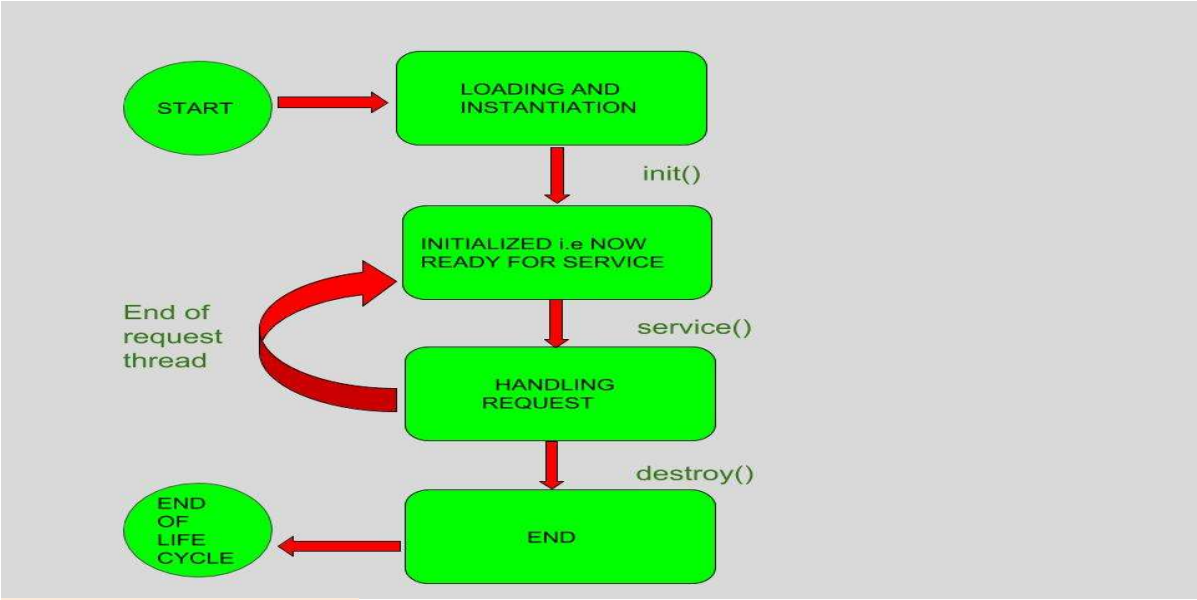
Let's look at each of these stages in details:

Loading a Servlet: The first stage of the Servlet lifecycle involves loading and initializing the Servlet by the Servlet container. The Web container or Servlet Container can load the Servlet at either of the following two stages :

- Initializing the context, on configuring the Servlet with a zero or positive integer value.
- If the Servlet is not preceding stage, it may delay the loading process until the Web container determines that this Servlet is needed to service a request.

The Servlet container performs two operations in this stage :

- **Loading :** Loads the Servlet class.
- **Instantiation :** Creates an instance of the Servlet. To create a new instance of the Servlet, the container uses the no-argument constructor.



Initializing a Servlet: After the Servlet is instantiated successfully, the Servlet container initializes the instantiated Servlet object. The container initializes the Servlet object by invoking the **Servlet.init(ServletConfig)** method which accepts ServletConfig object reference as parameter.
The Servlet container invokes the **Servlet.init(ServletConfig)** method only once, immediately after the **Servlet.init(ServletConfig)** object is instantiated successfully. This method is used to initialize the resources, such as JDBC datasource.
Now, if the Servlet fails to initialize, then it informs the Servlet container by throwing the **ServletException** or **UnavailableException**.
Handling request: After initialization, the Servlet instance is ready to serve the client requests. The Servlet container performs the following operations when the Servlet instance is located to service a request :

- It creates the **ServletRequest** and **ServletResponse** objects. In this case, if this is a HTTP request, then the Web container creates **HttpServletRequest** and **HttpServletResponse** objects which are subtypes of the **ServletRequest** and **ServletResponse** objects respectively.
- After creating the request and response objects it invokes the Servlet.service(ServletRequest, ServletResponse) method by passing the request and response objects.
  The **service()** method while processing the request may throw the **ServletException** or **UnavailableException** or **IOException**.

Destroying a Servlet: When a Servlet container decides to destroy the Servlet, it performs the following operations,

- It allows all the threads currently running in the service method of the Servlet instance to complete their jobs and get released.
- After currently running threads have completed their jobs, the Servlet container calls the **destroy()** method on the Servlet instance.
  After the **destroy()** method is executed, the Servlet container releases all the references of this Servlet instance so that it becomes eligible for garbage collection.

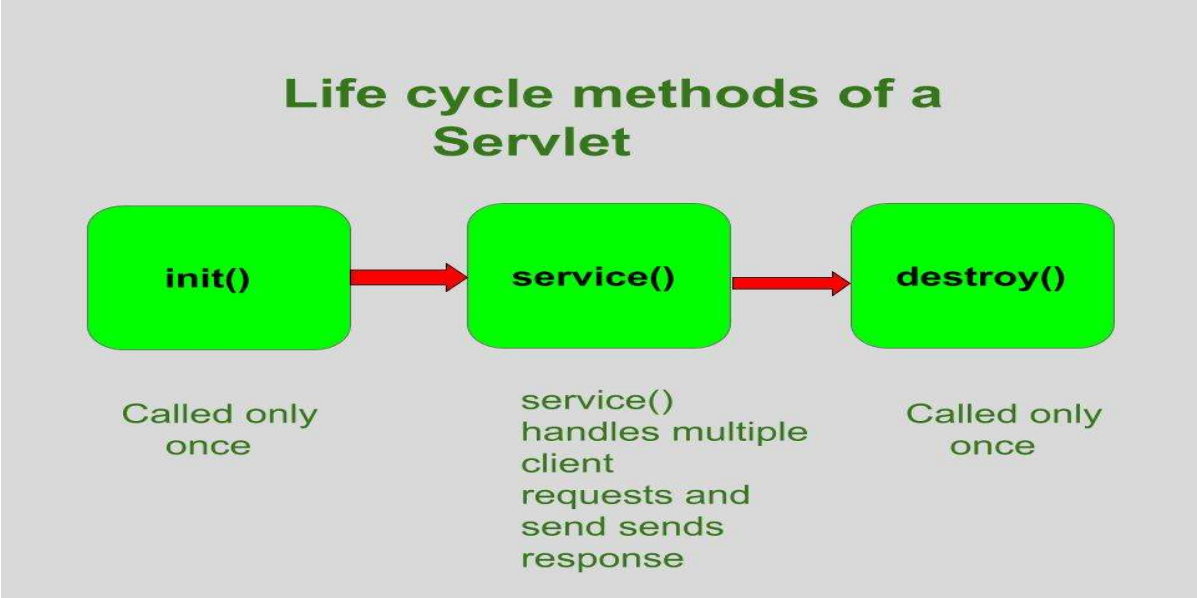
Servlet Life Cycle Methods

There are three life cycle methods of a Servlet :

- init()
- service()
- destroy()

Life cycle methods are those methods which are used to control the life cycle of the servlet. These methods are called in specific order during the servlets's entire life cycle.

The class **Servlet** provides the methods to control and supervise the life cycle of servlet. There are three life cycle methods in the Servlet interface. There are as follows:



Let's look at each of these methods in details:

- **init() method**: The **Servlet.init()** method is called by the Servlet container to indicate that this Servlet instance is instantiated successfully and is about to put into service.

```
//init() method

public class MyServlet implements Servlet{
   public void init(ServletConfig config) throws ServletException {
        //initialization code
   }
    //rest of code
}
```

A servlet's life begins here .
This method is called only once to load the servlet.Since it is called only once in it's lifetime,therefore "connected architecture" code is written inside it because we only want once to get connected with the database.
This method receives only one parameter, i.e **ServletConfig** object.
This method has the possibility to throw the ServletException.
Once the servlet is initialized, it is ready to handle the client request.
The prototype for the init() method:

```
public void init(ServletConfig con)throws ServletException{ }
```
where **con** is ServletConfig object

- **service() method**: The **service()** method of the Servlet is invoked to inform the Servlet about the client requests.
  - This method uses **ServletRequest** object to collect the data requested by the client.
  - This method uses **ServletResponse** object to generate the output content.

```
// service() method

public class MyServlet implements Servlet{
    public void service(ServletRequest res, ServletResponse res)
    throws ServletException, IOException {
            // request handling code
    }
    // rest of code
}
```

1. The service() method is the most important method to perform that provides the connection between client and server.
2. The web server calls the service() method to handle requests coming from the client(web browsers) and to send response back to the client.
3. This method determines the type of Http request (GET, POST, PUT, DELETE, etc.) .
4. This method also calls various other methods such as doGet(), doPost(), doPut(), doDelete(), etc. as required.
5. This method accepts two parameters.
6. The prototype for this method:
   ```
   public void service(ServletRequest req, ServletResponse resp)
   throws ServletException, IOException { }
   ```
   where

- **req** is the ServletRequest object which encapsulates the connection from client to server
- **resp** is the ServletResponse object which encapsulates the connection from server back to the client

- **destroy() method**: The **destroy()** method runs only once during the lifetime of a Servlet and signals the end of the Servlet instance.

```
//destroy() method

public void destroy()
```

As soon as the **destroy()** method is activated, the Servlet container releases the Servlet instance.

- The destroy() method is called only once.
- It is called at the end of the life cycle of the servlet.
- This method performs various tasks such as closing connection with the database, releasing memory allocated to the servlet, releasing resources that are allocated to the servlet and other cleanup activities.

- When this method is called, the garbage collector comes into action.
- The prototype for this method is:
  `public void destroy() { // Finalization code...}`
- Below is a sample program to illustrate Servlet in Java:

```
// Java program to show servlet example
// Importing required Java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class AdvanceJavaConcepts extends HttpServlet
{
   private String output;

   // Initializing servlet
   public void init() throws ServletException
   {
      output = "Advance Java Concepts";
   }

   // Requesting and printing the output
   public void doGet(HttpServletRequest req,
                     HttpServletResponse resp)
      throws ServletException, IOException
      {
         resp.setContentType("text/html");
         PrintWriter out = resp.getWriter();
         out.println(output);
      }

      public void destroy()
      {
         System.out.println("Over");
      }
}
```

From <https://www.geeksforgeeks.org/life-cycle-of-a-servlet/>

? Servlet API & Deployment

Servlets are the Java programs that run on the Java-enabled web server or application server. They are used to handle the request obtained from the webserver, process the request, produce the response, then send a response back to the webserver. In Java, to create web applications we use Servlets. To create Java Servlets, we need to use Servlet API which contains all the necessary interfaces and classes. Servlet API has 2 packages namely,

- javax.servlet
- javax.servlet.http

javax.servlet

- This package provides the number of interfaces and classes to support Generic servlet which is protocol independent.
- These interfaces and classes describe and define the contracts between a servlet class and the runtime environment provided by a servlet container.

Classes available in javax.servlet package:

Class Name	Description
GenericServlet	To define a generic and protocol-independent servlet.
ServletContextAttributeEvent	To generate notifications about changes to the attributes of the servlet context of a web application.
ServletContextEvent	To generate notifications about changes to the servlet context of a web application.
ServletInputStream	This class provides an input stream to read binary data from a client request.
ServletOutputStream	This class provides an output stream for sending binary data to the client.
ServletRequestAttributeEvent	To generate notifications about changes to the attributes of the servlet request in an application.
ServletRequestEvent	To indicate lifecycle events for a ServletRequest.
ServletRequestWrapper	This class provides the implementation of the ServletRequest interface that can be subclassed by developers to adapt the request to a Servlet.
ServletResponseWrapper	This class provides the implementation of the ServletResponse interface that can be subclassed by developers to adapt the response from a Servlet.

Interfaces available in javax.servlet package:

Interface Name	Description
Filter	To perform filtering tasks on either the request to a resource, or on the response from a resource, or both.
FilterChain	To provide a view into the invocation chain of a filtered request for a resource to the developer by the servlet container.
FilterConfig	To pass information to a filter during initialization used by a servlet container.
RequestDispatcher	It defines an object to dispatch the request and response to any other resource, means it receives requests from the client and sends them to a servlet/HTML file/JSP file on the server.
Servlet	This is the main interface that defines the methods in which all the servlets must implement. To implement this interface, write a generic servlet that extends javax.servlet.GenericServlet or an HTTP servlet that extends javax.servlet.http.HttpServlet.
ServletConfig	It defines an object created by a servlet container at the time of servlet instantiation and to pass information to the servlet during initialization.
ServletContext	It defines a set of methods that a servlet uses to communicate with its servlet container. The information related to the web application available in web.xml file is stored in ServletContext object created by container.
ServletContextAttributeListener	The classes that implement this interface receive notifications of changes to the attribute list on the servlet context of a web application.
ServletContextListener	The classes that implement this interface receive notifications about changes to the servlet context of the web application they are part of.
ServletRequest	It defines an object that is created by servlet container to pass client request information to a servlet.
ServletRequestAttributeListener	To generate the notifications of request attribute changes while the request is within the scope of the web application in which the listener is registered.

ServletRequestListener	To generate the notifications of requests coming in and out of scope in a web component.
ServletResponse	It defines an object created by servlet container to assist a servlet in sending a response to the client.

Exceptions in javax.servlet package:

Exception Name	Description
ServletException	A general exception thrown by a servlet when it encounters difficulty.
UnavailableException	Thrown by a servlet or filter to indicate that it is permanently or temporarily unavailable.

javax.servlet.http

- This package provides the number of interfaces and classes to support HTTP servlet which is HTTP protocol dependent.
- These interfaces and classes describe and define the contracts between a servlet class running under HTTP protocol and the runtime environment provided by a servlet container.

Classes available in javax.servlet.http package:

Class Name	Description
Cookie	Creates a cookie object. It is a small amount of information sent by a servlet to a Web browser, saved by the browser, and later sent back to the server used for session management.
HttpServlet	Provides an abstract class that defines methods to create an HTTP suitable servlet for a web application.
HttpServletRequestWrapper	This class provides implementation of the HttpServletRequest interface that can be subclassed to adapt the request to a Servlet.
HttpServletResponseWrapper	This class provides implementation of the HttpServletResponse interface that can be subclassed to adapt the response from a Servlet.
HttpSessionBindingEvent	This events are either sent to an object that implements HttpSessionBindingListener when it is bound or unbound from a session, or to a HttpSessionAttributeListener that has been configured in the deployment descriptor when any attribute is bound, unbound or replaced in a session.
HttpSessionEvent	To represent event notifications for changes to sessions within a web application.

Interfaces available in javax.servlet.http package:

Interface Name	Description
HttpServletRequest	To provide client HTTP request information for servlets. It extends the ServletRequest interface.
HttpServletResponse	To provide HTTP-specific functionality in sending a response to client. It extends the ServletResponse interface.
HttpSession	It provides a way to identify a user across web application/web site pages and to store information about that user.
HttpSessionActivationListener	Container to notify all the objects that are bound to a session that sessions will be passivated and that session will be activated.
HttpSessionAttributeListener	To get notifications of changes to the attribute lists of sessions within this web application, this listener interface can be implemented.
HttpSessionBindingListener	It causes an object to be notified by an HttpSessionBindingEvent object, when it is bound to or unbound from a session.

HttpSessionListener	To receive notification events related to the changes to the list of active sessions in a web application.

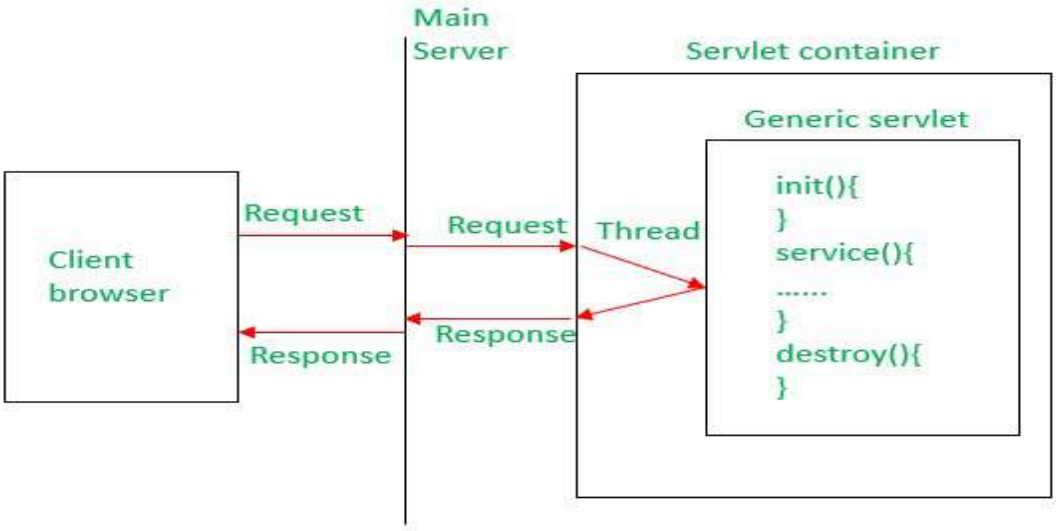
GenericServlet Class

Servlet API provide ***GenericServlet*** class in javax.servlet package.

- Java

```
public abstract class GenericServlet
extends java.lang.Object
implements Servlet, ServletConfig, java.io.Serializable
```

- An abstract class that implements most of the servlet basic methods.
- Implements the Servlet, ServletConfig, and Serializable interfaces.
- Protocol-independent servlet.



- Makes writing servlets easier by providing simple versions of the lifecycle methods init() and destroy().
- To write a generic servlet, you need to extend ***javax.servlet.GenericServlet*** class and need to override the abstract service() method.

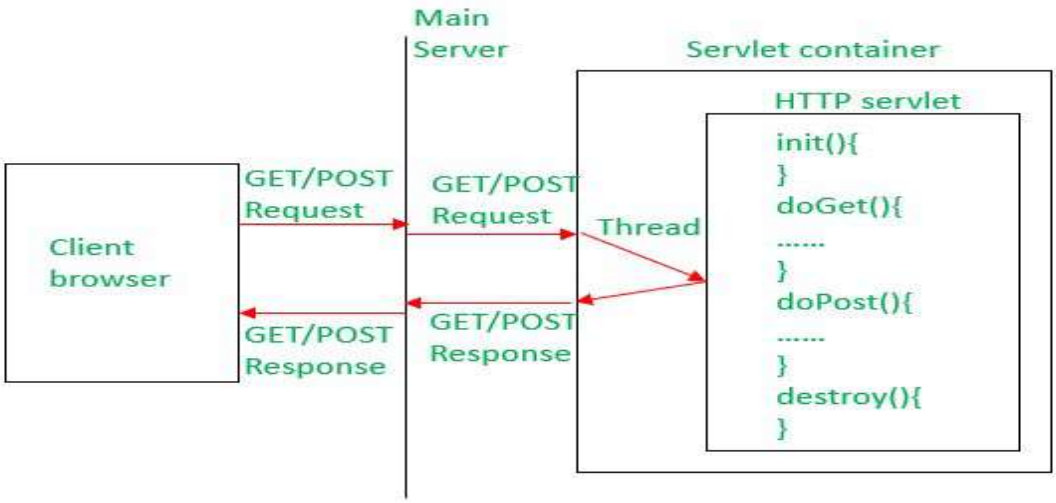
HttpServlet Class

Servlet API provides ***HttpServlet*** class in javax.servlet.http package.

- Java

```
public abstract class HttpServlet
extends GenericServlet
implements java.io.Serializable
```

- An abstract class to be subclassed to create an HTTP-specific servlet that is suitable for a Web site/Web application.
- Extends Generic Servlet class and implements Serializable interface.
- HTTP Protocol-dependent servlet.

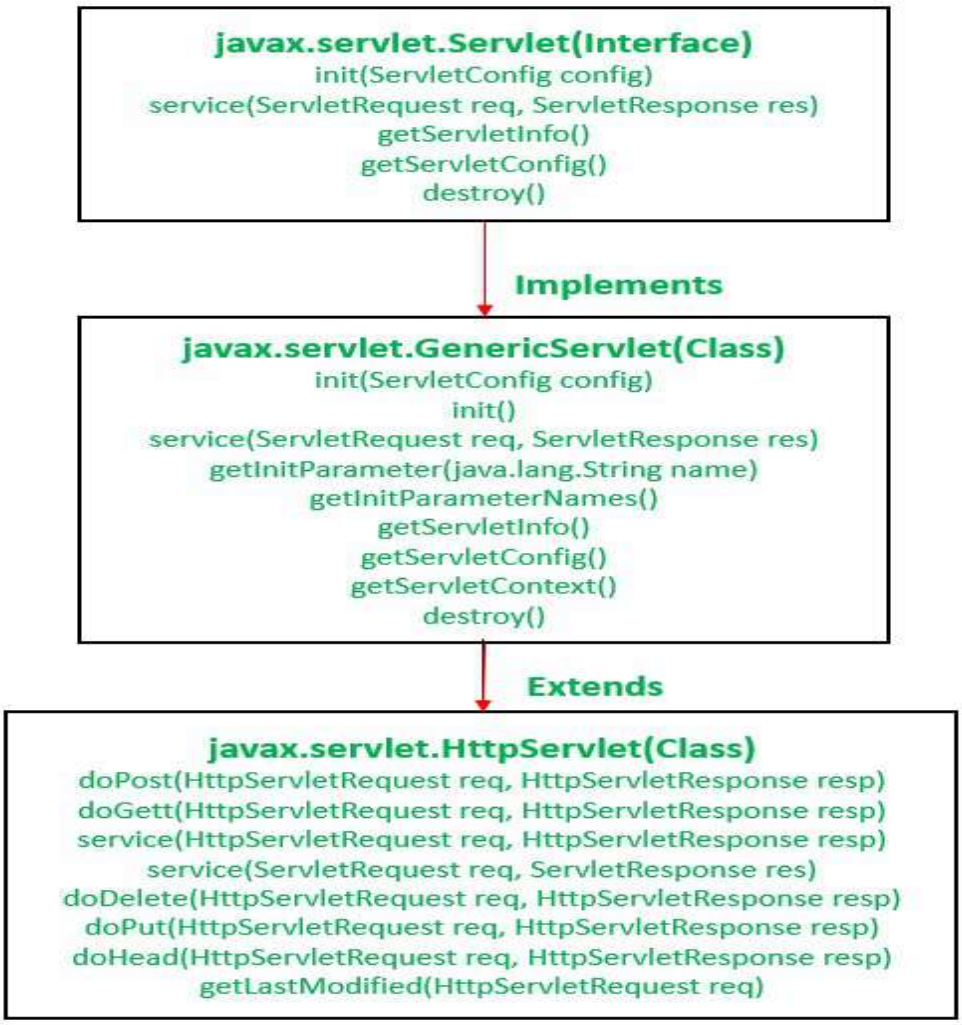


To write a Http servlet, you need to extend ***javax.servlet.http.HttpServlet*** class and must override at least one of the below methods,

- doGet() – to support HTTP GET requests by the servlet.
- doPost() – to support HTTP POST requests by the servlet.
- doPut() – to support HTTP PUT requests by the servlet.
- doDelete() – to support HTTP DELETE requests by the servlet.
- init() and destroy() – to manage resources held in the life of the servlet.
- getServletInfo() – To provide information about the servlet itself like the author of servlet or version of it etc.

Servlet API Package Hierarchy

Below is the package hierarchy of Generic Servlet and HTTP Servlet.



Servlet API provides all the required interfaces, classes, and methods to develop a web application. we need to add the ***servlet-api.jar*** file in our web application to use the servlet functionality. We can download this jar file from Maven Repository.

From <https://www.geeksforgeeks.org/servlet-api/>

Servlet Annotations

Package javax.servlet.annotation

The javax.servlet.annotation package contains a number of annotations that allow users to use annotations to declare servlets, filters, listeners and specify the metadata for the declared component

Annotation Types Summary	
HandlesTypes	This annotation is used to declare the class types that a ServletContainerInitializer can handle.
HttpConstraint	This annotation is used within the ServletSecurity annotation to represent the security constraints to be applied to all HTTP protocol methods for which a corresponding HttpMethodConstraint element does NOT occur within the ServletSecurity annotation.

HttpMethodConstraint	This annotation is used within the ServletSecurity annotation to represent security constraints on specific HTTP protocol messages.
MultipartConfig	Annotation that may be specified on a Servlet class, indicating that instances of the Servlet expect requests that conform to the multipart/form-data MIME type.
ServletSecurity	This annotation is used on a Servlet implementation class to specify security constraints to be enforced by a Servlet container on HTTP protocol messages.
WebFilter	Annotation used to declare a servlet filter.
WebInitParam	This annotation is used on a Servlet or Filter implementation class to specify an initialization parameter.
WebListener	This annotation is used to declare a WebListener.
WebServlet	Annotation used to declare a servlet.

From <https://docs.oracle.com/javaee/6/api/javax/servlet/annotation/package-summary.html>

? The Servlet interface

Interface javax.servlet.Servlet

This interface is for developing servlets. A servlet is a body of Java code that is loaded into and runs inside a servlet engine, such as a web server. It receives and responds to requests from clients. For example, a client may need information from a database; a servlet can be written that receives the request, gets and processes the data as needed by the client, and then returns it to the client.

All servlets implement this interface. Servlet writers typically do this by subclassing either GenericServlet, which implements the Servlet interface, or by subclassing GenericServlet's descendent, HttpServlet. Developers need to directly implement this interface only if their servlets cannot (or choose not to) inherit from GenericServlet or HttpServlet. For example, RMI or CORBA objects that act as servlets will directly implement this interface.

The Servlet interface defines methods to initialize a servlet, to receive and respond to client requests, and to destroy a servlet and its resources. These are known as life-cycle methods, and are called by the network service in the following manner:

1. Servlet is created then **init**ialized.
2. Zero or more **service** calls from clients are handled
3. Servlet is **destroy**ed then garbage collected and finalized

Initializing a servlet involves doing any expensive one-time setup, such as loading configuration data from files or starting helper threads. Service calls from clients are handled using a request and response paradigm. They rely on the underlying network transport to provide quality of service guarantees, such as reordering, duplication, message integrity, privacy, etc. Destroying a servlet involves undoing any initialization work and synchronizing persistent state with the current in-memory state.

In addition to the life-cycle methods, the Servlet interface provides for a method for the servlet to use to get any startup information, and a method that allows the servlet to return basic information about itself, such as its author, version and copyright.

From <https://docs.oracle.com/cd/E17802_01/products/products/servlet/2.1/api/javax.servlet.Servlet.html>

? The HttpServlet,

HttpServlet class

The HttpServlet class extends the GenericServlet class and implements Serializable interface. It provides http specific methods such as doGet, doPost, doHead, doTrace etc.

Methods of HttpServlet class

There are many methods in HttpServlet class. They are as follows:

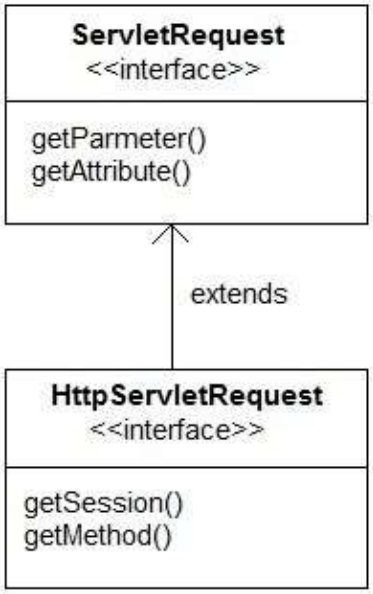
1. **public void service(ServletRequest req,ServletResponse res)** dispatches the request to the protected service method by converting the request and response object into http type.
2. **protected void service(HttpServletRequest req, HttpServletResponse res)** receives the request from the service method, and dispatches the request to the doXXX() method depending on the incoming http request type.
3. **protected void doGet(HttpServletRequest req, HttpServletResponse res)** handles the GET request. It is invoked by the web container.
4. **protected void doPost(HttpServletRequest req, HttpServletResponse res)** handles the POST request. It is invoked by the web container.
5. **protected void doHead(HttpServletRequest req, HttpServletResponse res)** handles the HEAD request. It is invoked by the web container.
6. **protected void doOptions(HttpServletRequest req, HttpServletResponse res)** handles the OPTIONS request. It is invoked by the web container.
7. **protected void doPut(HttpServletRequest req, HttpServletResponse res)** handles the PUT request. It is invoked by the web container.
8. **protected void doTrace(HttpServletRequest req, HttpServletResponse res)** handles the TRACE request. It is invoked by the web container.
9. **protected void doDelete(HttpServletRequest req, HttpServletResponse res)** handles the DELETE request. It is invoked by the web container.
10. **protected long getLastModified(HttpServletRequest req)** returns the time when HttpServletRequest was last modified since midnight January 1, 1970 GMT

From <https://www.javatpoint.com/HttpServlet-class>

HttpServletRequest,

HttpServletRequest interface

HttpServletRequest interface adds the methods that relates to the **HTTP** protocol.



Some important methods of HttpServletRequest

Methods	Description
String getContextPath()	returns the portion of the request URI that indicates the context of the request
Cookies getCookies()	returns an array containing all of the Cookie objects the client sent with this request
String getQueryString()	returns the query string that is contained in the request URL after the path
HttpSession getSession()	returns the current HttpSession associated with this request or, if there is no current session and create is true, returns a new session
String getMethod()	Returns the name of the HTTP method with which this request was made, for example, GET, POST, or PUT.
Part getPart(String name)	gets the Part with the given name
String getPathInfo()	returns any extra path information associated with the URL the client sent when it made this request.
String getServ	returns the part of this request's URL that calls the

letPath()	servlet

Example demonstrating Servlet Request

In this example, we will show how a parameter is passed to a Servlet in a request object from HTML page.

index.html

```
<formmethod="post"action="check">Name <inputtype="text"name="user">
<inputtype="submit"value="submit"></form>
```

Copy

web.xml

```
<servlet><servlet-name>check</servlet-name><servlet-class>MyServlet</servlet-class></servlet>
<servlet-mapping><servlet-name>check</servlet-name><url-pattern>/check</url-pattern></servlet-
mapping>
```

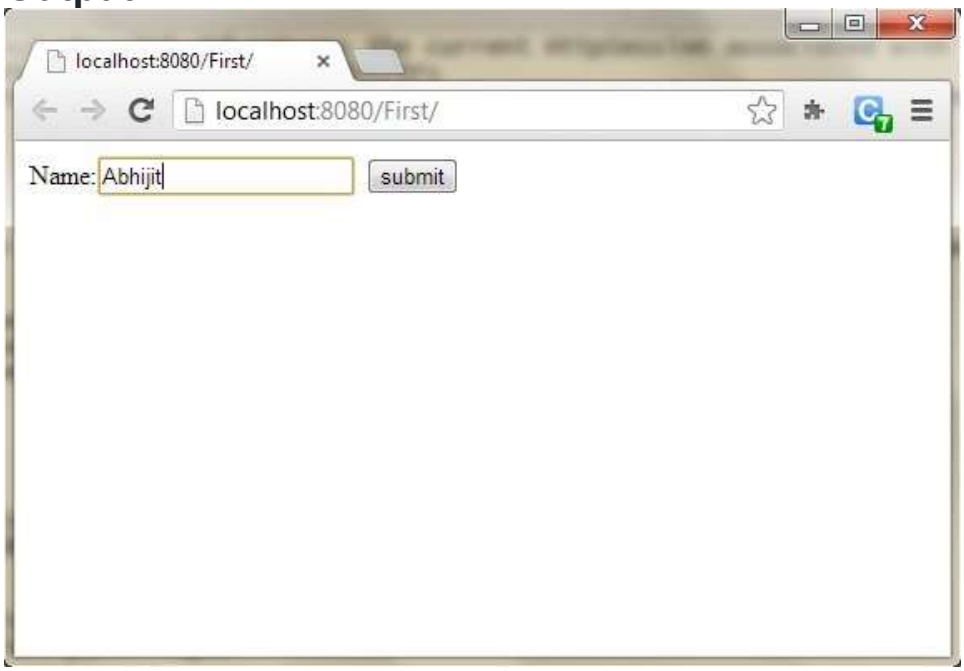
Copy

MyServlet.java

```
importjava.io.*;importjavax.servlet.*;importjavax.servlet.http.
*;publicclassMyServletextendsHttpServlet{protectedvoiddoPost(HttpServletRequestrequest,HttpServletRe
sponseresponse)
throwsServletException,IOException{response.setContentType("text/html;charset=UTF-8");PrintWriterout
=response.getWriter();try{Stringuser =request.getParameter("user");out.println("<h2> Welcome
"+user+"</h2>");}finally{out.close();}}}
```

Copy

Output :

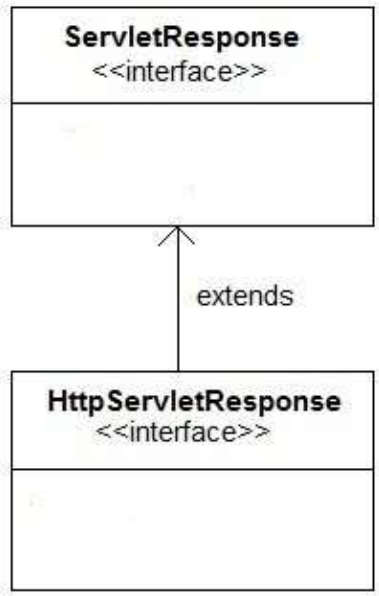


From <https://www.studytonight.com/servlet/servlet-request.php>

HttpServletResponse

HttpServletResponse Interface

HttpServletResponse interface adds the methods that relate to the **HTTP** response. It extends the ServletResponse interface. The object of HttpServletResponse is created in servlet container.



Some Important Methods of HttpServletResponse

Methods	Description
void addCookie(Cookie cookie)	adds the specified cookie to the response.
void sendRedirect(String location)	Sends a temporary redirect response to the client using the specified redirect location URL and clears the buffer
int getStatus()	gets the current status code of this response
String getHeader(String name)	gets the value of the response header with the given name.
void setHeader(String name, String value)	sets a response header with the given name and value
void setStatus(int sc)	sets the status code for this response
void sendError(int sc, String msg)	sends an error response to the client using the specified status and clears the buffer

From <https://www.studytonight.com/servlet/servlet-response.php>

? Exception Handling (ref. core java)

Keywords for Exception Handling in Java

The following are the primary keywords used in the process of Exception handling in Java.

Keyword	Description
try	The “try” keyword is used to specify the exception block
catch	The “catch” keyword is used to specify the solution
finally	The “finally” keyword has a mandatorily executable code
throw	The “throw” keyword throws an exception
throws	The “throws” keyword is used to declare an exception

With the keywords discussed, let us get into the central part of Exception Handling in Java. We will learn the methods of Exception Handling in Java.

Methods for Exception Handling in Java

There are two ways for Exception Handling in Java. They are as follows.

- Default Exception Handling in Java
- User-Defined Exception Handling in Java

Default Exception Handling in Java

Java Virtual machine handles default exceptions. The compiler identifies the presence of an exception, it quickly packs the recognized exception in the form of an object.

The compiler sends the exception object to the JVM during the run-time. This process is called throwing an Exception.

After the exception is thrown, many methods could get executed in response to the thrown exception. The list is known as the Call Stack. The Call Stack is described as follows.

- The thrown exception reaches the call stack, and the call stack responds with an exception handler that could handle the thrown exception.
- If the run-time system fails to recognize the appropriate exception handler for the thrown exception, then the run-time system sends the exception object to the default exception handler.
- The default exception handler results in an abnormal output that reads out a report related to the bizarre exception encounter.

User-Defined Exception Handling in Java

The customized/user-defined exception handling in Java is managed by using the exception handling keywords. They are:

- try
- catch
- throw
- throws
- finally

In customized exception handling, the user should recognize/expect an exception at a specific part of the code segment.

Once the location of the exception is finalized, then, the code block should be enclosed inside the try and catch blocks.

With this setup, whenever the code throws an exception, it gets handled by the appropriate catch block. The "throw" keyword is used to manually throw an exception. When an exception is thrown from the program, it is identified by the "throws" clause.

Now that we have clarity on the exceptions and procedures for Exception Handling in Java, we will directly deal with some of the frequently faced exceptions in Java and resolve them programmatically.

From <https://www.simplilearn.com/tutorials/java-tutorial/exception-handling-in-java>

? DAO

DAO stands for Data Access Object. DAO Design Pattern is used to separate the data

persistence logic in a separate layer. This way, the service remains completely in dark about how the low-level operations to access the database is done. This is known as the principle of **Separation of Logic**.

DAO Design Pattern

With DAO design pattern, we have following components on which our design depends:

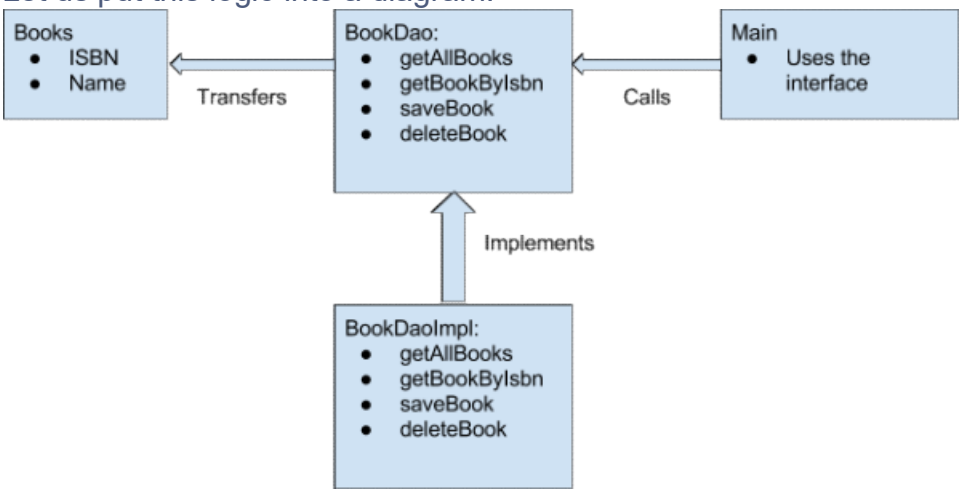
- The model which is transferred from one layer to the other.
- The interfaces which provides a flexible design.
- The interface implementation which is a concrete implementation of the persistence logic.

Implementing DAO pattern

With above mentioned components, let's try to implement the DAO pattern. We will use 3 components here:

- The `Book` model which is transferred from one layer to the other.
- The `BookDao` interface that provides a flexible design and API to implement.
- `BookDaoImpl` concrete class that is an implementation of the `BookDao` interface.

Let us put this logic into a diagram:



Advantages of DAO pattern

There are many advantages for using DAO pattern. Let's state some of them here:

- While changing a persistence mechanism, service layer doesn't even have to know where the data comes from. For example, if you're thinking of shifting from using MySQL to MongoDB, all changes are needed to be done in the DAO layer only.
- DAO pattern emphasis on the low coupling between different components of an application. So, the View layer have no dependency on DAO layer and only Service layer depends on it, even that with the interfaces and not from concrete implementation.
- As the persistence logic is completely separate, it is much easier to write Unit tests for individual components. For example, if you're using JUnit and Mockito for testing frameworks, it will be easy to mock the individual components of your application.
- As we work with interfaces in DAO pattern, it also emphasizes the style of "work with interfaces instead of implementation" which is an excellent OOPs style of programming.

DAO Pattern Conclusion

In this article, we learned how we can put DAO design pattern to use to emphasize on keeping persistence logic separate and so, our components loosely coupled. Design patterns are just based on a way of programming and so, is language and framework independent.

From <https://www.digitalocean.com/community/tutorials/dao-design-pattern>

POJO

POJO in Java stands for Plain Old Java Object. It is an ordinary object, which is not bound by any special restriction. The POJO file does not require any special classpath. It increases the readability & re-usability of a Java program.
POJOs are now widely accepted due to their easy maintenance. They are easy to read and write. A POJO class does not have any naming convention for properties and methods. It is not tied to any Java Framework; any Java Program can use it.
The term POJO was introduced by **Martin Fowler** (An American software developer) in 2000. it is available in Java from the EJB 3.0 by sun microsystem.
Generally, a POJO class contains variables and their Getters and Setters.
The POJO classes are similar to Beans as both are used to define the objects to increase the readability and re-usability. The only difference between them that Bean Files have some restrictions but, the POJO files do not have any special restrictions.

Example:

POJO class is used to define the object entities. For example, we can create an Employee POJO class to define its objects.
Below is an example of Java POJO class:
Employee.java:

```
// POJO class Exmaple
package Jtp.PojoDemo;
public class Employee {
private String name;
private String id;
private double sal;
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public String getId() {
    return id;
}
public void setId(String id) {
    this.id = id;
}
public double getSal() {
    return sal;
}
public void setSal(double sal) {
    this.sal = sal;
}
}
```

The above employee class is an example of an employee POJO class. If you are working on Eclipse, you can easily generate Setters and Getters by right click on the Java Program and navigate to **Source-> Generate Getters and Setters.**

```java
package Jtp.PojoDemo;

public class Employee {
String name;
String id;
double sal;
}
```

Right-click on the Java Program and Select Generate Getters and Setters.

Now, click on the **Generate** option given at the bottom of the Generate window. It will auto-generate setters and getters.

Properties of POJO class

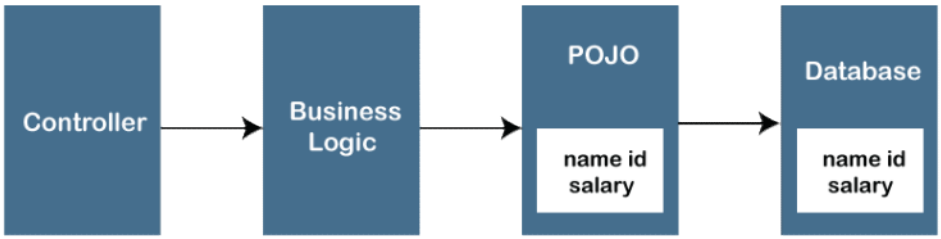
Below are some properties of the POJO class:

- The POJO class must be public.
- It must have a public default constructor.
- It may have the arguments constructor.
- All objects must have some public Getters and Setters to access the object values by other Java Programs.
- The object in the POJO Class can have any access modifies such as private, public, protected. But, all instance variables should be private for improved security of the project.
- A POJO class should not extend predefined classes.
- It should not implement prespecified interfaces.
- It should not have any prespecified annotation.

Working of POJO Class

The POJO class is an object class that encapsulates the Business logic. In an MVC architecture, the Controller interacts with the business logic, which contacts with POJO class to access the data.
Below is the working of the POJO class.

Java Beans



How to use POJO class in a Java Program

The POJO class is created to use the objects in other Java Programs. The major advantage of the POJO class is that we will not have to create objects every time in other Java programs. Simply we can access the objects by using the get() and set() methods.

To access the objects from the POJO class, follow the below steps:

- Create a POJO class objects
- Set the values using the set() method
- Get the values using the get() method

For example, create a MainClass.java class file within the same package and write the following code in it:

MainClass.java:

```
//Using POJO class objects in MainClass Java program
package Jtp.PojoDemo;
public class MainClass {
    public static void main(String[] args) {
        // Create an Employee class object
Employee obj= new Employee();
obj.setName("Alisha"); // Setting the values using the set() method
obj.setId("A001");
obj.setSal(200000);
System.out.println("Name: "+ obj.getName()); //Getting the values using the get() metho
d
System.out.println("Id: " + obj.getId());
System.out.println("Salary: " +obj.getSal());
    }
}
```

Output:

```
Name: Alisha
Id: A001
Salary: 200000.0
```

From the above example, we can see we have accessed the POJO class properties in MainClass.java.

POJO is similar to Bean Class, so people often get confused between them; let's see the difference between the POJO and Bean.

Java Bean

Java Bean class is also an object class that encapsulates several objects into a single

file (Bean Class File). There are some differences between POJO and Bean.

Java POJO and Bean in a nutshell:

- All the Bean files can be POJOs, but not all the POJOs are beans.
- All Bean files can implement a Serializable interface but, not all the POJOs can implement a Serializable interface.
- Both properties should be private to have complete control of fields.
- Properties must have the getters and setter to access them in other Java programs.
- The Bean class is a sub-set of the POJO class.
- There is no major disadvantage of using the POJO, but few disadvantages may be using the Bean class.
- The POJO is used when we want to provide full access to users and restrict our members. And, the Bean is used when we want to provide partial access to the members.

POJO Vs. Bean

POJO	Bean
In Pojo, there are no special restrictions other than Java conventions.	It is a special type of POJO files, which have some special restrictions other than Java conventions.
It provides less control over the fields as compared to Bean.	It provides complete protection on fields.
The POJO file can implement the Serializable interface; but, it is not mandatory.	The Bean class should implement the Serializable interface.
The POJO class can be accessed by using their names.	The Bean class can only be accessed by using the getters and setters.
Fields may have any of the access modifiers such as public, private, protected.	Fields can only have private access.
In POJO, it is not necessary to have a no-arg constructor; it may or may not have it.	It must have a no-arg constructor.
There is not any disadvantage to using the POJO	The disadvantage of using the Bean is that the Default constructor and public setter can change the object state when it should be immutable.

From <https://www.javatpoint.com/pojo-in-java>

? Session

? Session Management

? Session Tracking with
o Cookies
o HttpSession

HTTP is a "stateless" protocol which means each time a client retrieves a Web page, the client opens a separate connection to the Web server and the server automatically does not keep any record of previous client request.
Still there are following three ways to maintain session between web client and web server:

Cookies

A webserver can assign a unique session ID as a cookie to each web client and for subsequent requests from the client they can be recognized using the recieved cookie.
This may not be an effective way because many time browser does not support a cookie, so I would not recommend to use this procedure to maintain the sessions.

Hidden Form Fields

A web server can send a hidden HTML form field along with a unique session ID as follows:
<input type="hidden" name="sessionid" value="12345">
This entry means that, when the form is submitted, the specified name and value are automatically included in the GET or POST data. Each time when web browser sends request back, then session_id value can be used to keep the track of different web browsers.
This could be an effective way of keeping track of the session but clicking on a regular (<A HREF...>) hypertext link does not result in a form submission, so hidden form fields also cannot support general session tracking.

URL Rewriting

You can append some extra data on the end of each URL that identifies the session, and the server can associate that session identifier with data it has stored about that session.
For example, with http://tutorialspoint.com/file.htm;sessionid=12345, the session identifier is attached as sessionid=12345 which can be accessed at the web server to identify the client.
URL rewriting is a better way to maintain sessions and it works even when browsers don't support cookies. The drawback of URL re-writing is that you would have to generate every URL dynamically to assign a session ID, even in case of a simple static HTML page.

The HttpSession Object

Apart from the above mentioned three ways, servlet provides HttpSession Interface which provides a way to identify a user across more than one page request or visit to a Web site and to store information about that user.
The servlet container uses this interface to create a session between an HTTP client and an HTTP server. The session persists for a specified time period, across more than one connection or page request from the user.
You would get HttpSession object by calling the public method getSession() of HttpServletRequest, as below:

```
HttpSession session = request.getSession();
```

You need to call request.getSession() before you send any document content to the client. Here is a summary of the important methods available through HttpSession object:

S.N.	Method & Description
1	**public Object getAttribute(String name)** This method returns the object bound with the specified name in this session, or null if no object is bound under the name.
2	**public Enumeration getAttributeNames()** This method returns an Enumeration of String objects containing the names of all the objects bound to this session.
3	**public long getCreationTime()** This method returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.
4	**public String getId()** This method returns a string containing the unique identifier assigned to this session.
5	**public long getLastAccessedTime()** This method returns the last accessed time of the session, in the format of milliseconds since midnight January 1, 1970 GMT.
6	**public int getMaxInactiveInterval()** This method returns the maximum time interval (seconds), that the servlet container will keep the session open between client accesses.
7	**public void invalidate()** This method invalidates this session and unbinds any objects bound to it.
8	**public boolean isNew(** This method returns true if the client does not yet know about the session or if the client chooses not to join the session.
9	**public void removeAttribute(String name)** This method removes the object bound with the specified name from this session.
10	**public void setAttribute(String name, Object value)** This method binds an object to this session, using the name specified.
11	**public void setMaxInactiveInterval(int interval)** This method specifies the time, in seconds, between client requests before the servlet container will invalidate this session.

Session Tracking Example

This example describes how to use the HttpSession object to find out the creation time and the last-accessed time for a session. We would associate a new session with the request if one does not already exist.

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

// Extend HttpServlet class
public class SessionTrack extends HttpServlet {

  public void doGet(HttpServletRequest request,
                    HttpServletResponse response)
            throws ServletException, IOException
  {
      // Create a session object if it is already not  created.
      HttpSession session = request.getSession(true);
      // Get session creation time.
      Date createTime = new Date(session.getCreationTime());
      // Get last access time of this web page.
      Date lastAccessTime =
                        new Date(session.getLastAccessedTime());

      String title = "Welcome Back to my website";
      Integer visitCount = new Integer(0);
      String visitCountKey = new String("visitCount");
```

```
String userIDKey = new String("userID");
String userID = new String("ABCD");

// Check if this is new comer on your web page.
if (session.isNew()){
   title = "Welcome to my website";
   session.setAttribute(userIDKey, userID);
} else {
   visitCount = (Integer)session.getAttribute(visitCountKey);
   visitCount = visitCount + 1;
   userID = (String)session.getAttribute(userIDKey);
}
session.setAttribute(visitCountKey,  visitCount);

// Set response content type
response.setContentType("text/html");
PrintWriter out = response.getWriter();

String docType =
"<!doctype html public \"-//w3c//dtd html 4.0 " +
"transitional//en\">\n";
out.println(docType +
          "<html>\n" +
          "<head><title>" + title + "</title></head>\n" +
          "<body bgcolor=\"#f0f0f0\">\n" +
          "<h1 align=\"center\">" + title + "</h1>\n" +
           "<h2 align=\"center\">Session Infomation</h2>\n" +
          "<table border=\"1\" align=\"center\">\n" +
          "<tr bgcolor=\"#949494\">\n" +
          "  <th>Session info</th><th>value</th></tr>\n" +
          "<tr>\n" +
          "  <td>id</td>\n" +
          "  <td>" + session.getId() + "</td></tr>\n" +
          "<tr>\n" +
          "  <td>Creation Time</td>\n" +
          "  <td>" + createTime +
```

```
                "  </td></tr>\n" +
                "<tr>\n" +
                "  <td>Time of Last Access</td>\n" +
                "  <td>" + lastAccessTime +
                "  </td></tr>\n" +
                "<tr>\n" +
                "  <td>User ID</td>\n" +
                "  <td>" + userID +
                "  </td></tr>\n" +
                "<tr>\n" +
                "  <td>Number of visits</td>\n" +
                "  <td>" + visitCount + "</td></tr>\n" +
                "</table>\n" +
                "</body></html>");
  }
}
```

Compile the above servlet **SessionTrack** and create appropriate entry in web.xml file. Now running *http://localhost:8080/SessionTrack* would display the following result when you would run for the first time:

Welcome Back to my website

Session Infomation

info type	value
id	0AE3EC93FF44E3C525B4351B77ABB2D5
Creation Time	Tue Jun 08 17:26:40 GMT+04:00 2010
Time of Last Access	Tue Jun 08 17:26:40 GMT+04:00 2010
User ID	ABCD
Number of visits	1

The timeout is expressed as minutes, and overrides the default timeout which is 30 minutes in Tomcat. The getMaxInactiveInterval( ) method in a servlet returns the timeout period for that session in seconds. So if your session is configured in web.xml for 15 minutes, getMaxInactiveInterval( ) returns 900.

Difference between Session and Cookies

1. Session :

A session is used to save information on the server momentarily so that it may be utilized across various pages of the website. It is the overall amount of time spent on an activity. The user session begins when the user logs in to a specific network application and ends when the user logs out of the program or shuts down the machine.

Session values are far more secure since they are saved in binary or encrypted form and can only be decoded at the server. When the user shuts down the machine or logs out of the program, the session values are automatically deleted.

We must save the values in the database to keep them forever.

2. Cookie :

A cookie is a small text file that is saved on the user's computer. The maximum file size for a cookie is 4KB. It is also known as an HTTP cookie, a web cookie, or an internet cookie. When a user first visits a website, the site sends data packets to the user's computer in the form of a cookie.

The information stored in cookies is not safe since it is kept on the client-side in a text format that anybody can see. We can activate or disable cookies based on our needs.

Difference Between Session and Cookies :

Cookie	Session
Cookies are client-side files on a local computer that hold user information.	Sessions are server-side files that contain user data.
Cookies end on the lifetime set by the user.	When the user quits the browser or logs out of the programmed, the session is over.
It can only store a certain amount of info.	It can hold an indefinite quantity of data.
The browser's cookies have a maximum capacity of 4 KB.	We can keep as much data as we like within a session, however there is a maximum memory restriction of 128 MB that a script may consume at one time.
Because cookies are kept on the local computer, we don't need to run a function to start them.	To begin the session, we must use the session start() method.
Cookies are not secured.	Session are more secured compare than cookies.
Cookies stored data in text file.	Session save data in encrypted form.
Cookies stored on a limited data.	Session stored a unlimited data.
In PHP, to get the data from Cookies , $_COOKIES the global variable is used	In PHP , to set the data from Session, $_SESSION the global variable is used
We can set an expiration date to delete the cookie's data. It will automatically delete the data at that specific time.	In PHP, to destroy or remove the data stored within a session, we can use the session_destroy() function, and to unset a specific variable, we can use the unset() function.

From <https://www.geeksforgeeks.org/difference-between-session-and-cookies/>

o Cookies

Cookies are text files stored on the client computer and they are kept for various information tracking purpose. Java Servlets transparently supports HTTP cookies.

There are three steps involved in identifying returning users:

- Server script sends a set of cookies to the browser. For example name, age, or identification number etc.
- Browser stores this information on local machine for future use.
- When next time browser sends any request to web server then it sends those cookies information to the server and server uses that information to identify the user.

This chapter will teach you how to set or reset cookies, how to access them and how to delete them.

The Anatomy of a Cookie

Cookies are usually set in an HTTP header (although JavaScript can also set a cookie directly on a browser). A servlet that sets a cookie might send headers that look something like this:

```
HTTP/1.1 200 OK
Date: Fri, 04 Feb 2000 21:03:38 GMT
Server: Apache/1.3.9 (UNIX) PHP/4.0b3
Set-Cookie: name=xyz; expires=Friday, 04-Feb-07 22:03:38 GMT;
                path=/; domain=tutorialspoint.com
Connection: close
Content-Type: text/html
```

As you can see, the Set-Cookie header contains a name value pair, a GMT date, a path and a domain. The name and value will be URL encoded. The expires field is an instruction to the browser to "forget" the cookie after the given time and date.

If the browser is configured to store cookies, it will then keep this information until the expiry date. If the user points the browser at any page that matches the path and domain of the cookie, it will resend the cookie to the server. The browser's headers might look something like this:

```
GET / HTTP/1.0
Connection: Keep-Alive
User-Agent: Mozilla/4.6 (X11; I; Linux 2.2.6-15apmac ppc)
```

```
Host: zink.demon.co.uk:1126
Accept: image/gif, */*
Accept-Encoding: gzip
Accept-Language: en
Accept-Charset: iso-8859-1,*,utf-8
Cookie: name=xyz
```

A servlet will then have access to the cookie through the request method *request.getCookies()* which returns an array of *Cookie* objects.

Servlet Cookies Methods

Following is the list of useful methods which you can use while manipulating cookies in servlet.

S.N.	Method & Description
1	**public void setDomain(String pattern)** This method sets the domain to which cookie applies, for example tutorialspoint.com.
2	**public String getDomain()** This method gets the domain to which cookie applies, for example tutorialspoint.com.
3	**public void setMaxAge(int expiry)** This method sets how much time (in seconds) should elapse before the cookie expires. If you don't set this, the cookie will last only for the current session.
4	**public int getMaxAge()** This method returns the maximum age of the cookie, specified in seconds, By default, -1 indicating the cookie will persist until browser shutdown.
5	**public String getName()** This method returns the name of the cookie. The name cannot be changed after creation.
6	**public void setValue(String newValue)** This method sets the value associated with the cookie.
7	**public String getValue()** This method gets the value associated with the cookie.
8	**public void setPath(String uri)** This method sets the path to which this cookie applies. If you don't specify a path, the cookie is returned for all URLs in the same directory as the current page as well as all subdirectories.
9	**public String getPath()** This method gets the path to which this cookie applies.
10	**public void setSecure(boolean flag)** This method sets the boolean value indicating whether the cookie should only be sent over encrypted (i.e. SSL) connections.
11	**public void setComment(String purpose)** This method specifies a comment that describes a cookie's purpose. The comment is useful if the browser presents the cookie to the user.
12	**public String getComment()**

	This method returns the comment describing the purpose of this cookie, or null if the cookie has no comment.

Setting Cookies with Servlet

Setting cookies with servlet involves three steps:

(1) Creating a Cookie object: You call the Cookie constructor with a cookie name and a cookie value, both of which are strings.

```
Cookie cookie = new Cookie("key","value");
```

Keep in mind, neither the name nor the value should contain white space or any of the following characters:

```
[ ] ( ) = , " / ? @ : ;
```

(2) Setting the maximum age: You use setMaxAge to specify how long (in seconds) the cookie should be valid. Following would set up a cookie for 24 hours.

```
cookie.setMaxAge(60*60*24);
```

(3) Sending the Cookie into the HTTP response headers: You use response.addCookie to add cookies in the HTTP response header as follows:

```
response.addCookie(cookie);
```

Example

Let us modify our Form Example to set the cookies for first and last name.

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class HelloForm extends HttpServlet {

  public void doGet(HttpServletRequest request,
                    HttpServletResponse response)
            throws ServletException, IOException
  {
      // Create cookies for first and last names.
      Cookie firstName = new Cookie("first_name",
```

```
                    request.getParameter("first_name"));
      Cookie lastName = new Cookie("last_name",
                    request.getParameter("last_name"));

      // Set expiry date after 24 Hrs for both the cookies.
      firstName.setMaxAge(60*60*24);
      lastName.setMaxAge(60*60*24);

      // Add both the cookies in the response header.
      response.addCookie( firstName );
      response.addCookie( lastName );

      // Set response content type
      response.setContentType("text/html");

      PrintWriter out = response.getWriter();
      String title = "Setting Cookies Example";
      String docType =
      "<!doctype html public \"-//w3c//dtd html 4.0 " +
      "transitional//en\">\n";
      out.println(docType +
                "<html>\n" +
                "<head><title>" + title + "</title></head>\n" +
                "<body bgcolor=\"#f0f0f0\">\n" +
                "<h1 align=\"center\">" + title + "</h1>\n" +
                "<ul>\n" +
                "  <li><b>First Name</b>: "
                + request.getParameter("first_name") + "\n" +
                "  <li><b>Last Name</b>: "
                + request.getParameter("last_name") + "\n" +
                "</ul>\n" +
                "</body></html>");
  }
}
```

Compile the above servlet **HelloForm** and create appropriate entry in web.xml file and finally try following HTML page to call servlet.

```
<html>
<body>
<form action="HelloForm" method="GET">
First Name: <input type="text" name="first_name">
<br />
Last Name: <input type="text" name="last_name" />
<input type="submit" value="Submit" />
</form>
</body>
</html>
```

Keep above HTML content in a file Hello.htm and put it in <Tomcat-installation-directory>/webapps/ROOT directory. When you would access *http://localhost:8080/Hello.htm*, here is the actual output of the above form.

First Name:

Last Name: Submit

Try to enter First Name and Last Name and then click submit button. This would display first name and last name on your screen and same time it would set two cookies firstName and lastName which would be passed back to the server when next time you would press Submit button.

Next section would explain you how you would access these cookies back in your web application.

Reading Cookies with Servlet

To read cookies, you need to create an array of *javax.servlet.http.Cookie* objects by calling the **getCookies( )** method of *HttpServletRequest*. Then cycle through the array, and use getName() and getValue() methods to access each cookie and associated value.

Example

Let us read cookies which we have set in previous example:

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
```

```
    // Get an array of Cookies associated with this domain
    cookies = request.getCookies();

     // Set response content type
    response.setContentType("text/html");

    PrintWriter out = response.getWriter();
    String title = "Delete Cookies Example";
    String docType =
    "<!doctype html public \"-//w3c//dtd html 4.0 " +
    "transitional//en\">\n";
    out.println(docType +
              "<html>\n" +
              "<head><title>" + title + "</title></head>\n" +
              "<body bgcolor=\"#f0f0f0\">\n" );
     if( cookies != null ){
       out.println("<h2> Cookies Name and Value</h2>");
       for (int i = 0; i < cookies.length; i++){
          cookie = cookies[i];
          if((cookie.getName( )).compareTo("first_name") == 0 ){
               cookie.setMaxAge(0);
               response.addCookie(cookie);
               out.print("Deleted cookie : " +
                            cookie.getName( ) + "<br/>");
          }
          out.print("Name : " + cookie.getName( ) + ",  ");
          out.print("Value: " + cookie.getValue( )+" <br/>");
       }
    }else{
        out.println(
          "<h2>No cookies founds</h2>");
    }
    out.println("</body>");
    out.println("</html>");
}
```

```
   }
}
```

Compile above servlet **ReadCookies** and create appropriate entry in web.xml file. If you would have set first_name cookie as "John" and last_name cookie as "Player" then running *http:///localhost:8080/ReadCookies* would display the following result:

```
                Found Cookies Name and Value

Name : first_name, Value: John

Name : last_name,  Value: Player
```

Delete Cookies with Servlet

To delete cookies is very simple. If you want to delete a cookie then you simply need to follow up following three steps:

- Read an already existing cookie and store it in Cookie object.
- Set cookie age as zero using **setMaxAge()** method to delete an existing cookie.
- Add this cookie back into response header.

Example

The ollowing example would delete and existing cookie named "first_name" and when you would run ReadCookies servlet next time it would return null value for first_name.

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class DeleteCookies extends HttpServlet {

  public void doGet(HttpServletRequest request,
                    HttpServletResponse response)
            throws ServletException, IOException
  {
      Cookie cookie = null;

       Cookie[] cookies = null;
```

```
}
```

Compile above servlet **DeleteCookies** and create appropriate entry in web.xml file. Now running *http://localhost:8080/DeleteCookies* would display the following result:

```
                    Cookies Name and Value

Deleted cookie : first_name

Name : first_name, Value: John

Name : last_name,  Value: Player
```

Now try to run *http://localhost:8080/ReadCookies* and it would display only one cookie as follows:

```
                  Found Cookies Name and Value

Name : last_name,  Value: Player
```

You can delete your cookies in Internet Explorer manually. Start at the Tools menu and select Internet Options. To delete all cookies, press Delete Cookies.

? Request Dispatcher

RequestDispatcher in Servlet

The RequestDispatcher interface provides the facility of dispatching(to send something) the request to another resource it may be html, servlet or jsp. This interface can also be used to include the content of another resource also. It is one of the way of servlet collaboration.
There are two methods defined in the RequestDispatcher interface.

Methods of RequestDispatcher interface

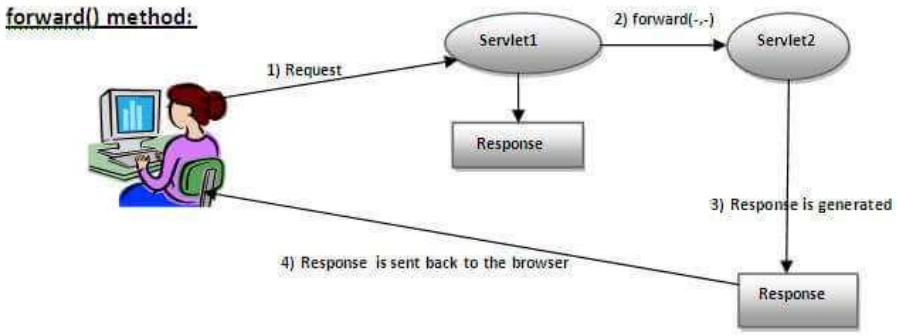
The RequestDispatcher interface provides two methods. They are:

1. **public void forward(ServletRequest request,ServletResponse response) throws ServletException,java.io.IOException:**

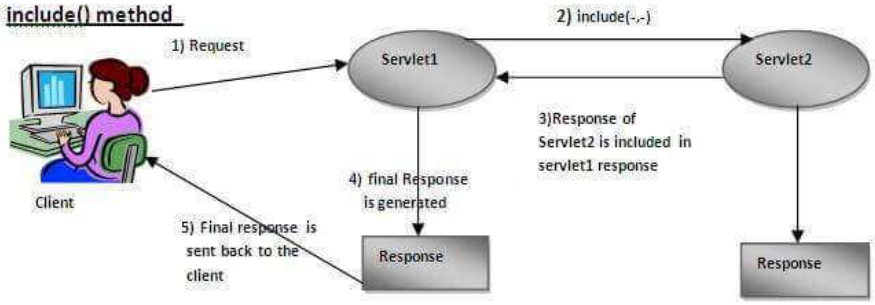
   Forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.

2. **public void include(ServletRequest request,ServletResponse response) throws ServletException,java.io.IOException:**

   Includes the content of a resource (servlet, JSP page, or HTML file) in the response.



As you see in the above figure, response of second servlet is sent to the client. Response of the first servlet is not displayed to the user.



As you can see in the above figure, response of second servlet is included in the response of the first servlet that is being sent to the client.

How to get the object of RequestDispatcher

The getRequestDispatcher() method of ServletRequest interface returns the object of RequestDispatcher. Syntax:

Syntax of getRequestDispatcher method

```
public RequestDispatcher getRequestDispatcher(String resource);
```

Example of using getRequestDispatcher method

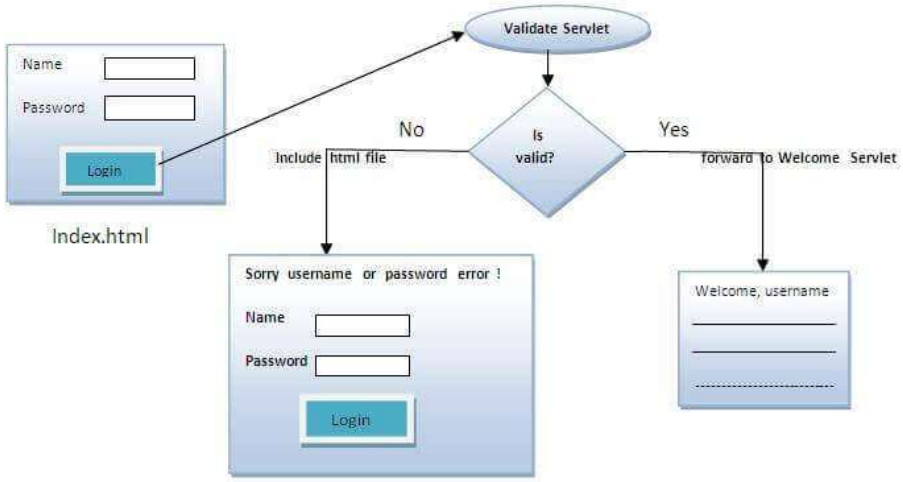
```
RequestDispatcher rd=request.getRequestDispatcher("servlet2");
//servlet2 is the url-pattern of the second servlet

rd.forward(request, response);//method may be include or forward
```

Example of RequestDispatcher interface

In this example, we are validating the password entered by the user. If password is servlet, it will forward the request to the WelcomeServlet, otherwise will show an error message: sorry username or password error!. In this program, we are cheking for hardcoded information. But you can check it to the database also that we will see in the

development chapter. In this example, we have created following files:

- **index.html file:** for getting input from the user.
- **Login.java file:** a servlet class for processing the response. If password is servet, it will forward the request to the welcome servlet.
- **WelcomeServlet.java file:** a servlet class for displaying the welcome message.
- **web.xml file:** a deployment descriptor file that contains the information about the servlet.



index.html

```
<form action="servlet1" method="post">
Name:<input type="text" name="userName"/><br/>
Password:<input type="password" name="userPass"/><br/>
<input type="submit" value="login"/>
</form>
```

Login.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;


public class Login extends HttpServlet {

public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    String n=request.getParameter("userName");
    String p=request.getParameter("userPass");

    if(p.equals("servlet")){
        RequestDispatcher rd=request.getRequestDispatcher("servlet2");
```

```
        rd.forward(request, response);
    }
    else{
        out.print("Sorry UserName or Password Error!");
        RequestDispatcher rd=request.getRequestDispatcher("/index.html");
        rd.include(request, response);

        }
    }

}
```

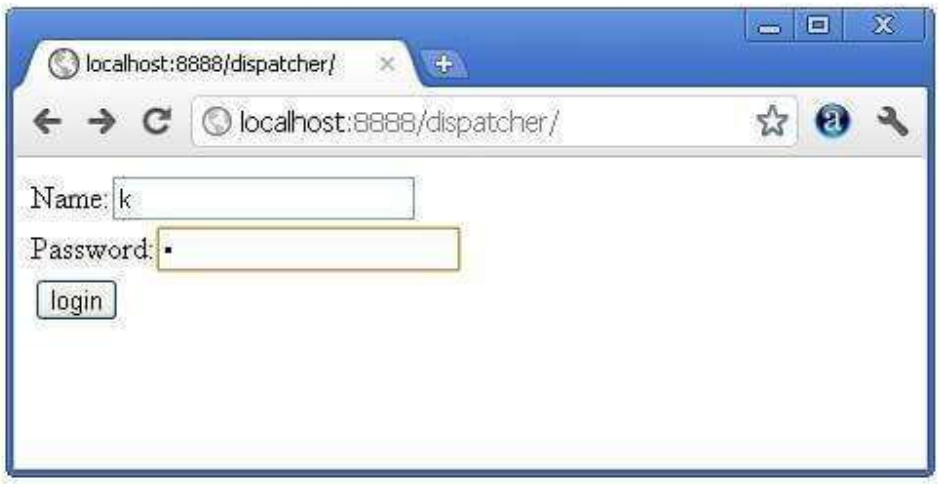
WelcomeServlet.java

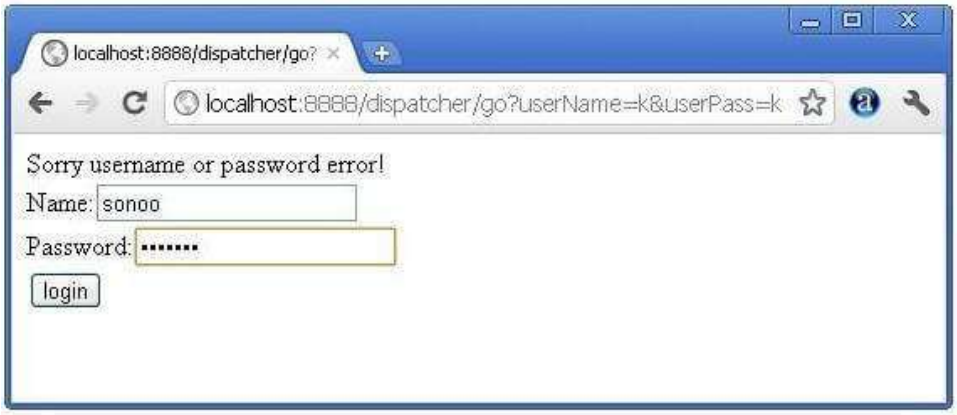
```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class WelcomeServlet extends HttpServlet {

    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    String n=request.getParameter("userName");
    out.print("Welcome "+n);
    }

}
```

web.xml

```
<web-app>
 <servlet>
    <servlet-name>Login</servlet-name>
    <servlet-class>Login</servlet-class>
  </servlet>
  <servlet>
    <servlet-name>WelcomeServlet</servlet-name>
    <servlet-class>WelcomeServlet</servlet-class>
  </servlet>


  <servlet-mapping>
    <servlet-name>Login</servlet-name>
    <url-pattern>/servlet1</url-pattern>
  </servlet-mapping>
  <servlet-mapping>
    <servlet-name>WelcomeServlet</servlet-name>
```

```
    <url-pattern>/servlet2</url-pattern>
  </servlet-mapping>

  <welcome-file-list>
   <welcome-file>index.html</welcome-file>
  </welcome-file-list>
</web-app>
```





From <https://www.javatpoint.com/requestdispatcher-in-servlet>

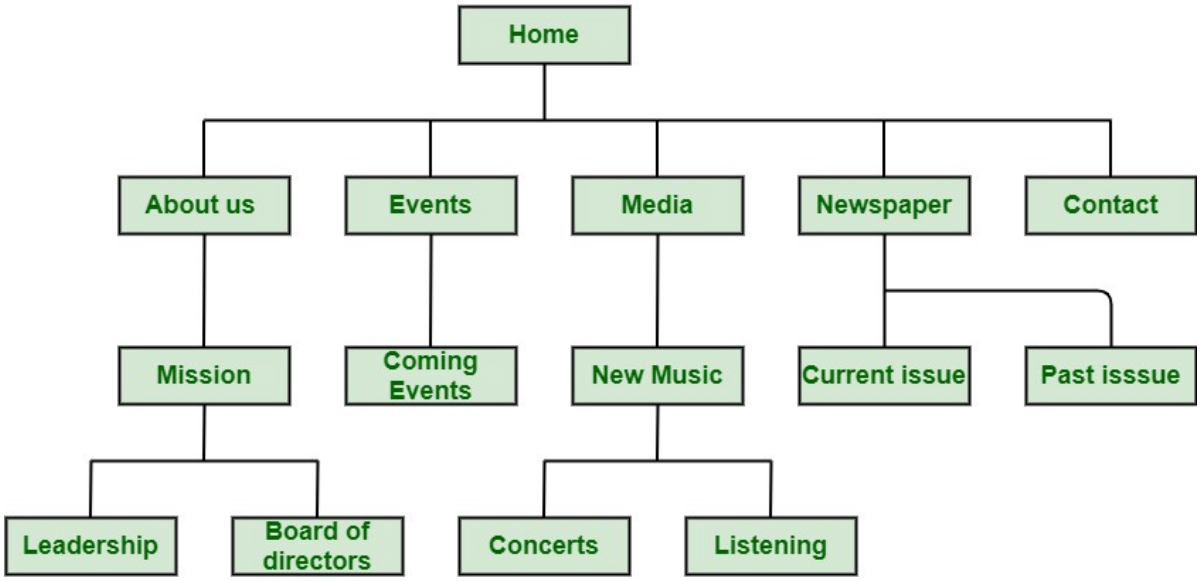
Page Navigation

Website Navigation and its Importance

Website Navigation, as name suggests, is a way to showing importance and relevance of pages, content, and information on website. It is way that helps users or visitors to find content that they want to see. It is simply a way of navigating pages, content and information on website using internet. It can be presented in different ways such as spider bars, menus in header or footer, etc. Simplicity is very essential and important for good website navigation.

Structure of Website Navigation

Structure of website navigation is very important as it have a greater impact on sales, user experience and SEO (Search Engine Optimization) rankings. It not only includes sketching pages but also include greater level of abstraction. It is not at all easy to design and organize website. Let us see an example:



In above diagram, about us, events, media, contact, newspaper, etc., are all linked to home menu. To have access to other pages such as mission, coming events, new music, etc., you have to first visit about us, events, media, etc. Then you can open whatever you want to.

Importance of website Navigation

Website navigation is way by which one can easily increase audience or number of visitors on website. If website navigation is not good and effective, there is not point of having a creative website because users won't be able to navigate easily around it to find information they want. So, it's very important to keep to simple, effective to increase user experience and keep clients and visitors happy. Below are given some reasons that will tell you why website navigation is important. Let's have a look.

- **Increase duration of visit:** Website navigation helps people to find information they want to see very easily and effectively. Good website navigation provides clear and simple navigation bars and links that encourages visitors or peoples to explore more and more. This in turn increase duration of their visit on website because navigation is very easy to use and understand and nowadays everyone wants it easy to get things.
- **Increased chance of sales:** Website navigation generally helps in showing visitors what website is actually about without any wastage of time. This in turn increase time spent by visitors on website and visitors stays on website for longer time. Visitors then go through website and purchase product effortlessly and easily. This will automatically increase purchase of products and sale.
- **Improved user experience:** Navigation is very important for website because without navigation, websites won't look good, and appear unorganized. Nowadays, no one to waste their time on determining how website works. Everyone wants to easily find what they want. Good navigation generally helps visitors to find information they want to search and increase speed of visitors search. Clear and simple navigation is simply to understand that in turn increase user experience.
- **Good Design:** Good navigation make website look more attractive, effective and good. Navigation has a greater impact on how website is viewed by users or visitors and how longer they will stay on website. Design of website navigation have a greater impact on success of website. It almost affects traffic, SEO ranking, user experience, conversions, sales, etc. So, good navigation is very useful to boost website design and brand of company or business. Good navigation keeps visitors happy and satisfied.

From <https://www.geeksforgeeks.org/website-navigation-and-its-importance/>

? Complete Case study Servlet Based

[MUST READ]

A Java Servlet e-Shop Case Study

From <https://www3.ntu.edu.sg/home/ehchua/programming/java/JavaServletCaseStudy.html>

Read "How to install Apache Tomcat server".

Sessions 5 & 6:

? JSP: Separating UI from Content generation code

Java Servlet technology and JavaServer Pages (JSP pages) are server-side technologies that have dominated the server-side Java technology market; they've become the standard way to develop commercial web applications. Java developers love these technologies for myriad reasons, including: the technologies are fairly easy to learn, and they bring the *Write Once, Run Anywhere* paradigm to web applications. More importantly, if used effectively by following best practices, servlets and JSP pages help separate presentation from content. *Best practices* are proven approaches for developing quality, reusable, and easily maintainable servlet- and JSP-based web applications. For instance, embedded Java code (scriptlets) in sections of HTML documents can result in complex applications that are not efficient, and difficult to reuse, enhance, and maintain. Best practices can change all that. In this article, I'll present important best practices for servlets and JSP pages; I assume that you have basic working knowledge of both technologies. This article:

- Presents an overview of Java servlets and JavaServer pages (JSP pages)
- Provides hints, tips, and guidelines for working with servlets and JSP pages
- Provides best practices for servlets and JSP pages

Overview of Servlets and JSP Pages

Similar to Common Gateway Interface (CGI) scripts, servlets support a request and response programming model. When a client sends a request to the server, the server sends the request to the servlet. The servlet then constructs a response that the server sends back to the client. Unlike CGI scripts, however, servlets run within the same process as the HTTP server.

In the simplest terms, servlets are Java classes that can generate dynamic HTML content using statements. What is important to note about servlets, however, is that they run in a container, and the APIs provide session and object life-cycle management. Consequently, when you use servlets, you gain all the benefits from the Java platform, which include the sandbox (security), database access API via JDBC, and cross-platform portability of servlets.

JavaServer Pages (JSP)

The JSP technology--which abstracts servlets to a higher level--is an open, freely available specification developed by Sun Microsystems as an alternative to Microsoft's Active Server Pages (ASP) technology, and a key component of the Java 2 Enterprise Edition (J2EE) specification. Many of the commercially available application servers (such as BEA WebLogic, IBM WebSphere, Live JRun, Orion, and so on) support JSP technology.

How Do JSP Pages Work?

A JSP page is basically a web page with traditional HTML and bits of Java code. The file extension of a JSP page is .jsp rather than .html or .htm, which tells the server that this page requires special handling that will be accomplished by a server extension or a plug-in. When a JSP page is called, it will be compiled (by the JSP engine) into a Java servlet. At this point the servlet is handled by the servlet engine, just like any other servlet. The servlet engine then loads the servlet class (using a class loader) and executes it to create dynamic HTML to be sent to the browser, as shown in Figure 1. The servlet creates any necessary object, and writes any object as a string to an output stream to the browser.

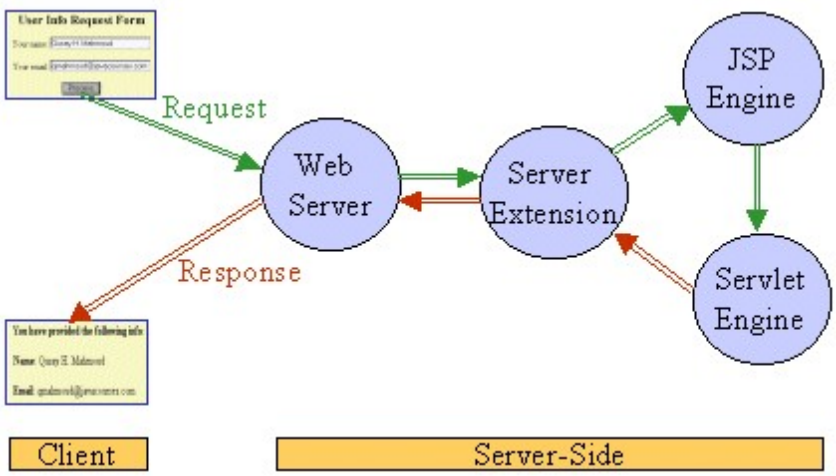


Figure 1: Request/Response flow calling a JSP page

The next time the page is requested, the JSP engine executes the already-loaded servlet unless the JSP page has changed, in which case it is automatically recompiled into a servlet and executed.

Best Practices

In this section, I present best practices for servlets and particularly JSP pages. The emphasis on JSP best practices is simply because JSP pages seem to be more widely used (probably because JSP technology promotes the separation of presentation from content). One best practice that combines and integrates the use of servlets and JSP pages is the Model View Controller (MVC) design pattern, discussed later in this article.

- **Don't overuse Java code in HTML pages**: Putting all Java code directly in the JSP page is OK for very simple applications. But overusing this feature leads to spaghetti code that is not easy to read and understand. One way to minimize Java code in HTML pages is to write separate Java classes that perform the computations. Once these classes are tested, instances can be created.

Integrating Servlets and JSP Pages

The JSP specification presents two approaches for building web applications using JSP pages:

JSP Model 1 and Model 2 architectures.

These two models differ in the location where the processing takes place.

In Model 1 architecture, as shown in Figure 2, the JSP page is responsible for processing requests and sending back replies to clients.

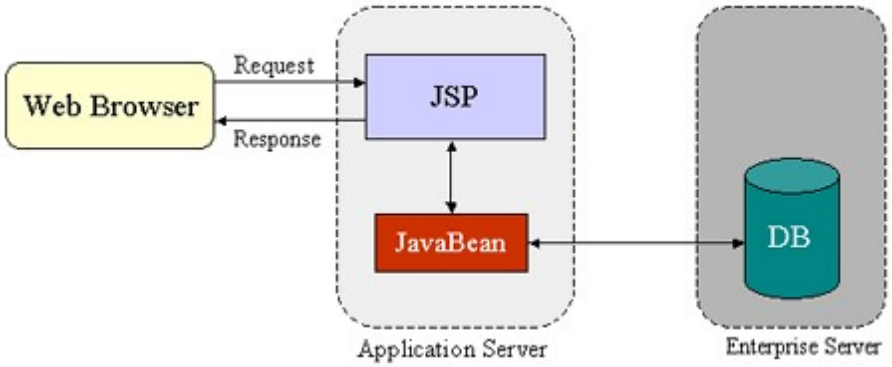


Figure 2: JSP Model 1 Architecture

The Model 2 architecture, as shown in Figure 3, integrates the use of both servlets and JSP pages.

In this mode, JSP pages are used for the presentation layer, and servlets for processing tasks. The servlet acts as a *controller* responsible for processing requests and creating any beans needed by the JSP page. The controller is also responsible for deciding to which JSP page to forward the request. The JSP page retrieves objects created by the servlet and extracts dynamic content for insertion within a template.

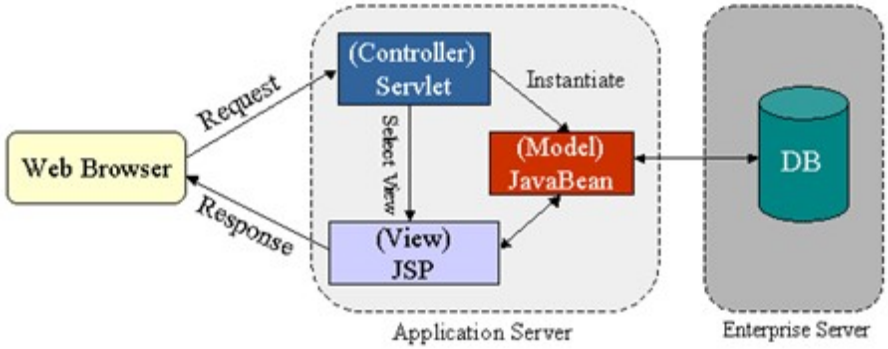


Figure 3: JSP Model 2 Architecture

This model promotes the use of the Model View Controller (MVC) architectural style design pattern. Note that several frameworks already exist that implement this useful design pattern, and that truly separate presentation from content. The Apache Struts is a formalized framework for MVC. This framework is best used for complex applications where a single request or form submission can result in substantially different-looking results.

From <https://www.oracle.com/technical-resources/articles/javase/servlets-jsp.html>

MVC architecture

What is MVC Framework?

The **Model-View-Controller (MVC)** framework is an architectural pattern that separates an application into three main logical components Model, View, and Controller. Hence the abbreviation MVC. Each architecture component is built to handle specific development aspect of an application. MVC separates the business logic and presentation layer from each other. It was traditionally used for desktop graphical user interfaces (GUIs). Nowadays, MVC architecture in web technology has become popular for designing web applications as well as mobile apps.

History of MVC

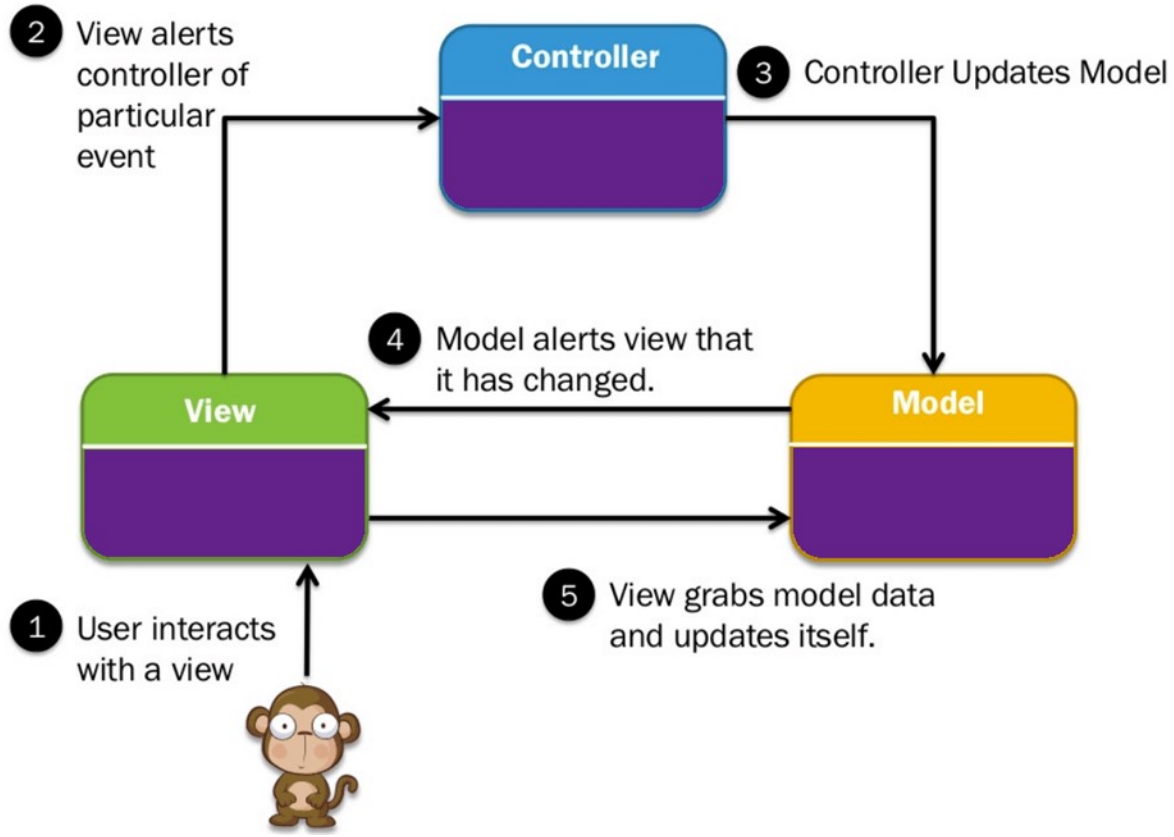
- MVC architecture was first discussed in 1979 by Trygve Reenskaug
- MVC model was first introduced in 1987 in the Smalltalk programming language.
- MVC was first time accepted as a general concept, in a 1988 article
- In the recent time, MVC pattern is widely used in modern web applications

Features of MVC

- Easy and frictionless testability. Highly testable, extensible and pluggable framework
- To design a web application architecture using the MVC pattern, it offers full control over your HTML as well as your URLs
- Leverage existing features provided by ASP.NET, JSP, Django, etc.
- Clear separation of logic: Model, View, Controller. Separation of application tasks viz. business logic, UI logic, and input logic
- URL Routing for SEO Friendly URLs. Powerful URL- mapping for comprehensible and searchable URLs
- Supports for Test Driven Development (TDD)

MVC Architecture

Here is the detailed architecture of MVC framework:



MVC Architecture Diagram

Three important MVC components are:

- Model: It includes all the data and its related logic
- View: Present data to the user or handles user interaction
- Controller: An interface between Model and View components

Let's see each other this component in detail:

View

A View is that part of the application that represents the presentation of data. Views are created by the data collected from the model data. A view requests the model to give information so that it presents the output presentation to the user.

The view also represents the data from charts, diagrams, and tables. For example, any customer view will include all the UI components like text boxes, drop downs, etc.

Controller

The Controller is that part of the application that handles the user interaction. The controller interprets the mouse and keyboard inputs from the user, informing model and the view to change as appropriate.

A Controller send's commands to the model to update its state(E.g., Saving a specific document). The controller also sends commands to its associated view to change the view's presentation (For example scrolling a particular document).

Model

The model component stores data and its related logic. It represents data that

is being transferred between controller components or any other related business logic. For example, a Controller object will retrieve the customer info from the database. It manipulates data and sends back to the database or uses it to render the same data.
It responds to the request from the views and also responds to instructions from the controller to update itself. It is also the lowest level of the pattern which is responsible for maintaining data.

MVC Examples

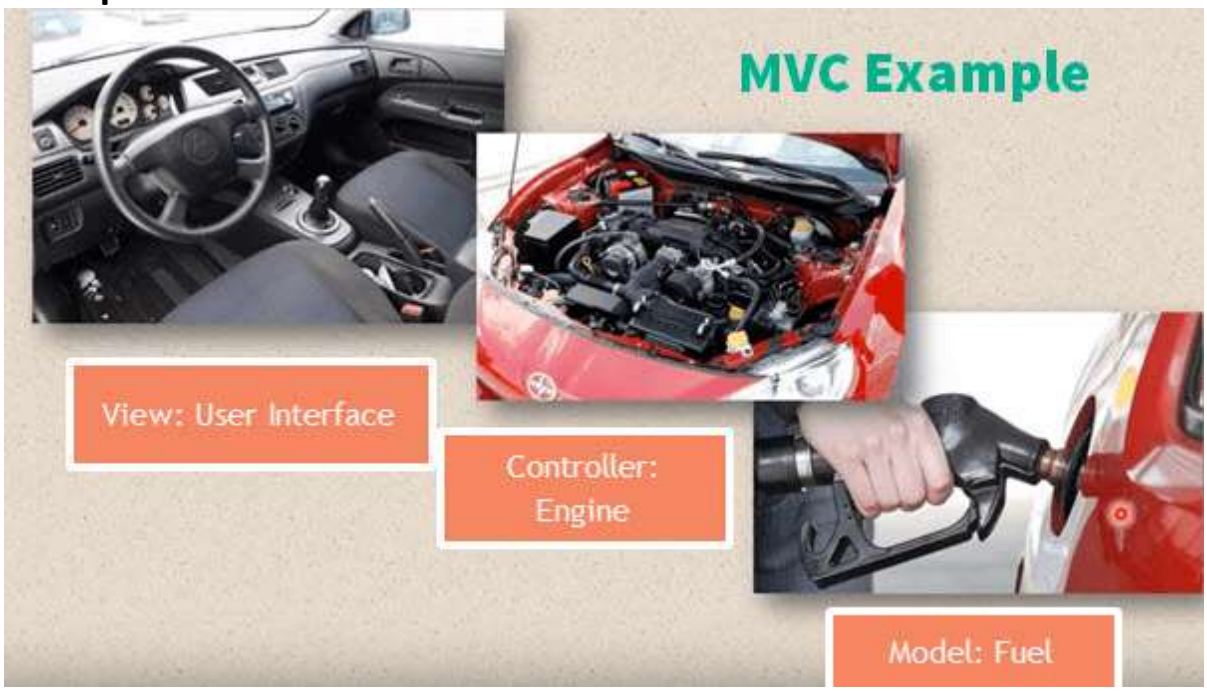
Let's see Model View Controller example from daily life:

Example 1:

- Let's assume you go to a restaurant. You will not go to the kitchen and prepare food which you can surely do at your home. Instead, you go there and wait for the waiter to come on.
- Now the waiter comes to you, and you order the food. The waiter doesn't know who you are and what you want he just written down the detail of your food order.
- Then, the waiter moves to the kitchen. In the kitchen, waiter does not prepare your food.
- The cook prepares your food. The waiter is given your order to him along with your table number.
- Cook then prepared food for you. He uses ingredients to cooks the food. Let's assume that your order a vegetable sandwich. Then he needs bread, tomato, potato, capsicum, onion, bit, cheese, etc. which he sources from the refrigerator
- Cook final hand over the food to the waiter. Now it is the job of the waiter to moves this food outside the kitchen.
- Now waiter knows which food you have ordered and how they are served.

In this MVC architecture example,
View= You
Waiter= Controller
Cook= Model
Refrigerator= Data
Let see one more MVC model example,

Example 2:



Car driving mechanism is another example of the MVC model.

- Every car consist of three main parts.
- View= User interface : (Gear lever, panels, steering wheel, brake, etc.)
- Controller- Mechanism (Engine)
- Model- Storage (Petrol or Diesel tank)

Car runs from engine take fuel from storage, but it runs only using mentioned user interface devices.

Popular MVC web frameworks

Here, is a list of some popular MVC frameworks:

- Ruby on Rails
- Django
- CakePHP
- Yii
- CherryPy
- Spring MVC
- Catalyst
- Rails
- Zend Framework
- CodeIgniter
- Laravel
- Fuel PHP
- Symphony

Advantages of MVC: Key Benefits

Here, are major benefits of using MVC architecture:

- Easy code maintenance which is easy to extend and grow
- MVC Model component can be tested separately from the user
- Easier support for new types of clients
- Development of the various components can be performed parallelly.
- It helps you to avoid complexity by dividing an application into the three units. Model, view, and controller

- It only uses a Front Controller pattern which process web application requests through a single controller.
- Offers the best support for test-driven development
- It works well for Web apps which are supported by large teams of web designers and developers.
- Provides clean separation of concerns(SoC).
- Search Engine Optimization (SEO) Friendly.
- All classes and objects are independent of each other so that you can test them separately.
- MVC design pattern allows logical grouping of related actions on a controller together.

Disadvantages of using MVC

- Difficult to read, change, unit test, and reuse this model
- The framework navigation can some time complex as it introduces new layers of abstraction which requires users to adapt to the decomposition criteria of MVC.
- No formal validation support
- Increased complexity and Inefficiency of data
- The difficulty of using MVC with the modern user interface
- There is a need for multiple programmers to conduct parallel programming.
- Knowledge of multiple technologies is required.
- Maintenance of lots of codes in Controller

3-tier Architecture vs. MVC Architecture

Parameter	3-Tier Architecture	MVC Architecture
Communication	This type of architecture pattern never communicates directly with the data layer.	All layers communicate directly using triangle topology.
Usage	3-tier: widely used in web applications where the client, data tiers, and middleware a run on physically separate platforms.	Generally used on applications that run on a single graphical workstation.

Summary

- The MVC is an architectural pattern that separates an application into 1) Model, 2) View and 3) Controller
- Model: It includes all the data and its related logic
- View: Present data to the user or handles user interaction
- Controller: An interface between Model and View components
- MVC architecture was first discussed in 1979 by Trygve Reenskaug
- MVC architecture in Java is a highly testable, extensible and pluggable framework
- Some popular MVC frameworks are Rails, Zend Framework, CodeIgniter, Laravel, Fuel PHP, etc.

From <https://www.guru99.com/mvc-tutorial.html>

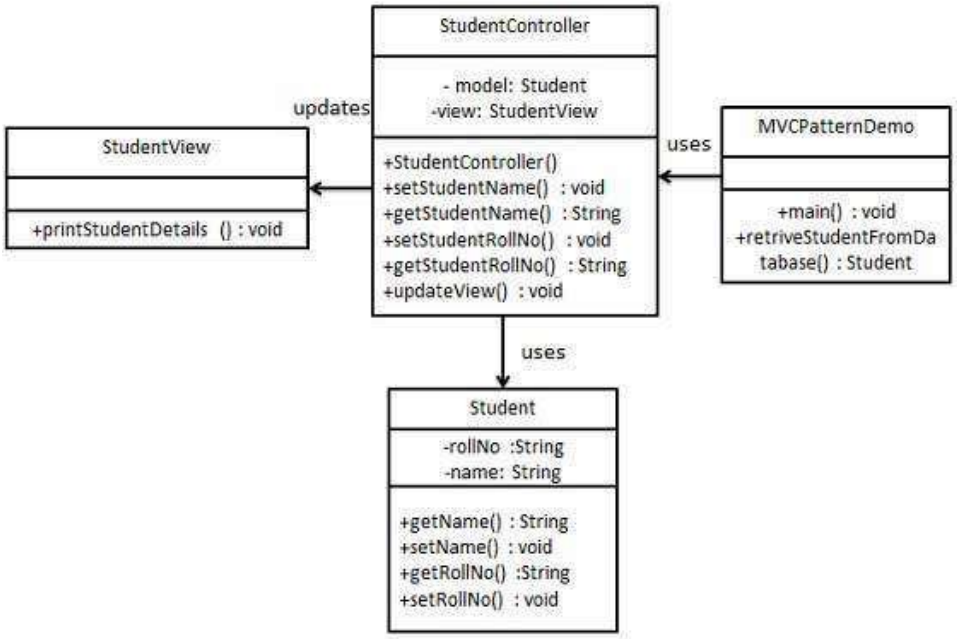
? Design Pattern: MVC Pattern

MVC Pattern stands for Model-View-Controller Pattern. This pattern is used to separate application's concerns.

- **Model** - Model represents an object or JAVA POJO carrying data. It can also have logic to update controller if its data changes.
- **View** - View represents the visualization of the data that model contains.
- **Controller** - Controller acts on both model and view. It controls the data flow into model object and updates the view whenever data changes. It keeps view and model separate.

Implementation

We are going to create a *Student* object acting as a model.*StudentView* will be a view class which can print student details on console and *StudentController* is the controller class responsible to store data in *Student* object and update view *StudentView* accordingly.
MVCPatternDemo, our demo class, will use *StudentController* to demonstrate use of MVC pattern.



Step 1

Create Model.
Student.java

```
public class Student {
  private String rollNo;
  private String name;

  public String getRollNo() {
    return rollNo;
  }

  public void setRollNo(String rollNo) {
    this.rollNo = rollNo;
  }
```

```
  public String getName() {
    return name;
  }

  public void setName(String name) {
    this.name = name;
  }
}
```

Step 2

Create View.

StudentView.java

```
public class StudentView {
  public void printStudentDetails(String studentName, String studentRollNo){
    System.out.println("Student: ");
    System.out.println("Name: " + studentName);
    System.out.println("Roll No: " + studentRollNo);
  }
}
```


Step 3

Create Controller.

StudentController.java

```
public class StudentController {
  private Student model;
  private StudentView view;
public StudentController(Student model, StudentView view){
    this.model = model;
    this.view = view;
  }
public void setStudentName(String name){
    model.setName(name);
  }
public String getStudentName(){
    return model.getName();
  }
public void setStudentRollNo(String rollNo){
    model.setRollNo(rollNo);
  }
public String getStudentRollNo(){
    return model.getRollNo();
  }
public void updateView(){
    view.printStudentDetails(model.getName(), model.getRollNo());
  }
}
```

Step 4

Use the *StudentController* methods to demonstrate MVC design pattern usage.

MVCPatternDemo.java

```
public class MVCPatternDemo {
  public static void main(String[] args) {
//fetch student record based on his roll no from the database
    Student model  = retriveStudentFromDatabase();
```

```
//Create a view : to write student details on console
    StudentView view = new StudentView();
StudentController controller = new StudentController(model, view);
controller.updateView();
//update model data
    controller.setStudentName("John");
controller.updateView();
  }
private static Student retriveStudentFromDatabase(){
    Student student = new Student();
    student.setName("Robert");
    student.setRollNo("10");
    return student;
  }
}
```

Step 5

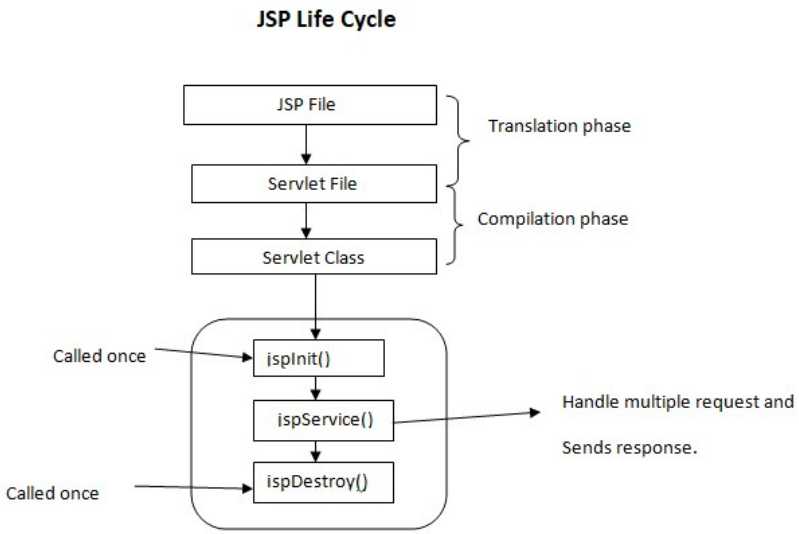
Verify the output.

```
Student:
Name: Robert
Roll No: 10
Student:
Name: John
Roll No: 10
```

From <https://www.tutorialspoint.com/design_pattern/mvc_pattern.htm>

? Life cycle of a JSP page

A Java Server Page life cycle is defined as the process that started with its creation which later translated to a servlet and afterward servlet lifecycle comes into play. This is how the process goes on until its destruction.



Following steps are involved in the JSP life cycle:

- Translation of JSP page to Servlet
- Compilation of JSP page(Compilation of JSP into test.java)
- Classloading (test.java to test.class)
- Instantiation(Object of the generated Servlet is created)
- Initialization(jspInit() method is invoked by the container)
- Request processing(_jspService()is invoked by the container)
- JSP Cleanup (jspDestroy() method is invoked by the container)

We can override jspInit(), jspDestroy() but we can't override _jspService() method.

Translation of JSP page to Servlet :

This is the first step of the JSP life cycle. This translation phase deals with the Syntactic correctness of JSP. Here test.jsp file is translated to test.java.

Compilation of JSP page :

Here the generated java servlet file (test.java) is compiled to a class file (test.class).

Classloading :

Servlet class which has been loaded from the JSP source is now loaded into the container.

Instantiation :

Here an instance of the class is generated. The container manages one or more instances by providing responses to requests.

Initialization :

jspInit() method is called only once during the life cycle immediately after the generation of Servlet instance from JSP.

Request processing :

_jspService() method is used to serve the raised requests by JSP. It takes request and response objects as parameters. This method cannot be overridden.

JSP Cleanup :

In order to remove the JSP from the use by the container or to destroy the method for servlets jspDestroy()method is used. This method is called once, if you need to perform any cleanup task like closing open files, releasing database connections jspDestroy() can be overridden.

From <https://www.geeksforgeeks.org/life-cycle-of-jsp/>

? Directives,

Directives in JSP

JSP directives are the elements of a JSP source code that guide the web container on how to translate the JSP page into it's respective servlet.

Syntax :

@

```
<%@ directive attribute = "value"%>
```

Directives can have a number of attributes which you can list down as **key-value pairs** and separated by commas. The blanks between the @ symbol and the directive name, and between the last attribute and the closing %>, are optional.

Different types of JSP directives :

There are three different JSP directives available. They are as follows:

- **Page Directives :** JSP page directive is used to define the properties applying the JSP page, such as the size of the allocated buffer, imported packages and classes/interfaces, defining what type of page it is etc. The syntax of JSP page directive is as follows:

```
<%@page attribute = "value"%>
```

- **Different properties/attributes :**
  The following are the different properties that can be defined using page directive :
- **import**: This tells the container what packages/classes are needed to be imported into the program.
  Syntax:
  `<%@page import = "value"%>`
- **Example :**
- html

```
<%-- JSP code to demonstrate how to use page
 directive to import a package --%>

<%@page import = "java.util.Date"%>
<%Date d = new Date();%>
<%=d%>
```

- **Output**:

- **contentType**: This defines the format of data that is being exchanged between the client and the server. It does the same thing as the setContentType method in servlet used to.
  Syntax:
  `<%@page contentType="value"%>`
- Usage Example:
- html

```
<%-- JSP code to demonstrate how to use
page directive to set the type of content --%>

<%@page contentType = "text/html" %>
<% = "This is sparta" %>
```

- **Output :**

- **info**: Defines the string which can be printed using the ‘getServletInfo()’ method.
  Syntax:
  `<%@page info="value"%>`
- **Usage Example**:
- html

```
<%-- JSP code to demonstrate how to use page
 directive to set the page information --%>

<%@page contentType = "text/html" %>
<% = getServletInfo()%>
```

- **Output**:

- **buffer**: Defines the size of the buffer that is allocated for handling the JSP page. The size is defined in Kilo Bytes.
  Syntax:
  `<%@page buffer = "size in kb"%>`
- **language**: Defines the scripting language used in the page. By default, this attribute contains the value ‘java’.
- **isELIgnored**: This attribute tells if the page supports expression language. By default, it is set to false. If set to true, it will disable expression language.
  Syntax:
  `<%@page isElIgnored = "true/false"%>`
- **Usage Example**:
- html

```
<%-- JSP code to demonstrate how to use page
directive to ignore expression language --%>

<%@page contentType = "text/html" %>
<%@page isELIgnored = "true"%>
<body bgcolor = "blue">
```

```
<c:out value = "${'This is sparta}"/>
</body>
```

- **Output**:
  (blank page)

- **errorPage**: Defines which page to redirect to, in case the current page encounters an exception.
  Syntax:
  `<%@page errorPage = "true/false"%>`
- **Usage Example:**
- html

```
//JSP code to divide two numbers
<%@ page errorPage = "error.jsp" %>

<%
// dividing the numbers
int z = 1/0;

 // result
out.print("division of numbers is: " + z);
%>
```

- **isErrorPage**: It classifies whether the page is an error page or not. By classifying a page as an error page, it can use the implicit object ‘exception’ which can be used to display exceptions that have occurred.
  Syntax:
  `<%@page isErrorPage="true/false"%>`
- **Usage example**:
- html

```
//JSP code for error page, which displays the exception
<%@ page isErrorPage = "true" %>

<h1>Exception caught</h1>

The exception is: <% = exception %>
Output:
```

- **Output**:

Exception caught

The exception is: java.lang.ArithmaticException:/ by zero

- **Include directive :**

- JSP include directive is used to include other files into the current jsp page. These files can be html files, other sp files etc. The advantage of using an include directive is that it allows code re-usability.
  The syntax of an include directive is as follows:

```
<@%include file = "file location"%>
```

- **Usage example**:
  In the following code, we're including the contents of an html file into a jsp page.
  a.html
- html

```
<h1>This is the content of a.html</h1>
```

- **index.jsp**
- html

```
<% = Local content%>
<%@include file = "a.html"%>
<% = local content%>
```

- **Output :**

- **Taglib Directive :** The taglib directive is used to mention the library whose custom-defined tags are being used in the JSP page. It's major application is JSTL(JSP standard tag library).
  Syntax:
  `<@%taglib uri = "library url" prefix="the prefix to identify the tags of this library with"%>`
- **Usage Example**:
- html

```
<%-- JSP code to demonstrate
taglib directive--%>
<%@ taglib uri = " http://java.sun.com/jsp/jstl/core"
prefix = "c" %>

<c:out value = "${'This is Sparta'}"/>
```

- In the above code, we've used to taglib directive to point to the JSTL library which is a set of some custom-defined tags in JSP that can be used in place of the scirptlet tag (<%..%>). The prefix attribute is used to define the prefix that is used to identify the tags of this library. Here, the prefix c is used in the <c:out> tag to tell the container that this tag belongs to the library mentioned above.
  Output:

From <https://www.geeksforgeeks.org/directives-in-jsp/>

Implicit and Explicit Objects,

What is JSP Implicit object?

- JSP implicit objects are created during the translation phase of JSP to the servlet.
- These objects can be directly used in scriplets that goes in the service method.
- They are created by the container automatically, and they can be accessed using objects.

How many Implicit Objects are available in JSP?

There are 9 types of implicit objects available in the container:

1. Out
2. Request
3. Response
4. Config
5. Application
6. Session
7. PageContext
8. Page
9. Exception

Lets study One By One

Out

- Out is one of the implicit objects to write the data to the buffer and send output to the client in response
- Out object allows us to access the servlet's output stream
- Out is object of javax.servlet.jsp.jspWriter class
- While working with servlet, we need printwriter object

Example:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Implicit Guru JSP1</title>
</head>
<body>
<% int num1=10;int num2=20;
out.println("num1 is " +num1);
```

```
out.println("num2 is "+num2);
%>
</body>
</html>
```

Explanation of the code:

Code Line 11-12– out is used to print into output stream

When we execute the above code, we get the following output:

Output:

- In the output, we get the values of num1 and num2

Request

- The request object is an instance of java.servlet.http.HttpServletRequest and it is one of the argument of service method
- It will be created by container for every request.
- It will be used to request the information like parameter, header information , server name, etc.
- It uses getParameter() to access the request parameter.

Example:

Implicit_jsp2.jsp(form from which request is sent to guru.jsp)

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Implicit Guru form JSP2</title>
</head>
<body>
<form action="guru.jsp">
<input type="text" name="username">
<input type="submit" value="submit">
</form>
</body>
</html>
```

Guru.jsp (where the action is taken)

```
1  <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2      pageEncoding="ISO-8859-1"%>
3  <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4  <html>
5  <head>
6  <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7  <title>form action</title>
8  </head>
9  <body>
10 <% String username = request.getParameter("username");
11 out.println("Welcome " +username);
12 %>
13 </body>
14 </html>
```

Explanation of code:

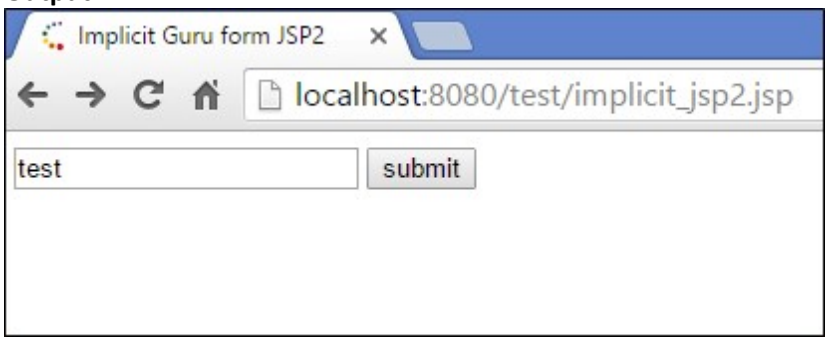
Code Line 10-13 : In implicit_jsp2.jsp(form) request is sent, hence the variable username is processed and sent to guru.jsp which is action of JSP.

Guru.jsp

Code Line10-11: It is action jsp where the request is processed, and username is taken from form jsp.

When you execute the above code, you get the following output

Output:



When you write test and click on the submit button, then you get the following output "Welcome Test."

Response

- "Response" is an instance of class which implements HttpServletResponse interface
- Container generates this object and passes to _jspservice() method as parameter
- "Response object" will be created by the container for each request.
- It represents the response that can be given to the client
- The response implicit object is used to content type, add cookie and redirect to response page

Example:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
```

```
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Implicit Guru JSP4</title>
</head>
<body>
<%response.setContentType("text/html"); %>
</body>
</html>
```

Explanation of the code:

Code Line 11: In the response object we can set the content type

Here we are setting only the content type in the response object. Hence, there is no output for this.

Config

- "Config" is of the type java.servlet.servletConfig
- It is created by the container for each jsp page
- It is used to get the initialization parameter in web.xml

Example:

Web.xml (specifies the name and mapping of the servlet)

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app id="WebApp_ID" version="2.4" xmlns="http://java.sun.com/xml/ns/j2ee" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
    <display-name>
    test</display-name>
    <servlet>
        <description>
        </description>
        <display-name>
        GuruServlet</display-name>
        <servlet-name>GuruServlet</servlet-name>
        <servlet-class>
        demotest.GuruServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>GuruServlet</servlet-name>
        <url-pattern>/GuruServlet</url-pattern>
    </servlet-mapping>
    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
        <welcome-file>index.htm</welcome-file>
        <welcome-file>index.jsp</welcome-file>
        <welcome-file>default.html</welcome-file>
        <welcome-file>default.htm</welcome-file>
        <welcome-file>default.jsp</welcome-file>
    </welcome-file-list>
</web-app>
```

Implicit_jsp5.jsp (getting the value of servlet name)

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Implicit Guru JSP5</title>
</head>
<body>
<% String servletName = config.getServletName();
out.println("Servlet Name is " +servletName);%>
</body>
</html>
```

Explanation of the code:

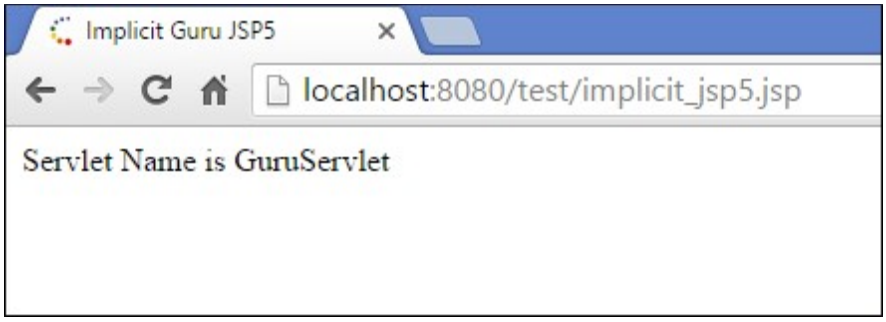
In web.xml

Code Line 14-17: In web.xml we have mapping of servlets to the classes.

Implicit_jsp5.jsp

Code Line 10-11: To get the name of the servlet in JSP, we can use config.getServletName, which will help us to get the name of the servlet.

When you execute the above code you get the following output:



Output:

- Servlet name is “GuruServlet” as the name is present in web.xml

Application

- Application object (code line 10) is an instance of javax.servlet.ServletContext and it is used to get the context information and attributes in JSP.
- Application object is created by container one per application, when the application gets deployed.
- Servletcontext object contains a set of methods which are used to interact with the servlet container.We can find information about the servlet container

Example:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Guru Implicit JSP6</title>
</head>
<body>
<% application.getContextPath(); %>
</body>
</html>
```

Explanation of the code:

- In the above code, application attribute helps to get the context path of the JSP page.

Session

- The session is holding “httpsession” object(code line 10).
- Session object is used to get, set and remove attributes to session scope and also used to get session information

Example:

Implicit_jsp7(attribute is set)

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Implicit JSP</title>
</head>
<body>
```

```
<% session.setAttribute("user","GuruJSP"); %>
<a href="implicit_jsp8.jsp">Click here to get user name</a>
</body>
</html>
```

Implicit_jsp8.jsp (getAttribute)

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>implicit Guru JSP8</title>
</head>
<body>
<% String name = (String)session.getAttribute("user");
out.println("User Name is " +name);
%>
</body>
</html>
```

Explanation of the code:

Implicit_jsp7.jsp

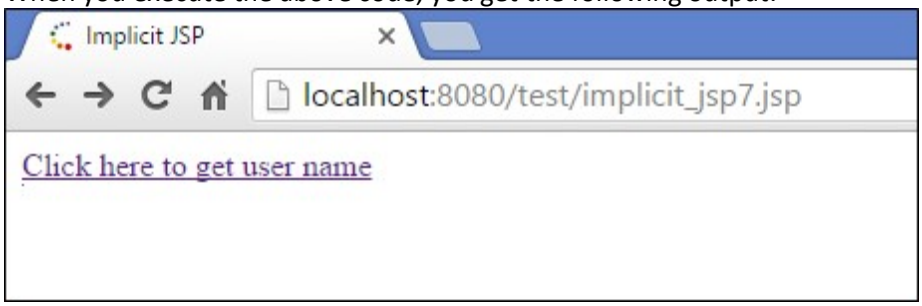
Code Line 11: we are setting the attribute user in the session variable, and that value can be fetched from the session in whichever jsp is called from that (_jsp8.jsp).

Code Line 12: We are calling another jsp on href in which we will get the value for attribute user which is set.

Implicit_jsp8.jsp

Code Line 11: We are getting the value of user attribute from session object and displaying that value

When you execute the above code, you get the following output:



When you click on the link for the username. You will get the following output.

Output:

- When we click on link given in implicit_jsp7.jsp then we are redirected to second jsp page, i.e (_jsp8.jsp) page and we get the value from session object of the user attribute (_jsp7.jsp).

PageContext

- This object is of the type of pagecontext.
- It is used to get, set and remove the attributes from a particular scope

Scopes are of 4 types:

- Page
- Request
- Session
- Application

Example:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Implicit Guru JSP9</title>
</head>
<body>
<% pageContext.setAttribute("student","gurustudent",pageContext.PAGE_SCOPE);
String name = (String)pageContext.getAttribute("student");
out.println("student name is " +name);
%>
</body>
</html>
```

Explanation of the code:

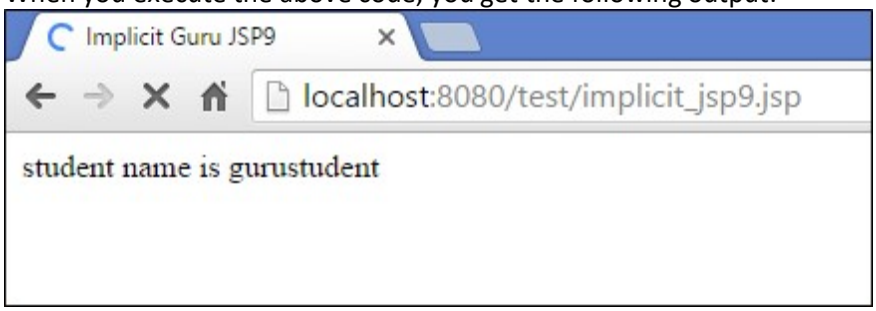
Code Line 11: we are setting the attribute using pageContext object, and it has three parameters:

- Key
- Value
- Scope

In the above code, the key is student and value is “gurustudent” while the scope is the page scope. Here the scope is “page” and it can get using page scope only.

Code Line 12: We are getting the value of the attribute using pageContext

When you execute the above code, you get the following output:



Output:

- The output will print “student name is gurustudent”.

Page

- Page implicit variable holds the currently executed servlet object for the corresponding jsp.
- Acts as this object for current jsp page.

Example:

In this example, we are using page object to get the page name using toString method

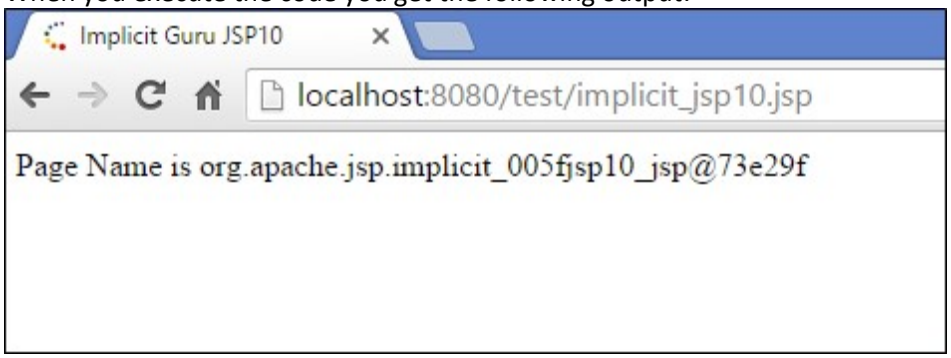
```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
```

```
<title>Implicit Guru JSP10</title>
</head>
<body>
<% String pageName = page.toString();
out.println("Page Name is " +pageName);%>
</body>
</html>
```

Explanation of the code:

Code Line 10-11: In this example, we are trying to use the method toString() of the page object and trying to get the string name of theJSP Page.

When you execute the code you get the following output:



Output:

- Output is string name of above jsp page

Exception

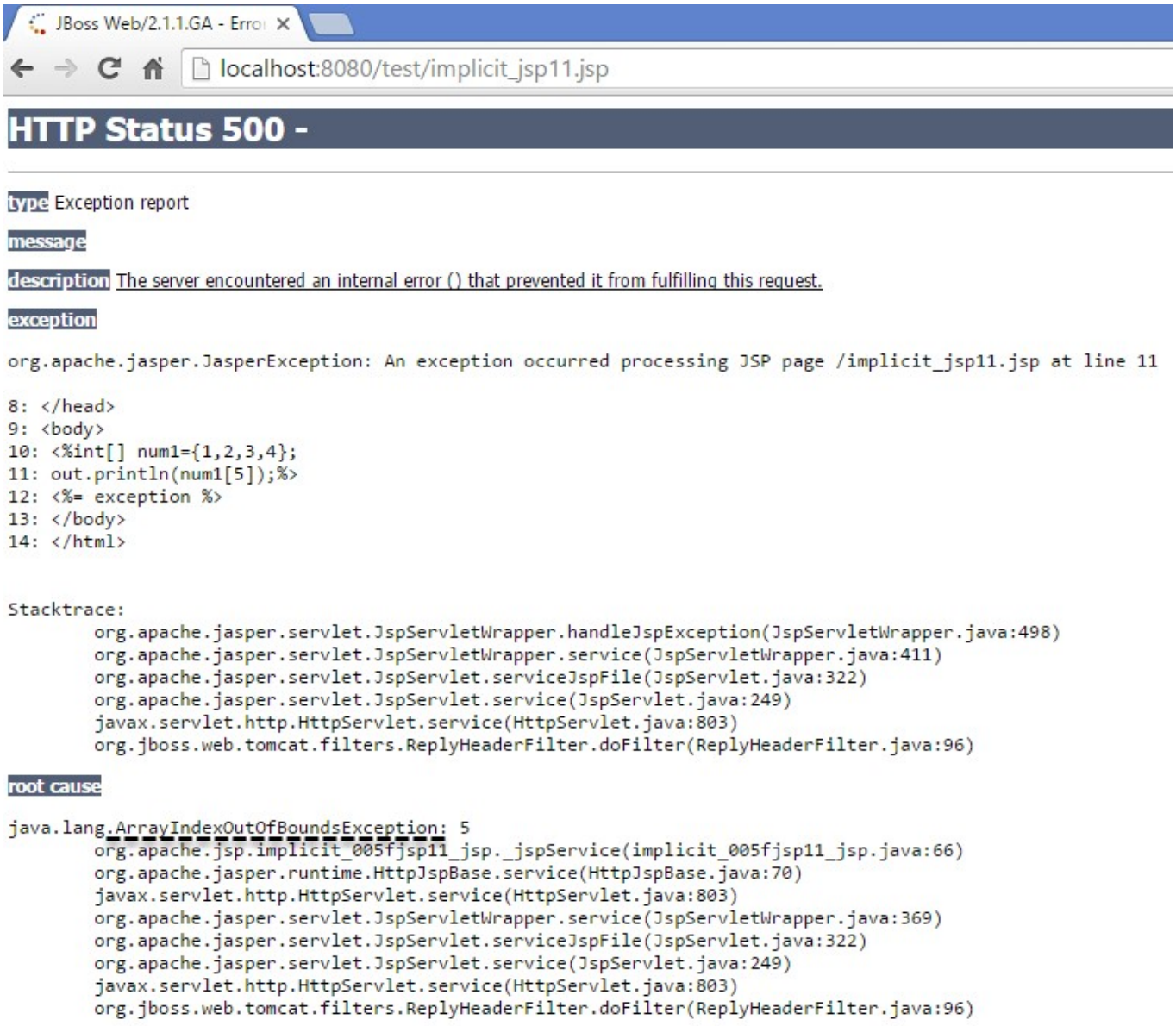
- Exception is the implicit object of the throwable class.
- It is used for exception handling in JSP.
- The exception object can be only used in error pages.**Example:**

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1" isErrorPage="true"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Implicit Guru JSP 11</title>
</head>
<body>
<%int[] num1={1,2,3,4};
out.println(num1[5]);%>
<%= exception %>
</body>
</html>
```

Explanation of the code:

Code Line 10-12 – It has an array of numbers, i.e., num1 with four elements. In the output, we are trying to print the fifth element of the array from num1, which is not declared in the array list. So it is used to get exception object of the jsp.

Output:



We are getting ArrayIndexOfBoundsException in the array where we are getting a num1 array of the fifth element.

From <https://www.guru99.com/jsp-implicit-objects.html>

Explicit Objects

Explicit objects are typically JavaBean instances declared and created in jsp:useBean action statements. The jsp:useBean statement and other action statements are described in "JSP Actions and the <jsp: > Tag Set", but an example is also shown here:
<jsp:useBean id="pageBean" class="mybeans.NameBean" scope="page" />

This statement defines an instance, pageBean, of the NameBean class that is in the mybeans package. The scope parameter is discussed in the next section, "Object Scopes".

You can also create objects within Java scriptlets or declarations, just as you would create Java class instances in any Java program.

From <https://docs.oracle.com/cd/A97336_01/buslog.102/a83726/genlovw3.htm>

Object Scopes

Objects in a JSP, whether explicit or implicit, are accessible within a particular *scope*. In the case of explicit objects, such as a JavaBean instance created in a jsp:useBean action statement, you can explicitly set the scope with the following syntax (as in the example in the preceding section, "Explicit Objects"):
scope="*scopevalue*"

There are four possible scopes:

- scope="page"--The object is accessible only from within the JSP page where it was created. Note that when the user refreshes the page while executing a JSP page, new instances will be created of all page-scope objects.
- scope="request"--The object is accessible from any JSP page servicing the same HTTP request that is serviced by the JSP page that created the object.
- scope="session"--The object is accessible from any JSP page sharing the same HTTP session as the JSP page that created the object.
- scope="application"--The object is accessible from any JSP page used in the same Web application (within any single Java virtual machine) as the JSP page that created the object

From <https://docs.oracle.com/cd/A97336_01/buslog.102/a83726/genlovw3.htm>

Scriptlets,

JSP Scriptlet Tag

Scriptlet Tag allows you to write java code inside JSP page. Scriptlet tag implements the _jspService method functionality by writing script/java code. Syntax of Scriptlet Tag is as follows :

```
<% JAVA CODE %>
```

JSP: Example of Scriptlet

In this example, we will show number of page visit.

```
<html>
<head>
<title>My First JSP Page</title>
</head>
<%
      int count = 0;
   %>
   <body>Page Count is
<% out.println(++count); %>
   </body>
</html>
```

Copy

We have been using the above example since last few lessons and in this scriptlet tags are used. Everything written inside the scriptlet tag is compiled as java code. Like in the above example, we initialize count variable of type int with a value of 0. And then we print it while using ++ operator to perform addition on it.
JSP makes it so easy to perform calculations, database interactions etc directly from inside the HTML code. Just write your java code inside the scriptlet tags.

Example of JSP Scriptlet Tag

In this example, we will create a simple JSP page which retrieves the name of the user from the request parameter. The **index.html** page will get the username from the user.

index.html

```
<form method="POST" action="welcome.jsp">
   Name <input type="text" name="user" >
   <input type="submit" value="Submit">
</form>
```

In the above HTML file, we have created a form, with an input text field for user to enter his/her name, and a Submit button to submit the form. On submission an HTTP Post request (method="**POST**") is made to the welcome.jsp file (action="**welcome.jsp**"), with the form values.

welcome.jsp

```
<html>
   <head>
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
      <title>Welcome Page</title>
   </head>
   <%
    String user = request.getParameter("user");
   %>

   <body>
      Hello, <% out.println(user); %>
   </body>
</html>
```

As we know that a JSP code is translated to Servlet code, in which _jspService method is executed which has **HttpServletRequest** and **HttpServletResponse** as argument. So in the **welcome.jsp** file, **request** is the HTTP Request and it has all the parameters sent from the form in index.html page, which we can be easily get using getParameter() with name of parameter as argument, to get its value.

Mixing scriptlet Tag and HTML

Let's see how we can utilize the power of JSP scripting with HTML to build dynamic webpages with help of a few examples.
If we want to create a Table in HTML with some dynamic data, for

example by reading data from some MySQL table or file. How to do that? Here we will describe you the technique by creating a table with numbers 1 to n.

```
<tableborder=1><%
    for ( int i = 0; i < n; i++ ) {
        %>
        <tr><td>Number</td><td><%= i+1 %></td></tr><%
    }
%>
</table>
```

Copy

The above piece of code will go inside the `<body>` tag of the JSP file and will work when you initialize `n` with some value.
Also, observer closely we have only included the java code inside the scriptlet tag, and all the HTML part is outside of it. Similarly we can do plenty of stuff.
Here is one more very simple example :

```
<%
    if ( hello ) {
        %>
        <p>Hello, world</p><%
    } else {
        %>
        <p>Goodbye, world</p><%
    }
%>
```

Copy

Above code is using if-else condition to evaluate what to show, based on the value of a **boolean** variable named `hello`.
You can even ask user to enter the value of `hello`, using HTML Form and evaluate your JSP code based on that.

From <https://www.studytonight.com/jsp/jsp-scriptlet-tag.php>

Expressions,

Expression tag is one of the scripting elements in JSP. Expression Tag in JSP is used for writing your content on the client-side. We can use this tag for displaying information on the client's browser. The JSP Expression tag transforms the code into an expression statement that converts into a value in the form of a string object and inserts into the implicit output object.

Syntax: JSP tag

- html

```
<%= expression %>
```

Difference between Scriptlet Tag and Expression Tag

- In the Scriptlet tag, it's Evaluated a Java expression. Does not display any result in

the HTML produced. Variables are declared to have only local scope, so cannot be accessed from elsewhere in the .jsp. but in Expression Tag it's Evaluates a Java expression. Inserts the result (as a string) into the HTML in the .js

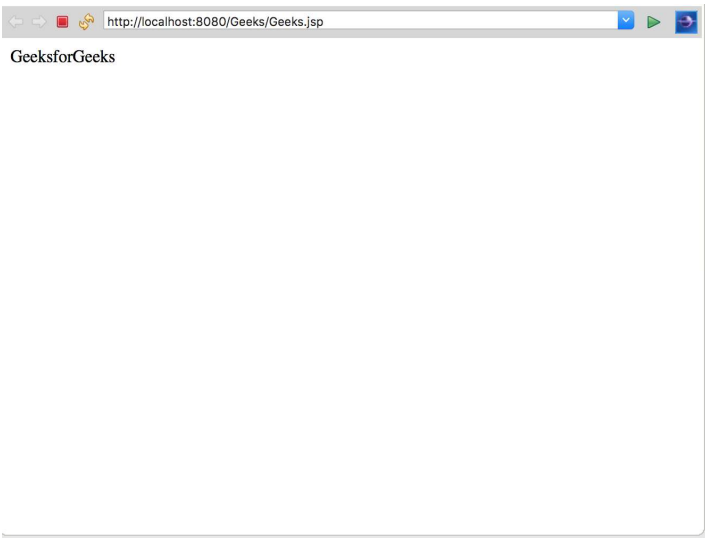
- We don't need to write out.println in Expression tag for printing anything because these are converted into out.print() statement and insert it into the _jspService(-, -) of the servlet class by the container.

Example

- html

```
<html>
<body>
<%= GeeksforGeeks %>  <!-- Expression tag -->
</body>
</html>
```

Output



Using expression tag:

- html

```
<%@ page language="java" contentType="text/html;
charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
 "http://www.w3.org/TR/html4/loose.dtd">
<html>

<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>GeeksforGeeks</title>
</head>

<body>
<% out.println("Hello Geeks "); %> <!-- Sriptlet Tag-->
<% int n1=10; int n2=30; %><!-- Sriptlet Tag-->
<% out.println("<br>sum of n1 and n2 is "); %> <!-- Sriptlet Tag-->
<%= n1+n2 %> <!-- Expression tag -->
</body>
```

```
</html>
```

Output

Hello Geeks
sum of n1 and n2 is 40

Let us take one more example.

Here we are creating an HTML file to take the username from user.save this file as index.html

- HTML

```
<!--index.html -->
<!-- Example of JSP code which prints the Username -->
<html>
<body>
<form action="Geeks.jsp">
<!-- move the control to Geeks.jsp when Submit button is click -->

Enter Username:
<input type="text" name="username">
<input type="submit" value="Submit"><br/>

</form>
</body>
</html>
```

Output:

Here we are creating A jsp file names as Geeks.jsp

- HTML

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
" http://www.w3.org/TR/html4/loose.dtd">
<html>

<head>
<meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">
<title>Insert title here</title>
</head>

<body>
<%= String name=request.getParameter("username");
out.print("Hello "+name);
%>
</body>

</html>
```

Output:

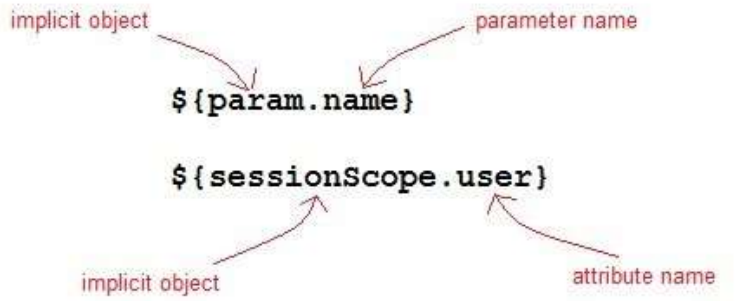
From <https://www.geeksforgeeks.org/jsp-expression-tag/>

Expression Language

Expression Language(EL) was added to JSP 2.0 specification. The purpose of EL is to produce scriptless JSP pages. The syntax of EL in a JSP is as follows:

`${expr}`

Here expr is a valid EL expression. An expression can be mixed with static text/values and can also be combined with other expressions to form larger expression.



How EL expression is used?

EL expression can be used in two ways in a JSP page

1. As attribute values in standard and custom tags. Example:
   ```
   <jsp:includepage="${location}">
   ```
   Copy
   Where location variable is separately defines in the jsp page. Expressions can also be used in jsp:setProperty to set a properties value, using other bean properties like : If we have a bean named Square with properties length, breadth and area.
   ```
   <jsp:setPropertyname="square"property="area"value="${square.length*square.breadth}"/>
   ```
   Copy

2. To output in HTML tag:
   ```
   <h1>Welcome ${name}</h1>
   ```
   Copy

To deactivate the evaluation of EL expressions, we specify the isELIgnored attribute of the page directive as below:

```
<%@ page isELIgnored ="true|false" %>
```

Copy

JSP EL Implicit Objects

Following are the implicit objects in EL :

Implicit Object	Description
pageContext	It represents the PageContext object.
pageScope	It is used to access the value of any variable which is set in the Page scope
requestScope	It is used to access the value of any variable which is set in the Request

	scope.
sessionScope	It is used to access the value of any variable which is set in the Session scope
applicationScope	It is used to access the value of any variable which is set in the Application scope
param	Map a request parameter name to a single value
paramValues	Map a request parameter name to corresponding array of string values.
header	Map containing header names and single string values.
headerValues	Map containing header names to corresponding array of string values.
cookie	Map containing cookie names and single string values.

Example of JSP EL

Let's take a simple example for understanding the JSP expression language,

index.jsp

```
<formmethod="POST"action="welcome.jsp">
Name <inputtype="text"name="user">
<inputtype="submit"value="Submit">
</form>
```

Copy

welcome.jsp

```
<html>
<head>
<title>Welcome Page</title>
</head>
<body>
<h1>Welcome ${param.name}</h1>
</body>
</html>
```

Copy

Arithmetic Operations available in EL

Following are the arithmetic operators avilable in EL:

Arithmetic Operation	Operator
Addition	+
Substraction	-
Multiplication	*
Division	/ and div

Remainder	% and mod

Logical and Relational Operators available in EL

Following are the logical operator and comparators avilable in EL:

Logical and Relational Operator	Operator
Equals	== and eq
Not equals	!= and ne
Less Than	< and lt
Greater Than	> and gt
Greater Than or Equal	>= and ge
Less Than or Equal	<= and le
and	&& and and
or	|| and or
not	! and not

From <https://www.studytonight.com/jsp/expression-language.php>

? Scope

One of the most powerful features of JSP is that a JSP page can access, create, and modify data objects on the server. You can then make these objects visible to JSP pages. When an object is created, it defines or defaults to a given scope. The container creates some of these objects, and the JSP designer creates others.

The *scope* of an object describes how widely it's available and who has access to it. For example, if an object is defined to have page scope, then it's available only for the duration of the current request on that page before being destroyed by the container. In this case, only the current page has access to this data, and no one else can read it. At the other end of the scale, if an object has application scope, then any page may use the data because it lasts for the duration of the application, which means until the container is switched off.

Page Scope

Objects with *page scope* are accessible only within the page in which they're created. The data is valid only during the processing of the current response; once the response is sent back to the browser, the data is no longer valid. If the request is forwarded to another page or the browser makes another request as a result of a redirect, the data is also lost.

```
//Example of JSP Page Scope
<jsp:useBean id="employee" class="EmployeeBean" scope="page" />
```

Request Scope

Objects with *request scope* are accessible from pages processing the same request in which they were created. Once the container has processed the request, the data is released. Even if the request is forwarded to another page, the data is still available though not if a redirect is required.

```
//Example of JSP Request Scope
<jsp:useBean id="employee" class="EmployeeBean" scope="request" />
```

Session Scope

Objects with *session scope* are accessible from pages processing requests that are in the same session as the one in which they were created. A *session* is the time users spend using the application, which ends when they close their browser, when they go to another Web site, or when the application designer wants (after a logout, for instance). So, for example, when users log in, their username could be stored in the session and displayed on every page they access. This data lasts until they leave the Web site or log out.

```
//Example of JSP Session Scope
<jsp:useBean id="employee" class="EmployeeBean" scope="session" />
```

Application Scope

Objects with *application scope* are accessible from JSP pages that reside in the same application. This creates a global object that's available to all pages.

Application scope uses a single namespace, which means all your pages should be careful not to duplicate the names of application scope objects or change the values when they're likely to be read by another page (this is called *thread safety*). Application scope variables are typically created and populated when an application starts and then used as read-only for the rest of the application.

```
//Example of JSP Application Scope
<jsp:useBean id="employee" class="EmployeeBean" scope="application" />
```

From <https://www.java-samples.com/showtutorial.php?tutorialid=1009>

? JSP Error Page handling

Exceptions in JSP occur when there is an error in the code either by the developer or internal error from the system. Exception handling in JSP is the same as in Java where we manage exceptions using Try Catch blocks. Unlike Java, there are exceptions in JSP also when there is no error in the code.

Types of Exceptions in JSP

Exceptions in JSP are of three types:

1. Checked Exception
2. Runtime Exception
3. Error Exception

Checked Exceptions

It is normally a user error or problems which are not seen by the developer are termed as checked exceptions.
Some Checked exception examples are:

1. **FileNotFoundException**: This is a checked exception (where it tries to find a file when the file is not found on the disk).

2. **IO Exception**: This is also checked exception if there is any exception occurred during reading or writing of a file then the IO exception is raised.
3. **SQLException**: This is also a checked exception when the file is connected with SQL database, and there is issue with the connectivity of the SQL database then SQLException is raised

Runtime Exceptions

Runtime exceptions are the one which could have avoided by the programmer. They are ignored at the time of compilation.

Some Runtime exception examples are:

1. **ArrayIndexOutOfBoundsException**: This is a runtime exception when array size exceeds the elements.
2. **ArithmeticException**: This is also a runtime exception when there are any mathematical operations, which are not permitted under normal conditions, for example, dividing a number by 0 will give an exception.
3. **NullPointer Exception**: This is also a runtime exception which is raised when a variable or an object is null when we try to access the same. This is a very common exception.

Errors:

The problem arises due to the control of the user or programmer. If stack overflows, then error can occur.

Some examples of the error are listed below:

1. **Error**: This error is a subclass of throwable which indicates serious problems that an application cannot catch.
2. **Instantiation Error**: This error occurs when we try to instantiate an object, and it fails to do that.
3. **Internal Error**: This error occurs when there is an error occurred from JVM i.e. Java Virtual Machine.

Error Exceptions

It is an instance of the throwable class, and it is used in error pages.

Some methods of throwable class are:

- **Public String getMessage()** – returns the message of the exception.
- **Public throwablegetCause()** – returns cause of the exception
- **Public printStackTrace()**– returns the stacktrace of the exception.

How to Handle Exception in JSP

Here is an example of how to handle exception in JSP:

Exception_example.jsp

```
<%@ page errorPage="guru_error.jsp" %>
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Exception Guru JSP1</title>
</head>
<body>
<%
 int num = 10;
   if (num == 10)
```

```
  {
    throw new RuntimeException("Error condition!!!");
  }
 %>
  </body>
</html>
```

Guru_error.jsp

```
<%@ page isErrorPage="true" %>
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Guru Exception Page</title>
</head>
<body>
<p>Guru Exception has occurred</p>
<% exception.printStackTrace(response.getWriter()); %>
</body>
</html>
```

Explanation of the code:

Exception_example.jsp

Code Line 1: Here we are setting error page to guru_error.jsp which will be used when the error will be redirected.

Code Line 15: we are taking a variable num and setting it to 10 and checking a condition if num is 10 then to throw a Runtime Exception with the message as Error Condition.

Guru_error.jsp

Code Line 1: Here we are setting isErrorPageattribute to true.

Code Line 12: The exception has been raised in exception_example.jsp using throw object and that exception will be shown here as IsErrorPage attribute is marked as true. Using the exception (this is an object which allows the exception data to be accessed by the JSP.) object we are trying to print the stacktrace of the error which was occurred in exception_example.jsp.

When you execute the above code you get the following output:

Output:

The exception has been raised which was thrown from exception_example.jsp using throw object of runtime exception and we get the above code.

Also guru_error.jsp is called from which Guru Exception has occurred from this file.

Summary

- Exceptions in JSP occur when there is an error in the code either by the

developer or internal error from the system.

- Exceptions in JSP are of 3 types: Checked Exceptions, Runtime Exceptions, and Error Exceptions
- Checked exception is normally a user error or problems which are not seen by the developer are termed as checked exceptions.
- Runtime exceptions are the one which could have avoided by the programmer. They are ignored at the time of compilation.
- Error exception is an instance of the throwable class, and it is used in error pages.

From <https://www.guru99.com/jsp-exception-handling.html>

? JSTL

1. Overview

JavaServer Pages Tag Library (JSTL) is a set of tags that can be used for implementing some common operations such as looping, conditional formatting, and others.

In this tutorial, we'll be discussing how to setup JSTL and how to use its numerous tags.

2. Setup

To enable JSTL features, we'd have to add the library to our project. For a Maven project, we add the dependency in *pom.xml* file:

```
<dependency>
<groupId>javax.servlet</groupId>
<artifactId>jstl</artifactId>
<version>1.2</version>
</dependency>
```

With the library added to our project, the final setup will be to add the core JSTL tag and any other tags' namespace file to our JSP using the taglib directive like this:

```
<%@ taglib prefix="c"uri="http://java.sun.com/jsp/jstl/core"%>
```

Next, we'll take a look at these tags which are broadly grouped into five categories.

From <https://www.baeldung.com/jstl>

Classification of The JSTL Tags

The JSTL tags can be classified, according to their functions, into the following JSTL tag library groups that can be used when creating a JSP page –

- **Core Tags**
- **Formatting tags**
- **SQL tags**
- **XML tags**
- **JSTL Functions**

Core Tags

The core group of tags are the most commonly used JSTL tags. Following is the syntax to include the JSTL Core library in your JSP –

```
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
```

Following table lists out the core JSTL Tags –

S.No.	Tag & Description
1	<c:out> Like <%= ... >, but for expressions.
2	<c:set > Sets the result of an expression evaluation in a **'scope'**
3	<c:remove > Removes a **scoped variable** (from a particular scope, if specified).
4	<c:catch> Catches any **Throwable** that occurs in its body and optionally exposes it.
5	<c:if> Simple conditional tag which evalutes its body if the supplied condition is true.
6	<c:choose> Simple conditional tag that establishes a context for mutually exclusive conditional operations, marked by **<when>** and **<otherwise>**.
7	<c:when> Subtag of **<choose>** that includes its body if its condition evalutes to **'true'**.
8	<c:otherwise > Subtag of **<choose>** that follows the **<when>** tags and runs only if all of the prior conditions evaluated to **'false'**.
9	<c:import> Retrieves an absolute or relative URL and exposes its contents to either the page, a String in **'var'**, or a Reader in **'varReader'**.
10	<c:forEach > The basic iteration tag, accepting many different collection types and supporting subsetting and other functionality .
11	<c:forTokens> Iterates over tokens, separated by the supplied delimeters.
12	<c:param> Adds a parameter to a containing **'import'** tag's URL.
13	<c:redirect > Redirects to a new URL.
14	<c:url> Creates a URL with optional query parameters

Formatting Tags

The JSTL formatting tags are used to format and display text, the date, the time, and numbers for internationalized Websites. Following is the syntax to include Formatting library in your JSP –

```
<%@ taglib prefix = "fmt" uri = "http://java.sun.com/jsp/jstl/fmt" %>
```

Following table lists out the Formatting JSTL Tags –

S.No.	Tag & Description
1	<fmt:formatNumber>

	To render numerical value with specific precision or format.
2	<fmt:parseNumber> Parses the string representation of a number, currency, or percentage.
3	<fmt:formatDate> Formats a date and/or time using the supplied styles and pattern.
4	<fmt:parseDate> Parses the string representation of a date and/or time
5	<fmt:bundle> Loads a resource bundle to be used by its tag body.
6	<fmt:setLocale> Stores the given locale in the locale configuration variable.
7	<fmt:setBundle> Loads a resource bundle and stores it in the named scoped variable or the bundle configuration variable.
8	<fmt:timeZone> Specifies the time zone for any time formatting or parsing actions nested in its body.
9	<fmt:setTimeZone> Stores the given time zone in the time zone configuration variable
10	<fmt:message> Displays an internationalized message.
11	<fmt:requestEncoding> Sets the request character encoding

SQL Tags

The JSTL SQL tag library provides tags for interacting with relational databases (RDBMSs) such as **Oracle, mySQL**, or **Microsoft SQL Server**.

Following is the syntax to include JSTL SQL library in your JSP –

```
<%@ taglib prefix = "sql" uri = "http://java.sun.com/jsp/jstl/sql" %>
```

Following table lists out the SQL JSTL Tags –

S.No.	Tag & Description
1	<sql:setDataSource> Creates a simple DataSource suitable only for prototyping
2	<sql:query> Executes the SQL query defined in its body or through the sql attribute.
3	<sql:update> Executes the SQL update defined in its body or through the sql attribute.
4	<sql:param> Sets a parameter in an SQL statement to the specified value.
5	<sql:dateParam> Sets a parameter in an SQL statement to the specified java.util.Date value.
6	<sql:transaction > Provides nested database action elements with a shared Connection, set up to execute all statements as one transaction.

XML tags

The JSTL XML tags provide a JSP-centric way of creating and manipulating the XML documents. Following is the syntax to include the JSTL XML library in your JSP.

The JSTL XML tag library has custom tags for interacting with the XML data. This

includes parsing the XML, transforming the XML data, and the flow control based on the XPath expressions.

```
<%@ taglib prefix = "x"
uri = "http://java.sun.com/jsp/jstl/xml" %>
```

Before you proceed with the examples, you will need to copy the following two XML and XPath related libraries into your **<Tomcat Installation Directory>\lib** −

- **XercesImpl.jar** − Download it from https://www.apache.org/dist/xerces/j/
- **xalan.jar** − Download it from https://xml.apache.org/xalan-j/index.html

Following is the list of XML JSTL Tags −

S.No.	Tag & Description
1	<x:out> Like <%= ... >, but for XPath expressions.
2	<x:parse> Used to parse the XML data specified either via an attribute or in the tag body.
3	<x:set > Sets a variable to the value of an XPath expression.
4	<x:if > Evaluates a test XPath expression and if it is true, it processes its body. If the test condition is false, the body is ignored.
5	<x:forEach> To loop over nodes in an XML document.
6	<x:choose> Simple conditional tag that establishes a context for mutually exclusive conditional operations, marked by **<when>** and **<otherwise>** tags.
7	<x:when > Subtag of **<choose>** that includes its body if its expression evalutes to 'true'.
8	<x:otherwise > Subtag of **<choose>** that follows the **<when>** tags and runs only if all of the prior conditions evaluates to 'false'.
9	<x:transform > Applies an XSL transformation on a XML document
10	<x:param > Used along with the **transform** tag to set a parameter in the XSLT stylesheet

JSTL Functions

JSTL includes a number of standard functions, most of which are common string manipulation functions. Following is the syntax to include JSTL Functions library in your JSP −

```
<%@ taglib prefix = "fn"
uri = "http://java.sun.com/jsp/jstl/functions" %>
```

Following table lists out the various JSTL Functions −

S.No.	Function & Description
1	fn:contains() Tests if an input string contains the specified substring.
2	fn:containsIgnoreCase() Tests if an input string contains the specified substring in a case insensitive way.
3	fn:endsWith() Tests if an input string ends with the specified suffix.

4	fn:escapeXml() Escapes characters that can be interpreted as XML markup.
5	fn:indexOf() Returns the index withing a string of the first occurrence of a specified substring.
6	fn:join() Joins all elements of an array into a string.
7	fn:length() Returns the number of items in a collection, or the number of characters in a string.
8	fn:replace() Returns a string resulting from replacing in an input string all occurrences with a given string.
9	fn:split() Splits a string into an array of substrings.
10	fn:startsWith() Tests if an input string starts with the specified prefix.
11	fn:substring() Returns a subset of a string.
12	fn:substringAfter() Returns a subset of a string following a specific substring.
13	fn:substringBefore() Returns a subset of a string before a specific substring.
14	fn:toLowerCase() Converts all of the characters of a string to lower case.
15	fn:toUpperCase() Converts all of the characters of a string to upper case.
16	fn:trim() Removes white spaces from both ends of a string.

From <https://www.tutorialspoint.com/jsp/jsp_standard_tag_library.htm>

Sessions 7 & 8:

JDBC & Transaction Management

- Introduction to JDBC API
- JDBC Architecture

JDBC Tutorial – JDBC

Architecture, Components and Working

In this JDBC tutorial, we will learn about the Performing Database Operations in Java using the JDBC API (SQL CREATE, INSERT, UPDATE, DELETE, and SELECT).
We will look at the process of connecting Java with the database using JDBC. We will implement each example of performing operations on the database. In this JDBC tutorial, we will also discuss the composition and architecture of JDBC in Java. We will also see all the classes and interfaces used in the JDBC API. So, let us start the JDBC tutorial.

What is JDBC?

The term JDBC stands for Java Database Connectivity. JDBC is a specification from Sun microsystems. JDBC is an API(Application programming interface) in Java that helps users to interact or communicate with various databases. The classes and interfaces of JDBC API allow the application to send the request to the specified database.
Using JDBC, we can write programs required to access databases. JDBC and the database driver are capable of accessing databases and spreadsheets. JDBC API is also helpful in accessing the enterprise data stored in a relational database(RDB).

Purpose of JDBC

There are some enterprise applications created using the JAVA EE(Enterprise Edition) technology. These applications need to interact with databases to store application-specific information.
Interacting with the database requires efficient database connectivity, which we can achieve using ODBC(Open database connectivity) driver. We can use this ODBC Driver with the JDBC to interact or communicate with various kinds of databases, like Oracle, MS Access, Mysql, and SQL, etc.

Applications of JDBC

JDBC is fundamentally a specification that provides a complete set of interfaces. These interfaces allow for portable access to an underlying database.
We can use Java to write different types of executables, such as:

- Java Applications
- Java Applets
- Enterprise JavaBeans (EJBs)
- Java Servlets
- Java ServerPages (JSPs)

All these different executables can use a JDBC driver to access a database, and

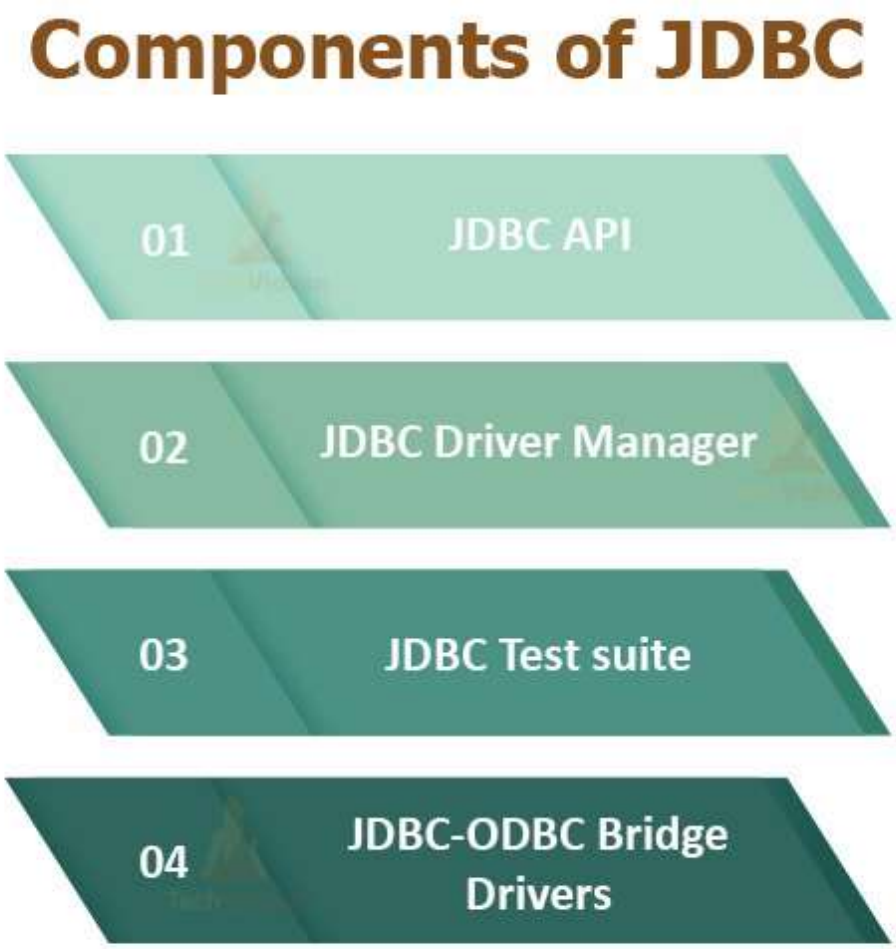
take advantage of the stored data. JDBC provides similar capabilities as ODBC by allowing Java programs to contain database-independent code.

The JDBC 4.0 Packages

There are two primary packages for JDBC 4.0: java.sql and javax.sql. JDBC 4.0 is the latest JDBC version at the time of writing this article. These packages offer the main classes for interacting with data sources.
The new features in these packages include changes in the following areas:

- Automatic database driver loading.
- Exception handling improvements.
- National character set support.
- SQL ROWID access.
- Enhanced BLOB/CLOB functionality.
- Connection and statement interface enhancements.
- SQL 2003 XML data type support.
- Annotations.

Components of JDBC



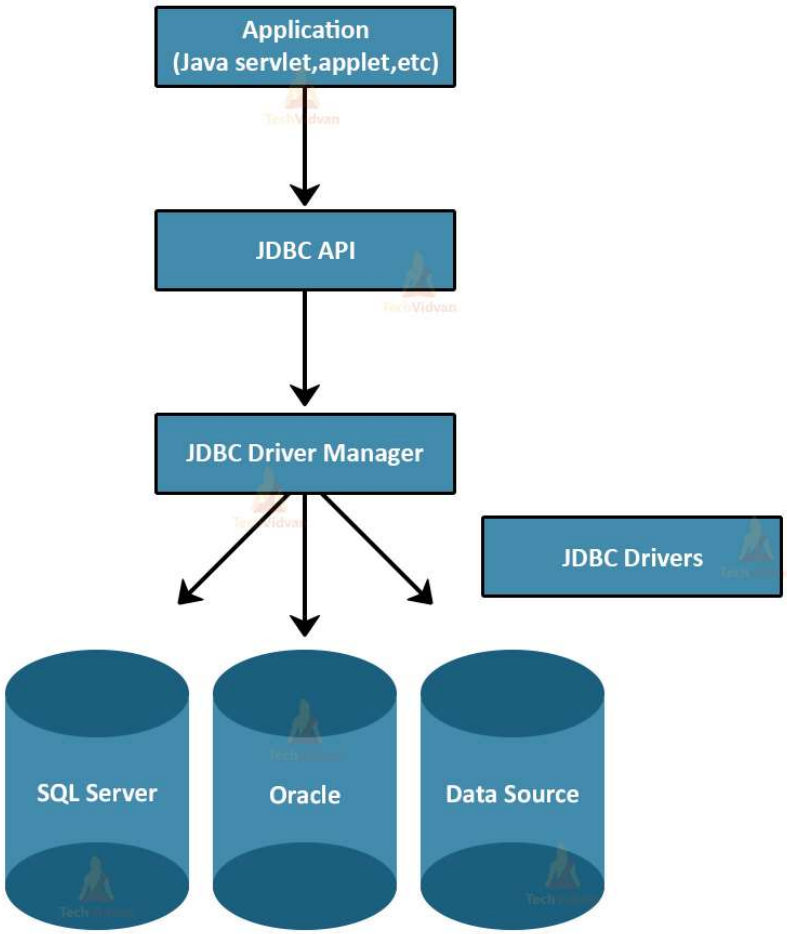
Let us move further in the JDBC Tutorial and learn the JDBC components.

There are mainly four main components of JDBC. These components help us to interact with a database. The components of JDBC are as follows:
1. JDBC API: The JDBC API provides various classes, methods, and interfaces that are helpful in easy communication with the database. It also provides two packages that contain the Java SE(Standard Edition) and Java EE(Enterprise Edition) platforms to exhibit the WORA(write once run everywhere) capabilities.
There is also a standard in JDBC API to connect a database to a client application.
2. JDBC Driver Manager: The Driver Manager of JDBC loads database-specific drivers in an application. This driver manager establishes a connection with a database. It also makes a database-specific call to the database so that it can process the user request.
3. JDBC Test suite: The Test Suite of JDBC helps to test the operation such as insertion, deletion, updation that the JDBC Drivers perform.
4. JDBC-ODBC Bridge Drivers: The JDBC-ODBC Bridge Driver connects the database drivers to the database. This bridge driver translates the JDBC method call to the ODBC method call. It uses a package in which there is a native library to access ODBC characteristics.

Architecture of JDBC

The following figure shows the JDBC architecture:

Architecture of JDBC



Description of the Architecture:
1. Application: Application in JDBC is a Java applet or a Servlet that communicates with a data source.
2. JDBC API: JDBC API provides classes, methods, and interfaces that allow Java programs to execute SQL statements and retrieve results from the database. Some important classes and interfaces defined in JDBC API are as follows:
A list of popular *interfaces* of JDBC API is given below:

- Driver interface
- Connection interface
- Statement interface
- PreparedStatement interface
- CallableStatement interface
- ResultSet interface
- ResultSetMetaData interface
- DatabaseMetaData interface
- RowSet interface

Classes of JDBC API

A list of popular *classes* of JDBC API is given below:

- DriverManager class
- Blob class
- Clob class
- Types class

3. Driver Manager: The Driver Manager plays an important role in the JDBC architecture. The Driver manager uses some database-specific drivers that effectively connect enterprise applications to databases.
4. JDBC drivers: JDBC drivers help us to communicate with a data source through JDBC. We need a JDBC driver that can intelligently interact with the respective data source.

Types of JDBC Architecture

There are two types of processing models in JDBC architecture: two-tier and three-tier. These models help us to access a database. They are:

1. Two-tier model

In this model, a Java application directly communicates with the data source. JDBC driver provides communication between the application and the data source. When a user sends a query to the data source, the answers to those queries are given to the user in the form of results.
We can locate the data source on a different machine on a network to which a user is connected. This is called a client/server configuration, in which the user machine acts as a client, and the machine with the data source acts as a server.

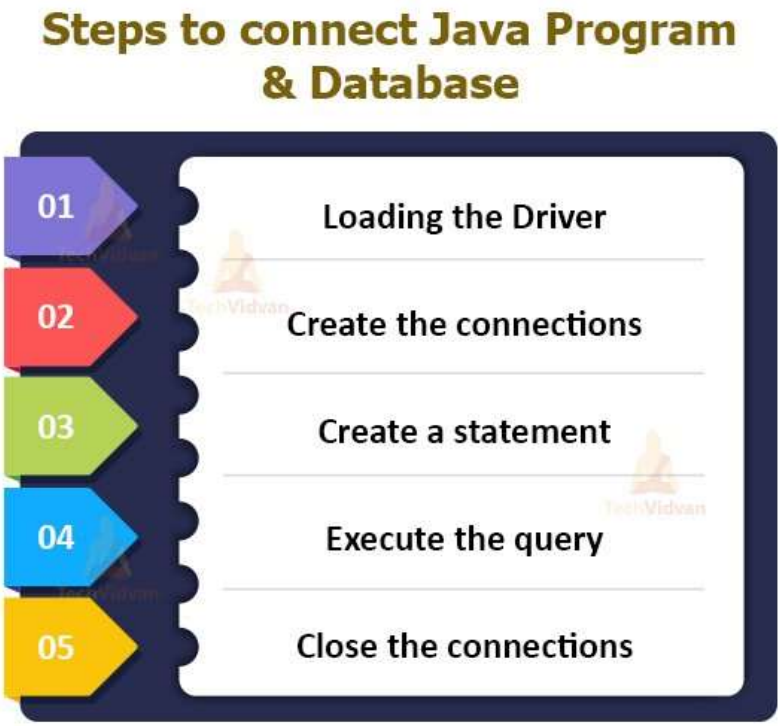
2. Three-tier model

In the three-tier model, the query of the user queries goes to the middle-tier services. From the middle-tier service, the commands again reach the data source. The results of the query go back to the middle tier.
From there, it finally goes to the user. This type of model is beneficial for management information system directors.

Prerequisites of JDBC

- JDK(Java Development Kit)
- Oracle Database: Download it from http://www.oracle.com/technetwork/database/database-technologies/express-edition/downloads/index.html
- JDBC driver for Oracle Database: Download it from http://www.oracle.com/technetwork/apps-tech/jdbc-112010-090769.html. Add ojdbc6.jar to the project library.

Steps to connect Java program and database:



1. Loading the Driver

We first need to load the driver or register it before using it in the program. There should be registration once in your program. We can register a driver in any of the two ways:
a. Class.forName(): In this, we load the driver's class file into memory during runtime. There is no need to use a new operator for the creation of an object. The following shows the use of Class.forName() to load the Oracle driver:

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

b. DriverManager.registerDriver(): DriverManager is an inbuilt class of Java that comes with a static member register. We call the drivers class' constructor at compile-time. The following example shows the use of DriverManager.registerDriver() to register the Oracle driver:

```
DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver())
```

2. Create the connections

After loading the driver, we need to establish connections using the following code:

```
Connection con = DriverManager.getConnection(url, user, password)
```

- user: username from which sql command prompt can be accessed.
- password: password from which sql command prompt can be accessed.

- con: reference to Connection interface.
- url : Uniform Resource Locator. We can create it as follows:
- String url = " jdbc:oracle:thin:@localhost:1521:xe"

Where oracle is the database, thin is the driver, @localhost is the IP Address where the database is stored, 1521 is the port number, and xe is the service provider. All three parameters are of String type and the programmer should declare them before calling the function.

3. Create a statement

Once you establish a connection, you can interact with the database. The JDBCStatement, CallableStatement, and PreparedStatement interfaces define the methods that allow us to send the SQL commands and receive data from the database.
Use of JDBC Statement is as follows:

```
Statement statement = con.createStatement()
```

Here, con is a reference to the Connection interface that we used in the previous step.

4. Execute the query

The most crucial part is executing the query. Here, Query is an SQL Query. Now, as we know that we can have multiple types of queries. Some of them are as follows:

- The query for updating or inserting tables in a database.
- The query for retrieving data from the database.

The executeQuery() method of the Statement interface executes queries of retrieving values from the database. The executeQuery() method returns the object of ResultSet that we can use to get all the records of a table.

5. Close the connections

Till now, we have sent the data to the specified location. Now, we are about to complete our task. We need to close the connection. By closing the connection, objects of Statement and ResultSet interface are automatically closed. The close() method of Connection interface closes the connection.
Example :

```
con.close();
```

Implementation

```
package com.techvidvan.jdbctutorial;
import java.sql. * ;
import java.util. * ;
class JDBCTutorial {
 public static void main(String a[]) {
 //Creating the connection
String url = "jdbc:oracle:thin:@localhost:1521:xe";
 String user = "system";
 String pass = "12345";
```

```
 //Entering the data
Scanner sc = new Scanner(System. in );
 System.out.println("Enter name:");
 String name = sc.next();
 System.out.println("Enter Roll number:");
 int rollNumber = sc.nextInt();
 System.out.println("Enter class:");
 String cls = sc.next();

 //Inserting data using SQL query
String sql = "insert into student values('" + name + "'," + rollNumber + ",'" + cls + "')";
 Connection con = null;
 try {
 DriverManager.registerDriver(new oracle.jdbc.OracleDriver());

 //Reference to connection interface
con = DriverManager.getConnection(url, user, pass);

 Statement st = con.createStatement();
 int result = st.executeUpdate(sql);
 if (result == 1) System.out.println("Inserted successfully: " + sql);
 else System.out.println("Insertion failed");
 con.close();
 }
 catch(Exception e) {
 System.err.println(e);
 }
 }
}
```

Output:

Enter name:

Shreya

Enter Roll number:

123

Enter class:

8C

Inserted successfully: insert into student values('Shreya', '123', '8C')

Working of JDBC

Now, moving ahead in this JDBC Tutorial, let us learn working of JDBC. A Java application that communicates with the database requires programming using JDBC API.

We need to add Supporting data sources of JDBC Driver such as Oracle and SQL server in Java application for JDBC support. We can do this dynamically at run time. This JDBC driver intelligently interacts with the respective data source.

Creating a simple JDBC application

```
package com.techvidvan.jdbctutorial;
import java.sql. * ;
public class JDBCTutorial {
 public static void main(String args[]) throws ClassNotFoundException,
 SQLException,
 {
 String driverName = "sun.jdbc.odbc.JdbcOdbcDriver";
 String url = "jdbc:odbc:XE";
 String username = "John";
 String password = "john12";
 String query1 = "insert into students values(101, 'Pooja')";

 //Load the driver class
Class.forName(driverName);

 //Obtaining a connection
Connection con = DriverManager.getConnection(url, username, password);

 //Obtaining a statement
Statement stmt = con.createStatement();

 //Executing the query
int count = stmt.executeUpdate(query1);
 System.out.println("The number of rows affected by this query= " + count);

 //Closing the connection
con.close();
 }
}
```

The above example shows the basic steps to access a database using JDBC. We used the JDBC-ODBC bridge driver to connect to the database. We have to import the java.sql package that provides basic SQL functionality.

Principal JDBC Interfaces and Classes

Let us take an overview look at the principal interfaces and classes of JDBC. They are all present in the java.sql package.

1. Class.forName()

This method loads the driver's class file into memory at runtime. There is no need to use new or creation of objects.

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

2. DriverManager

The DriverManager class registers drivers for a specific database type. For example, Oracle Database in this tutorial. This class also establishes a database connection with the server using its getConnection() method.

3. Connection

The Connection interface represents an established database connection. Using this connection, we can create statements to execute queries and retrieve results. We can also get metadata about the database, close the

connection, etc.

```
Connection con = DriverManager.getConnection
("jdbc:oracle:thin:@localhost:1521:orcl", "login1", "pwd1");
```

4. Statement and PreparedStatement

The Statement and PreparedStatement interfaces execute a static SQL query and parameterized SQL queries. The statement interface is the super interface of the PreparedStatement interface. The commonly used methods of these interfaces are:

a. boolean execute(String sql): This method executes a general SQL statement. It returns true if the query returns a ResultSet and false if the query returns nothing. We can use this method with a Statement only.
b. int executeUpdate(String sql): This method executes an INSERT, UPDATE, or DELETE statement. It then returns an updated account showing the number of rows affected. For example, 1 row inserted, or 2 rows updated, or 0 rows affected, etc.
c. ResultSet executeQuery(String sql): This method executes a SELECT statement and returns an object of ResultSet. This returned object contains results returned by the query.

5. ResultSet

The ResultSet is an interface that contains table data returned by a SELECT query. We use the object of ResultSet to iterate over rows using the next() method.

6. SQLException

The SQLException class is a checked exception. We declare it to so all the above methods can throw this exception. We have to provide a mechanism to explicitly catch this exception when we call the methods of the above classes.

Implementing Insert Statement in JDBC

```
package com.techvidvan.jdbctutorial;
import java.sql. * ;
public class InsertStatementDemo {
 public static void main(String args[]) {
 String id = "id1";
 String password = "pswd1";
 String fullname = "TechVidvan";
 String email = "techvidvan.com";

 try {
 Class.forName("oracle.jdbc.driver.OracleDriver");
 Connection con = DriverManager.getConnection("
jdbc:oracle:thin:@localhost:1521:orcl", "login1", "pswd1");
 Statement stmt = con.createStatement();

 // Inserting data in database
String s1 = "insert into userid values('" + id + "', '" + password + "', '" + fullname + "', '" + email + "')";
 int result = stmt.executeUpdate(s1);
 if (result > 0) System.out.println("Successfully Registered");
```

```
 else System.out.println("Insertion Failed");
 con.close();
 }
 catch(Exception e) {
 System.out.println(e);
 }
 }
}
```

Output:

```
Successfully Registered
```

Implementing Update Statement in JDBC

```
package com.techvidvan.jdbctutorial;
import java.sql. * ;
public class UpdateStatementDemo {
 public static void main(String args[]) {
 String id = "id1";
 String password = "pswd1";
 String newPassword = "newpswd";
 try {
 Class.forName("oracle.jdbc.driver.OracleDriver");
 Connection con = DriverManager.getConnection("
jdbc:oracle:thin:@localhost:1521:orcl", "login1", "pswd1");
 Statement stmt = con.createStatement();

 // Updating database
String s1 = "UPDATE userid set password = '" + newPassword + "' WHERE id = '" + id + "' AND password
= '" + password + "'";
 int result = stmt.executeUpdate(s1);

 if (result > 0) System.out.println("Password Updated Successfully ");
 else System.out.println("Error Occured!!Could not update");
 con.close();
 }
 catch(Exception e) {
 System.out.println(e);
 }
 }
}
```

Output:

```
Password Updated Successfully
```

Implementing Delete Statement in JDBC

```
package com.techvidvan.jdbctutorial;
import java.sql. * ;
public class DeleteStatementDemo {
 public static void main(String args[]) {
 String id = "id2";
 String password = "pswd";
 try {
 Class.forName("oracle.jdbc.driver.OracleDriver");
 Connection con = DriverManager.getConnection("
jdbc:oracle:thin:@localhost:1521:orcl", "login1", "pswd1");
 Statement stmt = con.createStatement();

 //Deleting from database
String s1 = "DELETE from userid WHERE id = '" + id + "' AND password = '" + pswd + "'";
```

```
int result = stmt.executeUpdate(s1);

if (result > 0) System.out.println("One User Successfully Deleted");
else System.out.println("Error Occured!!Could not delete");

con.close();
}
catch(Exception e) {
System.out.println(e);
}
}
}
```

Output:

One User Successfully Deleted

Implementing Select Statement in JDBC

```
package com.techvidvan.jdbctutorial;
import java.sql. * ;
public class SelectStatementDemo {
 public static void main(String args[]) {
 String id = "id1";
 String password = "pwd1";
 try {
 Class.forName("oracle.jdbc.driver.OracleDriver");
 Connection con = DriverManager.getConnection("
jdbc:oracle:thin:@localhost:1521:orcl", "login1", "pwd1");
 Statement stmt = con.createStatement();

 //SELECT query
String s1 = "select * from userid WHERE id = '" + id + "' AND pwd = '" + pwd + "'";
 ResultSet rs = stmt.executeQuery(s1);
 if (rs.next()) {
 System.out.println("User-id: " + rs.getString(1));
 System.out.println("Full Name: " + rs.getString(3));
 System.out.println("E-mail: " + rs.getString(4));
 }
 else {
 System.out.println("This id is already registered");
 }
 con.close();
 }
 catch(Exception e) {
 System.out.println(e);
 }
 }
}
```

Output:

User-Id: id1

Full Name: TechVidvan

E-mail: techvidvan.com

Conclusion

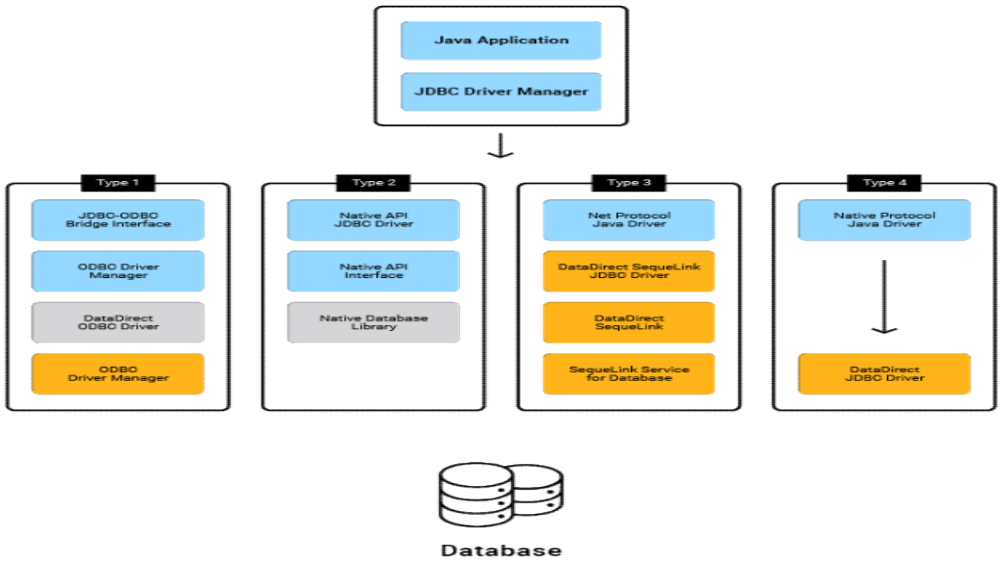
In this JDBC tutorial, we learned how to perform the various database operations in Java. We also discussed various programs and steps of Connecting to a database.
Then we learned to execute the INSERT statement, the SELECT Statement, the UPDATE statement, AND the DELETE statement with an example program. We covered the architecture and components of JDBC.
We hope this article will surely help you in performing database operations in Java.

From <https://techvidvan.com/tutorials/jdbc-tutorial/>

? JDBC Drivers

Today, there are five types of JDBC drivers in use:

- **Type 1:** JDBC-ODBC bridge
- **Type 2:** partial Java driver
- **Type 3:** pure Java driver for database middleware
- **Type 4:** pure Java driver for direct-to-database
- **Type 5:** highly-functional drivers with superior performance



JDBC Driver is a software component that enables java application to interact with the database. There are 4 types of JDBC drivers:

1. JDBC-ODBC bridge driver
2. Native-API driver (partially java driver)
3. Network Protocol driver (fully java driver)
4. Thin driver (fully java driver)

1) JDBC-ODBC bridge driver

The JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. This is now discouraged because of thin driver.

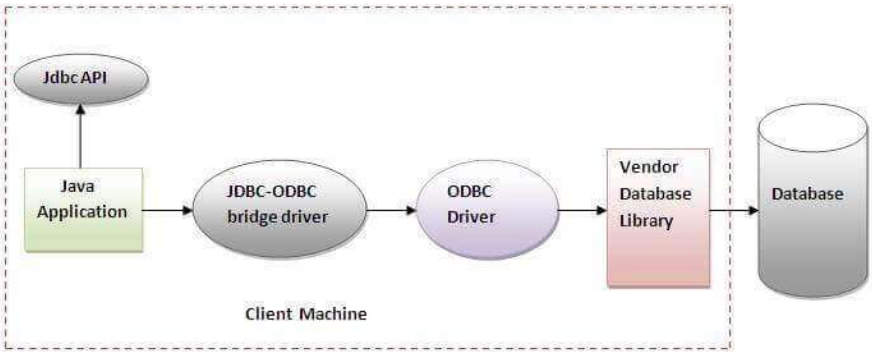


Figure- JDBC-ODBC Bridge Driver

In Java 8, the JDBC-ODBC Bridge has been removed.
Oracle does not support the JDBC-ODBC Bridge from Java 8. Oracle recommends that you use JDBC drivers provided by the vendor of your database instead of the JDBC-ODBC Bridge.

Advantages:

- easy to use.
- can be easily connected to any database.

Disadvantages:

- Performance degraded because JDBC method call is converted into the ODBC function calls.
- The ODBC driver needs to be installed on the client machine.

2) Native-API driver

The Native API driver uses the client-side libraries of the database. The driver converts JDBC method calls into native calls of the database API. It is not written entirely in java.

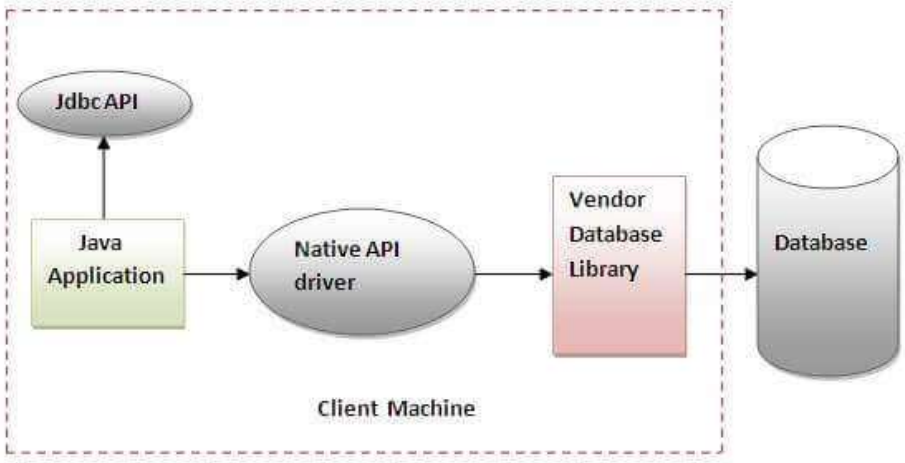


Figure- Native API Driver

Advantage:

- performance upgraded than JDBC-ODBC bridge driver.

Disadvantage:

- The Native driver needs to be installed on the each client machine.
- The Vendor client library needs to be installed on client machine.

3) Network Protocol driver

The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is fully written in java.

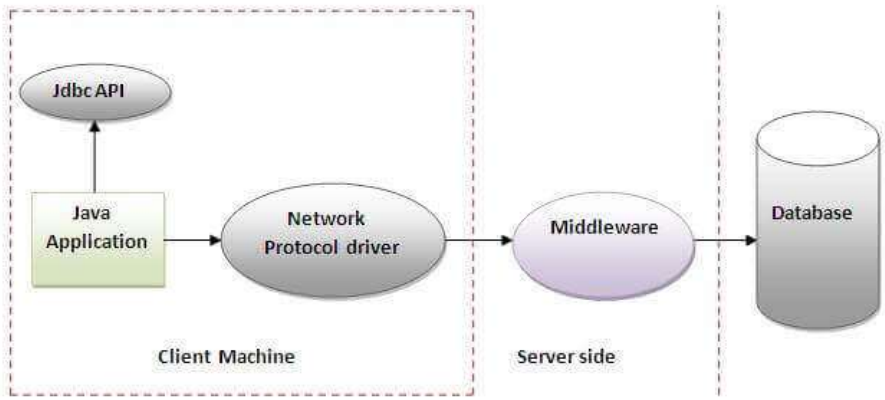


Figure- Network Protocol Driver

Advantage:

- No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.

Disadvantages:

- Network support is required on client machine.
- Requires database-specific coding to be done in the middle tier.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

4) Thin driver

The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as thin driver. It is fully written in Java language.

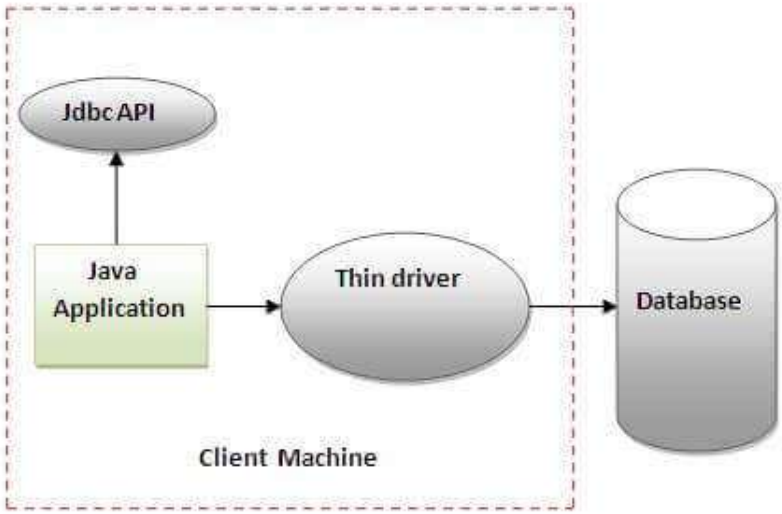


Figure- Thin Driver

Advantage:

- Better performance than all other drivers.
- No software is required at client side or server side.

Disadvantage:

- Drivers depend on the Database.

From <https://www.javatpoint.com/jdbc-driver>

? JDBC Classes& Interfaces: Driver, Connection, Statement, PreparedStatement, ResultSet and their relationship to provider implementations

JDBC API is available in two packages java.sql, core API and javax.sql JDBC optional

packages. Following are the important classes and interfaces of JDBC.

Class/interface	Description
DriverManager	This class manages the JDBC drivers. You need to register your drivers to this.It provides methods such as registerDriver() and getConnection().
Driver	This interface is the Base interface for every driver class i.e. If you want to create a JDBC Driver of your own you need to implement this interface. If you load a Driver class (implementation of this interface), it will create an instance of itself and register with the driver manager.
Statement	This interface represents a static SQL statement. Using the Statement object and its methods, you can execute an SQL statement and get the results of it.It provides methods such as execute(), executeBatch(), executeUpdate() etc. To execute the statements.
PreparedStatement	This represents a precompiled SQL statement. An SQL statement is compiled and stored in a prepared statement and you can later execute this multiple times. You can get an object of this interface using the method of the Connection interface named prepareStatement(). This provides methods such as executeQuery(), executeUpdate(), and execute() to execute the prepared statements and getXXX(), setXXX() (where XXX is the datatypes such as long int float etc..) methods to set and get the values of the bind variables of the prepared statement.
CallableStatement	Using an object of this interface you can execute the stored procedures. This returns single or multiple results. It will accept input parameters too. You can create a CallableStatement using the prepareCall() method of the Connection interface.Just like Prepared statement, this will also provide setXXX() and getXXX() methods to pass the input parameters and to get the output parameters of the procedures.
Connection	This interface represents the connection with a specific database. SQL statements are executed in the context of a connection.This interface provides methods such as close(), commit(), rollback(), createStatement(), prepareCall(), prepareStatement(), setAutoCommit() setSavepoint() etc.
ResultSet	This interface represents the database result set, a table which is generated by executing statements. This interface provides getter and update methods to retrieve and update its contents respectively.
ResultSetMetaData	This interface is used to get the information about the result set such as, number of columns, name of the column, data type of the column, schema of the result set, table name, etcIt provides methods such as getColumnCount(), getColumnName(), getColumnType(), getTableName(), getSchemaName() etc.

From <https://www.tutorialspoint.com/what-are-the-main-classes-and-interfaces-of-jdbc>

? Stored procedures and functions Invocation

Java CallableStatement Interface

CallableStatement interface is used to call the **stored procedures and functions**. We can have business logic on the database by the use of stored procedures and functions that will make the performance better because these are precompiled. Suppose you need the get the age of the employee based on the date of birth, you may create a function that receives date as the input and returns age of the employee as the output.

What is the difference between stored procedures and functions.

The differences between stored procedures and functions are given below:

Stored Procedure	Function
is used to perform business logic.	is used to perform calculation.
must not have the return type.	must have the return type.
may return 0 or more values.	may return only one values.
We can call functions from the procedure.	Procedure cannot be called from function.

Procedure supports input and output parameters.	Function supports only input parameter.
Exception handling using try/catch block can be used in stored procedures.	Exception handling using try/catch can't be used in user defined functions.

How to get the instance of CallableStatement?

The prepareCall() method of Connection interface returns the instance of CallableStatement. Syntax is given below:

```
public CallableStatement prepareCall("{ call procedurename(?,?...?)}");
```

The example to get the instance of CallableStatement is given below:

```
CallableStatement stmt=con.prepareCall("{call myprocedure(?,?)}");
```

It calls the procedure myprocedure that receives 2 arguments.

Full example to call the stored procedure using JDBC

To call the stored procedure, you need to create it in the database. Here, we are assuming that stored procedure looks like this.

```
create or replace procedure "INSERTR"
(id IN NUMBER,
name IN VARCHAR2)
is
begin
insert into user420 values(id,name);
end;
```

The table structure is given below:

```
create table user420(id number(10), name varchar2(200));
```

In this example, we are going to call the stored procedure INSERTR that receives id and name as the parameter and inserts it into the table user420. Note that you need to create the user420 table as well to run this application.

```
import java.sql.*;
public class Proc {
public static void main(String[] args) throws Exception{

Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection(  "jdbc:oracle:thin:
@localhost:1521:xe","system","oracle");
CallableStatement stmt=con.prepareCall("{call insertR(?,?)}");
stmt.setInt(1,1011);
stmt.setString(2,"Amit");
stmt.execute();

System.out.println("success");
}
```

```
- }
```
Now check the table in the database, value is inserted in the user420 table.

Example to call the function using JDBC

In this example, we are calling the sum4 function that receives two input and returns the sum of the given number. Here, we have used the **registerOutParameter** method of CallableStatement interface, that registers the output parameter with its corresponding type. It provides information to the CallableStatement about the type of result being displayed.
The **Types** class defines many constants such as INTEGER, VARCHAR, FLOAT, DOUBLE, BLOB, CLOB etc.
Let's create the simple function in the database first.

```
- create or replace function sum4
- (n1 in number,n2 in number)
- return number
- is
- temp number(8);
- begin
- temp :=n1+n2;
- return temp;
- end;
```

Now, let's write the simple program to call the function.

```
- import java.sql.*;
-
- public class FuncSum {
- public static void main(String[] args) throws Exception{
-
- Class.forName("oracle.jdbc.driver.OracleDriver");
- Connection con=DriverManager.getConnection(
- "jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
-
- CallableStatement stmt=con.prepareCall("{?= call sum4(?,?)}");
- stmt.setInt(2,10);
- stmt.setInt(3,43);
- stmt.registerOutParameter(1,Types.INTEGER);
- stmt.execute();
-
- System.out.println(stmt.getInt(1));
-
- }
- }
```
Output: 53

From <https://www.javatpoint.com/CallableStatement-interface>

? SQL Injection overview and prevention

SQL injection (SQLi) is a technique used to inject malicious code into existing SQL statements.These injections make it possible for malicious users to bypass existing security controls and gain unauthorized access to obtain, modify, and extract data, including customer records, intellectual property, or personal information. Attackers can also use this technique to locate the credentials of administrators and gain complete control over affected websites, applications, and database servers.

SQL injection attacks can affect any application that uses a SQL database and handles data, including websites, desktops, and phone apps—with extremely serious consequences.

How Does SQL Injection Work?

SQL injections are typically performed via web page or application input. These input forms are often found in features like search boxes, form fields, and URL parameters.
To perform an SQL injection attack, bad actors need to identify vulnerabilities within a web page or application. After locating a target, attackers create malicious payloads and send their input content to execute malicious commands.
In some cases, bad actors may simply leverage an automated program to carry out an SQLi for them—all they need to provide is the URL of the target website to obtain stolen data from the victim.

Types of SQL Injection Attacks

Injections were listed as the number one threat to web application security in the **OWASP Top 10**, and SQL injection vulnerabilities can be exploited in a variety of different ways.
A few common methods for SQL injections include executing commands on the database server, retrieving data based on errors, or interfering with the query logic.

1Union-Based SQL Injection

This type of SQL injection is the most popular method performed by attackers.
This injection technique allows bad actors to extract data from the database by extending the results from the original query. It uses the UNION SQL operator to integrate two SELECT statements into a single result, then returns it as part of the response.

2Blind SQL Injection

Typically more sophisticated and difficult to perform than other varieties of injections, attackers perform blind SQL injections when generic error messages are received from the target.
Blind SQL injections differentiate themselves from regular SQL injections in the method that they retrieve information from the database. In this technique, bad actors query the database for true or false questions, then determine the answer based on the response, as well as the time it takes to retrieve a server response when using it with time-based attacks.

3Boolean-Based SQL Injection

This type of attack overwrites the logic and conditions of the query to its own. It is commonly used in permission or authentication queries, where they trick the database into thinking they have elevated permissions or correct credentials.
Boolean-based SQL injections are also used in blind SQL injections, where they

proceed by elimination to extract data from the database. By sending tons of requests, each with a condition slightly different from the precedents, attackers can figure out what is the data stored based on the result of the operation.

4Error-Based SQL Injection

In an error based SQL injection, attackers exploit database errors from a web page or application that have been triggered by unsanitized inputs.
During an attack, this technique uses error messages to return full query results and reveal confidential information from the database. This method can also be used to identify if a website or web application is vulnerable and obtain additional information to restructure malicious queries.

5Time-Based SQL Injection

During a normal SQL injection, bad actors can simply read text as it is returned. However, when attackers are unable to retrieve information from a database server, they will often employ time-based SQL injections to achieve their results. This works by using operations which take a long time to complete—often many seconds.
Time-based SQL injections are commonly used when determining if vulnerabilities are present on a web application or website, as well as in conjunction with Boolean-based techniques during Blind SQL injections.

SQL Injection Example

In the following SQL injection example, we try to login by comparing the user input (username and password) to those stored in the database.

*This is an example of what NOT to do—this query has multiple flaws by design. Notably, it is vulnerable to SQL injection, and does not use **hashed** and **salted** passwords.*

```
$mysqli = new mysqli(/*...*/);

$username = $_POST['username'];
$password = $_POST['password'];

$query = "SELECT * FROM user WHERE user.username = '$username' AND user.password = '$password'";
$result = $mysqli->query($query);
```

In this example, the query is built by concatenating the user-specified values (username and password) directly into the query.
This makes it easy for an attacker to escape the quotes and inject more SQL operations. By having the following username admin' or true — it is possible for an attacker to login with the account of his choice.

```
$mysqli = new mysqli(/*...*/);

$username = $_POST['username'];
$password = $_POST['password'];

$query = "SELECT * FROM user WHERE user.username = 'admin' or true --' AND user.password = '$password'";
$result = $mysqli->query($query);
```

How to Detect an SQL Injection

SQL injections are notoriously difficult to detect. Unlike cross-site scripting, remote code injection, and other types of infections, SQL injections are vulnerabilities that do not leave traces on the server. Instead, the exploit executes genuine queries on the database. As a result, the majority of attacks are detected once an attacker has used a vulnerability to perform malicious actions or gained administrative access.
By taking precautionary measures and **actively monitoring your database** and its

queries, you can identify if an attacker is running malicious injections on your website.

How to Prevent SQL Injection Attacks

Attackers frequently target websites that use known vulnerabilities. Undisclosed, unpatched, or **zero-day vulnerabilities** also account for a large percentage of SQL injections during targeted attacks.
The easiest way to **protect your website against SQL injections** is to keep all of your third party software and components up to date. However, a number of techniques exist that you can use to help prevent SQL injection vulnerabilities.

1Use Prepared Statements with Parameterized Queries

Prepared statements are used to ensure none of the dynamic variables you need in a query can escape their position. The core query is defined beforehand, with the arguments and their types afterward.
Since the query knows the type of data that is expected, such as string or number, they know exactly how to integrate them to the query without causing issues.

```
$mysqli = new mysqli(/*...*/);

$username = $_POST['username'];
$password = $_POST['password'];

$query = "SELECT * FROM user WHERE user.username = ? AND user.password = ?";
$statement = $mysqli->prepare($query);
$statement->bind_param('s', $username);
$statement->bind_param('s', $password);
$statement->execute();
$result = $statement->get_results();
```

In this example, even if the username or password variables attempt to escape their query, the prepared statements properly escape their characters to prevent unexpected behavior or SQLi.

2Use Stored Procedures

Stored procedures are frequent SQL operations that are stored on the database itself, varying only with their arguments. Stored procedures make it much more difficult for attackers to execute their malicious SQL, as it is unable to be dynamically inserted within queries.

3Allowlist Input Validation

As a rule of thumb, don’t trust user-submitted data. You can perform allowlist validation to test user input against an existing set of known, approved, and defined input. Whenever data is received that doesn’t meet the assigned values, it is rejected—protecting the application or website from malicious SQL injections in the process.

4Enforce the Principle of Least Privilege

The Principle of Least Privilege is a computer science principle that strengthens access controls to your website to mitigate security threats.
To implement this principle and defend against SQL injections:

- Use the minimum set of privileges on your systems to perform an actions.
- Grant privileges only for the time that the action is necessary.
- Do not assign admin type access rights to application accounts.
- Minimize the privileges to every database account in your environment.

5Escape User Supplied Input

During a normal SQL injection, bad actors can simply read text as it is returned. However, when attackers are unable to retrieve information from a database server, they will often employ time-based SQL injections to achieve their results. This works by using operations which take a long time to complete—often many seconds.
Time-based SQL injections are commonly used when determining if vulnerabilities are present on a web application or website, as well as in conjunction with Boolean-based techniques during Blind SQL injections.

6Use a Web Application Firewall

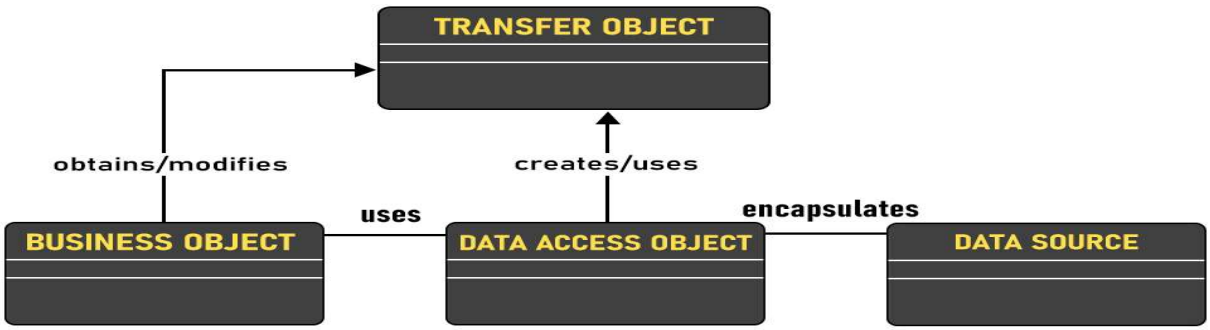
You can protect against generic **SQL injections** with a web application firewall. By filtering potentially dangerous web requests, web application firewalls can catch and prevent SQL injections.

From <https://sucuri.net/guides/what-is-sql-injection/>

? Design Pattern: Data Access Object Pattern

Data Access Object Pattern or DAO pattern is used to separate low-level data accessing API or operations from high-level business services. Following are the participants in Data Access Object Pattern.

UML Diagram Data Access Object Pattern



Advantages

1. The advantage of using data access objects is the relatively simple and rigorous separation between two important parts of an application that can but should not know anything of each other, and which can be expected to evolve frequently and independently.

2. If we need to change the underlying persistence mechanism we only have to change the DAO layer and not all the places in the domain logic where the DAO layer is used from.

Disadvantages

1. Potential disadvantages of using DAO is a leaky abstraction, code duplication, and abstraction inversion.

Design components

- **BusinessObject:** The BusinessObject represents the data client. It is the object that requires access to the data source to obtain and store data. A BusinessObject may be implemented as a session bean, entity bean, or some other Java object in addition to a servlet or helper bean that accesses the data source.
- **DataAccessObject:** The DataAccessObject is the primary object of this pattern. The DataAccessObject abstracts the underlying data access implementation for the BusinessObject to enable transparent access to the data source.
- **DataSource:** This represents a data source implementation. A data source could be a database such as an RDBMS, OODBMS, XML repository, flat file system, and so forth. A data source can also be another system service or some kind of repository.
- **TransferObject:** This represents a Transfer Object used as a data carrier. The DataAccessObject may use a Transfer Object to return data to the client. The DataAccessObject may also receive the data from the client in a Transfer Object to update the data in the data source.
  Example:

- Java

```java
// Java program to illustrate Data Access Object Pattern

// Importing required classes
import java.util.ArrayList;
import java.util.List;

// Class 1
// Helper class
class Developer {

    private String name;
    private int DeveloperId;

    // Constructor of Developer class
    Developer(String name, int DeveloperId)
    {

        // This keyword refers to current instance itself
        this.name = name;
        this.DeveloperId = DeveloperId;
    }

    // Method 1
    public String getName() { return name; }

    // Method 2
    public void setName(String name) { this.name = name; }

    // Method 3
    public int getDeveloperId() { return DeveloperId; }

    // Method 4
    public void setDeveloperId(int DeveloperId)
    {
        this.DeveloperId = DeveloperId;
    }
```

```
}

// Interface
interface DeveloperDao {
    public List<Developer> getAllDevelopers();
    public Developer getDeveloper(int DeveloperId);
    public void updateDeveloper(Developer Developer);
    public void deleteDeveloper(Developer Developer);
}

// Class 2
// Implementing above defined interface
class DeveloperDaoImpl implements DeveloperDao {

    List<Developer> Developers;

    // Method 1
    public DeveloperDaoImpl()
    {
        Developers = new ArrayList<Developer>();
        Developer Developer1 = new Developer("Kushagra", 0);
        Developer Developer2 = new Developer("Vikram", 1);
        Developers.add(Developer1);
        Developers.add(Developer2);
    }

    // Method 2
    @Override
    public void deleteDeveloper(Developer Developer)
    {
        Developers.remove(Developer.getDeveloperId());
        System.out.println("DeveloperId "
                           + Developer.getDeveloperId()
                           + ", deleted from database");
    }

    // Method 3
    @Override public List<Developer> getAllDevelopers()
    {
        return Developers;
    }

    // Method 4
    @Override public Developer getDeveloper(int DeveloperId)
    {
        return Developers.get(DeveloperId);
    }

    // Method 5
    @Override
    public void updateDeveloper(Developer Developer)
    {
        Developers.get(Developer.getDeveloperId())
            .setName(Developer.getName());
        System.out.println("DeveloperId "
                           + Developer.getDeveloperId()
```

```
                            + ", updated in the database");
    }
}

// Class 3
// DaoPatternDemo
class GFG {

    // Main driver method
    public static void main(String[] args)
    {

        DeveloperDao DeveloperDao = new DeveloperDaoImpl();

        for (Developer Developer :
             DeveloperDao.getAllDevelopers()) {
            System.out.println("DeveloperId : "
                               + Developer.getDeveloperId()
                               + ", Name : "
                               + Developer.getName());
        }

        Developer Developer
            = DeveloperDao.getAllDevelopers().get(0);

        Developer.setName("Lokesh");
        DeveloperDao.updateDeveloper(Developer);

        DeveloperDao.getDeveloper(0);
        System.out.println(
            "DeveloperId : " + Developer.getDeveloperId()
            + ", Name : " + Developer.getName());
    }
}
```

Output

```
DeveloperId : 0, Name : Kushagra
DeveloperId : 1, Name : Vikram
DeveloperId 0, updated in the database
DeveloperId : 0, Name : Lokesh
```

From <https://www.geeksforgeeks.org/data-access-object-pattern/>

Sessions 9, 10 & 11:

? Hibernate Framework
o Introduction to Hibernate Framework

Need of Hibernate Framework

Hibernate is used to overcome the limitations of JDBC like:

- JDBC code is dependent upon the Database software being used i.e. our persistence logic is dependent, because of using JDBC. Here we are inserting a record into Employee table but our query is Database software-dependent i.e. Here we are using MySQL. But if we change our Database then this query won't work.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
public class InsertDataIntoEmp
{
    public static void main(String[] args)
    {
        final String URL="jdbc:mysql://localhost:3306?user=root&&password=root";
        String query="insert into EmployeeDB.Employee values(1,'Bikash Dubey','300000','CSE')";
        try
        {
            Class.forName("com.mysql.jdbc.Driver");
            Connection con=DriverManager.getConnection(URL);
            System.out.println("Connection established");
            Statement st=con.createStatement();
            st.execute(query);
            System.out.println("Data inserted");
            st.close();
            con.close();
        }catch(SQLException | ClassNotFoundException e)
        {
            e.printStackTrace();
        }
    }
}
```

- If working with JDBC, changing of Database in middle of the project is very costly.
- JDBC code is not portable code across the multiple database software.
- In JDBC, Exception handling is mandatory. Here We can see that we are handling lots of Exception for connection.

```
import java.io.IOException;
import java.sql.*;
public class Demo
{
    public static void main(String[] args) throws IOException, SQLException
    {
        try {
            /*loading & register of the driver class*/
            Class.forName("com.mysql.jdbc.Driver");
            Connection con=DriverManager.getConnection("jdbc:mysql://localhost:3306","root", "root");
            System.out.println("Driver class loaded successfully");
        } catch (ClassNotFoundException e)
        {
            System.out.println("Driver Class not loaded");
        }
    }
}
```

- While working with JDBC, There is no support Object-level relationship.
- In JDBC, there occurs a Boilerplate problem i.e. For each and every project we have to write the below code. That increases the code length and reduce the readability.

```
try
{
    Class.forName("com.mysql.jdbc.Driver");
    Connection con=DriverManager.getConnection("jdbc:mysql://localhost:3306?user=root&&password=root");
    System.out.println("Connection established");
    con.close();
}catch(SQLException | ClassNotFoundException e)
{
    e.printStackTrace();
}
```

To overcome the above problems we use ORM tool i.e. nothing but Hibernate framework. By using Hibernate we can avoid all the above problems and we can enjoy some additional set of functionalities.

About Hibernate Framework

Hibernate is a framework which provides some **abstraction layer**, meaning that the programmer does not have to worry about the implementations, Hibernate does the implementations for you internally like **Establishing a connection with the database, writing query to perform CRUD operations etc**.

It is a **java framework** which is used to develop persistence logic. Persistence logic means to store and process the data for long use. More precisely Hibernate is an open-source, non-invasive, light-weight java ORM(Object-relational mapping) framework to develop objects which are independent of the database software and make independent persistence logic in all JAVA, JEE.

Framework means it is special install-able software that provides an abstraction layer on one or more technologies like JDBC, Servlet, etc to simplify or reduce the complexity for the development process.

Open Source means:

- Hibernate framework is available for everyone without any cost.
- The source code of Hibernate is also available on the Internet and we can also modify the code.

Light-weight means:

- Hibernate is less in size means the installation package is not big is size.
- Hibernate does not require any heavy container for execution.
- It does not require POJO and POJI model programming.
- Hibernate can be used alone or we can use Hibernate with other java technology and framework.

Non-invasive means:

- The classes of Hibernate application development are loosely coupled classes with respect to Hibernate API i.e. Hibernate class need not implement hibernate API interfaces and need not extend from Hibernate API classes.

Functionalities supported by Hibernate framework

- Hibernate framework support **Auto DDL** operations. In JDBC manually we have to create table and declare the data-type for each and every column. But Hibernate can do **DDL operations** for you internally like creation of table,drop a table,alter a table etc.
- Hibernate supports **Auto Primary key generation**. It means in JDBC we have to manually set a primary key for a table. But Hibernate can this task for you.
- Hibernate framework is independent of Database because it supports **HQL (Hibernate Query Language)** which is not specific to any database, whereas JDBC is database dependent.
- In Hibernate, **Exception Handling is not mandatory**, whereas In JDBC exception handling is mandatory.
- Hibernate supports **Cache Memory** whereas JDBC does not support cache memory.
- Hibernate is a **ORM tool** means it support Object relational mapping. Whereas JDBC is not object oriented moreover we are dealing with values means primitive data. In hibernate each record is represented as a Object but in JDBC each record is nothing but a data which is nothing but primitive values.

From <https://www.geeksforgeeks.org/introduction-to-hibernate-framework/>

o Architecture

Hibernate: Hibernate is a framework which is used to develop persistence logic which is independent of Database software. In JDBC to develop persistence logic we deal with primitive types. Whereas Hibernate framework we use Objects to develop persistence logic which are independent of database software.

Hibernate Architecture:

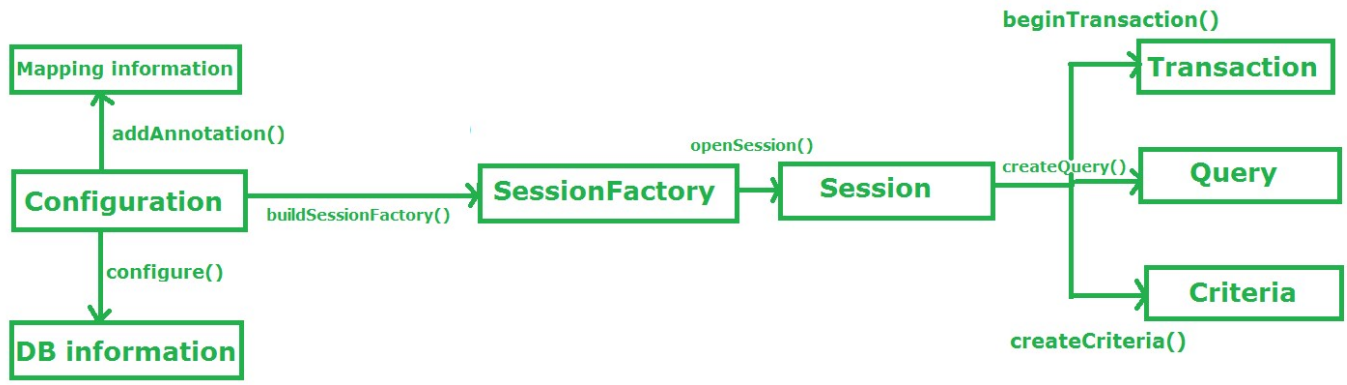


Fig: Hibernate Architecture

Configuration:

- Configuration is a class which is present in org.hibernate.cfg package. It activates Hibernate framework. It reads both configuration file and mapping files.

```
It activate Hibernate Framework
Configuration cfg=new Configuration();
It read both cfg file and mapping files
```

```
cfg.configure();
```

- It checks whether the config file is syntactically correct or not.
- If the config file is not valid then it will throw an exception. If it is valid then it creates a meta-data in memory and returns the meta-data to object to represent the config file.

SessionFactory:

- SessionFactory is an Interface which is present in org.hibernate package and it is used to create Session Object.
- It is immutable and thread-safe in nature.

```
buildSessionFactory() method gathers the meta-data which is in the cfg Object.
From cfg object it takes the JDBC information and create a JDBC Connection.
SessionFactory factory=cfg.buildSessionFactory();
```

Session:

- Session is an interface which is present in org.hibernate package. Session object is created based upon SessionFactory object i.e. factory.
- It opens the Connection/Session with Database software through Hibernate Framework.
- It is a light-weight object and it is not thread-safe.
- Session object is used to perform CRUD operations.

```
Session session=factory.buildSession();
```

Transaction:

- Transaction object is used whenever we perform any operation and based upon that operation there is some change in database.
- Transaction object is used to give the instruction to the database to make the changes that happen because of operation as a permanent by using commit() method.

```
Transaction tx=session.beginTransaction();
tx.commit();
```

Query:

- Query is an interface that present inside org.hibernate package.
- A Query instance is obtained by calling Session.createQuery().
- This interface exposes some extra functionality beyond that provided by Session.iterate() and Session.find():
  1. A particular page of the result set may be selected by calling setMaxResults(), setFirstResult().
  2. Named query parameters may be used.

```
Query query=session.createQuery();
```

Criteria:

- Criteria is a simplified API for retrieving entities by composing Criterion objects.

- The Session is a factory for Criteria. Criterion instances are usually obtained via the factory methods on Restrictions.
  `Criteria criteria=session.createCriteria();`

Flow of working during operation in Hibernate Framework :

Suppose We want to insert an Object to the database. Here Object is nothing but persistence logic which we write on java program and create an object of that program. If we want to insert that object in the database or we want to retrieve the object from the database. Now the question is that how hibernate save the Object to the database or retrieve the object from the database. There are several layers through which Hibernate framework go to achieve the above task. Let us understathe nd the layers/flow of Hibernate framework during performing operations:

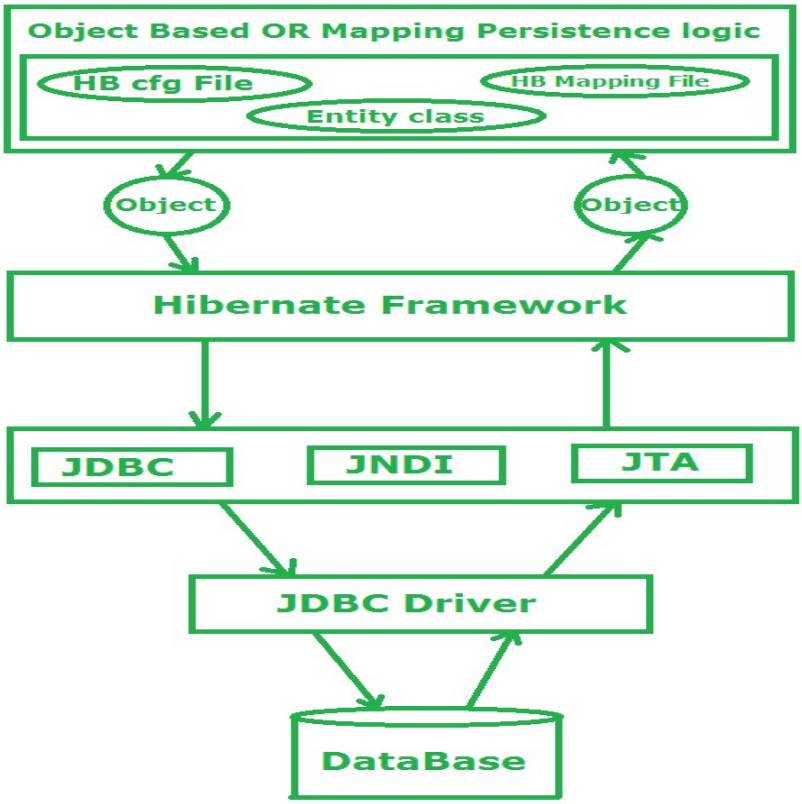


Fig: Working Flow of Hibernate framework to save/retrieve the data from the database in form of Object

Stage I: In first stage, we will write the persistence logic to perform some specific operations to the database with the help of Hibernate Configuration file and Hibernate mapping file. And after that we create an object of the particular class on which we wrote the persistence logic.

Stage II: In second stage, our class which contains the persistence logic will interact with the hibernate framework where hibernate framework gives some abstraction do perform some task. Now here the picture of java class is over. Now Hibernate is responsible to perform the persistence logic with the help of layers which is below of Hibernate framework or we can say that the layers which are the internal implementation of Hibernate.

Stage III: In third stage, our hibernate framework interact which JDBC, JNDI, JTA etc to go to the database to perform that persistence logic.

Stage IV & V: In fourth & fifth stage, hibernate is interact with Database with the help of JDBC driver. Now here hibernate perform that persistence logic which is nothing but **CRUD** operation. If our persistence logic is to retrieve an record then in the reverse order it will display on the console of our java program in terms of Object.

From <https://www.geeksforgeeks.org/hibernate-architecture/>

? Hibernate in IDE

o Creating web application using Hibernate API

What is Hibernate framework?

Hibernate is a Java-based framework that aims at relieving developers from common database programming tasks such as writing SQL queries manually with JDBC. Thus developers can focus more on business logic of the application. Hibernate encapsulates common database operations (CRUD – create, retrieve, update and delete) via a set of useful APIs.
Hibernate is also an Object/Relational Mapping (ORM) framework which maps tables from database with Java objects, allows developers to build database relationship and access data easily through its mapping, configuration and APIs . It is widely used by Java developers for Java applications ranging from desktop to enterprise.
If you are new to Hibernate and can't wait to learn it, this tutorial is to you. We will get you familiar with Hibernate through a hand-on exercise which involves in developing a simple Java application to manage data of a table in database (IDE to use is Eclipse). You will see how simple Hibernate makes to database development.

About the sample Hibernate application:

In this tutorial, we will develop a console application that manages a list of contacts in MySQL database. Hibernate will be used to perform the CRUD operations: create, read, update, and delete. Let's start!

Download Hibernate and MySQL JDBC driver:

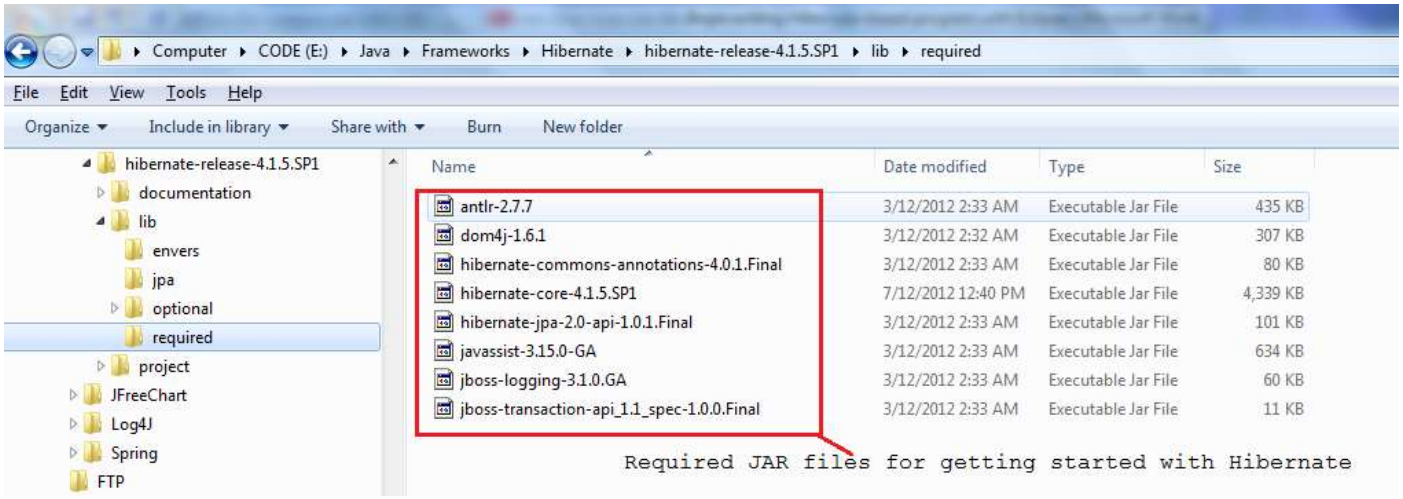
You can obtain the latest distribution of Hibernate framework at:

http://hibernate.org/downloads

You are redirected to SourceForge download page:

http://sourceforge.net/projects/hibernate/files/hibernate4/

At the time of writing this tutorial, the latest version of Hibernate is 4.1.5.SP1. Download the zip archive `hibernate-release-4.1.5.SP1.zip` and extract it to a desired location on your computer. The distribution has a lot of stuff but we are interested in only the required JAR files which are under *lib \required* directory:



Those JAR files are minimum requirement for getting started with Hibernate.
In addition, we need to download JDBC driver library for MySQL, called Connector/J at: http://www.mysql.com/downloads/connector/j/
Extract the `mysql-connector-java-5.1.21.zip` file and pick up the JAR file `mysql-connector-java-5.1.21-bin.jar`.
Notes: For more information about downloading and using MySQL JDBC driver, see the tutorial: Connect to MySQL via JDBC.

1. Setup database

Suppose that you have MySQL server installed on your computer. Execute the following SQL script to create a table called *contact* in database *contactdb*:

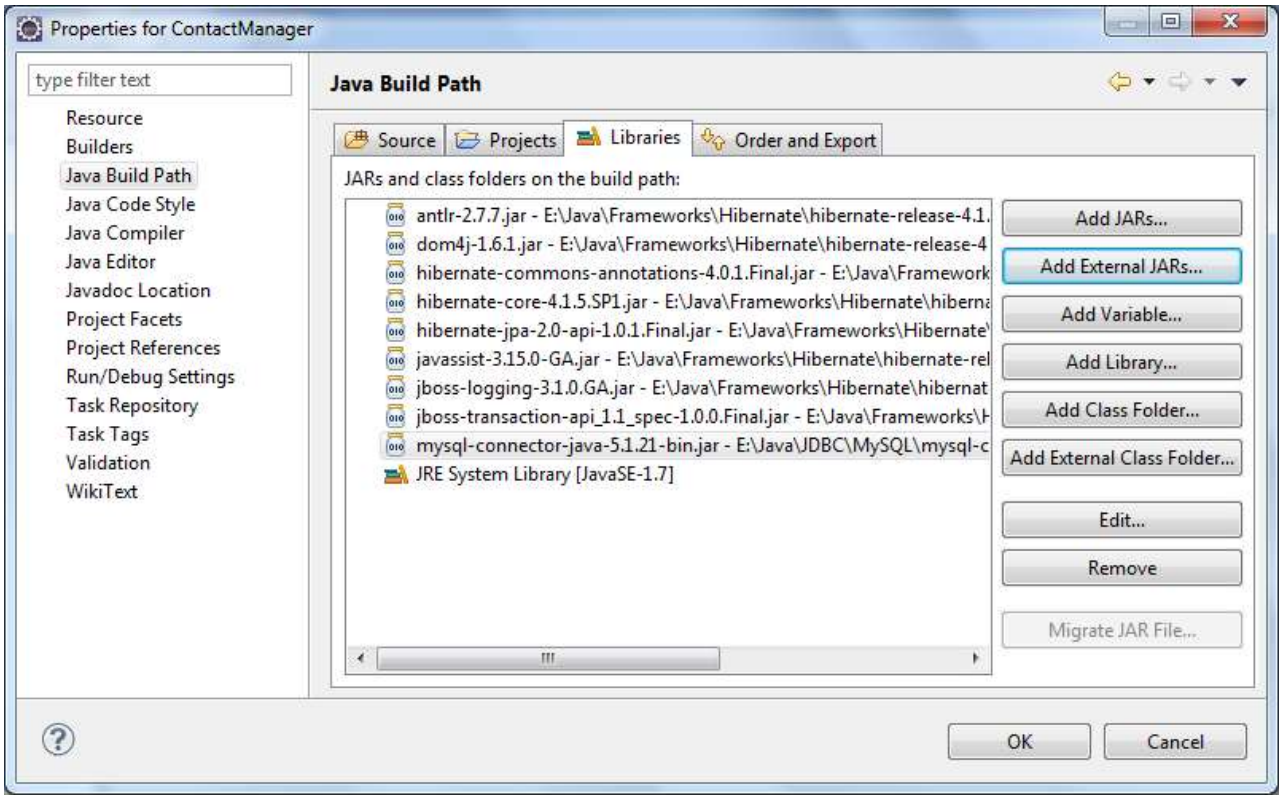
```
create database contactdb;
use contactdb;
CREATE  TABLE `contact` (
  `contact_id` INT NOT NULL ,
  `name` VARCHAR(45) NOT NULL ,
  `email` VARCHAR(45) NOT NULL ,
  `address` VARCHAR(45) NOT NULL ,
  `telephone` VARCHAR(45) NOT NULL ,
  PRIMARY KEY (`contact_id`) )
DEFAULT CHARACTER SET = utf8
COLLATE = utf8_general_ci;
```

From the above script, we can see that a contact has following information:

- contact_id: primary key of the table
- name
- email
- address
- telephone.

2. Setup Eclipse project

Create a new Java project in Eclipse entitled *ContactManager*.
Select **Project** ⮚ **Properties** from main menu.
The dialog *Properties for ContactManager* appears, select *Java Build Path* on the left.
Under *Libraries* tab on the right, click **Add External JARs**.
In the dialog *Jar Selection*, select all JAR files under the directory *{Hibernate_Home}\lib \required* where *{Hibernate_Home}* is the directory where you extracted Hibernate distribution previously.
Click **Open** to add these JAR files to project's classpath, as shown in the following screen-shot:



In addition, since we use MySQL database, it's necessary to include a MySQL JDBC driver library to the project as well. See this article for how to download and extract MySQL JDBC driver JAR file.

Under `src`directory, create a new Java package called `com.mycompany`

3. Create Hibernate configuration file

First of all, you need to tell Hibernate some information regarding the database system you intend to use, such as host name, port number, JDBC driver class name, database connection URL… via some kind of configuration. Flexibly, Hibernate provides several ways for supplying database configuration:

- via properties file.
- via XML file.
- via JNDI data source.
- Programmatic configuration.
- Setting system properties.

We choose XML configuration file. In Eclipse, create a new XML file under `src` directory with this name: `hibernate.cfg.xml`. Put some configuration information in this file as follows:

```
1  <?xml version='1.0' encoding='utf-8'?>
2  <!DOCTYPE hibernate-configuration PUBLIC
3          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4          "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
5  <hibernate-configuration>
6    <session-factory>
7      <!-- Database connection settings -->
8      <property name="connection.driver_class">
9  com.mysql.jdbc.Driver</property>
10     <property name="connection.url">
11 jdbc:mysql://localhost:3306/contactdb</property>
12     <property name="connection.username">root</property>
13     <property name="connection.password">P@ssw0rd</property>
14
     </session-factory>
   </hibernate-configuration>
```

We specify JDBC class driver name (for MySQL), connection URL (which also specifies host name, port number, and database name), user name and password to connect to the database.
The class `org.hibernate.cfg.Configuration` is used to load the configuration file:

```
Configuration conf = new Configuration().configure();
```

The `configure()` method reads configuration from `hibernate.cfg.xml` file under program's classpath. There are also some overloaded versions of `configure()` method which allow you to load configuration file in different ways.
The above configuration is only for necessary information. In addition, Hibernate provides a variety of settings such as connection pooling, transaction, debugging… however we don't cover all of them in this simple tutorial. Just add this line to the `hibernate.cfg.xml` file:

```
<property name="show_sql">true</property>
```

which tells Hibernate to show all SQL statements to the standard output.

4. Create Java entity class

Next, we will create a plain old java object (POJO) class which represents a table in the database. Since we created a table called Contact, we also create a new Java class with the same name, `Contact.java`, under package `com.mycompany`. Write code for the Contact class as following:

```
package com.mycompany;
public class Contact {
    private int id;
    private String name;
    private String email;
    private String address;
    private String telephone;
    public Contact() {
    }
    public Contact(int id, String name, String email, String address,
            String telephone) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.address = address;
        this.telephone = telephone;
    }
    public Contact(String name, String email, String address, String
telephone) {
        this.name = name;
        this.email = email;
        this.address = address;
        this.telephone = telephone;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
```

```
    public void setName(String name) {
        this.name = name;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }
    public String getTelephone() {
        return telephone;
    }
    public void setTelephone(String telephone) {
        this.telephone = telephone;
    }
}
```

It's just a simple class following JavaBean naming convention.

5. Create Hibernate mapping file

Hibernate can detect Java classes which map with tables in the database via some kind of mappings. A mapping tells Hibernate how to map between a Java class and a table in the database. For example, the class maps with which table, which class' member maps with which field in that table, what kind of data types, etc.

Hibernate allows developer to specify mapping through XML file or annotations embedded right in the Java source file. We go with the XML mapping.

In Eclipse, create a new XML file called `Contact.hbm.xml` under package `com.mycompany`, with the following content:

```
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC
        "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
        "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping package="com.mycompany">
    <class name="Contact" table="CONTACT">
        <id name="id" column="CONTACT_ID">
            <generator class="increment"/>
        </id>
        <property name="name" type="string" column="NAME"/>
        <property name="email"/>
        <property name="address"/>
        <property name="telephone"/>

    </class>
</hibernate-mapping>
```

As you can see:

- The `package` property of the root element `<hibernate-mapping>` specifies under which package Hibernate finds Java classes.
- The `class` element specifies which Java class (via property `name`) maps with which table (via property `table`) in the database.
- The `id` element specifies which property in the Java class map to primary key column in the database. The `generator` element tells Hibernate how the primary key's value is created, and we

specify `increment` which means the value is incremented automatically.

- The `property`element specifies how a property in the Java class maps with a column in the database:
- the `name` property specifies name of a property in Java class.
- the `column` property specifies mapping column in database.
- the `type` property specifies data type of the mapping column. By default, Hibernate can detect and create mapping automatically based on property `name` and the JavaBean class, so you can see, we do not specify `type` and `column` for the last three properties.

After defining the mapping file, you need to tell Hibernate to parse the mapping file by adding the following entry in the `hibernate.cfg.xml` file:

```
<mapping resource="com/mycompany/Contact.hbm.xml"/>
```

The `hibernate.cfg.xml` file should look like this till now:

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <!-- Database connection settings -->
    <property name="connection.driver_class">
com.mysql.jdbc.Driver</property>
    <property name="connection.url">
jdbc:mysql://localhost:3306/contactdb</property>
    <property name="connection.username">root</property>
    <property name="connection.password">P@ssw0rd</property>

    <property name="show_sql">true</property>

    <mapping resource="com/mycompany/Contact.hbm.xml"/>

  </session-factory>
</hibernate-configuration>
```

6. Working with Hibernate session

Hibernate provides its database services via a session. In other words, all database operations are executed under context of a Hibernate session. The session manages mapped objects and offers database's CRUD (create, read, update, delete) operations. Working with Hibernate's API involves in working with its Session and Session's relevant methods.
The following code obtains a new session from a session factory:

```
Configuration configuration = new Configuration().configure();
ServiceRegistryBuilder registry = new ServiceRegistryBuilder();
registry.applySettings(configuration.getProperties());
ServiceRegistry serviceRegistry = registry.buildServiceRegistry();
SessionFactory sessionFactory =
configuration.buildSessionFactory(serviceRegistry);
Session session = sessionFactory.openSession();
```

The `Session`interface defines a number of methods for executing CRUD operations with mapped objects. The following table lists the most common methods provided by the session:

#	Method name	Return	Description	Issued SQL statement
1	`beginTransaction()`	`Transaction`	Creates a Transaction object or returns an existing one, for working under context of a transaction.	
2	`getTransaction()`	`Transaction`	Returns the current transaction.	
3	`get(Class class, Serializable id)`	`Object`	Loads a persistent instance of the given class with the given id, into the session.	SELECT
4	`load(Class class, Serializable id)`	`Object`	Does same thing as `get()` method, but throws an `ObjectNotFound` error if no row with the given id exists.	SELECT
5	`persist(Object)`	`void`	saves a mapped object as a row in database	INSERT
6	`save(Object)`	`Serializable`	Does same thing as `persist()` method, plus returning a generated identifier.	INSERT
7	`update(Object)`	`void`	Updates a detached instance of the given object and the underlying row in database.	UPDATE
8	`saveOrUpdate(Object)`	`void`	Saves the given object if it does not exist, otherwise updates it.	INSERT or UPDATE
9	`delete(Object)`	`void`	Removes a persistent object and the underlying row in database.	DELETE
10	`close()`	`void`	Ends the current session.	
11	`flush()`	`void`	Flushes the current session. This method should be called before committing the transaction and closing the session.	
12	`disconnect()`	`void`	Disconnects the session from current JDBC connection.	

7. Working with Hibernate transaction

Multiple database operations should be executed inside a transaction in order to protect consistence of data in the database in case of exceptions or errors occurred. The following skeleton code illustrates working with a transaction in Hibernate:

```
session.beginTransaction();
try {
    session.persist(Object);
    session.update(Object);
    session.delete(Object);
} catch (Exception ex) {
    session.getTransaction().rollback();
}
session.getTransaction().commit();
session.close();
```

When working with transaction, it requires committing the transaction to make permanent changes to the database, and finally, close the session.

8. Perform CRUD operations with Hibernate

Create:

To persist an object into the session (equivalent to inserting a record into the database), either `persist()` or `save()` method can be used. For example:

```
Contact contact1 = new
Contact("Nam", "hainatuatgmail.com", "Vietnam", "0904277091");
session.persist(contact1);
Contact contact2 = new Contact("Bill", "billatgmail.com", "USA", "18001900");
Serializable id = session.save(contact2);
System.out.println("created id: " + id);
```

The save method returns id of the new record inserted in the database.

Read:

To retrieve a persistent object from the database into the session (equivalent to selecting a record from the database), use the `get()` method. For example:

```
1 Contact contact = (Contact) session.get(Contact.class, new Integer(2));
2 if (contact == null) {
3     System.out.println("There is no Contact object with id=2");
4 } else {
5     System.out.println("Contact3's name: " + contact.getName());
6 }
```

The `get()` method returns null if there is no row with the given id exists in the database, whereas the `load()` method throws an `ObjectNotFound` error. Therefore, use the `load()` method only when the object is assumed exists.

Update:

To update a persistent object which is already loaded in the session (attached instance), simply update the mapped object. Hibernate will update changes to the database then the session is flushed. For example:

```
1 Contact contact = (Contact) session.load(Contact.class, new
2 Integer(5));
3 contact.setEmail("infoatcompany.com");
  contact.setTelephone("1234567890");
```

You can call the `update()` method explicitly (but not required):

```
1 session.update(contact);
```

To update a detached instance, call either the `update()` or `saveOrUpdate()` method, for example:

```
1 Contact contact = new
2 Contact("Jobs", "jobsatapplet.com", "Cupertino", "0123456789");
  session.update(contact);
```

Both the `update()` or `saveOrUpdate()` methods throws a `NonUniqueObjectException` exception if there is a different object with the same identifier value was already associated with the session. For example, the `update()` statement in the following code throws such exception:

```
1 Contact contact = (Contact) session.load(Contact.class, new
2 Integer(3));
3 Contact contactUpdate = new
  Contact(3, "Jobs", "jobsatapplet.com", "Cupertino", "0123456789");
  session.update(contactUpdate);    // will throw
  NonUniqueObjectException
```

Delete:

To delete an object with a given identifier, use the `delete()` method, for example:

```
1 Contact contactToDelete = new Contact();
2 contactToDelete.setId(4);
3 session.delete(contactToDelete);
```

or delete a loaded instance:

```
1 Contact contactToDelete = (Contact) session.load(Contact.class, new
```

```
2 Integer(4));
  session.delete(contactToDelete);
```

The `delete()` method throws a `StaleStateException` exception if there is no row with the given identifier exists in the database, and a `NonUniqueObjectException` exception if there is a different object with the same identifier value was already associated with the session.

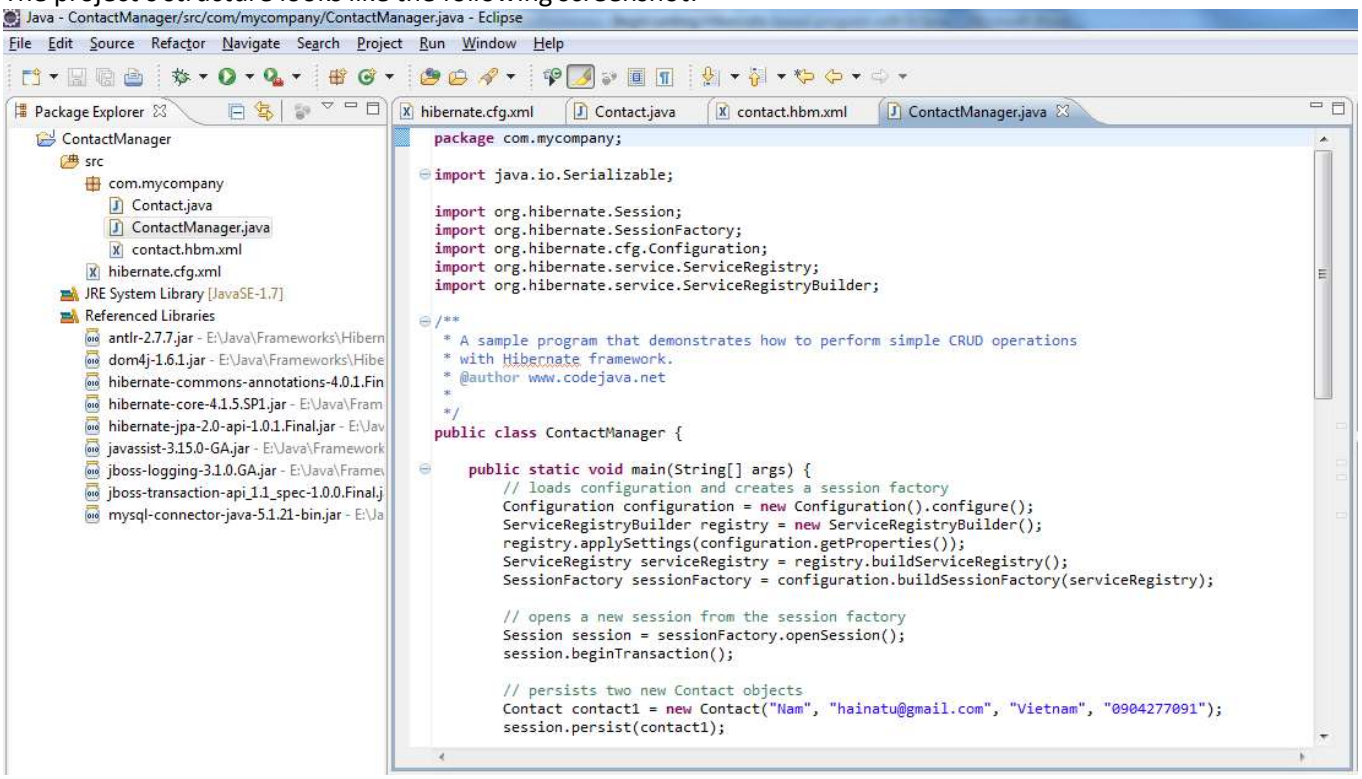
9. Code Hibernate sample program

Finally, we write the sample program to demonstrate how to perform CRUD operations with Hibernate. In Eclipse IDE, create a new class called `ContactManager`under package `com.mycompany` with the following code:

```
1  package com.mycompany;
2
3  import java.io.Serializable;
4  import org.hibernate.Session;
5  import org.hibernate.SessionFactory;
6  import org.hibernate.cfg.Configuration;
7  import org.hibernate.service.ServiceRegistry;
8  import org.hibernate.service.ServiceRegistryBuilder;
9
10 /**
11  * A sample program that demonstrates how to perform simple CRUD operations
12  * with Hibernate framework.
13  * @author www.codejava.net
14  *
15  */
16 public class ContactManager {
17     public static void main(String[] args) {
18         // loads configuration and creates a session factory
19         Configuration configuration = new Configuration().configure();
20         ServiceRegistryBuilder registry = new ServiceRegistryBuilder();
21         registry.applySettings(configuration.getProperties());
22         ServiceRegistry serviceRegistry = registry.buildServiceRegistry();
23         SessionFactory sessionFactory =
24 configuration.buildSessionFactory(serviceRegistry);
25
26         // opens a new session from the session factory
27         Session session = sessionFactory.openSession();
28         session.beginTransaction();
29         // persists two new Contact objects
30         Contact contact1 = new
31 Contact("Nam", "hainatuatgmail.com", "Vietnam", "0904277091");
32         session.persist(contact1);
33         Contact contact2 = new
34 Contact("Bill", "billatgmail.com", "USA", "18001900");
35         Serializable id = session.save(contact2);
36         System.out.println("created id: " + id);
37
38         // loads a new object from database
39         Contact contact3 = (Contact) session.get(Contact.class, new
40 Integer(1));
41         if (contact3 == null) {
42             System.out.println("There is no Contact object with id=1");
43         } else {
44             System.out.println("Contact3's name: " + contact3.getName());
45         }
46
47         // loads an object which is assumed exists
```

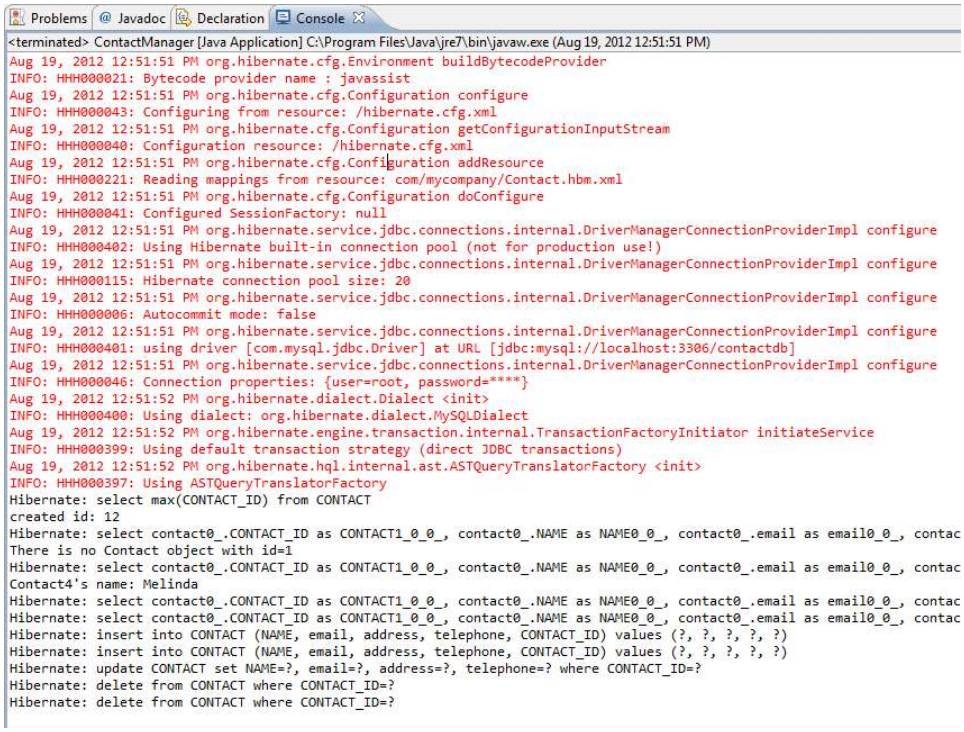
```
        Contact contact4 = (Contact) session.load(Contact.class, new
Integer(4));
        System.out.println("Contact4's name: " + contact4.getName());

        // updates a loaded instance of a Contact object
        Contact contact5 = (Contact) session.load(Contact.class, new
Integer(5));
        contact5.setEmail("info1atcompany.com");
        contact5.setTelephone("1234567890");
        session.update(contact5);
        // updates a detached instance of a Contact object
        Contact contact6 = new
Contact(3, "Jobs", "jobsatapplet.com", "Cupertino", "0123456789");
        session.update(contact6);

        // deletes an object
        Contact contact7 = new Contact();
        contact7.setId(7);
        session.delete(contact7);
        // deletes a loaded instance of an object
        Contact contact8 = (Contact) session.load(Contact.class, new
Integer(8));
        session.delete(contact8);

        // commits the transaction and closes the session
        session.getTransaction().commit();
        session.close();

    }
}
```

As you can see, there are no SQL statements have to be written to interact with the database, because Hibernate hides all the complex details and exposes its simple and intuitive API. Thus you are free of writing repetitive SQL queries and check their syntax. Working with Hibernate will involve mainly with defining the entities, the mapping, and the association among entities in your application.
The project's structure looks like the following screenshot:



Run the program:

Right click on the `ContactManager.java` file, select **Run As > Java Application**. The program produces some output in Eclipse's **Console** view, as in the following screenshot:



As you can see, the text in red is the debugging information of Hibernate, the text in black is the program's output, and we can see Hibernate issues these SQL statements: select, insert, update and delete.

You can download the sample application as an Eclipse project in the attachment section at the end of this tutorial.

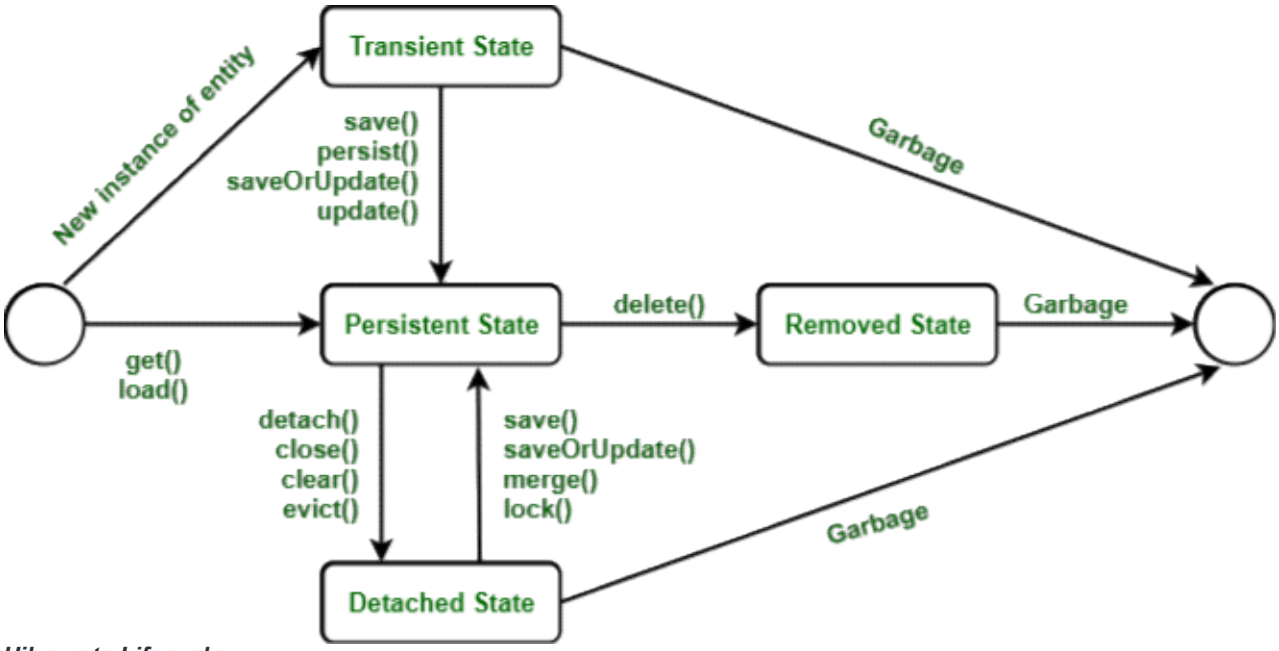
From <https://www.codejava.net/frameworks/hibernate/writing-a-basic-hibernate-based-program-with-eclipse?showall=1&limitstart=>

o Lifecycle of Hibernate Entities

Here we will learn about Hibernate Lifecycle or in other words, we can say that we will learn about the lifecycle of the mapped instances of the entity/object classes in hibernate. In Hibernate, we can either create a new object of an entity and store it into the database, or we can fetch the existing data of an entity from the database. These entity is connected with the lifecycle and each object of entity passes through the various stages of the lifecycle.

There are mainly four states of the Hibernate Lifecycle :

1. Transient State
2. Persistent State
3. Detached State
4. Removed State

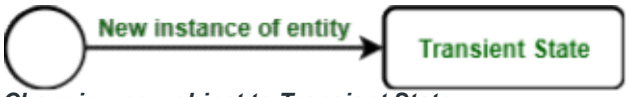


Hibernate Lifecycle

As depicted from the above media one can co-relate how they are plotted in order to plot better in our mind. Now we will be discussing the states to better interpret hibernate lifecycle. It is as follows:

State 1: Transient State

The transient state is the first state of an entity object. When we instantiate an object of a POJO class using the new operator then the object is in the transient state. This object is not connected with any hibernate session. As it is not connected to any Hibernate Session, So this state is not connected to any database table. So, if we make any changes in the data of the POJO Class then the database table is not altered. Transient objects are independent of Hibernate, and they exist in the **heap memory**.



Changing new object to Transient State

There are two layouts in which transient state will occur as follows:

1. When objects are generated by an application but are not connected to any session.
2. The objects are generated by a closed session.

Here, we are creating a new object for the Employee class. Below is the code which shows the initialization of the Employee object :

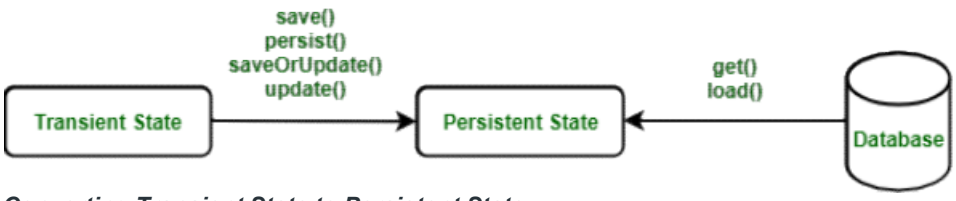
```
//Here, The object arrives in the transient state.
Employee e = new Employee();
e.setId(21);
e.setFirstName("Neha");
e.setMiddleName("Shri");
e.setLastName("Rudra");
```

State 2: Persistent State

Once the object is connected with the Hibernate Session then the object moves into the Persistent State. So, there are two ways to convert the Transient State to the Persistent State :

a. Using the hibernated session, save the entity object into the database table.
b. Using the hibernated session, load the entity object into the database table.

In this state. each object represents one row in the database table. Therefore, if we make any changes in the data then hibernate will detect these changes and make changes in the database table.



Converting Transient State to Persistent State

Following are the methods given for the persistent state:

- session.persist(e);
- session.save(e);
- session.saveOrUpdate(e);
- session.update(e);
- session.merge(e);
- session.lock(e);

Example:

```
// Transient State
Employee e = new Employee("Neha Shri Rudra", 21, 180103);
//Persistent State
session.save(e);
```

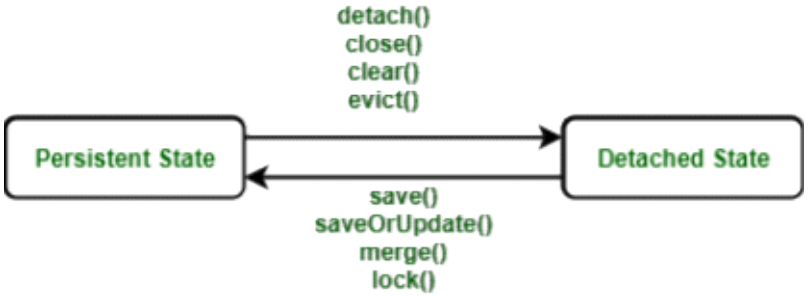
State 3: Detached State

For converting an object from Persistent State to Detached State, we either have to close the session or we have to clear its cache. As the session is closed here or the cache is cleared, then any changes made to the data will not affect the database table. Whenever needed, the detached object can be reconnected to a new hibernate session. To reconnect the detached object to a new hibernate session, we will use the following methods as follows:

- merge()
- update()
- load()
- refresh()
- save()
- update()

Following are the methods used for the detached state :

- session.detach(e);
- session.evict(e);
- session.clear();
- session.close();



Converting Persistent State to Detached State

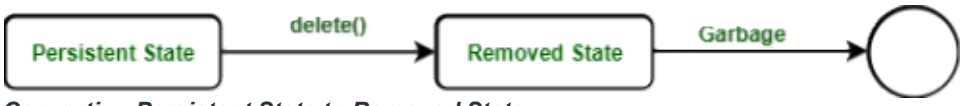
Example

```
// Transient State
Employee e = new Employee("Neha Shri Rudra", 21, 180103);
// Persistent State
session.save(e);

// Detached State
session.close();
```

State 4: Removed State

In the hibernate lifecycle it is the last state. In the removed state, when the entity object is deleted from the database then the entity object is known to be in the removed state. It is done by calling the ***delete() operation***. As the entity object is in the removed state, if any change will be done in the data will not affect the database table.

Note: *To make a removed entity object we will call* ***session.delete()****.*



Converting Persistent State to Removed State

Example

```
// Java Pseudo code to Illustrate Remove State

// Transient State
Employee e = new Employee();
Session s = sessionfactory.openSession();
e.setId(01);
// Persistent State
session.save(e)

// Removed State
session.delete(e);
```

From <https://www.geeksforgeeks.org/hibernate-lifecycle/>

? HB with annotation example

The hibernate application can be created with annotation. There are many annotations

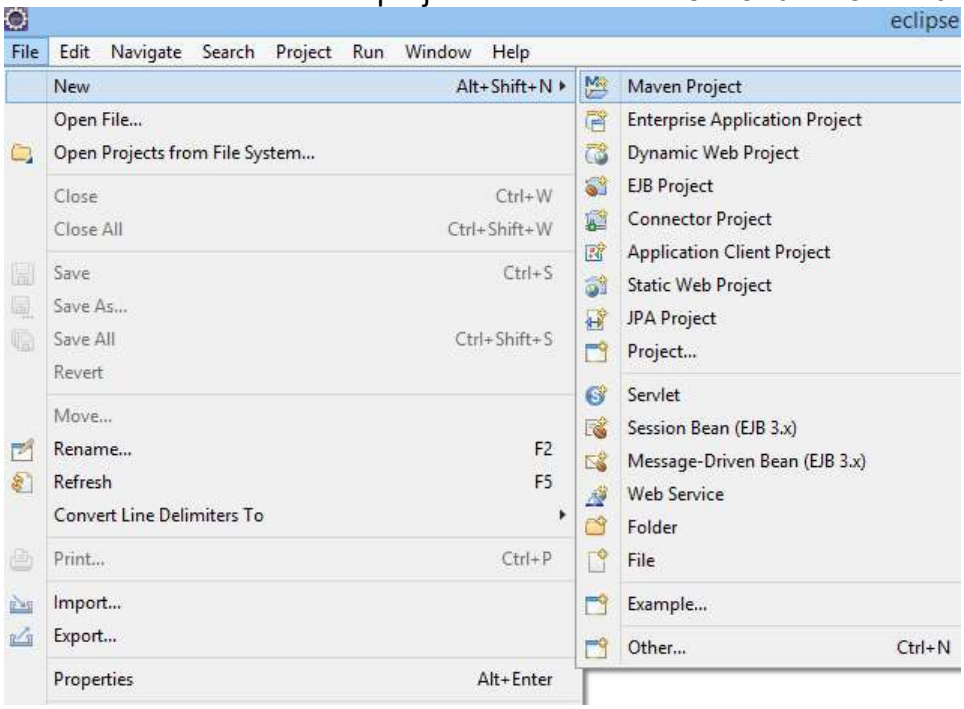
that can be used to create hibernate application such as @Entity, @Id, @Table etc. Hibernate Annotations are based on the JPA 2 specification and supports all the features.
All the JPA annotations are defined in the **javax.persistence** package. Hibernate EntityManager implements the interfaces and life cycle defined by the JPA specification. The core advantage of using hibernate annotation is that you don't need to create mapping (hbm) file. Here, hibernate annotations are used to provide the meta data.

Example to create the hibernate application with Annotation

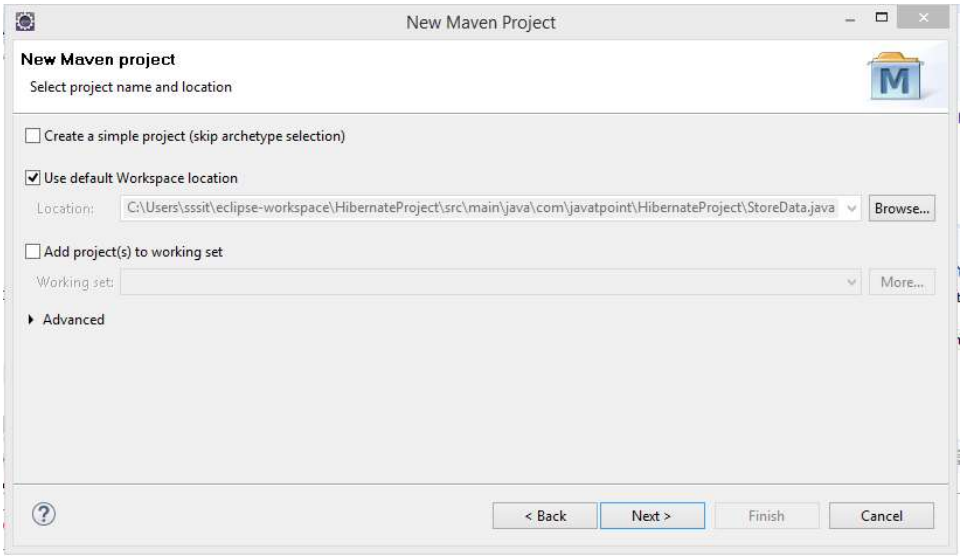
Here, we are going to create a maven based hibernate application using annotation in eclipse IDE. For creating the hibernate application in Eclipse IDE, we need to follow the below steps:

1) Create the Maven Project

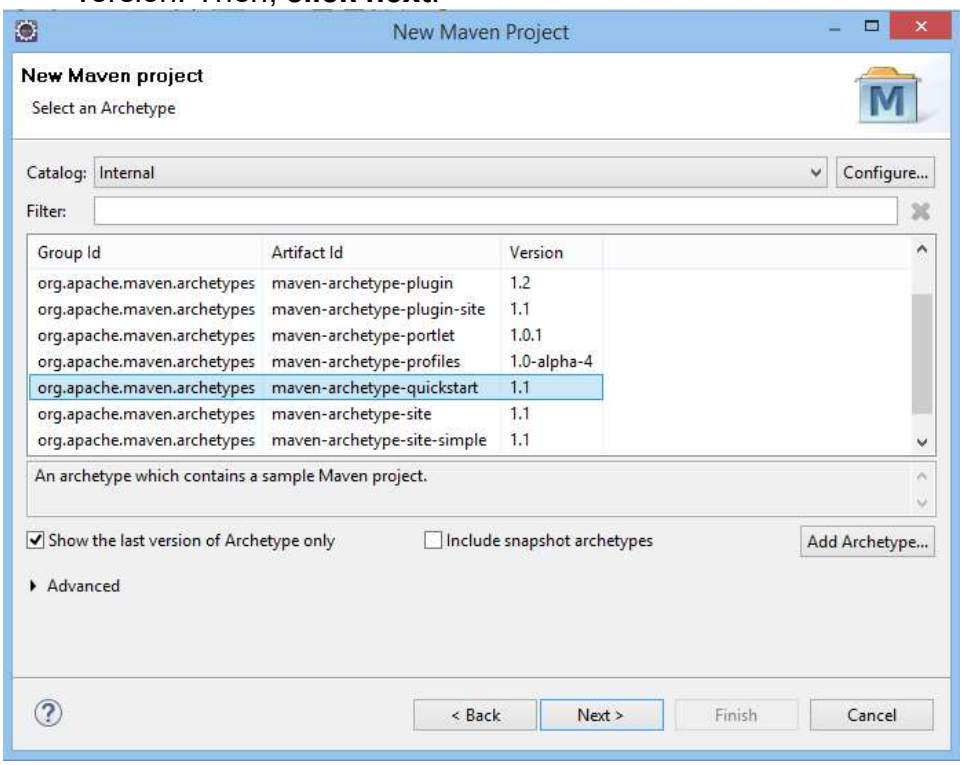
- To create the maven project left click on **File Menu - New- Maven Project**.



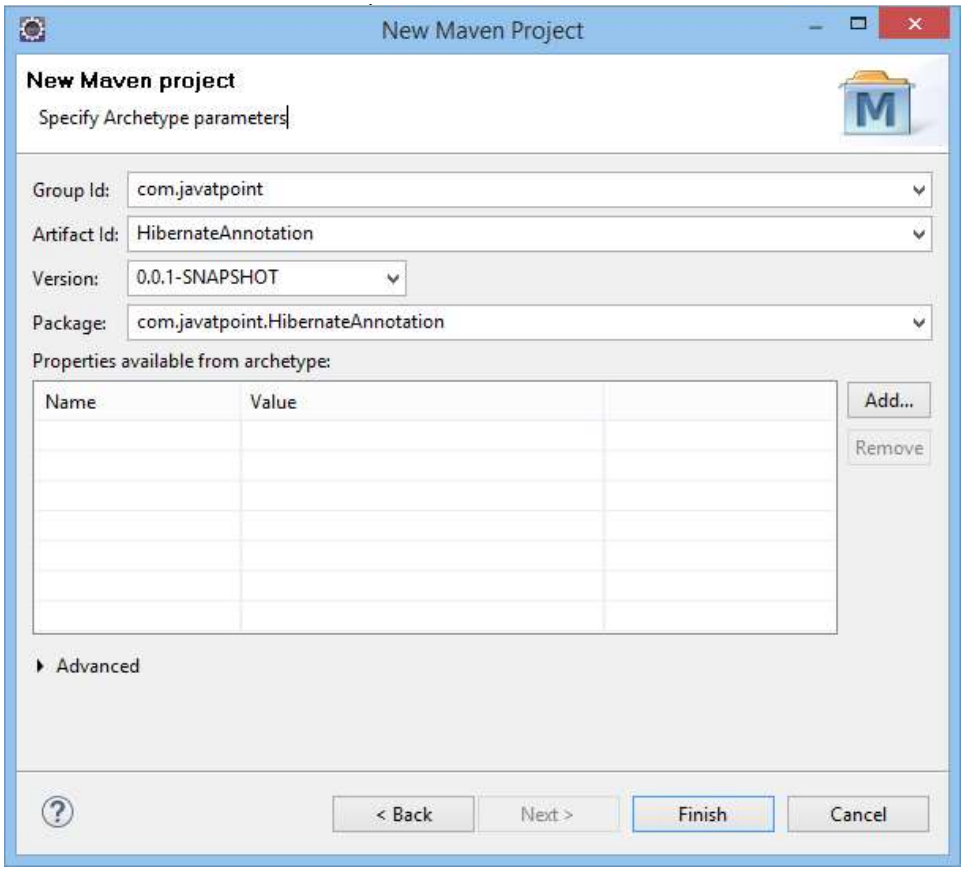
- The new maven project opens in your eclipse. **Click Next**.



- Now, select catalog type: internal and maven archetype - **quickstart** of 1.1 version. Then, **click next**.



- Now, specify the name of Group Id and Artifact Id. The Group Id contains package name (e.g. com.javatpoint) and Artifact Id contains project name (e.g. HibernateAnnotation). Then **click Finish**.



2) Add project information and configuration in pom.xml file.

Open pom.xml file and click source. Now, add the below dependencies between <dependencies>....</dependencies> tag. These dependencies are used to add the jar files in Maven project.

```
1. <dependency>
2.     <groupId>org.hibernate</groupId>
3.     <artifactId>hibernate-core</artifactId>
4.     <version>5.3.1.Final</version>
5. </dependency>
6. 
7. <dependency>
8.     <groupId>com.oracle</groupId>
9.     <artifactId>ojdbc14</artifactId>
10.    <version>10.2.0.4.0</version>
11. </dependency>
```

Due to certain license issues, Oracle drivers are not present in public Maven repository. We can install it manually. To install Oracle driver into your local Maven repository, follow the following steps:

- Install Maven
- Run the command : install-file -Dfile=Path/to/your/ojdbc14.jar -DgroupId=com.oracle -DartifactId=ojdbc14 -Dversion=12.1.0 -Dpackaging=jar

3) Create the Persistence class.

Here, we are creating the same persistent class which we have created in the previous topic. But here, we are using annotation.

@Entity annotation marks this class as an entity.
@Table annotation specifies the table name where data of this entity is to be persisted. If you don't use @Table annotation, hibernate will use the class name as the table name by default.
@Id annotation marks the identifier for this entity.
@Column annotation specifies the details of the column for this property or field. If @Column annotation is not specified, property name will be used as the column name by default.
To create the Persistence class, right click on **src/main/java - New - Class -** specify the class name with package - **finish**.
Employee.java

```
1.  package com.javatpoint.mypackage;
2.
3.  import javax.persistence.Entity;
4.  import javax.persistence.Id;
5.  import javax.persistence.Table;
6.
7.  @Entity
8.  @Table(name= "emp500")
9.  public class Employee {
10.
11. @Id
12. private int id;
13. private String firstName,lastName;
14.
15. public int getId() {
16.     return id;
17. }
18. public void setId(int id) {
19.     this.id = id;
20. }
21. public String getFirstName() {
22.     return firstName;
23. }
24. public void setFirstName(String firstName) {
25.     this.firstName = firstName;
26. }
27. public String getLastName() {
28.     return lastName;
29. }
30. public void setLastName(String lastName) {
31.     this.lastName = lastName;
32. }
33. }
```

4) Create the Configuration file

To create the configuration file, right click on **src/main/java - new - file -** specify the file name (e.g. hibernate.cfg.xml) - **Finish**.
hibernate.cfg.xml

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
```

```
        "-//Hibernate/Hibernate Configuration DTD 5.3//EN"
        "http://www.hibernate.org/dtd/hibernate-configuration-5.3.dtd">
<hibernate-configuration>
   <session-factory>

   <property name="hbm2ddl.auto">update</property>
      <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
      <property name="connection.url">jdbc:oracle:thin:
@localhost:1521:xe</property>
      <property name="connection.username">system</property>
      <property name="connection.password">jtp</property>
      <property name="connection.driver_class">
oracle.jdbc.driver.OracleDriver</property>

      <mapping class="com.javatpoint.mypackage.Employee"/>
   </session-factory>
</hibernate-configuration>
```

5) Create the class that retrieves or stores the persistent object.

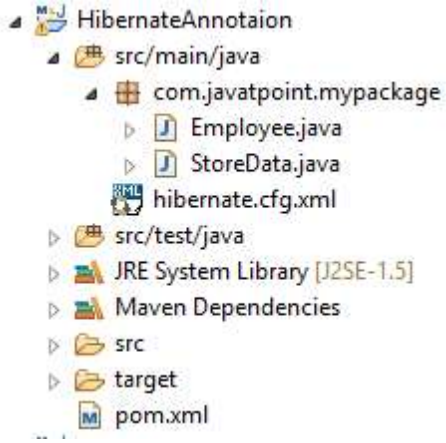
StoreData.java

```
package com.javatpoint.mypackage;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.boot.Metadata;
import org.hibernate.boot.MetadataSources;
import org.hibernate.boot.registry.StandardServiceRegistry;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;


public class StoreData {
public static void main(String[] args) {

   StandardServiceRegistry ssr = new StandardServiceRegistryBuilder().configure(
"hibernate.cfg.xml").build();
   Metadata meta = new MetadataSources(ssr).getMetadataBuilder().build();

SessionFactory factory = meta.getSessionFactoryBuilder().build();
Session session = factory.openSession();
Transaction t = session.beginTransaction();

   Employee e1=new Employee();
   e1.setId(101);
   e1.setFirstName("Gaurav");
   e1.setLastName("Chawla");

   session.save(e1);
   t.commit();
   System.out.println("successfully saved");
```

```
30.     factory.close();
31.     session.close();
32.
33. }
34. }
```

6) Run the application

Before running the application, determine that the directory structure is like this.



To run the hibernate application, right click on the **StoreData - Run As - Java Application**.

From <https://www.javatpoint.com/hibernate-with-annotation>

? Hibernate Mappings and Relationships

Using hibernate, if we want to put relationship between two entities [objects of two pojo classes], then in the database tables, there must exist foreign key relationship, we call it as Referential integrity.

The main advantage of putting relation ship between objects is, we can do operation on one object, and the same operation can transfer onto the other object in the database [remember, object means one row in hibernate terminology]

While selecting, it is possible to get data from multiple tables at a time if there exits relationship between the tables, nothing but in hibernate relationships between the objects

Using hibernate we can put the following 4 types of relationships

- One-To-One
- Many-To-One
- Many-To-Many
- One-To-Many

https://www.java4s.com/hibernate/hibernate-one-to-one-mapping-using-annotations/

From <https://www.java4s.com/hibernate/hibernate-relationships-in-depth/>

? Collection and Component Mapping

A **Component** mapping is a mapping for a class having a reference to another class as a member variable.

Collection elements are much needed to have one-to-many, many-to-many, relationships, etc., Any one type from below can be used to declare the type of collection in the Persistent class. Persistent class from one of the following types:

- java.util.List
- java.util.Set
- java.util.SortedSet
- java.util.Map
- java.util.SortedMap
- java.util.Collection

Let us see where can we use List, Set, and Map by seeing the differences among them

List	Set	Map
Duplicates are allowed.	Duplicates are not allowed.	Duplicates are not allowed.
Insertion order is maintained.	Insertion order is not maintained.	Insertion order is not maintained.
Index element helps to identify an element in the list. Hence need to use this whenever the index is possible and it is mandatory.	For uniquely maintaining the collection of elements, we can go for the set.	As a key/value pair pattern means we can go for Map.

? HQL,

Hibernate Query Language (HQL) is same as SQL (Structured Query Language) but it doesn't depends on the table of the database. Instead of table name, we use class name in HQL. So it is database independent query language.

Advantage of HQL

There are many advantages of HQL. They are as follows:

- database independent
- supports polymorphic queries
- easy to learn for Java Programmer

Query Interface

It is an object oriented representation of Hibernate Query. The object of Query can be obtained by calling the createQuery() method Session interface.
The query interface provides many methods. There is given commonly used methods:

a. **public int executeUpdate()** is used to execute the update or delete query.
b. **public List list()** returns the result of the ralation as a list.
c. **public Query setFirstResult(int rowno)** specifies the row number from where record will be retrieved.
d. **public Query setMaxResult(int rowno)** specifies the no. of records to be retrieved from the relation (table).
e. **public Query setParameter(int position, Object value)** it sets the value to the JDBC style query parameter.
f. **public Query setParameter(String name, Object value)** it sets the value to a named query parameter.

Example of HQL to get all the records

```
1. Query query=session.createQuery("from Emp");//here persistent class name is Em
   p
2. List list=query.list();
```

Example of HQL to get records with pagination

```
3. Query query=session.createQuery("from Emp");
4. query.setFirstResult(5);
5. query.setMaxResult(10);
6. List list=query.list();//will return the records from 5 to 10th number
```

Example of HQL update query

```
7. Transaction tx=session.beginTransaction();
8. Query q=session.createQuery("update User set name=:n where id=:i");
9. q.setParameter("n","Udit Kumar");
10. q.setParameter("i",111);
11.
12. int status=q.executeUpdate();
13. System.out.println(status);
14. tx.commit();
```

Example of HQL delete query

```
15. Query query=session.createQuery("delete from Emp where id=100");
16. //specifying class name (Emp) not tablename
17. query.executeUpdate();
```

HQL with Aggregate functions

You may call avg(), min(), max() etc. aggregate functions by HQL. Let's see some common examples:

Example to get total salary of all the employees

```
18. Query q=session.createQuery("select sum(salary) from Emp");
19. List<Integer> list=q.list();
20. System.out.println(list.get(0));
```

Example to get maximum salary of employee

```
21. Query q=session.createQuery("select max(salary) from Emp");
```

Example to get minimum salary of employee

```
22. Query q=session.createQuery("select min(salary) from Emp");
```

Example to count total number of employee ID

```
23. Query q=session.createQuery("select count(id) from Emp");
```

Example to get average salary of each employees

```
24. Query q=session.createQuery("select avg(salary) from Emp");
```

From <https://www.javatpoint.com/hql>
https://www.geeksforgeeks.org/hql-introduction/

Named Queries,

A named query is a predefined query that you create and associate with a container-managed entity.

Using Annotations

Example 8-1 Implementing a Query Using @NamedQuery

```
@Entity
@NamedQuery(
   name="findAllEmployeesByFirstName",
   queryString="SELECT OBJECT(emp) FROM Employee emp WHERE emp.firstName = 'John'"
)
public class Employee implements Serializable {
...
}
```

Example 8-2 Implementing a Query With Parameters Using @NamedQuery

```
@Entity
@NamedQuery(
   name="findAllEmployeesByFirstName",
   queryString="SELECT OBJECT(emp) FROM Employee emp WHERE emp.firstName = :firstname"
)
public class Employee implements Serializable {
...
}
```

Example 8-3 Setting Parameters in a Named Query

```
Query queryEmployeesByFirstName = em.createNamedQuery("findAllEmployeesByFirstName");
queryEmployeeByFirstName.setParameter("firstName", "John");
Collection employees = queryEmployessByFirstName.getResultList();
```

From <https://docs.oracle.com/cd/E16439_01/doc.1013/e13981/ent30qry001.htm>

Criteria Queries

Hibernate provides alternate ways of manipulating objects and in turn data available in RDBMS tables. One of the methods is Criteria API, which allows you to build up a criteria query object programmatically where you can apply filtration rules and logical conditions.

The Hibernate **Session** interface provides **createCriteria()** method, which can be used to create a **Criteria** object that returns instances of the persistence object's class when your application executes a criteria query.

Following is the simplest example of a criteria query is one, which will simply return every object that corresponds to the Employee class.

```
Criteria cr = session.createCriteria(Employee.class);
List results = cr.list();
```

Restrictions with Criteria

You can use **add()** method available for **Criteria** object to add restriction for a criteria query. Following is the example to add a restriction to return the records with salary is equal to 2000 –

```
Criteria cr = session.createCriteria(Employee.class);
cr.add(Restrictions.eq("salary", 2000));
List results = cr.list();
```

Following are the few more examples covering different scenarios and can be used as per the requirement –

```
Criteria cr = session.createCriteria(Employee.class);

// To get records having salary more than 2000
cr.add(Restrictions.gt("salary", 2000));
// To get records having salary less than 2000
cr.add(Restrictions.lt("salary", 2000));
// To get records having fistName starting with zara
cr.add(Restrictions.like("firstName", "zara%"));
// Case sensitive form of the above restriction.
cr.add(Restrictions.ilike("firstName", "zara%"));
// To get records having salary in between 1000 and 2000
cr.add(Restrictions.between("salary", 1000, 2000));
// To check if the given property is null
cr.add(Restrictions.isNull("salary"));
// To check if the given property is not null
cr.add(Restrictions.isNotNull("salary"));
// To check if the given property is empty
cr.add(Restrictions.isEmpty("salary"));
// To check if the given property is not empty
cr.add(Restrictions.isNotEmpty("salary"));
```

You can create AND or OR conditions using LogicalExpression restrictions as follows –

```
Criteria cr = session.createCriteria(Employee.class);
Criterion salary = Restrictions.gt("salary", 2000);
Criterion name = Restrictions.ilike("firstNname","zara%");
// To get records matching with OR conditions
LogicalExpression orExp = Restrictions.or(salary, name);
cr.add( orExp );
// To get records matching with AND conditions
LogicalExpression andExp = Restrictions.and(salary, name);
cr.add( andExp );
List results = cr.list();
```

From <https://www.tutorialspoint.com/hibernate/hibernate_criteria_queries.htm>

Sessions 12, 13 & 14:

? What is Spring Framework

Introduction

Prior to the advent of Enterprise Java Beans (EJB), Java developers needed to use JavaBeans to create Web applications. Although JavaBeans helped in the development of user interface (UI) components, they were not able to provide services, such as transaction management and security, which were required for developing robust and secure enterprise applications. The advent of EJB was seen as a solution to this problem EJB extends the Java components, such as Web and enterprise components, and provides services that help in enterprise application development. However, developing an enterprise application with EJB was not easy, as the developer needed to perform various tasks, such as creating Home and Remote interfaces and implementing lifecycle callback methods which lead to the complexity of providing code for EJBs Due to this complication, developers started looking for an easier way to develop enterprise applications.

The Spring framework(which is commonly known as Spring) has emerged as a solution to all these complications This framework uses various new techniques such as Aspect-Oriented Programming (AOP), Plain Old Java Object (POJO), and dependency injection (DI), to develop enterprise applications, thereby removing the complexities involved while developing enterprise applications using EJB, Spring is an open source lightweight framework that allows Java EE 7 developers to build simple, reliable, and scalable enterprise applications. This framework mainly focuses on providing various ways to help you manage your business objects. It made the development of Web applications much easier as compared to classic Java frameworks and Application Programming Interfaces (APIs), such as Java database connectivity(JDBC), JavaServer Pages(JSP), and Java Servlet.

The Spring framework can be considered as a collection of sub-frameworks, also called layers, such as

- Spring AOP.

- Spring Object-Relational Mapping (Spring ORM).
- Spring Web Flow, and
- Spring Web MVC.

It is a lightweight application framework used for developing enterprise applications. You can use any of these modules separately while constructing a Web application. The modules may also be grouped together to provide better functionalities in a Web application. Spring framework is loosely coupled because of dependency Injection.

Features of the Spring Framework
The features of the Spring framework such as IoC, AOP, and transaction management, make it unique among the list of frameworks. Some of the most important features of the Spring framework are as follows:

- **IoC container:**
  Refers to the core container that uses the DI or IoC pattern to implicitly provide an object reference in a class during runtime. This pattern acts as an alternative to the service locator pattern. The IoC container contains assembler code that handles the configuration management of application objects.
  The Spring framework provides two packages, namely org.springframework.beans and org.springframework.context which helps in providing the functionality of the IoC container.
- **Data access framework:**
  Allows the developers to use persistence APIs, such as JDBC and Hibernate, for storing persistence data in database. It helps in solving various problems of the developer, such as how to interact with a database connection, how to make sure that the connection is closed, how to deal with exceptions, and how to implement transaction management It also enables the developers to easily write code to access the persistence data throughout the application.
- **Spring MVC framework:**
  Allows you to build Web applications based on MVC architecture. All the requests made by a user first go through the controller and are then dispatched to different views, that is, to different JSP pages or Servlets. The form handling and form validating features of the Spring MVC framework can be easily integrated with all popular view technologies such as ISP, Jasper Report, FreeMarker, and Velocity.
- **Transaction management:**
  Helps in handling transaction management of an application without affecting its code. This framework provides Java Transaction API (JTA) for global transactions managed by an application server and local transactions managed by using the JDBC Hibernate, Java Data Objects (JDO), or other data access APIs. It enables the developer to model a wide range of transactions on the basis of Spring's declarative and programmatic transaction management.
- **Spring Web Service:**
  Generates Web service endpoints and definitions based on Java classes, but it is difficult to manage them in an application. To solve this problem, Spring Web Service provides layered-based approaches that are separately managed by Extensible Markup Language (XML) parsing (the technique of reading and manipulating XML). Spring provides effective mapping for transmitting incoming XML message request to an object and the developer to easily distribute XML message (object) between two machines.
- **JDBC abstraction layer:**

Helps the users in handling errors in an easy and efficient manner. The JDBC programming code can be reduced when this abstraction layer is implemented in a Web application. This layer handles exceptions such as DriverNotFound. All SQLExceptions are translated into the DataAccessException class. Spring's data access exception is not JDBC specific and hence Data Access Objects (DAO) are not bound to JDBC only.

- **Spring TestContext framework:**
  Provides facilities of unit and integration testing for the Spring applications. Moreover, the Spring TestContext framework provides specific integration testing functionalities such as context management and caching DI of test fixtures, and transactional test management with default rollback semantics.

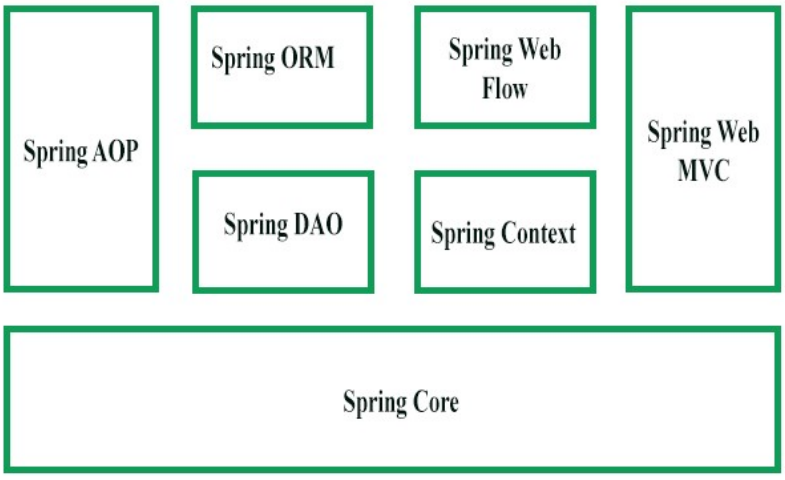
Evolution of Spring Framework

The Spring Framework was first released in 2004. After that there has been a significant major revision, such as

- Spring 2.0 provided XML namespaces and AspectJ support,
- Spring 2.5 provide annotation-driven configuration,
- Spring 3.0 provided a Java-based @Configuration model.
- The latest release of the spring framework is 4.0. it is released with the support for Java 8 and Java EE 7 technologies.

Though you can still use Spring with an older version of java, the minimum requirement is restricted to Java SE 6. Spring 4.0 also supports Java EE 7 technologies, such as java message service (JMS) 2.0, java persistence API (JPA) 2.1, Bean validation 1.1, servlet 3.1, and JCache.

Spring Framework Architecture



The Spring framework consists of seven modules which are shown in the above Figure. These modules are Spring Core, Spring AOP, Spring Web MVC, Spring DAO, Spring ORM, Spring context, and Spring Web flow. These modules provide different platforms to develop different enterprise applications; for example, you can use Spring Web MVC module for developing MVC-based applications.

Spring Framework Modules

- **Spring Core Module:**

The Spring Core module, which is the core component of the Spring framework, provides the IoC container There are two types of implementations of the Spring container, namely, bean factory and application context. Bean factory is defined using the org.springframework.beans.factory.BeanFactory interface and acts as a container for beans. The Bean factory container allows you to decouple the configuration and specification of dependencies from program logic. In the Spring framework, the Bean factory acts as a central IoC container that is responsible for instantiating application objects. It also configures and assembles the dependencies between these objects. There are numerous implementations of the BeanFactory interface. The XmlBeanFactory class is the most common implementation of the BeanFactory interface. This allows you to express the object to compose your application and remove interdependencies between application objects.

- **Spring AOP Module:**
  Similar to Object-Oriented Programming (OOP), which breaks down the applications into hierarchy of objects, AOP breaks down the programs into aspects or concerns. Spring AOP module allows you to implement concerns or aspects in a Spring application in Spring AOP, the aspects are the regular Spring beans or regular classes annotated with @Aspect annotation. These aspects help in transaction management and logging and failure monitoring of an application. For example, transaction management is required in bank operations such as transferring an amount from one account to another Spring AOP module provides a transaction management abstraction layer that can be applied to transaction APIs.
- **Spring ORM Module:**
  The Spring ORM module is used for accessing data from databases in an application. It provides APIs for manipulating databases with JDO, Hibernate, and iBatis. Spring ORM supports DAO, which provides a convenient way to build the following DAOs-based ORM solutions:
  - Simple declarative transaction management
  - Transparent exception handling
  - Thread-safe, lightweight template classes
  - DAO support classes
  - Resource management
- **Spring Web MVC Module:**
  The Web MVC module of Spring implements the MVC architecture for creating Web applications. It separates the code of model and view components of a Web application. In Spring MVC, when a request is generated from the browser, it first goes to the DispatcherServlet class (Front Controller), which dispatches the request to a controller (SimpleFormController class or AbstractWizardformController class) using a set of handler mappings. The controller extracts and processes the information embedded in a request and sends the result to the DispatcherServlet class in the form of the model object. Finally, the DispatcherServlet class uses ViewResolver classes to send the results to a view, which displays these results to the users.
- **Spring Web Flow Module:**
  The Spring Web Flow module is an extension of the Spring Web MVC module. Spring Web MVC framework provides form controllers, such as class SimpleFormController and AbstractWizardFormController class, to implement predefined workflow. The Spring Web Flow helps in defining XML file or Java

Class that manages the workflow between different pages of a Web application. The Spring Web Flow is distributed separately and can be downloaded through http://www.springframework.org website.
The following are the advantages of Spring Web Flow:

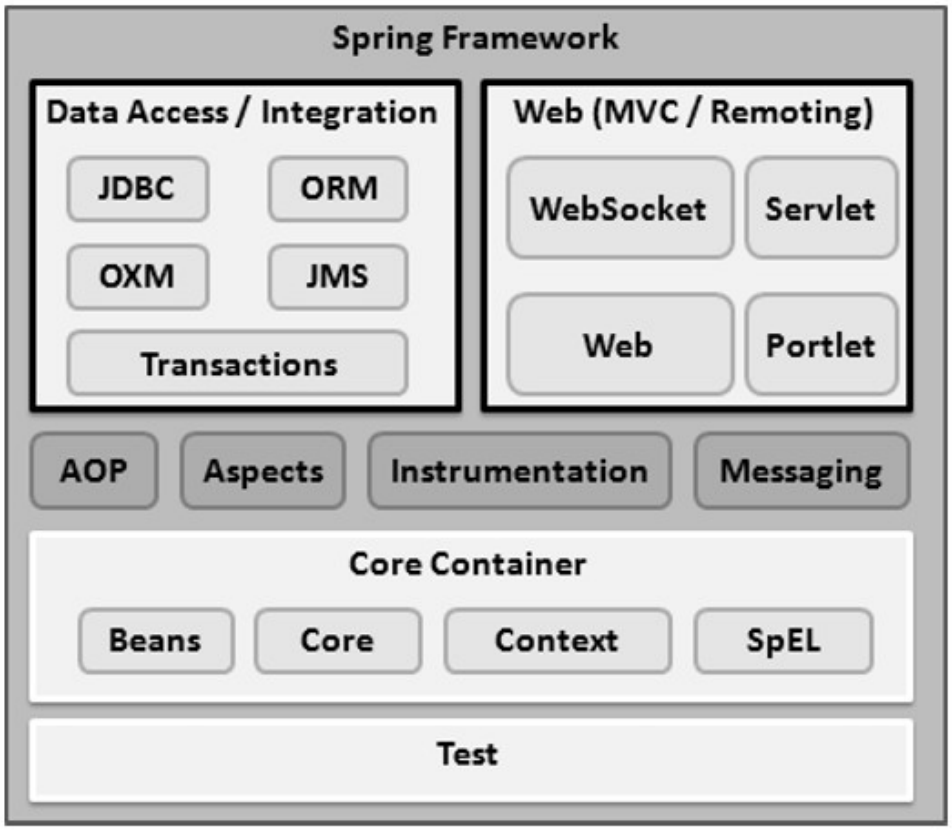
- The flow between different UIs of the application is clearly provided by defining Web flow in XML file.
- Web flow definitions help you to virtually split an application in different modules and reuse these modules in multiple situations.

- **Spring Web DAO Module:**
The DAO package in the Spring framework provides DAO support by using data access technologies such as JDBC, Hibernate, or JDO. This module introduces a JDBC abstraction layer by eliminating the need for providing tedious JDBC coding. It also provides programmatic as well as declarative transaction management classes. Spring DAO package supports heterogeneous Java Database Connectivity and O/R mapping, which helps Spring work with several data access technologies. For easy and quick access to database resources, the Spring framework provides abstract DAO base classes. Multiple implementations are available for each data access technology supported by the Spring framework. For example, in JDBC, the JdbcDaoSupport class and its methods are used to access the DataSource instance and a preconfigured JdbcTemplate instance. You need to simply extend the JdbcDaoSupport class and provide a mapping to the actual DataSource instance in an application context configuration to access a DAO-based application.
- **Spring Application Context Module:**
The Spring Application context module is based on the Core module. Application context org.springframework.context.ApplicationContext is an interface of BeanFactory. This module derives its feature from the org.springframework.beans package and also supports functionalities such as internationalization (I18N), validation, event propagation, and resource loading. The Application context implements MessageSource interface and provides the messaging functionality to an application.

From <https://www.geeksforgeeks.org/introduction-to-spring-framework/>

? Overview of Spring Architecture

Spring could potentially be a one-stop shop for all your enterprise applications. However, Spring is modular, allowing you to pick and choose which modules are applicable to you, without having to bring in the rest. The following section provides details about all the modules available in Spring Framework.
The Spring Framework provides about 20 modules which can be used based on an application requirement.



Core Container

The Core Container consists of the Core, Beans, Context, and Expression Language modules the details of which are as follows –

- The **Core** module provides the fundamental parts of the framework, including the IoC and Dependency Injection features.
- The **Bean** module provides BeanFactory, which is a sophisticated implementation of the factory pattern.
- The **Context** module builds on the solid base provided by the Core and Beans modules and it is a medium to access any objects defined and configured. The ApplicationContext interface is the focal point of the Context module.
- The **SpEL** module provides a powerful expression language for querying and manipulating an object graph at runtime.

Data Access/Integration

The Data Access/Integration layer consists of the JDBC, ORM, OXM, JMS and Transaction modules whose detail is as follows –

- The **JDBC** module provides a JDBC-abstraction layer that removes the need for tedious JDBC related coding.
- The **ORM** module provides integration layers for popular object-relational mapping APIs, including JPA, JDO, Hibernate, and iBatis.
- The **OXM** module provides an abstraction layer that supports Object/XML mapping implementations for JAXB, Castor, XMLBeans, JiBX and XStream.
- The Java Messaging Service **JMS** module contains features for producing and consuming messages.
- The **Transaction** module supports programmatic and declarative transaction management for classes that implement special interfaces and for all your POJOs.

Web

The Web layer consists of the Web, Web-MVC, Web-Socket, and Web-Portlet modules the details of which are as follows –

- The **Web** module provides basic web-oriented integration features such as multipart file-upload functionality and the initialization of the IoC container using servlet listeners and a web-oriented application context.
- The **Web-MVC** module contains Spring's Model-View-Controller (MVC) implementation for web applications.
- The **Web-Socket** module provides support for WebSocket-based, two-way communication between the client and the server in web applications.
- The **Web-Portlet** module provides the MVC implementation to be used in a portlet environment and mirrors the functionality of Web-Servlet module.

Miscellaneous

There are few other important modules like AOP, Aspects, Instrumentation, Web and Test modules the details of which are as follows –

- The **AOP** module provides an aspect-oriented programming implementation allowing you to define method-interceptors and pointcuts to cleanly decouple code that implements functionality that should be separated.
- The **Aspects** module provides integration with AspectJ, which is again a powerful and mature AOP framework.
- The **Instrumentation** module provides class instrumentation support and class loader implementations to be used in certain application servers.
- The **Messaging** module provides support for STOMP as the WebSocket sub-protocol to use in applications. It also supports an annotation programming model for routing and processing STOMP messages from WebSocket clients.
- The **Test** module supports the testing of Spring components with JUnit or TestNG frameworks.

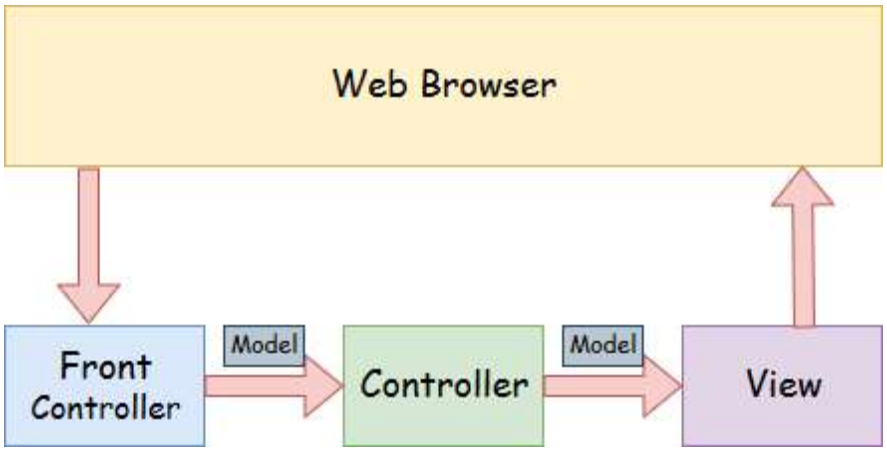
From <https://www.tutorialspoint.com/spring/spring_architecture.htm>

□ Spring MVC architecture

A Spring MVC is a Java framework which is used to build web applications. It follows the Model-View-Controller design pattern. It implements all the basic features of a core spring framework like Inversion of Control, Dependency Injection.

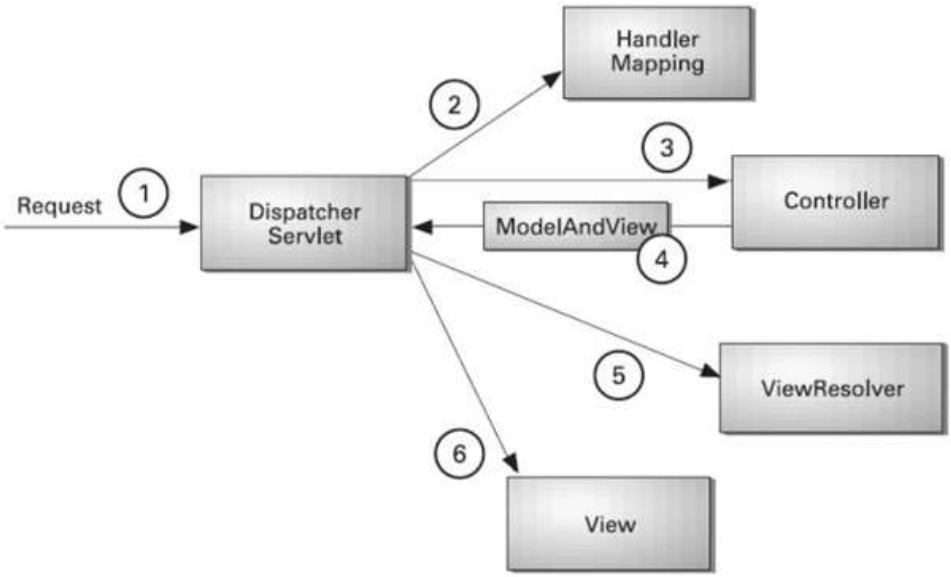
A Spring MVC provides an elegant solution to use MVC in spring framework by the help of **DispatcherServlet**. Here, **DispatcherServlet** is a class that receives the incoming request and maps it to the right resource such as controllers, models, and views.

Spring Web Model-View-Controller



- **Model** - A model contains the data of the application. A data can be a single object or a collection of objects.
- **Controller** - A controller contains the business logic of an application. Here, the @Controller annotation is used to mark the class as the controller.
- **View** - A view represents the provided information in a particular format. Generally, JSP+JSTL is used to create a view page. Although spring also supports other view technologies such as Apache Velocity, Thymeleaf and FreeMarker.
- **Front Controller** - In Spring Web MVC, the DispatcherServlet class works as the front controller. It is responsible to manage the flow of the Spring MVC application.

Understanding the flow of Spring Web MVC



- As displayed in the figure, all the incoming request is intercepted by the DispatcherServlet that works as the front controller.
- The DispatcherServlet gets an entry of handler mapping from the XML file

and forwards the request to the controller.

- The controller returns an object of ModelAndView.
- The DispatcherServlet checks the entry of view resolver in the XML file and invokes the specified view component.

Advantages of Spring MVC Framework

Let's see some of the advantages of Spring MVC Framework:-

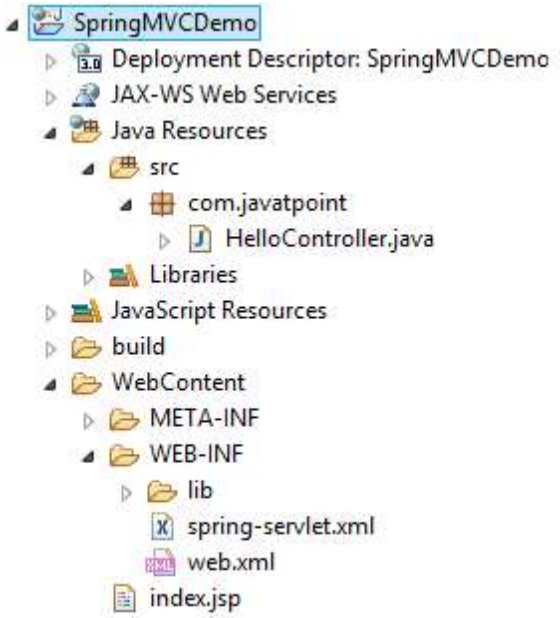
- **Separate roles** - The Spring MVC separates each role, where the model object, controller, command object, view resolver, DispatcherServlet, validator, etc. can be fulfilled by a specialized object.
- **Light-weight** - It uses light-weight servlet container to develop and deploy your application.
- **Powerful Configuration** - It provides a robust configuration for both framework and application classes that includes easy referencing across contexts, such as from web controllers to business objects and validators.
- **Rapid development** - The Spring MVC facilitates fast and parallel development.
- **Reusable business code** - Instead of creating new objects, it allows us to use the existing business objects.
- **Easy to test** - In Spring, generally we create JavaBeans classes that enable you to inject test data using the setter methods.
- **Flexible Mapping** - It provides the specific annotations that easily redirect the page.

Spring Web MVC Framework Example

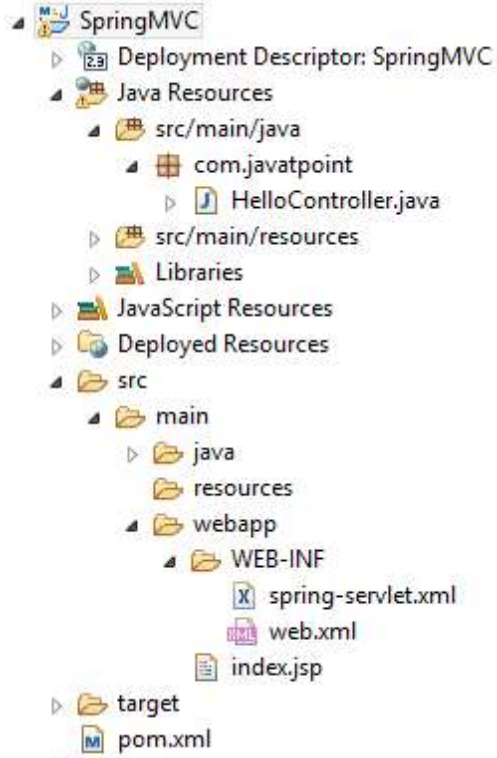
Let's see the simple example of a Spring Web MVC framework. The steps are as follows:

- Load the spring jar files or add dependencies in the case of Maven
- Create the controller class
- Provide the entry of controller in the web.xml file
- Define the bean in the separate XML file
- Display the message in the JSP page
- Start the server and deploy the project

Directory Structure of Spring MVC



Directory Structure of Spring MVC using Maven



Required Jar files or Maven Dependency

To run this example, you need to load:

- Spring Core jar files
- Spring Web jar files
- JSP + JSTL jar files (If you are using any another view technology then load the corresponding jar files).

Download Link: Download all the jar files for spring including JSP and JSTL.

If you are using Maven, you don't need to add jar files. Now, you need to add maven dependency to the pom.xml file.

1. Provide project information and configuration in the pom.xml file.

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.javatpoint</groupId>
  <artifactId>SpringMVC</artifactId>
  <packaging>war</packaging>
  <version>0.0.1-SNAPSHOT</version>
  <name>SpringMVC Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.springframework/spring-webmvc -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.1.1.RELEASE</version>
</dependency>

<!-- https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>servlet-api</artifactId>
    <version>3.0-alpha-1</version>
</dependency>

  </dependencies>
  <build>
    <finalName>SpringMVC</finalName>
  </build>
</project>
```

2. Create the controller class

To create the controller class, we are using two annotations @Controller and @RequestMapping.

The @Controller annotation marks this class as Controller.

The @Requestmapping annotation is used to map the class with the specified URL name.

HelloController.java

```
package com.javatpoint;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
@Controller
public class HelloController {
@RequestMapping("/")
   public String display()
   {
      return "index";
   }
}
```

3. Provide the entry of controller in the web.xml file

In this xml file, we are specifying the servlet class DispatcherServlet that acts as the front controller in Spring Web MVC. All the incoming request for the html file will be forwarded to the DispatcherServlet.

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://java.sun.com/xml/ns/javaee" xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd" id="WebApp_ID" version="3.0">
  <display-name>SpringMVC</display-name>
   <servlet>
    <servlet-name>spring</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>spring</servlet-name>
    <url-pattern>/</url-pattern>
</servlet-mapping>
</web-app>
```

4. Define the bean in the xml file

This is the important configuration file where we need to specify the View components.

The context:component-scan element defines the base-package where DispatcherServlet will search the controller class.

This xml file should be located inside the WEB-INF directory.

spring-servlet.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:mvc="http://www.springframework.org/schema/mvc"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
```

```
        http://www.springframework.org/schema/context/spring-context.xsd
        http://www.springframework.org/schema/mvc
        http://www.springframework.org/schema/mvc/spring-mvc.xsd">

    <!-- Provide support for component scanning -->
    <context:component-scan base-package="com.javatpoint" />

    <!--Provide support for conversion, formatting and validation -->
    <mvc:annotation-driven/>

</beans>
```

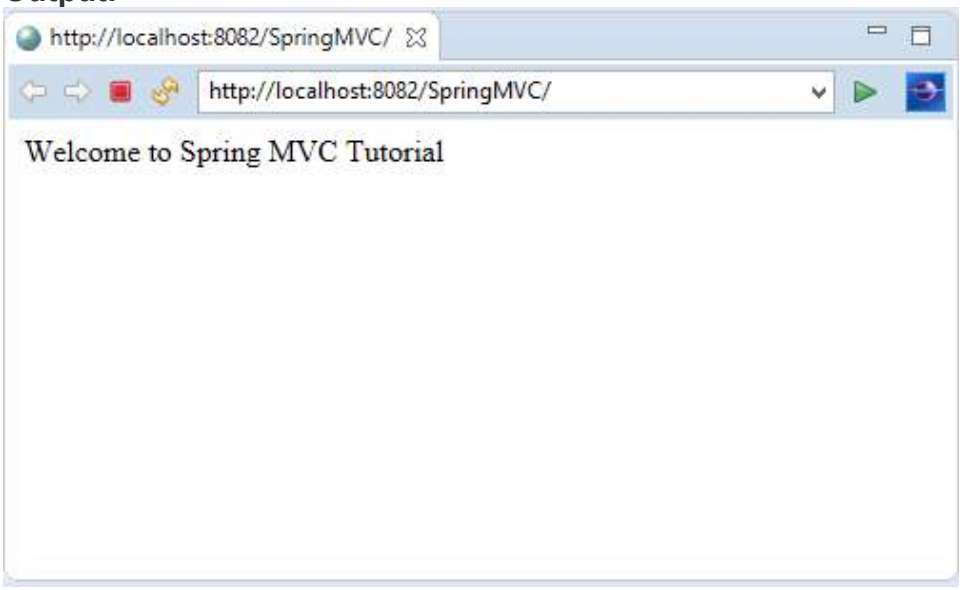
5. Display the message in the JSP page

This is the simple JSP page, displaying the message returned by the Controller.

index.jsp

```
<html>
<body>
<p>Welcome to Spring MVC Tutorial</p>
</body>
</html>
```

Output:

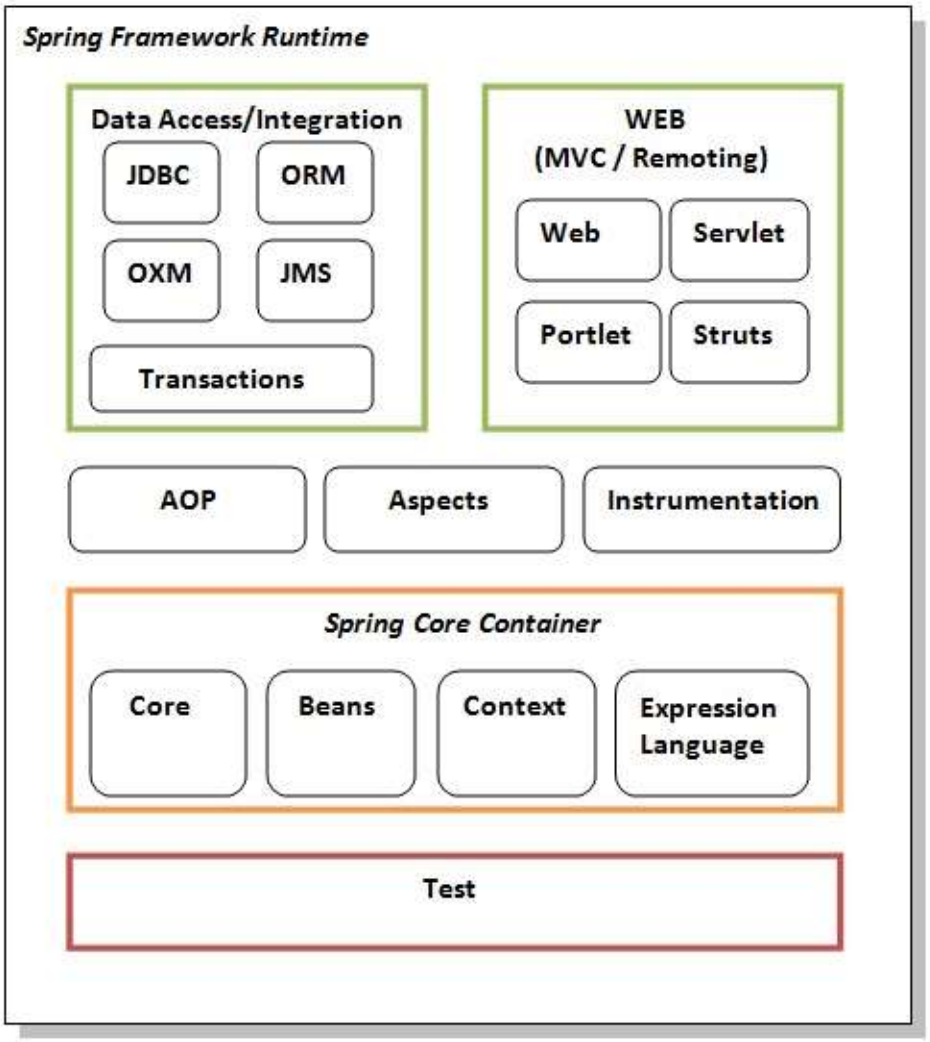


From <https://www.javatpoint.com/spring-mvc-tutorial>

? Spring Modules Overview

Spring Modules

The Spring framework comprises of many modules such as core, beans, context, expression language, AOP, Aspects, Instrumentation, JDBC, ORM, OXM, JMS, Transaction, Web, Servlet, Struts etc. These modules are grouped into Test, Core Container, AOP, Aspects, Instrumentation, Data Access / Integration, Web (MVC / Remoting) as displayed in the following diagram.



Test

This layer provides support of testing with JUnit and TestNG.

Spring Core Container

The Spring Core container contains core, beans, context and expression language (EL) modules.

Core and Beans

These modules provide IOC and Dependency Injection features.

Context

This module supports internationalization (I18N), EJB, JMS, Basic Remoting.

Expression Language

It is an extension to the EL defined in JSP. It provides support to setting and getting property values, method invocation, accessing collections and indexers, named variables, logical and arithmetic operators, retrieval of objects by name etc.

AOP, Aspects and Instrumentation

These modules support aspect oriented programming implementation where you can use Advices, Pointcuts etc. to decouple the code.
The aspects module provides support to integration with AspectJ.

The instrumentation module provides support to class instrumentation and classloader implementations.

Data Access / Integration

This group comprises of JDBC, ORM, OXM, JMS and Transaction modules. These modules basically provide support to interact with the database.

Web

This group comprises of Web, Web-Servlet, Web-Struts and Web-Portlet. These modules provide support to create web application.

From <https://www.javatpoint.com/spring-modules>

? Understanding Spring 4 annotations (Basic Introduction)

Annotation	Package Detail/Import statement
@Service	import org.springframework.stereotype.Service;
@Repository	import org.springframework.stereotype.Repository;
@Component	import org.springframework.stereotype.Component;
@Autowired	import org.springframework.beans.factory.annotation.Autowired;
@Transactional	import org.springframework.transaction.annotation.Transactional;
@Scope	import org.springframework.context.annotation.Scope;
Spring MVC Annotations	
@Controller	import org.springframework.stereotype.Controller;
@RequestMapping	import org.springframework.web.bind.annotation.RequestMapping;
@PathVariable	import org.springframework.web.bind.annotation.PathVariable;
@RequestParam	import org.springframework.web.bind.annotation.RequestParam;
@ModelAttribute	import org.springframework.web.bind.annotation.ModelAttribute;
@SessionAttributes	import org.springframework.web.bind.annotation.SessionAttributes;
Spring Security Annotations	
@PreAuthorize	import org.springframework.security.access.prepost.PreAuthorize;

For spring to process annotations, add the following lines in your application-context.xml file.

```
<context:annotation-config />
<context:component-scan base-package="...specify your package name..." />
```

Spring supports both Annotation based and XML based configurations. You can even mix them together. Annotation injection is performed before XML injection, thus the latter configuration will override the former for properties wired through both approaches.

@Service

Annotate all your service classes with @Service. All your business logic should be in Service classes.

```
@Service
public class CompanyServiceImpl implements CompanyService {
...
}
```

@Repository

Annotate all your DAO classes with @Repository. All your database access logic should be in DAO classes.

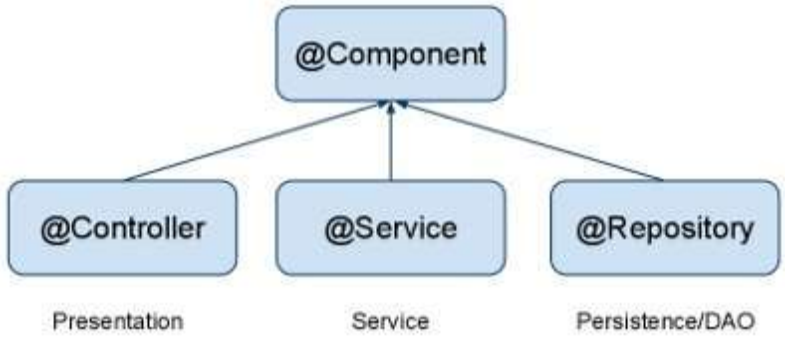
```
@Repository
public class CompanyDAOImpl implements CompanyDAO {
...
}
```

@Component

Annotate your other components (for example REST resource classes) with @Component.

```
@Component
public class ContactResource {
...
}
```

@Component is a generic stereotype for any Spring-managed component. @Repository, @Service, and @Controller are specializations of @Component for more specific use cases, for example, in the persistence, service, and presentation layers, respectively.



@Autowired

Let Spring auto-wire other beans into your classes using @Autowired annotation.

```
@Service
public class CompanyServiceImpl implements CompanyService {
@Autowired
 private CompanyDAO companyDAO;

 ...
}
```

Spring beans can be wired by name or by type.

- @Autowire by default is a type driven injection. @Qualifier spring annotation can be used to further fine-tune autowiring.
- @Resource (javax.annotation.Resource) annotation can be used for

wiring by name.

Beans that are themselves defined as a collection or map type cannot be injected through @Autowired, because type matching is not properly applicable to them. Use @Resource for such beans, referring to the specific collection or map bean by unique name.

@Transactional

Configure your transactions with @Transactional spring annotation.

```
@Service
public class CompanyServiceImpl implements CompanyService {
@Autowired
 private CompanyDAO companyDAO;
@Transactional
 public Company findByName(String name) {
Company company = companyDAO.findByName(name);
   return company;
 }
 ...
}
```

To activate processing of Spring's @Transactional annotation, use the <tx:annotation-driven/> element in your spring's configuration file.

The default @Transactional settings are as follows:

- Propagation setting is PROPAGATION_REQUIRED.
- Isolation level is ISOLATION_DEFAULT.
- Transaction is read/write.
- Transaction timeout defaults to the default timeout of the underlying transaction system, or to none if timeouts are not supported.
- Any RuntimeException triggers rollback, and any checked Exception does not.

These default settings can be changed using various properties of the @Transactional spring annotation.

Specifying the @Transactional annotation on the bean class means that it applies to all applicable business methods of the class. Specifying the annotation on a method applies it to that method only. If the annotation is applied at both the class and the method level, the method value overrides if the two disagree.

@Scope

As with Spring-managed components in general, the default and most common scope for autodetected components is singleton. To change this default behavior, use @Scope spring annotation.

```
@Component
@Scope("request")
public class ContactResource {
...
}
```

Similarly, you can annotate your component with @Scope("prototype") for beans with prototype scopes.

Please note that the dependencies are resolved at instantiation time. For prototype scope, it does NOT create a new instance at runtime more than once. It is only during instantiation that each bean is injected with a separate instance of prototype bean.

Spring MVC Annotations

@Controller

Annotate your controller classes with @Controller.

```
@Controller
public class CompanyController {
...
}
```

@RequestMapping

You use the @RequestMapping spring annotation to map URLs onto an entire class or a particular handler method. Typically the class-level annotation maps a specific request path (or path pattern) onto a form controller, with additional method-level annotations narrowing the primary mapping.

```
@Controller
@RequestMapping("/company")
public class CompanyController {
@Autowired
 private CompanyService companyService;
...
}
```

@PathVariable

You can use the @PathVariable spring annotation on a method argument to bind it to the value of a URI template variable. In our example below, a request path of /company/techferry will bind companyName variable with 'techferry' value.

```
@Controller
@RequestMapping("/company")
public class CompanyController {
@Autowired
 private CompanyService companyService;
@RequestMapping("{companyName}")
 public String getCompany(Map<String, Object> map,
                          @PathVariable String companyName) {
   Company company = companyService.findByName(companyName);
   map.put("company", company);
   return "company";
 }
...
}
```

@RequestParam

You can bind request parameters to method variables using spring annotation @RequestParam.

```
@Controller
@RequestMapping("/company")
public class CompanyController {
@Autowired
 private CompanyService companyService;
```

```
@RequestMapping("/companyList")
 public String listCompanies(Map<String, Object> map,
                             @RequestParam int pageNum) {
   map.put("pageNum", pageNum);
   map.put("companyList", companyService.listCompanies(pageNum));
   return "companyList";
 }
...
}
```

Similarly, you can use spring annotation @RequestHeader to bind request headers.

@ModelAttribute

An @ModelAttribute on a method argument indicates the argument should be retrieved from the model. If not present in the model, the argument should be instantiated first and then added to the model. Once present in the model, the argument's fields should be populated from all request parameters that have matching names. This is known as data binding in Spring MVC, a very useful mechanism that saves you from having to parse each form field individually.

```
@Controller
@RequestMapping("/company")
public class CompanyController {
@Autowired
 private CompanyService companyService;
@RequestMapping("/add")
 public String saveNewCompany(@ModelAttribute Company company) {
   companyService.add(company);
   return "redirect:" + company.getName();
 }
...
}
```

@SessionAttributes

@SessionAttributes spring annotation declares session attributes. This will typically list the names of model attributes which should be transparently stored in the session, serving as form-backing beans between subsequent requests.

```
@Controller
@RequestMapping("/company")
@SessionAttributes("company")
public class CompanyController {
@Autowired
 private CompanyService companyService;
...
}
```

@SessionAttribute works as follows:

- @SessionAttribute is initialized when you put the corresponding attribute into model (either explicitly or using @ModelAttribute-annotated method).
- @SessionAttribute is updated by the data from HTTP parameters when controller method with the corresponding model attribute in its signature is invoked.
- @SessionAttributes are cleared when you call setComplete() on SessionStatus object passed into controller method as an argument.

The following listing illustrate these concepts. It is also an example for pre-populating Model objects.

```
@Controller
@RequestMapping("/owners/{ownerId}/pets/{petId}/edit")
@SessionAttributes("pet")
```

```
public class EditPetForm {
@ModelAttribute("types")

	public Collection<PetType> populatePetTypes() {
		return this.clinic.getPetTypes();
	}

	@RequestMapping(method = RequestMethod.POST)
	public String processSubmit(@ModelAttribute("pet") Pet pet,
			BindingResult result, SessionStatus status) {
		new PetValidator().validate(pet, result);
		if (result.hasErrors()) {
			return "petForm";
		}else {
			this.clinic.storePet(pet);
			status.setComplete();
			return "redirect:owner.do?ownerId="
				+ pet.getOwner().getId();
		}
	}
}
```

Spring Security Annotations

@PreAuthorize

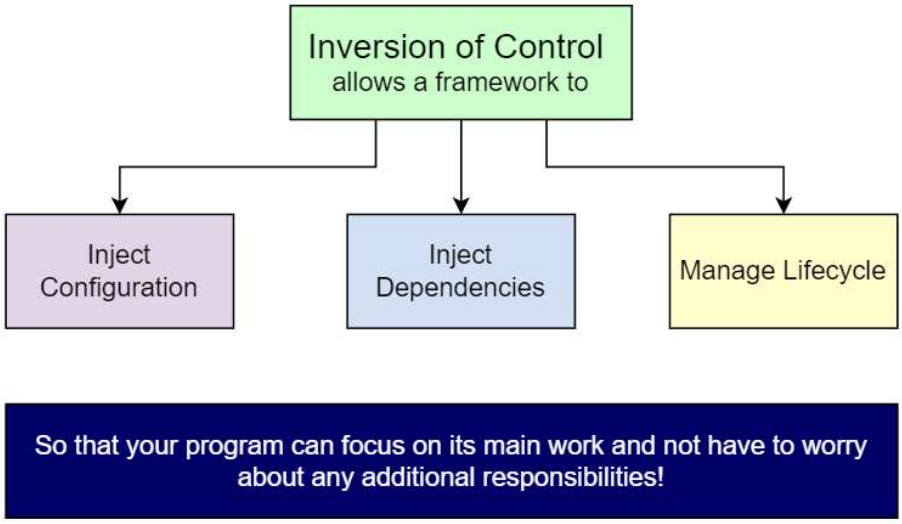
Using Spring Security @PreAuthorize annotation, you can authorize or deny a functionality. In our example below, only a user with Admin role has the access to delete a contact.

```
 @Transactional
 @PreAuthorize("hasRole('ROLE_ADMIN')")
 public void removeContact(Integer id) {
  contactDAO.removeContact(id);
 }
```

From <https://www.techferry.com/articles/spring-annotations.html>

? What is IoC (Inversion of Control)

One of the major principles of Software Engineering is that classes should have minimum interdependence, i.e., low coupling. **Inversion of Control (IoC)** is a design principle that allows classes to be loosely coupled and, therefore, easier to test and maintain. IoC refers to transferring the control of objects and their dependencies from the main program to a container or framework. IoC is a principle, not a design pattern – the implementation details depend on the developer. All IoC does is provide high-level guidelines.



Inversion of Control and Dependency Injection are often used interchangeably. However, Dependency Injection is only one implementation of IoC. Other popular implementations of IoC are:

- Service Locator Pattern
- Event-driven programs, as opposed to sequential programs

Benefits of IoC

- Reduces amount of application code
- Decreases coupling between classes
- Makes the application easier to test and maintain

From <https://www.educative.io/answers/what-is-inversion-of-control>

? IOC container

The Spring framework can be considered as a collection of sub-frameworks, also referred to as layers, such as Spring AOP, Spring ORM, Spring Web Flow, and Spring Web MVC. You can use any of these modules separately while constructing a Web application. The modules may also be grouped together to provide better functionalities in a web application.

The IoC container is responsible to instantiate, configure and assemble the objects. The IoC container gets informations from the XML file and works accordingly. The main tasks performed by IoC container are:

- to instantiate the application class
- to configure the object
- to assemble the dependencies between the objects

There are two types of IoC containers. They are:

1. **BeanFactory**
2. **ApplicationContext**

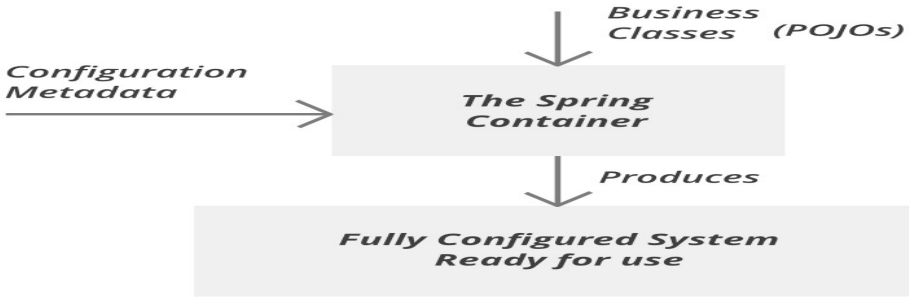
The features of the Spring framework such as IoC, AOP, and transaction management, make it unique among the list of frameworks. Some of the most important features of the Spring framework are as follows:

a. **IoC container**
b. Data Access Framework
c. Spring MVC
d. Transaction Management
e. Spring Web Services
f. JDBC abstraction layer
g. Spring TestContext framework

Spring IoC Container is the core of Spring Framework. It creates the objects, configures and assembles their dependencies, manages their entire life cycle. The Container uses Dependency Injection(DI) to manage the components that make up the application. It gets the information about the objects from a configuration file(XML) or Java Code or Java Annotations and Java POJO class. These objects are called Beans. Since the Controlling of Java objects and their lifecycle is not done by the developers, hence the name **Inversion Of Control**.

Note: *Spring IoC generally directly refers to a core container that uses the DI/DC pattern to implicitly provide an object reference in a class during runtime. The IoC container contains assembler code that handles the configuration management of application objects.*

The following diagram depicts how the Container makes use of Configuration metadata and Java POJO classes to manage beans.



So finally let us discuss out some major differences between BeanFactory vs ApplicationContext in order to get a clear cutaway understanding of the spring IoC container which is as shown below in a tabular format below as follows:

Feature	BeanFactory	ApplicationContext
Annotation Support	No	Yes
Bean Instantiation/Wiring	Yes	Yes
Internationalization	No	Yes
Enterprise Services	No	Yes

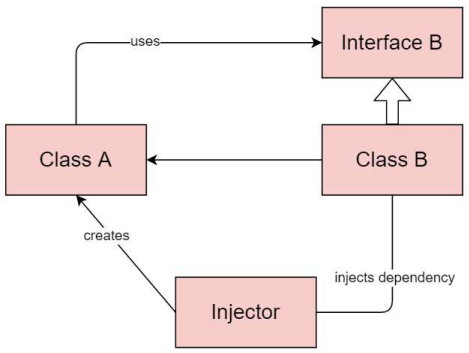
ApplicationEvent publication	No	Yes
Automatic BeanPostProcessor registration	No	Yes
Loading Mechanism	Lazy loading	Aggressive loading
Automatic BeanFactoryPostProcessor registration	No	Yes

From <https://www.geeksforgeeks.org/spring-ioc-container/>

? Dependency Injection

Dependency injection is a technique that allows objects to be separated from the objects they depend upon. For example, suppose object A requires a method of object B to complete its functionality. Well, dependency injection suggests that instead of creating an instance of class B in class A using the new operator, the object of class B should be injected in class A using one of the following methods:

- Constructor injection
- Setter injection
- Interface injection



From <https://www.educative.io/answers/what-is-inversion-of-control>

? Spring Beans

? Autowiring Beans

? Bean Scopes

Introduction

The bean is an important concept of the Spring Framework and as such, if you want to use Spring, it is fundamental to understand what it is and how it works. By definition, a Spring bean is an object that form

the backbone of your application and that is managed by the Spring IoC container. A bean is an object that is instantiated, assembled, and otherwise managed by a Spring IoC container. Otherwise, a bean is simply one of many objects in your application. Beans, and the dependencies among them, are reflected in the configuration metadata used by a container.

Inversion of Control (IoC)

Inside Spring, a bean exploits the Inversion of Control feature by which an object defines its dependencies without creating them. This object delegates the job of constructing and instantiating such dependencies to an IoC container, the Spring lightweight container.

For example, if we have a class `Person` which has a property of class `Animal` as following :

```
1  public class Person {
2      private Animal animal;
3
4      public Person(Animal animal) {
5          this.animal = animal;
6      }
7
8      // getter, setter
9  }
```

Person.java hosted with ❤ by GitHub

Suppose that Animal has two properties : name and age. To instantiate a Person object normally we have to do the following things :

```
1   Animal animal = new Animal("fuffy", 5);
2   Person person = new Person(animal);
```

Fuffy.java hosted with ❤ by GitHub

This can be surely done, but what if we could manage this kind of instantiation in a better way? Actually, this means that the two classes are tightly coupled, and if the `Animal` class change for some reason (i.e. a property is added), the Person class will probably change too. And this could become very difficult to manage dozens of Person instantiation.

Here comes in handy Spring and its container because, with them in action, an object can retrieve its dependencies from an IoC container, instead of constructing dependencies by itself. All we need to do is to provide the container with appropriate configuration metadata.

Now we can do the following :

```
1   @Component
2   public class Person {
3
4       @Autowired
5       private Animal animal;
6
7       // ...
8   }
```

And instantiating the animal with the bean annotation as follows :

```java
@Configuration
public class Config {
    @Bean
    public Animal getAnimal() {
        return new Animal("Fuffy", 5);
    }
}
```

Now an instance of Person get already injected an animal as instantiated (with name Fuffy and age 5). In the next paragraph, we will see how we can retrieve these beans, once instantiated.

Get Beans

Basically, the method responsible for retrieving a bean instance from the Spring container is the *BeanFactory.getBean()* method. Usually in the place of BeanFactory, we use one of the known implementing classes as by documentation here. Whatever implementation you use, there are (by BeanFactory) different signatures by which we can retrieve a bean from the Spring container :

1. By name : given a name, returns an Object if a bean with that name exists in the application context, otherwise throw *NoSuchBeanDefinitionException*.

```
#by name

Object animal = context.getBean("animal");
```

2. By name and type : given a name and a type, returns a bean of the given type if does exist in the application context; it may be useful because avoid to cast the object returned. This method can throw *NoSuchBeanDefinitionException* (if there is no such bean) or BeanNotOfRequiredTypeException (if the bean isn't of the required

type)
#by name and type

```
Animal animal = context.getBean("animal", Animal.class);
```

3. By type : it is similar to the previous, with the difference that we can find more beans with the same type instantiated; in this case we would get a *NoUniqueBeanDefinitionException*
#by type

```
Animal animal = context.getBean(Animal.class);
```

4. By name with constructor parameters : we can pass the bean name and constructor parameters as follows :
#by name and constructor parameters

```
Animal fuffyAnimal = (Animal) context.getBean("animal", "Fuffy", 5);
```

In this case we get an animal named Fuffy with age 5. But if the bean is singleton (i.e. it's not prototype) we can
get *BeanDefinitionStoreException*.

5. By type with constructor parameters : similar to the previous but instead of the name we have to pass the type and the bean cannot be singleton :
#by type and constructor parameters

```
Animal fuffyAnimal = (Animal) context.getBean(Animal.class, "Fuffy", 5);
```

These are the main methods that the `BeanFactory` interface gives us to get beans from the container and usually, we use the ApplicationContext to call these methods. Moreover, to avoid mixing logic with a framework-related way of getting beans we rely on autowiring and dependency injection to get the beans inside another Spring bean.

Most used Spring Bean annotations

I already listed and deeply discussed in another article the most frequently used Spring annotations in general. Here I want to discuss the most common related to beans. All of these annotations are found through the Spring component scanning feature and registered as beans in the Spring container.

1. **@Component :** class-level annotation which by default denotes a bean with the same name as the class name with a lowercase first letter. It can be specified a different name using the value argument of the annotation
2. **@Repository** : class-level annotation representing the DAO layer of an application; it enables an automatic persistence exception translation.
3. **@Service** : class-level annotation representing where the business logic usually resides and can flavor the reuse of the code.
4. **@Controller** : class-level annotation representing the controllers in Spring MVC (Model-View-Controller). See also @RestController for the REST “mode”.
5. **@Configuration** : class-level annotation to say that it can contain bean definition methods annotated with **@Bean**

Bean Scopes

The scope of a bean defines the lifecycle and visibility of that bean in the contexts in which it is used. The scopes of a bean can be separated into basic scopes and web-aware scopes: basic scopes are singleton and prototype while web-aware scopes are request, session, application and websocket.

1. Singleton : it means the container creates a single instance of that

bean and when requested (by name or type) the same object will be returned

```
@Bean
@Scope("singleton")
public Animal theAnimal() {
    return new Animal();
}
```

2. Prototype : a different instance is created every time it is requested from the container.

```
@Bean
@Scope("prototype")
public Animal protoAnimal() {
    return new Animal();
}
```

3. Request : creates a bean instance for a single HTTP request

```
@Bean
@RequestScope
/*@Scope(value = WebApplicationContext.SCOPE_REQUEST,
    proxyMode = ScopedProxyMode.TARGET_CLASS)*/
public Animal requestAnimalBean() {
    return new Animal();
}
```

4. Session : creates a bean instance for each HTTP session

```
@Bean
@SessionScope
/*@Scope(value = WebApplicationContext.SCOPE_SESSION,
    proxyMode = ScopedProxyMode.TARGET_CLASS)*/
public Animal sessionAnimalBean() {
    return new Animal();
}
```

5. Application : creates a bean instance for the lifecycle of a ServletContext

```
@Bean
@ApplicationScope
/*@Scope(value = WebApplicationContext.SCOPE_APPLICATION,
    proxyMode = ScopedProxyMode.TARGET_CLASS)*/
public Animal applicationAnimalBean() {
    return new Animal();
}
```

6. WebSocket : creates a bean instance for a particular WebSocket session.

```
@Bean
@Scope(scopeName = "websocket", proxyMode = ScopedProxyMode.TARGET_CLASS)
public Animal websocketAnimalBean() {
    return new Animal();
}
```

For web-aware scopes, the *proxyMode* attribute is needed because when the web application context is instantiated, there is no active request. Spring will then create a proxy to be injected as dependency and instantiate the target bean when it is needed in a request.

From <https://medium.com/javarevisited/spring-beans-in-depth-a6d8b31db8a1>

Spring MVC

One of the most popular Java frameworks for developing web applications is Spring. Almost every web application integrates with **Spring Framework** because it doesn't require web server activation. With **Spring MVC**, this support is built-in. You are not bound to any container life cycle that you need to manipulate. In this Spring MVC Tutorial, I will tell you how to develop a Spring MVC web application using the Spring Tool Suite.

Below topics are covered in this article:

- What is Spring MVC?
- Spring Web Model View Controller
- Spring MVC Framework example
- Advantages of Spring MVC

Let's get started!

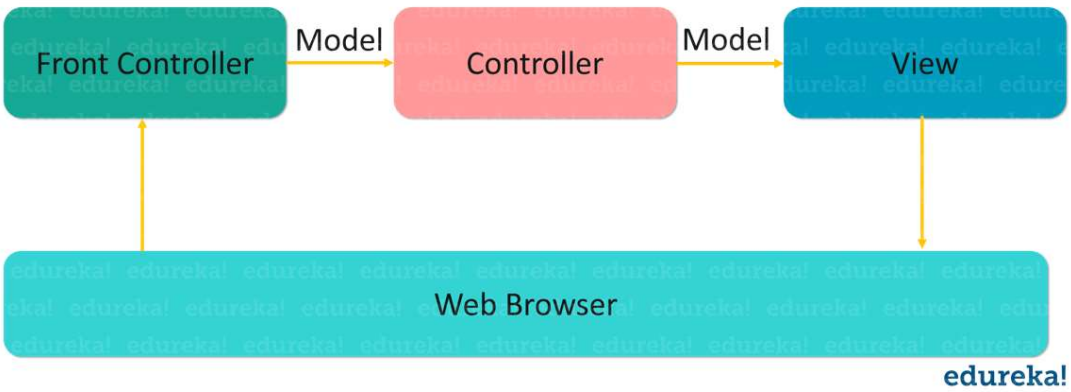
What is Spring MVC?

It is a Java framework which is used to build web applications. It follows the ***Model-View-Controller*** design pattern. Not just that, it also implements all the basic features of a core Spring Framework like Inversion of Control, Dependency Injection. Spring MVC provides a dignified solution to use MVC in Spring Framework by the help of **DispatcherServlet**. In this case, *DispatcherServlet* is a class that receives the incoming request and maps it to the right resource such as **Controllers, Models, and Views.**

Having understood this, now let's move further and understand the fundamentals of Spring Web MVC.

Spring Web Model View Controller

It comprises four main components as shown in the below figure:

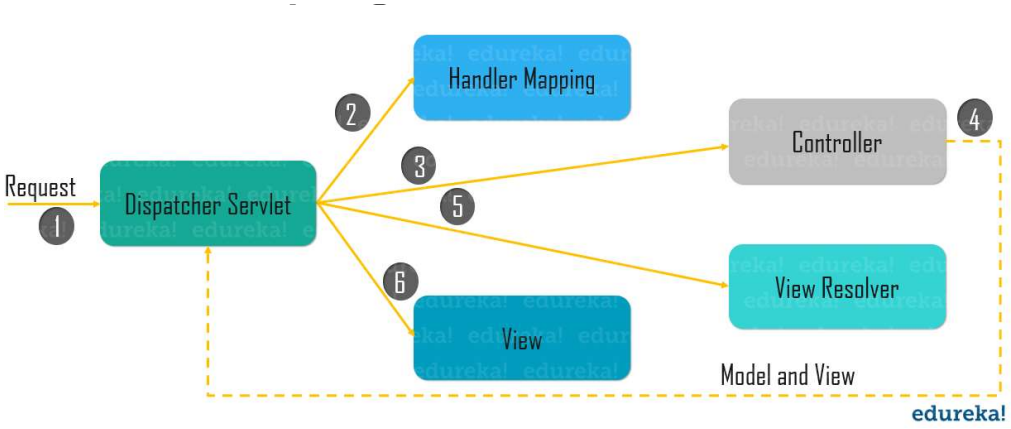


Now let's get into the details of each of these components:

- **Model** – Model contains the core data of the application. Data can either be a single object or a group of objects.
- **Controller** – It contains the business logic of an application. You can use ***@Controller*** annotation to mark the class as Controller.
- **View** – Basically, the View is used to represent the information in a particular format. Here, you can use **JSP+JSTL** to create a view page.
- **Front Controller** – In Spring Web MVC, the **DispatcherServlet** class works as the front Controller.

Now let's see how internally Spring integrates with a Model, View and Controller approach.

Workflow of Spring MVC



As shown in the figure, all the incoming requests are obstructed by the *DispatcherServlet* that works as the front Controller.

- This DispatcherServlet gets an entry of handler mapping from the XML file and forwards the request to the Controller.
- After that, the Controller returns an object of *ModelAndView*.
- Finally, the DispatcherServlet checks the entry of view resolver in the XML file and then invokes the specified view component.

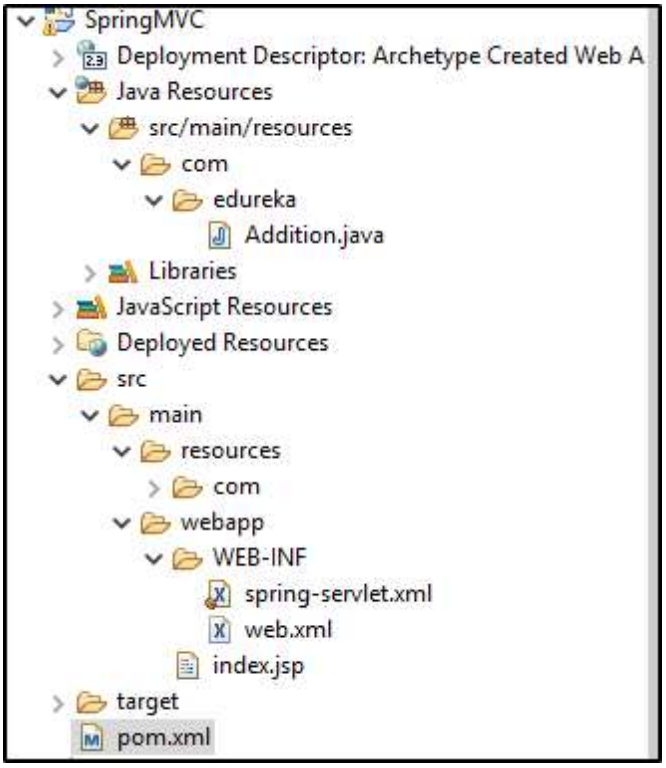
That was all about the workflow of Spring MVC. Now that you know how actually it works, let's dive deeper into the Spring MVC Tutorial article and know its working with the help of examples.

Spring MVC Framework example

To create a Spring MVC application, you need to follow the below steps:

STEP I: Creation of Maven Project

- Create a Maven Project and add the Spring Dependencies into the pom.xml file. If you wish to learn how to configure Spring Framework you can refer to this Spring Framework Tutorial.
- To create a Maven Project for Spring MVC, install Eclipse for JEE developers and follow these steps.
- *Click on File -> New -> Other-> Maven Project -> Next-> Choose maven-archetype-webapp-> Specify GroupID -> Artifact ID -> Package name and then click on finish.*
- The directory structure of your project should look like as shown below:



- Once you create a Maven Project, the next thing that you have to do is to add maven dependencies in the *pom.xml* file.
- Your pom.xml file should consist of the below-mentioned dependencies for Spring MVC.

```
<project xmlns="<a href="http://maven.apache.org/POM/4.0.0">
http://maven.apache.org/POM/4.0.0</a>" xmlns:xsi="<a
href="http://www.w3.org/2001/XMLSchema-instance">
http://www.w3.org/2001/XMLSchema-instance</a>" xsi:schemaLocation="<a
href="http://maven.apache.org/POM/4.0.0">
http://maven.apache.org/POM/4.0.0</a> <a href="http://maven.apache.org/maven-
v4_0_0.xsd">http://maven.apache.org/maven-v4_0_0.xsd</a>">
<modelVersion>4.0.0</modelVersion>
<groupId>com.edureka</groupId>
<artifactId>SpringMVC</artifactId>
<packaging>war</packaging>
<version>0.0.1-SNAPSHOT</version>
<name>SpringMVC Maven Webapp</name>
<url><a href="http://maven.apache.org">http://maven.apache.org</a></url>
<dependencies>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
<!-- <a href="https://mvnrepository.com/artifact/org.springframework/spring-
context">https://mvnrepository.com/artifact/org.springframework/spring-
context</a> -->
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
```

```
<version>5.1.8.RELEASE</version>
</dependency>
<!-- <a href="https://mvnrepository.com/artifact/org.springframework/spring-
webmvc">https://mvnrepository.com/artifact/org.springframework/spring-
webmvc</a> -->
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-webmvc</artifactId>
<version>5.1.8.RELEASE</version>
</dependency>
<!-- <a href="https://mvnrepository.com/artifact/mysql/mysql-connector-java">
https://mvnrepository.com/artifact/mysql/mysql-connector-java</a> -->
<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>8.0.16</version>
</dependency>
<dependency>
<groupId>javax.servlet</groupId>
<artifactId>jstl</artifactId>
<version>1.2</version>
</dependency>
</dependencies>
<build>
<finalName>SpringMVC</finalName>
</build>
</project>
```

- After configuring your *pom.xml* file, all the required *jar files will* be imported. You can also copy and paste the required jar files dependency code from the maven repository.

After this, the next step is to create a Controller class.

Step II: Create the controller class

In order to create a Controller class, I am using two annotations *@Controller and @RequestMapping.*

- The **@Controller** annotation marks this class as Controller.
- The **@RequestMapping** annotation is used to map the class with the specified URL name.

Now let's see how to do that with the help of the below code:

Addition.java

```
package com.edureka;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
@Controller
public class Addition {
@RequestMapping("/")
public void add(){
int i = Integer.parseInt(req.getParameter("num1"));
int j = Integer.parseInt(req.getParameter("num2");
int k= i+j;
System.out.println("Result is" + k) //returns the result from jsp file
}
}
```

Step III: Configure the web.xml file and provide entry for Controller class

In this XML file, I am specifying the Servlet class which is **DispatcherServlet** that acts as the front Controller in Spring Web MVC. All the incoming requests for the HTML file will be forwarded to the DispatcherServlet. Let's now write the web.xml file. This file will take the mappings and the URL pattern for executing the program.

web.xml

```
<!DOCTYPE web-app PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application
2.3//EN" "<a href="http://java.sun.com/dtd/web-app_2_3.dtd">
http://java.sun.com/dtd/web-app_2_3.dtd</a>" >
<web-app>
<display-name>Archetype Created Web Application</display-name>
<servlet>
<servlet-name>spring</servlet-name>
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
<load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
<servlet-name>spring</servlet-name>
<url-pattern>/add</url-pattern>
</servlet-mapping>
</web-app>
```

After this, the next step is to define the bean class file.

Step IV: Define bean in an XML file

This file is necessary to specify the View components. In this, the ***context:component-scan*** element defines the base-package where *DispatcherServlet* will search the Controller class. This file should be present inside the *WEB-INF Directory.*

add-servlet.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="<a href="http://www.springframework.org/schema/beans">
http://www.springframework.org/schema/beans</a>" xmlns:ctx="<a
href="http://www.springframework.org/schema/context">
http://www.springframework.org/schema/context</a>" xmlns:xsi="<a
href="http://www.w3.org/2001/XMLSchema-instance">
http://www.w3.org/2001/XMLSchema-instance</a>" xmlns:mvc="<a
href="http://www.springframework.org/schema/mvc">
http://www.springframework.org/schema/mvc</a>" xsi:schemaLocation="<a
href="http://www.springframework.org/schema/beans">
http://www.springframework.org/schema/beans</a> <a
href="http://www.springframework.org/schema/beans/spring-beans-2.5.xsd">
http://www.springframework.org/schema/beans/spring-beans-2.5.xsd</a> <a
href="http://www.springframework.org/schema/mvc">
http://www.springframework.org/schema/mvc</a> <a
href="http://www.springframework.org/schema/mvc/spring-mvc-3.0.xsd">
http://www.springframework.org/schema/mvc/spring-mvc-3.0.xsd</a> <a
href="http://www.springframework.org/schema/context">
http://www.springframework.org/schema/context</a> <a
href="http://www.springframework.org/schema/context/spring-context-2.5.xsd">
http://www.springframework.org/schema/context/spring-context-2.5.xsd</a> ">
<ctx:annotation-config></ctx:annotation-config>
<ctx:component-scan base-package="com.edureka">
</ctx:component-scan>
<mvc:annotation-driven/>
</beans>
```

Now the final step is to write the request in the index.jsp file.

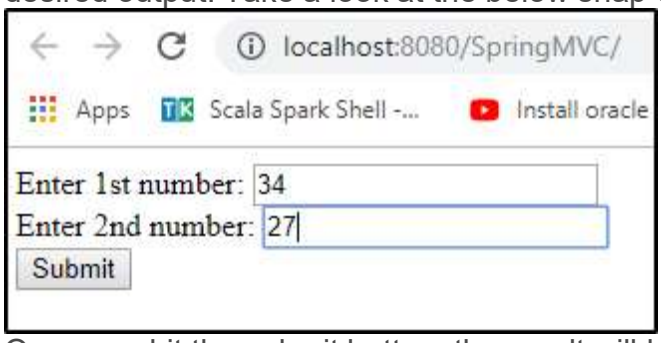
Step V. Create the JSP page

This is the simple JSP page, in which I will perform addition of 2 numbers.

```
<html>
<body>

<form action ="add">
Enter 1st number: <input type="text" name="num1">
Enter 2nd number: <input type="text" name="num2">
<input type ="submit">
</form>
```

```
9
10 </body>
11 </html>
```

After all this, you can run the program by starting the server. You will get the desired output. Take a look at the below snap-shot to refer the output:



Once you hit the submit button, the result will be displayed on the screen. Basically, that's how it works.
That was all about how to create a Spring MVC Application. Having understood this, let's move further in Spring MVC Tutorial, and know what are the advantages of using Spring MVC.

Advantages of Spring MVC

1. **Lightweight:** As Spring is a lightweight framework, there will not be any performance issues in Spring-based web application.
2. **High productive:** Spring MVC can boost your development process and hence is highly productive.
3. **Secure:** Most of the online banking web applications are developed using Spring MVC because it is highly secure. For enterprise-grade security implementation, Spring security is a great API.
4. **MVC Supported:** As it is based on MVC, it is a great way to develop modular web applications.
5. **Role Separation:** It comprises of a separate class for specific roles like Model, Command, Validator, etc.

These were some of the advantages of using Spring MVC Framework.

From <https://www.edureka.co/blog/spring-mvc-tutorial/>

□ Model, Model & View,

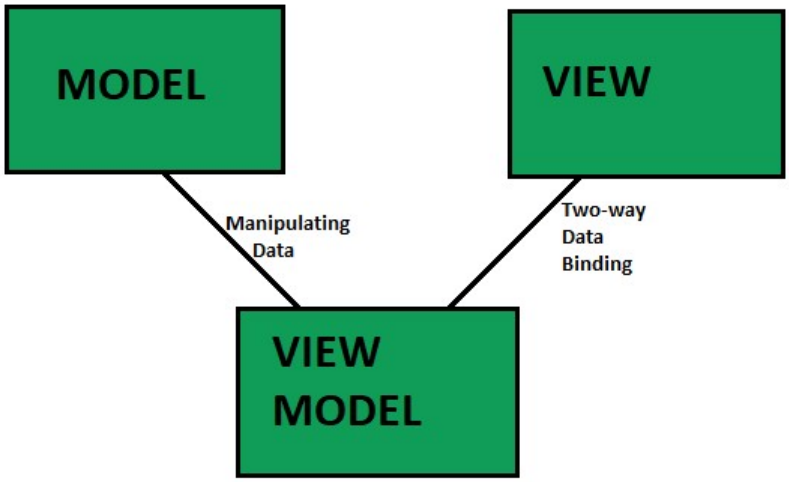
Introduction to Model View View Model (MVVM)

Description of Model is as follows:

- **MODEL:** (Reusable Code – DATA) Business Objects that encapsulate data and behavior of application domain, Simply hold the data.
- **VIEW:** (Platform Specific Code – USER INTERFACE) What the user sees, The Formatted data.
- **VIEWMODEL:** (Reusable Code – LOGIC) Link between Model and View OR It Retrieves data from Model and exposes it to the View. This is the model specifically designed for the View.

Note: *Link between Model and View Model is Manipulating Data and between ViewModel and View is 2-way Data Binding.*

BASIC INTRODUCTION: [Way to Structure Code]



FEATURES:

- Life Cycle state of Application will be maintained.
- The application will be in the same position as where the user left it.
- UI Components are kept away from Business Logic.
- Business Logic is kept away from Database operations.
- Easy to understand and read.

BASIC EXAMPLE: We want to display Name in Purple Color (not written in the proper format, proper length) or Display Purple Color if Age of a person is > 18 years, Display Pink Color if Age of a person is < 18 years, Then the Logic of Purple and Pink Color would be present in ViewModel.

SUMMARY: From Server, Get Data(available in Model Objects), View Model reads Model Objects and then facilitates the easy presentation of data on the view.

The primary differences between MVVM AND MVC are as follows:

MVVM	MVC
The Model is somewhat similar to MVC but here we have ViewModels which are passed to the view and all the logic is in the ViewModel and hence no controller is there. Example: Knockout.js	In this pattern, we have models which are basic objects with no code and just properties, views that contribute to presentation items (HTML, WinForms, etc), client-side deletes, and Controllers that focus on the logic part. Examples: ASP.NET MVC, Angular
In MVVM your DeletePerson would be called off of your view model	We have a PersonController with an Action DeletePerson that delete a person
We are on the client side so we can hold on to objects and do a lot more logic in a non-disconnected state.	MVC is typically used when things are transactional and disconnected as is the case with server-side web. In ASP MVC we send the view through the wire and then the transaction with the client is over.

ADVANTAGES:

- Maintainability – Can remain agile and keep releasing successive versions quickly.
- Extensibility – Have the ability to replace or add new pieces of code.
- Testability – Easier to write unit tests against a core logic.
- Transparent Communication – The view model provides a transparent interface to the view controller, which it uses to populate the view layer and interact with the model layer, which results in a transparent communication between the layers of your application.

DISADVANTAGES:

- Some people think that for simple UIs, MVVM can be overkill.
- In bigger cases, it can be hard to design the ViewModel.
- Debugging would be a bit difficult when we have complex data bindings.

From <https://www.geeksforgeeks.org/introduction-to-model-view-view-model-mvvm/>

HandlerMapping,

Spring MVC Handler Mapping

HandlerMapping is an Interface to be implemented by objects that define a mapping between requests and handler objects.

By default **DispatcherServlet** uses **BeanNameUrlHandlerMapping** and **DefaultAnnotationHandlerMapping**. In Spring we majorly use the below handler mappings

1. BeanNameUrlHandlerMapping
2. ControllerClassNameHandlerMapping
3. SimpleUrlHandlerMapping

- In **BeanNameUrlHandlerMapping** we will be mapping each request to a Bean directly.
- **ControllerClassNameHandlerMapping** uses a convention to map the requested URL to the Controller. It will take the **Controller name** and **converts them to lower case** with a **leading "/"**
- **SimpleUrlHandlerMapping** is the simplest of all handler mappings which allows you specify URL pattern and handler explicity

From <https://www.javainterviewpoint.com/spring-mvc-handler-mapping/>

ViewResolver

Spring MVC framework provides away with the help of which you can work with the views. It is done with the help of View Resolver, which does not need any specific view technology like JSP, Velocity, etc. The View Resolver maps the view names to the corresponding view. There are various View Resolvers available in Spring, such as InternalResourceViewResolver, XmlViewResolver, etc.

Given below are the various View Resolvers available in Spring MVC:

- AbstractCachingViewResolver: This Resolver provides caching when we extend this.
- XmlViewResolver: It accepts a configuration file written in XML with the same DTD as of Spring Bean factories, and the default configuration file is /WEB-INF/views.xml.
- ResourceBundleViewResolver: It uses bean definition in a ResourceBundle, which has a specific bundle name, and the default file name is views.properties.
- UrlBasedViewResolver: It affects the direct resolution of view names to the URLs.
- InternalResourceViewResolver: It is a subclass of UrlBasedViewResolver that supports InternalResourceView (Servlets and JSPs) and subclasses such as JstlView and TilesView. View Class can be specified using setViewClass().
- VelocityViewResolver / FreeMarkerViewResolver: It is a subclass of UrlBasedViewResolver that supports Velocity View or Free Marker View.
- ContentNegotiatingViewResolver: It resolves a view based on the request file name or Accept header.

From <https://www.h2kinfosys.com/blog/what-are-view-resolvers-in-spring/>

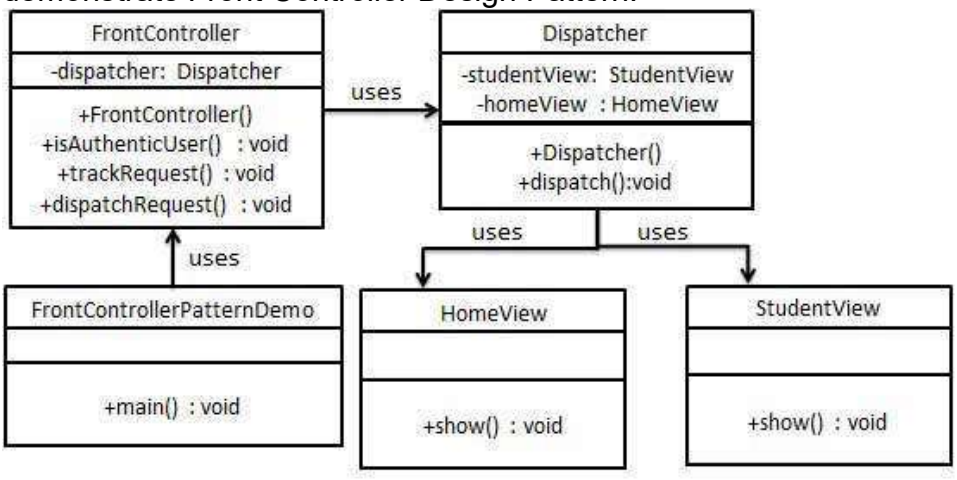
? Design Pattern: Front Controller Pattern

The front controller design pattern is used to provide a centralized request handling mechanism so that all requests will be handled by a single handler. This handler can do the authentication/ authorization/ logging or tracking of request and then pass the requests to corresponding handlers. Following are the entities of this type of design pattern.

- **Front Controller** - Single handler for all kinds of requests coming to the application (either web based/ desktop based).
- **Dispatcher** - Front Controller may use a dispatcher object which can dispatch the request to corresponding specific handler.
- **View** - Views are the object for which the requests are made.

Implementation

We are going to create a *FrontController* and *Dispatcher* to act as Front Controller and Dispatcher correspondingly. *HomeView* and *StudentView* represent various views for which requests can come to front controller.
FrontControllerPatternDemo, our demo class, will use *FrontController* to demonstrate Front Controller Design Pattern.



AD

Step 1

Create Views.

HomeView.java

```
public class HomeView {
  public void show(){
    System.out.println("Displaying Home Page");
  }
}
```

StudentView.java

```
public class StudentView {
  public void show(){
    System.out.println("Displaying Student Page");
  }
}
```

Step 2

Create Dispatcher.

Dispatcher.java

```
public class Dispatcher {
  private StudentView studentView;
  private HomeView homeView;

  public Dispatcher(){
    studentView = new StudentView();
    homeView = new HomeView();
  }
public void dispatch(String request){
    if(request.equalsIgnoreCase("STUDENT")){
      studentView.show();
    }
    else{
      homeView.show();
    }
  }
}
```


Step 3

Create FrontController

FrontController.java

```
public class FrontController {

  private Dispatcher dispatcher;
public FrontController(){
    dispatcher = new Dispatcher();
  }
private boolean isAuthenticUser(){
    System.out.println("User is authenticated successfully.");
    return true;
  }
private void trackRequest(String request){
    System.out.println("Page requested: " + request);
  }
public void dispatchRequest(String request){
```

```
    //log each request
    trackRequest(request);

    //authenticate the user
    if(isAuthenticUser()){
      dispatcher.dispatch(request);
    }
  }
}
```

Step 4

Use the *FrontController* to demonstrate Front Controller Design Pattern.

FrontControllerPatternDemo.java

```
public class FrontControllerPatternDemo {
  public static void main(String[] args) {

    FrontController frontController = new FrontController();
    frontController.dispatchRequest("HOME");
    frontController.dispatchRequest("STUDENT");
  }
}
```

Step 5

Verify the output.

```
Page requested: HOME
User is authenticated successfully.
Displaying Home Page
Page requested: STUDENT
User is authenticated successfully.
Displaying Student Page
```

From <https://www.tutorialspoint.com/design_pattern/front_controller_pattern.htm>

? Spring MVC Web application with JSP views (without Spring Boot)

https://crunchify.com/simplest-spring-mvc-hello-world-example-tutorial-spring-model-view-controller-tips/

? Using Thymleaf as alternate View Technology (only introduction)

What is Thymeleaf?

The **Thymeleaf** is an open-source Java library that is licensed under the **Apache License 2.0**. It is a **HTML5/XHTML/XML** template engine. It is a **server-side Java template** engine for both web (servlet-based) and non-web (offline) environments. It is perfect for modern-day HTML5 JVM web development. It provides full integration with Spring Framework.

It applies a set of transformations to template files in order to display data or text produced by the application. It is appropriate for serving XHTML/HTML5 in web applications.

The goal of Thymeleaf is to provide a **stylish** and **well-formed** way of creating templates. It is based on XML tags and attributes. These XML tags define the execution of predefined logic on the DOM (Document Object Model) instead of explicitly writing that logic as code inside the template. It is a substitute for **JSP**.

The architecture of Thymeleaf allows the **fast processing** of templates that depends on the caching of parsed files. It uses the least possible amount of I/O operations during execution.

Why we use Thymeleaf?

JSP is more or less similar to HTML. But it is not completely compatible with HTML like Thymeleaf. We can open and display a Thymeleaf template file normally in the browser while the JSP file does not.

Thymeleaf supports variable expressions (${...}) like Spring EL and executes on model attributes, asterisk expressions (*{...}) execute on the form backing bean, hash expressions (#{...}) are for internationalization, and link expressions (@{...}) rewrite URLs.

Like JSP, Thymeleaf works well for Rich HTML emails.

Thymeleaf can process six types of templates (also known as **Template Mode**) are as follows:

- XML
- Valid XML
- XHTML
- Valid XHTML
- HTML5
- Legacy HTML5

Except for the Legacy HTML5 mode, all the above modes refer to **well-defined XML** files. It allows us to process HTML5 files with features such as **standalone tags, tag attributes without value, or not written between quotes**.
To process files in this specific mode, Thymeleaf performs a transformation that converts files into a **well-formed XML** file (valid HTML5 file).

Thymeleaf Features

- It works on both web and non-web environments.
- Java template engine for HTML5/ XML/ XHTML.
- Its high-performance parsed template cache reduces I/O to the minimum.
- It can be used as a template engine framework if required.
- It supports several template modes: XML, XHTML, and HTML5.
- It allows developers to extend and create custom dialect.
- It is based on modular features sets called dialects.
- It supports internationalization.

From <https://www.javatpoint.com/spring-boot-thymeleaf-view>

? Spring Validations

The Spring MVC Validation is used to restrict the input provided by the user. To validate the user's input, the Spring 4 or higher version supports and use Bean Validation API. It can validate both server-side as well as client-side applications.

Bean Validation API

The Bean Validation API is a Java specification which is used to apply constraints on object model via annotations. Here, we can validate a length, number, regular expression, etc. Apart from that, we can also provide custom validations.
As Bean Validation API is just a specification, it requires an implementation. So, for that, it uses Hibernate Validator. The Hibernate Validator is a fully compliant JSR-303/309 implementation that allows to express and validate application constraints.

Validation Annotations

Let's see some frequently used validation annotations.

Annotation	Description
@NotNull	It determines that the value can't be null.
@Min	It determines that the number must be equal or greater than the specified value.
@Max	It determines that the number must be equal or less than the specified value.
@Size	It determines that the size must be equal to the specified value.
@Pattern	It determines that the sequence follows the specified regular expression.

From <https://www.javatpoint.com/spring-mvc-validation>

? Spring i18n, Localization, Properties

Welcome to the Spring Internationalization (i18n) tutorial. Any web application with users all around the world, **internationalization** (i18n) or **localization** (L10n) is very important for better user interaction. Most of the web application frameworks provide easy ways to localize the application based on user locale settings. Spring also follows the pattern and provides extensive support for internationalization (i18n) through the use of Spring interceptors, Locale Resolvers and Resource Bundles for different locales. Some earlier articles about i18n in java.

From <https://www.digitalocean.com/community/tutorials/spring-mvc-internationalization-i18n-and-localization-l10n-example>

? File Upload example

https://www.tutorialspoint.com/springmvc/springmvc_upload.htm

Sessions 15 & 16:

? Spring Boot essentials

Key Components and Internals of Spring Boot Framework

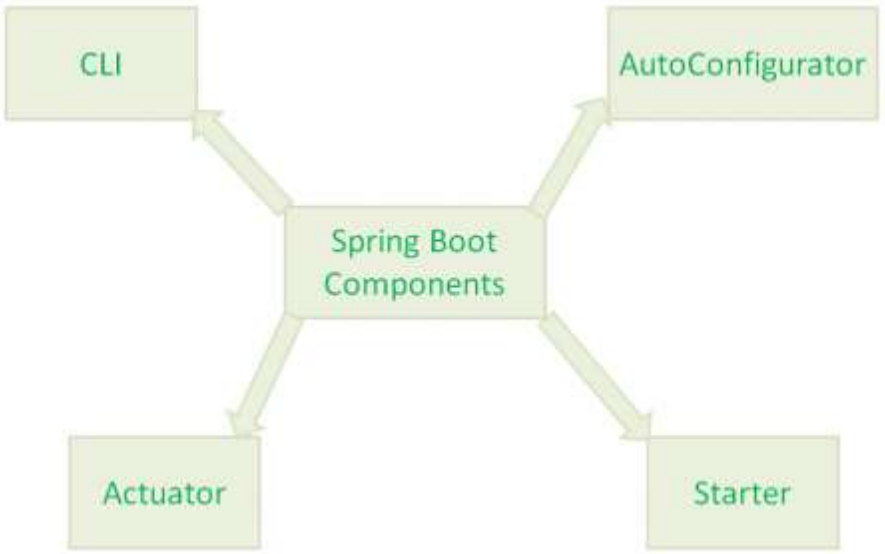
Key Components of Spring Boot Framework

Spring Boot Framework has mainly four major Components.

- Spring Boot Starters
- Spring Boot AutoConfigurator
- Spring Boot CLI
- Spring Boot Actuator

NOTE:- In addition to these four major components, there are two more Spring Boot components:

- Spring Initilizr
- Spring Boot IDEs



Now we will discuss these Spring Boot four components one by one in detail.

Spring Boot Starter

Spring Boot Starters is one of the major key features or components of Spring Boot Framework. The main responsibility of Spring Boot Starter is to combine a group of common or related dependencies into single dependencies. We will explore this statement in detail with one example. For instance, we would like to develop a Spring WebApplication with Tomcat WebServer. Then we need to add the following minimal jar dependencies in your Maven's pom.xml file or Gradle's build.gradle file

- Spring core Jar file(spring-core-xx.jar)
- Spring Web Jar file(spring-web-xx.jar)
- Spring Web MVC Jar file(spring-webmvc-xx.jar)
- Servlet Jar file(servlet-xx.jar)

If we want to add some database stuff, then we need to add database related jars like Spring JDBC jar file, Spring ORM jar files,Spring Transaction Jar file etc.

- Spring JDBC Jar file(spring-jdbc-xx.jar)
- Spring ORM Jar file(spring-orm-xx.jar)
- Spring Transaction Jar file(spring-transaction-xx.jar)

We need to define lot of dependencies in our build files. It is very tedious and cumbersome

tasks for a Developer. And also it increases our build file size. What is the solution to avoid this much dependencies definitions in our build files? The solution is Spring Boot Starter component.

Spring Boot Starter component combines all related jars into single jar file so that we can add only jar file dependency to our build files. Instead of adding above 4 jars files to our build file, we need to add one and only one jar file: "spring-boot-starter-web" jar file. When we add "spring-boot-starter-web" jar file dependency to our build file, then Spring Boot Framework will automatically download all required jars and add to our project classpath.

In the same way, "spring-boot-starter-logging" jar file loads all it's dependency jars like "jcl-over-slf4j, jul-to-slf4j,log4j-over-slf4j, logback-classic" to our project classpath.

Major Advantages of Spring Boot Starter

- Spring Boot Starter reduces defining many dependencies simplify project build dependencies.
- Spring Boot Starter simplifies project build dependencies.

That's it about Spring Boot Starter component. We will discuss some more details with some Spring Boot examples in coming posts.

Spring Boot AutoConfigurator

Another important key component of Spring Boot Framework is Spring Boot AutoConfigurator. Most of the Spring IO Platform (Spring Framework) Critics opinion is that "To develop a Spring-based application requires lot of configuration (Either XML Configuration of Annotation Configuration). Then how to solve this problem. The solution to this problem is Spring Boot AutoConfigurator.

The main responsibility of Spring Boot AutoConfigurator is to reduce the Spring Configuration. If we develop Spring applications in Spring Boot,then We dont need to define single XML configuration and almost no or minimal Annotation configuration. Spring Boot AutoConfigurator component will take care of providing those information. For instance, if we want to declare a Spring MVC application using Spring IO Platform, then we need to define lot of XML Configuration like views, view resolvers etc. But if we use Spring Boot Framework, then we dont need to define those XML Configuration. Spring Boot AutoConfigurator will take of this. If we use "spring-boot-starter-web" jar file in our project build file, then Spring Boot AutoConfigurator will resolve views, view resolvers etc. automatically. And also Spring Boot reduces defining of Annotation configuration. If we use @SpringBootApplication annotation at class level, then Spring Boot AutoConfigurator will automatically add all required annotations to Java Class ByteCode.

If we go through Spring Boot Documentation, we can find the following definition for @SpringBootApplication.

```
@Target(value=TYPE)
@Retention(value=RUNTIME)
@Documented
@Inherited
@Configuration
@EnableAutoConfiguration
@ComponentScan
public @interface SpringBootApplication
```

That is, @SpringBootApplication = @Configuration + @ComponentScan + @EnableAutoConfiration. That's it about Spring Boot AutoConfigurate component. We will discuss some more details with some Spring Boot examples in coming posts. **NOTE:-**

- In simple words, Spring Boot Starter reduces build's dependencies and Spring Boot AutoConfigurator reduces the Spring Configuration.
- As we discussed that Spring Boot Starter has a dependency on Spring Boot AutoConfigurator, Spring Boot Starter triggers Spring Boot AutoConfigurator automatically.

Spring Boot CLI

Spring Boot CLI(Command Line Interface) is a Spring Boot software to run and test Spring Boot applications from command prompt. When we run Spring Boot applications using CLI, then it internally uses Spring Boot Starter and Spring Boot AutoConfigurate components to resolve all dependencies and execute the application. We can run even Spring Web Applications with simple Spring Boot CLI Commands. Spring Boot CLI has introduced a new "spring" command to execute Groovy Scripts from command prompt. **spring command example:**

```
spring run HelloWorld.groovy
```

Here HelloWorld.groovy is a Groovy script FileName. Like Java source file names have *.java extension, Groovy script files have *.groovy extension. "spring" command executes HelloWorld.groovy and produces output. Spring Boot CLI component requires many steps like CLI Installation, CLI Setup, Develop simple Spring Boot application and test it. So we are going to dedicate another post to discuss it in details with some Spring Boot Examples. Please refer my next post on Spring Boot CLI.

Spring Boot Actuator

Spring Boot Actuator components gives many features, but two major features are

- Providing Management EndPoints to Spring Boot Applications.
- Spring Boot Applications Metrics.

When we run our Spring Boot Web Application using CLI, Spring Boot Actuator automatically provides hostname as "localhost" and default port number as "8080". We can access this application using " https://localhost:8080/" end point. We actually use HTTP Request methods like GET and POST to represent Management EndPoints using Spring Boot Actuator. We will discuss some more details about Spring Boot Actuator in coming posts.

From <https://www.digitalocean.com/community/tutorials/key-components-and-internals-of-spring-boot-framework>

? Why Spring boot

? Spring Boot Overview

Spring Boot uses completely new development model to make Java Development very easy by avoiding some tedious development steps and boilerplate code and

configuration.

What is Spring Boot?

Spring Boot is a Framework from “The Spring Team” to ease the bootstrapping and development of new Spring Applications. It provides defaults for code and annotation configuration to quick start new Spring projects within no time. It follows “Opinionated Defaults Configuration” Approach to avoid lot of boilerplate code and configuration to improve Development, Unit Test and Integration Test Process.

What is NOT Spring Boot?

Spring Boot Framework is not implemented from the scratch by The Spring Team, rather than implemented on top of existing Spring Framework (Spring IO Platform). It is not used for solving any new problems. It is used to solve same problems like Spring Framework.

Why Spring Boot?

- To ease the Java-based applications Development, Unit Test and Integration Test Process.
- To reduce Development, Unit Test and Integration Test time by providing some defaults.
- To increase Productivity.
  Don’t worry about what is “Opinionated Defaults Configuration” Approach at this stage. We will explain this in detail with some examples in coming posts.

Advantages of Spring Boot:

- It is very easy to develop Spring Based applications with Java or Groovy.
- It reduces lots of development time and increases productivity.
- It avoids writing lots of boilerplate Code, Annotations and XML Configuration.
- It is very easy to integrate Spring Boot Application with its Spring Ecosystem like Spring JDBC, Spring ORM, Spring Data, Spring Security etc.
- It follows “Opinionated Defaults Configuration” Approach to reduce Developer effort
- It provides Embedded HTTP servers like Tomcat, Jetty etc. to develop and test our web applications very easily.
- It provides CLI (Command Line Interface) tool to develop and test Spring Boot(Java or Groovy) Applications from command prompt very easily and quickly.
- It provides lots of plugins to develop and test Spring Boot Applications very easily using Build Tools like Maven and Gradle
- It provides lots of plugins to work with embedded and in-memory Databases very easily.
  In Simple Terminology, What Spring Boot means

That means Spring Boot is nothing but existing Spring Framework + Some Embedded HTTP Servers (Tomcat/Jetty etc.) - XML or Annotations Configurations. Here minus means we don’t need to write any XML Configuration and few Annotations only.

Main Goal of Spring Boot:

The main goal of Spring Boot Framework is to reduce Development, Unit Test and Integration Test time and to ease the development of Production ready web applications very easily compared to existing Spring Framework, which really takes more time.

- To avoid XML Configuration completely
- To avoid defining more Annotation Configuration(It combined some existing Spring Framework Annotations to a simple and single Annotation)

- To avoid writing lots of import statements
- To provide some defaults to quick start new projects within no time.
- To provide Opinionated Development approach.

By providing or avoiding these things, Spring Boot Framework reduces Development time, Developer Effort and increases productivity.

Limitation/Drawback of Spring Boot:

Spring Boot Framework has one limitation. It is some what bit time consuming process to convert existing or legacy Spring Framework projects into Spring Boot Applications but we can convert all kinds of projects into Spring Boot Applications. It is very easy to create brand new/Greenfield Projects using Spring Boot. To Start Opinionated Approach to create Spring Boot Applications, The Spring Team (The Pivotal Team) has provided the following three approaches.

- Using Spring Boot CLI Tool
- Using Spring STS IDE
- Using Spring Initializr Website

From <https://www.digitalocean.com/community/tutorials/spring-boot-tutorial>

? Basic Introduction of MAVEN

Maven is an automation and management tool developed by Apache Software Foundation. It is written in Java Language to build projects written in C#, Ruby, Scala, and other languages. It allows developers to create projects, dependency, and documentation using Project Object Model and plugins. It has a similar development process as ANT, but it is more advanced than ANT.

Maven can also build any number of projects into desired output such as jar, war, metadata.
It was initially released on 13 July 2004. In the Yiddish language, the meaning of Maven is "accumulator of knowledge."

How can Maven benefit my development process?

Maven helps the developer to create a java-based project more easily. Accessibility of new feature created or added in Maven can be easily added to a project in Maven configuration. It increases the performance of project and building process.
The main feature of Maven is that it can download the project dependency libraries automatically.
Below are the examples of some popular IDEs supporting development with Maven Framework:

- Eclipse
- IntelliJ IDEA
- JBuilder
- NetBeans
- MyEclipse

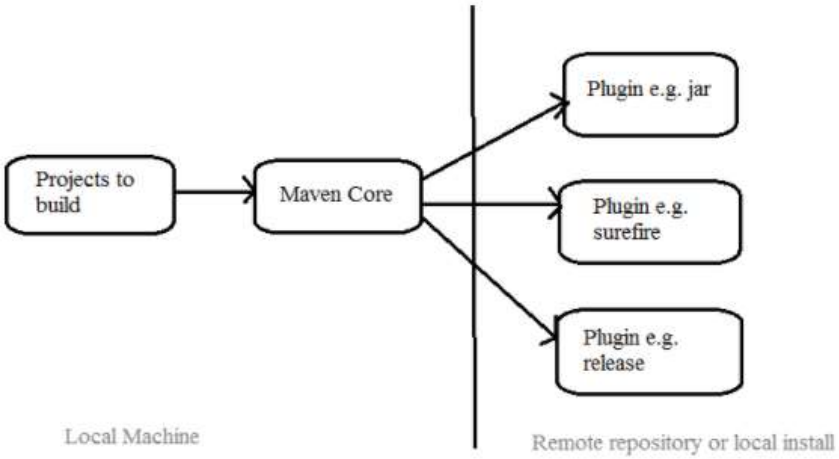
Processes which can manage using maven:

- Builds
- Documentation
- Reporting
- Dependencies
- SCMs
- Releases

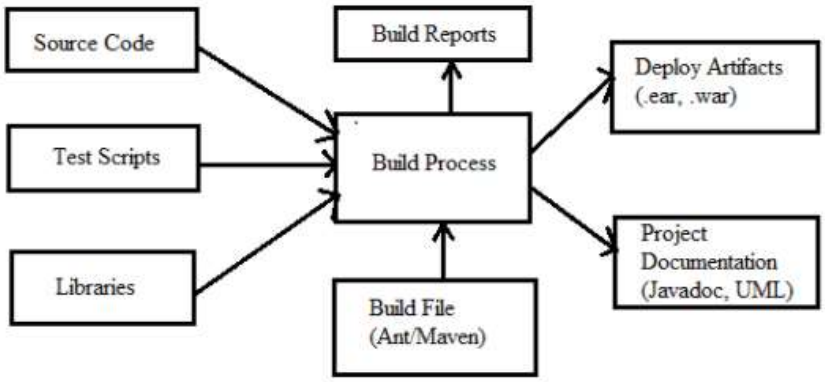
- Distribution
- mailing list

Maven Architecture

Maven Architecture includes plugin jar, code file etc.



Maven Architecture



How to use Maven

- To configure the Maven in Java, you need to use Project Object Model, which is stored in a pom.xml-file.
- POM includes all the configuration setting related to Maven. Plugins can be configured and edit in the <plugins> tag of a pom.xml file. And developer can use any plugin without much detail of each plugin.
- When user start working on Maven Project, it provides default setting of configuration, so the user does not need to add every configuration in pom.xml

Steps/process involved in building the project:

- Add / Write the code for application creation and process that into source code repository
- Edit configuration / pom.XML / plugin details
- Build the application
- Save the build process output as WAR or EAR file to a local location or server

- Get the file from local location or server and deploy the file to the production site or
- client site Updated the application document with date and updated version number of the application
- create and generate a report as per the application or requirement.

Summary:

- Maven is an automation and management tool.
- Maven tool is written in Java Language and used to build and manage projects written in C# (C Sharp), Ruby, Scala, and other languages.
- Developers can create a java-based project more easily with the use of Maven tool.
- To configure the Maven, you need to use Project Object Model, which is stored in a pom.xml-file.

From <https://www.guru99.com/maven-tutorial.html>
Click below for detail read.
https://www.simplilearn.com/tutorials/maven-tutorial/introduction-to-maven#:~:text=Maven%20is%20a%20popular%20open,manage%20any%20Java%2Dbased%20project.

? Building Spring Web application with Boot
? Spring Boot in detail (Use Spring Boot for all demo & assignments here onwards)
? Running a web application using Spring Boot with CRUD (with Static Data not DB)

- https://medium.com/@wiki.yasas/how-to-make-a-simple-crud-web-app-using-spring-boot-with-mysql-database-8c9b987ecd82
- https://www.codejava.net/frameworks/spring-boot/spring-boot-crud-web-application-with-jdbc-thymeleaf-oracle

? Spring Data JDBC

Spring Data JDBC, part of the larger Spring Data family, makes it easy to implement JDBC based repositories. This module deals with enhanced support for JDBC based data access layers. It makes it easier to build Spring powered applications that use data access technologies.
Spring Data JDBC aims at being conceptually easy. In order to achieve this it does NOT offer caching, lazy loading, write behind or many other features of JPA. This makes Spring Data JDBC a simple, limited, opinionated ORM.

From <https://spring.io/projects/spring-data-jdbc#overview>

Difference Between Spring JDBC and Spring Data JDBC

Spring JDBC	Spring Data JDBC
Spring JDBC is a Model class.	Spring Data JDBC is a POJO class.
Getter and setters are mandatory.	Getter and setters are not mandatory.
The parameterized constructor will be helpful.	The parameterized constructor may not be helpful.
There is no specific annotation is required. The only thing is we should have equal attributes match with the DB table and each attribute should have a getter and setter.	@Table, @ID, and @Column annotations are helpful to mention for direct connection with the database.
Data Access Layer is specified with the interface and its implementation.	Data Access Layer is simple and it omits lazy loading, cache implementation, etc., which is there in JPA(Java Persistence API).

From <https://www.geeksforgeeks.org/spring-boot-spring-jdbc-vs-spring-data-jdbc/>

Sessions 17 & 18:

Spring Data Module

Spring Data JPA (Repository support for JPA)

Spring Data JPA API provides JpaTemplate class to integrate spring application with JPA.
JPA (Java Persistent API) is the sun specification for persisting objects in the enterprise application. It is currently used as the replacement for complex entity beans.
The implementation of JPA specification are provided by many vendors such as:

- Hibernate
- Toplink
- iBatis
- OpenJPA etc.

Advantage of Spring JpaTemplate

You don't need to write the before and after code for persisting, updating, deleting or searching object such as creating Persistence instance, creating EntityManagerFactory instance, creating EntityTransaction instance, creating EntityManager instance, commiting EntityTransaction instance and closing EntityManager.
So, it **save a lot of code**.

From <https://www.javatpoint.com/spring-and-jpa-integration>

From <https://www.digitalocean.com/community/tutorials/spring-data-jpa>

? Crud Repository & JPA Repository

CrudRepository and JPA repository both are the interface of the spring data repository library. Spring data repository reduces the boilerplate code by providing some predefined finders to access the data layer for various persistence layers.

JPA repository extends CrudRepository and PagingAndSorting repository. It inherits some finders from crud repository such as findOne, gets and removes an entity. It also provides some extra methods related to JPA such as delete records in batch, flushing data directly to a database base and methods related to pagination and sorting.

We need to extend this repository in our application and then we can access all methods which are available in these repositories. We can also add new methods using named or native queries based on business requirements.

Sr. No.	Key	JPARepository	CrudRepository
1	Hierarchy	JPA extend crudRepository and PagingAndSorting repository	Crud Repository is the base interface and it acts as a marker interface.
2	Batch support	JPA also provides some extra methods related to JPA such as delete records in batch and flushing data directly to a database.	It provides only CRUD functions like findOne, saves, etc.
3	Pagination support	JPA repository also extends the PagingAndSorting repository. It provides all the method for which are useful for implementing pagination.	Crud Repository doesn't provide methods for implementing pagination and sorting.
4	Use Case	JpaRepository ties your repositories to the JPA persistence technology so it should be avoided.	We should use CrudRepository or PagingAndSortingRepository depending on whether you need sorting and paging or not.

Example of JpaRepository

```
@Repository
public interface BookDAO extends JpaRepository {
  Book findByAuthor(@Param("id") Integer id);
}
```

Example of CrudRepository

```
@Repository
public interface BookDAO extends CrudRepository {
  Book Event findById(@Param("id") Integer id);
}
```

From <https://www.tutorialspoint.com/difference-between-crudrepository-and-jparepository-in-java>

? Query methods

Spring Data JPA query methods are the most powerful methods, we can create query methods to select records from the database without writing SQL queries.

Behind the scenes, Spring Data JPA will create SQL queries based on the query method and execute the query for us.
The query generation from the method name is a query generation strategy where the invoked query is derived from the name of the query method.
We can create query methods for the repository using Entity fields and creating query methods is also called finder methods (findBy, findAll …)
For example: Consider we have UserRepository
with findByEmailAddressAndLastname() is a query method:

```
publicinterfaceUserRepositoryextendsRepository<User, Long>{
 List<User>findByEmailAddressAndLastname(StringemailAddress, Stringlastname);
}
```

Spring Data JPA behind the scene creates a query using the JPA criteria API from the above query method (findByEmailAddressAndLastname), but, essentially, this translates into the following JPQL query: **select u from User u where u.emailAddress = ?1 and u.lastname = ?2.**

Supported keywords inside method names

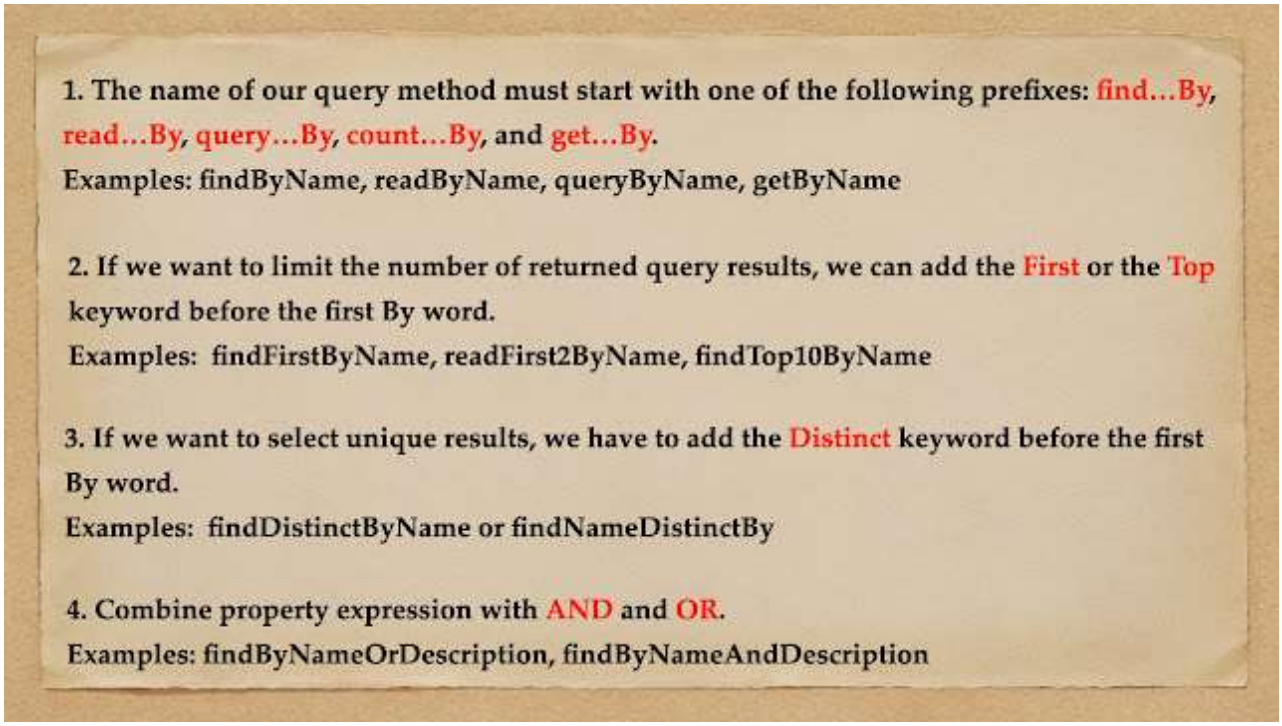
The following table describes the keywords supported for JPA and what a method containing that keyword translates to:

Keyword	Sample	JPQL snippet
And	findByLastnameAndFirstname	… where x.lastname = ?1 and x.firstname = ?2
Or	findByLastnameOrFirstname	… where x.lastname = ?1 or x.firstname = ?2
Is,Equals	findByFirstname,findByFirstnameIs,findByFirstnameEquals	… where x.firstname = ?1
Between	findByStartDateBetween	… where x.startDate between ?1 and ?2
LessThan	findByAgeLessThan	… where x.age < ?1
LessThanEqual	findByAgeLessThanEqual	… where x.age <= ?1
GreaterThan	findByAgeGreaterThan	… where x.age > ?1
GreaterThanEqual	findByAgeGreaterThanEqual	… where x.age >= ?1
After	findByStartDateAfter	… where x.startDate > ?1
Before	findByStartDateBefore	… where x.startDate < ?1
NotLike	findByFirstnameNotLike	… where x.firstname not like ?1
StartingWith	findByFirstnameStartingWith	… where x.firstname like ?1 (parameter bound with appended %)
EndingWith	findByFirstnameEndingWith	… where x.firstname like ?1 (parameter bound with prepended %)
Containing	findByFirstnameContaining	… where x.firstname like ?1 (parameter bound wrapped in %)
OrderBy	findByAgeOrderByLastnameDesc	… where x.age = ?1 order by x.lastname desc
Not	findByLastnameNot	… where x.lastname <> ?1
In	findByAgeIn(Collection<Age> ages)	… where x.age in ?1
NotIn	findByAgeNotIn(Collection<Age> ages)	… where x.age not in ?1
True	findByActiveTrue()	… where x.active = true
False	findByActiveFalse()	… where x.active = false
IgnoreCase	findByFirstnameIgnoreCase	… where UPPER(x.firstame) = UPPER(?1)

[Refer to official Spring Data JPA documentation] to know more about supported keywords.

Rules for Creating Query Methods

Let's take a look into the following few rules to create query methods using method names:



Now, we know the supported keywords and rules to create query methods from method names right.

From <https://www.javaguides.net/2018/11/spring-data-jpa-query-creation-from-method-names.html>

? Using custom query (@Query)

Well, the method names query generation strategy has the following weaknesses:

1. The features of the method name parser determine what kind of queries we can create. If the method name parser doesn't support the required keyword, we cannot use this strategy.
2. The method names of complex query methods are long and ugly.
3. There is no support for dynamic queries.

In this post, we can avoid those weaknesses by using the `@Query` annotation.

Creating Query Methods using @Query Annotation

We can configure the invoked database query by annotating the query method with the `@Query` annotation. It supports both **JPQL** and **SQL** queries, and the query that is specified by using the `@Query` annotation precedes all other query generation strategies.

Let's find out how we can create both **JPQL** and **SQL** queries with the `@Query` annotation.

Creating JPQL Queries

We can create a **JPQL** query with the `@Query` annotation by following these steps:

1. Add a query method to our repository interface.
2. Annotate the query method with the *@Query* annotation, and specify the invoked query by setting it as the value of the *@Query* annotation.

Let's look at the below examples that demonstrate the creation of **JPQL** Queries using *@Query* annotation.

The following example shows a query created with the *@Query* annotation:

```
publicinterfaceUserRepositoryextendsJpaRepository<User, Long>{
@Query("select u from User u where u.emailAddress = ?1")
 UserfindByEmailAddress(StringemailAddress);
}
```

Using Advanced LIKE Expressions - The query execution mechanism for manually defined queries created with *@Query* allows the definition of advanced LIKE expressions inside the query definition, as shown in the following example:

```
publicinterfaceUserRepositoryextendsJpaRepository<User, Long>{
@Query("select u from User u where u.firstname like %?1")
 List<User>findByFirstnameEndsWith(Stringfirstname);
}
```

In the preceding example, the LIKE delimiter character (%) is recognized, and the query is transformed into a valid JPQL query (removing the %). Upon query execution, the parameter passed to the method call gets augmented with the previously recognized LIKE pattern.

Creating SQL Queries

The *@Query* annotation allows for running native queries by setting the *nativeQuery* flag to **true**.

Let's follow the below steps to create a SQL query with the *@Query* annotation:

1. Add a query method to our repository interface.
2. Annotate the query method with the *@Query* annotation, and specify the invoked query by setting it as the value of the *@Query* annotation's value attribute.
3. Set the value of the *@Query* annotation's **nativeQuery** attribute to **true**.

```
importjava.util.List;
importorg.springframework.data.jpa.repository.JpaRepository;
importorg.springframework.data.jpa.repository.Query;
importorg.springframework.stereotype.Repository;
importnet.guides.springboot2.springboottestingexamples.model.User;
/** * UserRepository demonstrates the method name query generation. *  * @authorRamesh Fadatare *
*/@RepositorypublicinterfaceUserRepositoryextendsJpaRepository<User, Long>{
@Query(value="select * from users where first_name like %?1", nativeQuery=true)
 List<User>findByFirstnameEndsWith(Stringfirstname);
@Query(value="SELECT * FROM USERS WHERE EMAIL_ADDRESS = ?1", nativeQuery=true)
 UserfindByEmailAddress(StringemailAddress);
}
```

From <https://www.javaguides.net/2018/11/spring-data-jpa-creating-database-queries-using-query-annotation.html>

Session 19:

Spring AOP

? AOP Overview

? Spring AOP

? AOP Terminology and annotations: Advice, Join Points, Pointcuts, Aspects

Spring Framework is developed on two core concepts - Dependency Injection and Aspect Oriented Programming (Spring AOP).

Spring AOP

We have already seen how Spring Dependency Injection works, today we will look into the core concepts of Aspect-Oriented Programming and how we can implement it using Spring Framework.

Spring AOP Overview

Most of the enterprise applications have some common crosscutting concerns that are applicable to different types of Objects and modules. Some of the common crosscutting concerns are logging, transaction management, data validation, etc. In Object Oriented Programming, modularity of application is achieved by Classes whereas in Aspect Oriented Programming application modularity is achieved by Aspects and they are configured to cut across different classes. Spring AOP takes out the direct dependency of crosscutting tasks from classes that we can't achieve through normal object oriented programming model. For example, we can have a separate class for logging but again the functional classes will have to call these methods to achieve logging across the application.

Aspect Oriented Programming Core Concepts

Before we dive into the implementation of Spring AOP implementation, we should understand the core concepts of AOP.

1. **Aspect**: An aspect is a class that implements enterprise application concerns that cut across multiple classes, such as transaction management. Aspects can be a normal class configured through Spring XML configuration or we can use Spring AspectJ integration to define a class as Aspect using @Aspect annotation.
2. **Join Point**: A join point is a specific point in the application such as method execution, exception handling, changing object variable values, etc. In Spring AOP a join point is always the execution of a method.
3. **Advice**: Advices are actions taken for a particular join point. In terms of programming, they are methods that get executed when a certain join point with matching pointcut is reached in the application. You can think of Advices as Struts2 interceptors or Servlet Filters.
4. **Pointcut**: Pointcut is expressions that are matched with join points to determine whether

advice needs to be executed or not. Pointcut uses different kinds of expressions that are matched with the join points and Spring framework uses the AspectJ pointcut expression language.

5. **Target Object**: They are the object on which advices are applied. Spring AOP is implemented using runtime proxies so this object is always a proxied object. What is means is that a subclass is created at runtime where the target method is overridden and advice are included based on their configuration.
6. **AOP proxy**: Spring AOP implementation uses JDK dynamic proxy to create the Proxy classes with target classes and advice invocations, these are called AOP proxy classes. We can also use CGLIB proxy by adding it as the dependency in the Spring AOP project.
7. **Weaving**: It is the process of linking aspects with other objects to create the advised proxy objects. This can be done at compile time, load time or at runtime. Spring AOP performs weaving at the runtime.

AOP Advice Types

Based on the execution strategy of advice, they are of the following types.

1. **Before Advice**: These advices runs before the execution of join point methods. We can use @Before annotation to mark an advice type as Before advice.
2. **After (finally) Advice**: An advice that gets executed after the join point method finishes executing, whether normally or by throwing an exception. We can create after advice using @After annotation.
3. **After Returning Advice**: Sometimes we want advice methods to execute only if the join point method executes normally. We can use @AfterReturning annotation to mark a method as after returning advice.
4. **After Throwing Advice**: This advice gets executed only when join point method throws exception, we can use it to rollback the transaction declaratively. We use @AfterThrowing annotation for this type of advice.
5. **Around Advice**: This is the most important and powerful advice. This advice surrounds the join point method and we can also choose whether to execute the join point method or not. We can write advice code that gets executed before and after the execution of the join point method. It is the responsibility of around advice to invoke the join point method and return values if the method is returning something. We use @Around annotation to create around advice methods.

The points mentioned above may sound confusing but when we will look at the implementation of Spring AOP, things will be more clear. Let's start creating a simple Spring project with AOP implementations. Spring provides support for using AspectJ annotations to create aspects and we will be using that for simplicity. All the above AOP annotations are defined in org.aspectj.lang.annotation package. **Spring Tool Suite** provides useful information about the aspects, so I would suggest you use it.

From <https://www.digitalocean.com/community/tutorials/spring-aop-example-tutorial-aspect-advice-pointcut-joinpoint-annotations>

Sessions 20 & 21:

Building REST services with Spring

? Introduction to web services

The Internet is the worldwide connectivity of hundreds of thousands of computers of various types that belong to multiple networks. On the World Wide Web, a web service is a standardized method for propagating messages between client and server applications. A web service is a software module

that is intended to carry out a specific set of functions. Web services in cloud computing can be found and invoked over the network.

The web service would be able to deliver functionality to the client that invoked the web service.

A web service is a set of open protocols and standards that allow data to be exchanged between different applications or systems. Web services can be used by software programs written in a variety of programming languages and running on a variety of platforms to exchange data via computer networks such as the Internet in a similar way to inter-process communication on a single computer.

Any software, application, or cloud technology that uses standardized web protocols (HTTP or HTTPS) to connect, interoperate, and exchange data messages – commonly XML (Extensible Markup Language) – across the internet is considered a web service.

Web services have the advantage of allowing programs developed in different languages to connect with one another by exchanging data over a web service between clients and servers. A client invokes a web service by submitting an XML request, which the service responds with an XML response**.**

Functions of Web Services

- It's possible to access it via the internet or intranet networks.
- XML messaging protocol that is standardized.
- Operating system or programming language independent.
- Using the XML standard, it is self-describing.
- A simple location approach can be used to locate it.

Components of Web Service

XML and HTTP is the most fundamental web services platform. The following components are used by all typical web services:

SOAP (Simple Object Access Protocol)

SOAP stands for "Simple Object Access Protocol." It is a transport-independent messaging protocol. SOAP is built on sending XML data in the form of SOAP Messages. A document known as an XML document is attached to each message. Only the structure of the XML document, not the content, follows a pattern. The best thing about Web services and SOAP is that everything is sent through HTTP, the standard web protocol.

A root element known as the element is required in every SOAP document. In an XML document, the root element is the first element. The "envelope" is separated into two halves. The header comes first, followed by the body. The routing data, or information that directs the XML document to which client it should be sent to, is contained in the header. The real message will be in the body.

UDDI (Universal Description, Discovery, and Integration)

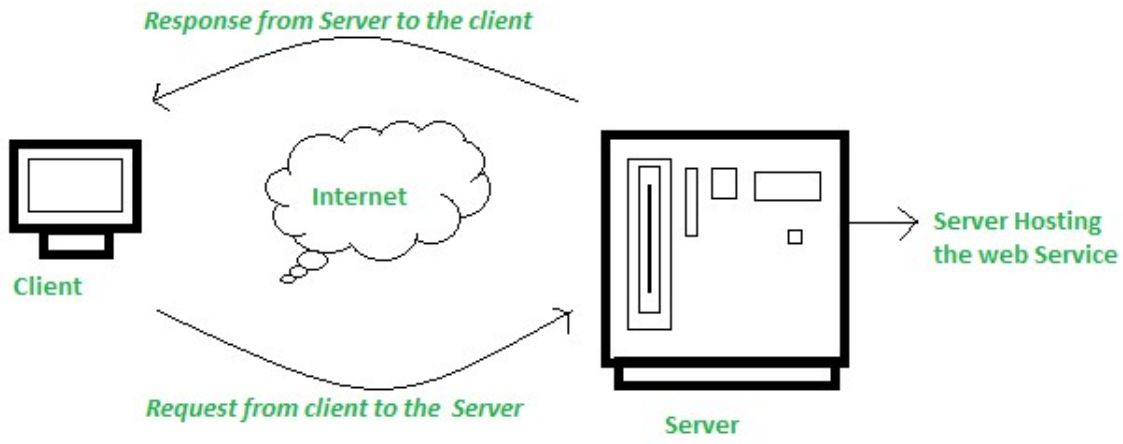
UDDI is a standard for specifying, publishing and discovering a service provider's online services. It provides a specification that aids in the hosting of data via web services.

WSDL (Web Services Description Language)

If a web service can't be found, it can't be used. The client invoking the web service should be aware of the location of the web service. Second, the client application must understand what the web service does in order to invoke the correct web service. The WSDL, or Web services description language, is used to accomplish this. The WSDL file is another XML-based file that explains what the web service does to the client application. The client application will be able to understand where the web service is located and how to use it by using the WSDL document.

How Does Web Service Work?

The diagram depicts a very simplified version of how a web service would function. The client would use requests to send a sequence of web service calls to a server that would host the actual web service.



Remote procedure calls are what are used to make these requests. Calls to methods hosted by the relevant web service are known as Remote Procedure Calls (RPC). Example: Flipkart offers a web service that displays prices for items offered on Flipkart.com. The front end or presentation layer can be written in .Net or Java, but the web service can be communicated using either programming language.

The data that is exchanged between the client and the server, which is XML, is the most important part of a web service design. XML (Extensible markup language) is a simple intermediate language that is understood by various programming languages. It is a counterpart to HTML. As a result, when programs communicate with one another, they do so using XML. This creates a common platform for applications written in different programming languages to communicate with one another.

For transmitting XML data between applications, web services employ SOAP (Simple Object Access Protocol). The data is sent using standard HTTP. A SOAP message is data that is sent from the web service to the application. An XML document is all that is contained in a SOAP message. The client application that calls the web service can be created in any programming language because the content is written in XML.

Features/Characteristics Of Web Service

Web services have the following features:

(a) XML Based.

(b) Loosely Coupled.

(c) Capability to be Synchronous or Asynchronous.

(d) Coarse-Grained.

(e) Supports Remote Procedural Call.

(f) Supports Document Exchanges

Advantages Of Web Service

Using web services has the following advantages:

(a) Business Functions can be exposed over the Internet.

(b) Interoperability.

(c) Communication with Low Cost.

(d) A Standard Protocol that Everyone Understands.

(e) Reusability: A single web service can be used simultaneously by several client applications.

From <https://www.geeksforgeeks.org/what-are-web-services/>

? SOAP Vs RESTful web services

What is SOAP?

SOAP is a protocol which was designed before REST and came into the picture. The main idea behind designing SOAP was to ensure that programs built on different platforms and programming languages could exchange data in an easy manner. SOAP stands for Simple Object Access Protocol.

What is REST?

REST was designed specifically for working with components such as media components, files, or even objects on a particular hardware device. Any web service that is defined on the principles of REST can be called a

RestFul web service. A Restful service would use the normal HTTP verbs of GET, POST, PUT and DELETE for working with the required components. REST stands for Representational State Transfer.

KEY DIFFERENCE

- SOAP stands for Simple Object Access Protocol whereas REST stands for Representational State Transfer.
- SOAP is a protocol whereas REST is an architectural pattern.
- SOAP uses service interfaces to expose its functionality to client applications while REST uses Uniform Service locators to access to the components on the hardware device.
- SOAP needs more bandwidth for its usage whereas REST doesn't need much bandwidth.
- Comparing SOAP vs REST API, SOAP only works with XML formats whereas REST work with plain text, XML, HTML and JSON.
- SOAP cannot make use of REST whereas REST can make use of SOAP.

Difference Between SOAP and REST

Each technique has its own advantages and disadvantages. Hence, it's always good to understand in which situations each design should be used. This REST and SOAP API difference tutorial will go into some of the key difference between REST and SOAP API as well as what challenges you might encounter while using them.

Below is the main difference between SOAP and REST API

SOAP	REST
• SOAP stands for Simple Object Access Protocol	• REST stands for Representational State Transfer
• SOAP is a protocol. SOAP was designed with a specification. It includes a WSDL file which has the required information on what the web service does in addition to the location of the web service.	• REST is an Architectural style in which a web service can only be treated as a RESTful service if it follows the constraints of being 1. Client Server 2. Stateless 3. Cacheable 4. Layered System 5. Uniform Interface
• SOAP cannot make use of REST since SOAP is a protocol and REST is an architectural pattern.	• REST can make use of SOAP as the underlying protocol for web services, because in the end it is just an architectural pattern.
• SOAP uses service interfaces to expose its functionality to	• REST use Uniform Service locators to access to the

client applications. In SOAP, the WSDL file provides the client with the necessary information which can be used to understand what services the web service can offer.	components on the hardware device. For example, if there is an object which represents the data of an employee hosted on a URL as http://demo.guru99 , the below are some of URI that can exist to access them. http://demo.guru99.com/Employee http://demo.guru99.com/Employee/1
• SOAP requires more bandwidth for its usage. Since SOAP Messages contain a lot of information inside of it, the amount of data transfer using SOAP is generally a lot. <?xml version="1.0"?> <SOAP-ENV:Envelope xmlns:SOAP-ENV ="http://www.w3.org/2001/12/soap-envelope" SOAP-ENV:encodingStyle =" http://www.w3.org/2001/12/soap-encoding"> <soap:Body> <Demo.guru99WebService xmlns="http://tempuri.org/"> <EmployeeID>int</EmployeeID> </Demo.guru99WebService> </soap:Body> </SOAP-ENV:Envelope>	• REST does not need much bandwidth when requests are sent to the server. REST messages mostly just consist of JSON messages. Below is an example of a JSON message passed to a web server. You can see that the size of the message is comparatively smaller to SOAP. {"city":"Mumbai","state":"Maharastra"}
• SOAP can only work with XML format. As seen from SOAP messages, all data passed is in XML format.	• REST permits different data format such as Plain text, HTML, XML, JSON, etc. But the most preferred format for transferring data is JSON.

From <https://www.guru99.com/comparison-between-web-services.html>

? RESTful web service introduction

Introduction to RESTful Web Services

REST stands for **REpresentational State Transfer**. It is developed by **Roy Thomas Fielding**, who also developed HTTP. The main goal of RESTful web services is to make web services **more effective**. RESTful web services try to

define services using the different concepts that are already present in HTTP.
REST is an **architectural approach**, not a protocol.
It does not define the standard message exchange format. We can build REST services with both XML and JSON. JSON is more popular format with REST.
The **key abstraction** is a resource in REST. A resource can be anything. It can be accessed through a **Uniform Resource Identifier (URI)**. For example:
The resource has representations like XML, HTML, and JSON. The current state capture by representational resource. When we request a resource, we provide the representation of the resource. The important methods of HTTP are:

- **GET:** It reads a resource.
- **PUT:** It updates an existing resource.
- **POST:** It creates a new resource.
- **DELETE:** It deletes the resource.

For example, if we want to perform the following actions in the social media application, we get the corresponding results.
POST /users: It creates a user.
GET /users/{id}: It retrieves the detail of a user.
GET /users: It retrieves the detail of all users.
DELETE /users: It deletes all users.
DELETE /users/{id}: It deletes a user.
GET /users/{id}/posts/post_id: It retrieve the detail of a specific post.
POST / users/{id}/ posts: It creates a post of the user.
Further, we will implement these URI in our project.
HTTP also defines the following standard status code:

- **404:** RESOURCE NOT FOUND
- **200:** SUCCESS
- **201:** CREATED
- **401:** UNAUTHORIZED
- **500:** SERVER ERROR

RESTful Service Constraints

- There must be a service producer and service consumer.
- The service is stateless.
- The service result must be cacheable.
- The interface is uniform and exposing resources.
- The service should assume a layered architecture.

Advantages of RESTful web services

- RESTful web services are **platform-independent**.
- It can be written in any programming language and can be executed on any platform.
- It provides different data format like **JSON, text, HTML,** and **XML**.
- It is fast in comparison to SOAP because there is no strict specification like

SOAP.

- These are **reusable**.
- They are **language neutral**.

From <https://www.javatpoint.com/restful-web-services-spring-boot>

⍰ Create RESTful web service in java using Spring Boot
⍰ RESTful web service JSON example
⍰ RESTful web service CRUD example
⍰ Using POSTMAN client to invoke REST API's
⍰ REST service invocation using REST Template

Spring Boot provides a very good support to building RESTful Web Services for enterprise applications. This chapter will explain in detail about building RESTful web services using Spring Boot.

Note – For building a RESTful Web Services, we need to add the Spring Boot Starter Web dependency into the build configuration file.

If you are a Maven user, use the following code to add the below dependency in your **pom.xml** file –

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

If you are a Gradle user, use the following code to add the below dependency in your **build.gradle** file.

```
compile('org.springframework.boot:spring-boot-starter-web')
```

The code for complete build configuration file **Maven build – pom.xml** is given below –

```
<?xml version = "1.0" encoding = "UTF-8"?>
<project xmlns = "http://maven.apache.org/POM/4.0.0"
  xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation = "http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>com.tutorialspoint</groupId>
  <artifactId>demo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>
  <name>demo</name>
  <description>Demo project for Spring Boot</description>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>1.5.8.RELEASE</version>
    <relativePath/>
  </parent>
```

```
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    <java.version>1.8</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>

</project>
```

The code for complete build configuration file **Gradle Build – build.gradle** is given below –

```
buildscript {
  ext {
    springBootVersion = '1.5.8.RELEASE'
  }
  repositories {
    mavenCentral()
  }
  dependencies {
    classpath("org.springframework.boot:spring-boot-gradle-plugin:${springBootVersion}")
  }
}
apply plugin: 'java'
apply plugin: 'eclipse'
apply plugin: 'org.springframework.boot'
group = 'com.tutorialspoint'
version = '0.0.1-SNAPSHOT'
sourceCompatibility = 1.8
repositories {
  mavenCentral()
}
dependencies {
  compile('org.springframework.boot:spring-boot-starter-web')
```

```
  testCompile('org.springframework.boot:spring-boot-starter-test')
}
```

Before you proceed to build a RESTful web service, it is suggested that you have knowledge of the following annotations –

Rest Controller

The @RestController annotation is used to define the RESTful web services. It serves JSON, XML and custom response. Its syntax is shown below –

```
@RestController
public class ProductServiceController {
}
```


Request Mapping

The @RequestMapping annotation is used to define the Request URI to access the REST Endpoints. We can define Request method to consume and produce object. The default request method is GET.

```
@RequestMapping(value = "/products")
public ResponseEntity<Object> getProducts() { }
```

Request Body

The @RequestBody annotation is used to define the request body content type.

```
public ResponseEntity<Object> createProduct(@RequestBody Product product) {}
```


Path Variable

The @PathVariable annotation is used to define the custom or dynamic request URI. The Path variable in request URI is defined as curly braces {} as shown below –

```
public ResponseEntity<Object> updateProduct(@PathVariable("id") String id) {}
```

Request Parameter

The @RequestParam annotation is used to read the request parameters from the Request URL. By default, it is a required parameter. We can also set default value for request parameters as shown here –

```
public ResponseEntity<Object> getProduct(
  @RequestParam(value = "name", required = false, defaultValue = "honey") String name) {}
```

GET API

The default HTTP request method is GET. This method does not require any Request Body. You can send request parameters and path variables to define the custom or dynamic URL.

The sample code to define the HTTP GET request method is shown below. In this example, we used HashMap to store the Product. Note that we used a POJO class as the product to be stored.

Here, the request URI is **/products** and it will return the list of products from HashMap repository. The controller class file is given below that contains GET method REST Endpoint.

```
package com.tutorialspoint.demo.controller;
import java.util.HashMap;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.tutorialspoint.demo.model.Product;
@RestController
```

```
public class ProductServiceController {
  private static Map<String, Product> productRepo = new HashMap<>();
  static {
    Product honey = new Product();
    honey.setId("1");
    honey.setName("Honey");
    productRepo.put(honey.getId(), honey);

    Product almond = new Product();
    almond.setId("2");
    almond.setName("Almond");
    productRepo.put(almond.getId(), almond);
  }
  @RequestMapping(value = "/products")
  public ResponseEntity<Object> getProduct() {
    return new ResponseEntity<>(productRepo.values(), HttpStatus.OK);
  }
}
```

POST API

The HTTP POST request is used to create a resource. This method contains the Request Body. We can send request parameters and path variables to define the custom or dynamic URL.

The following example shows the sample code to define the HTTP POST request method. In this example, we used HashMap to store the Product, where the product is a POJO class.

Here, the request URI is **/products**, and it will return the String after storing the product into HashMap repository.

```
package com.tutorialspoint.demo.controller;
import java.util.HashMap;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;
import com.tutorialspoint.demo.model.Product;
@RestController
public class ProductServiceController {
  private static Map<String, Product> productRepo = new HashMap<>();

  @RequestMapping(value = "/products", method = RequestMethod.POST)
  public ResponseEntity<Object> createProduct(@RequestBody Product product) {
    productRepo.put(product.getId(), product);
    return new ResponseEntity<>("Product is created successfully", HttpStatus.CREATED);
  }
}
```

PUT API

The HTTP PUT request is used to update the existing resource. This method contains a Request Body. We can send request parameters and path variables to define the custom or dynamic URL.

The example given below shows how to define the HTTP PUT request method. In this example, we used HashMap to update the existing Product, where the product

is a POJO class.
Here the request URI is **/products/{id}** which will return the String after a the product into a HashMap repository. Note that we used the Path variable **{id}** which defines the products ID that needs to be updated.

```
package com.tutorialspoint.demo.controller;
import java.util.HashMap;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;
import com.tutorialspoint.demo.model.Product;
@RestController
public class ProductServiceController {
  private static Map<String, Product> productRepo = new HashMap<>();

  @RequestMapping(value = "/products/{id}", method = RequestMethod.PUT)
  public ResponseEntity<Object> updateProduct(@PathVariable("id") String id, @RequestBody
Product product) {
    productRepo.remove(id);
    product.setId(id);
    productRepo.put(id, product);
    return new ResponseEntity<>("Product is updated successsfully", HttpStatus.OK);
  }
}
```

DELETE API

The HTTP Delete request is used to delete the existing resource. This method does not contain any Request Body. We can send request parameters and path variables to define the custom or dynamic URL.
The example given below shows how to define the HTTP DELETE request method. In this example, we used HashMap to remove the existing product, which is a POJO class.
The request URI is **/products/{id}** and it will return the String after deleting the product from HashMap repository. We used the Path variable **{id}** which defines the products ID that needs to be deleted.

```
package com.tutorialspoint.demo.controller;
import java.util.HashMap;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;
import com.tutorialspoint.demo.model.Product;
@RestController
public class ProductServiceController {
  private static Map<String, Product> productRepo = new HashMap<>();
```

```
  @RequestMapping(value = "/products/{id}", method = RequestMethod.DELETE)
  public ResponseEntity<Object> delete(@PathVariable("id") String id) {
    productRepo.remove(id);
    return new ResponseEntity<>("Product is deleted successsfully", HttpStatus.OK);
  }
}
```

This section gives you the complete set of source code. Observe the following codes for their respective functionalities –

The Spring Boot main application class – DemoApplication.java

```
package com.tutorialspoint.demo;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class DemoApplication {
  public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
  }
}
```

The POJO class – Product.java

```
package com.tutorialspoint.demo.model;
public class Product {
  private String id;
  private String name;
public String getId() {
    return id;
  }
  public void setId(String id) {
    this.id = id;
  }
  public String getName() {
    return name;
  }
  public void setName(String name) {
    this.name = name;
  }
}
```

The Rest Controller class – ProductServiceController.java

```
package com.tutorialspoint.demo.controller;
import java.util.HashMap;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;
import com.tutorialspoint.demo.model.Product;
@RestController
public class ProductServiceController {
  private static Map<String, Product> productRepo = new HashMap<>();
  static {
    Product honey = new Product();
```

```
    honey.setId("1");
    honey.setName("Honey");
    productRepo.put(honey.getId(), honey);

    Product almond = new Product();
    almond.setId("2");
    almond.setName("Almond");
    productRepo.put(almond.getId(), almond);
  }

  @RequestMapping(value = "/products/{id}", method = RequestMethod.DELETE)
  public ResponseEntity<Object> delete(@PathVariable("id") String id) {
    productRepo.remove(id);
    return new ResponseEntity<>("Product is deleted successsfully", HttpStatus.OK);
  }

  @RequestMapping(value = "/products/{id}", method = RequestMethod.PUT)
  public ResponseEntity<Object> updateProduct(@PathVariable("id") String id, @RequestBody
Product product) {
    productRepo.remove(id);
    product.setId(id);
    productRepo.put(id, product);
    return new ResponseEntity<>("Product is updated successsfully", HttpStatus.OK);
  }

  @RequestMapping(value = "/products", method = RequestMethod.POST)
  public ResponseEntity<Object> createProduct(@RequestBody Product product) {
    productRepo.put(product.getId(), product);
    return new ResponseEntity<>("Product is created successfully", HttpStatus.CREATED);
  }

  @RequestMapping(value = "/products")
  public ResponseEntity<Object> getProduct() {
    return new ResponseEntity<>(productRepo.values(), HttpStatus.OK);
  }
}
```

You can create an executable JAR file, and run the spring boot application by using the below Maven or Gradle commands as shown –

For Maven, use the command shown below –

```
mvn clean install
```

After “BUILD SUCCESS”, you can find the JAR file under the target directory.

For Gradle, use the command shown below –

```
gradle clean build
```

After “BUILD SUCCESSFUL”, you can find the JAR file under the build/libs directory.

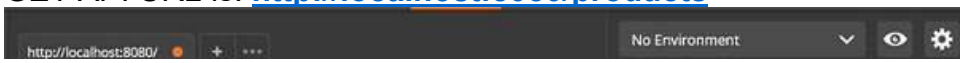
You can run the JAR file by using the command shown below –

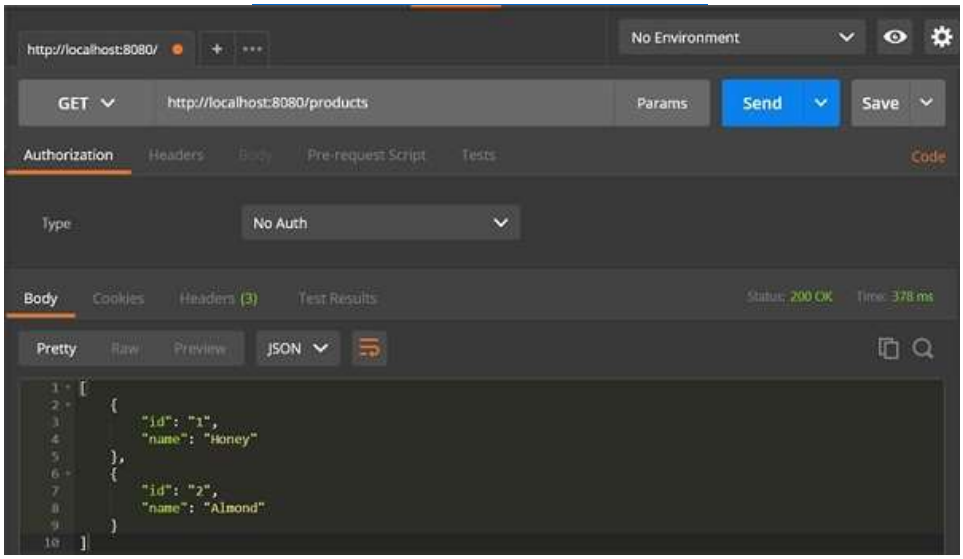
```
java –jar <JARFILE>
```

This will start the application on the Tomcat port 8080 as shown below –

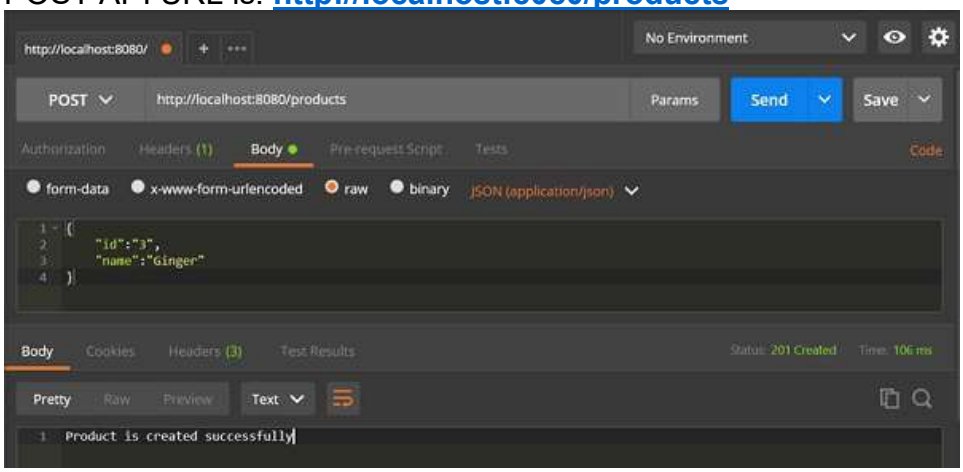
Now hit the URL shown below in POSTMAN application and see the output.

GET API URL is: **http://localhost:8080/products**

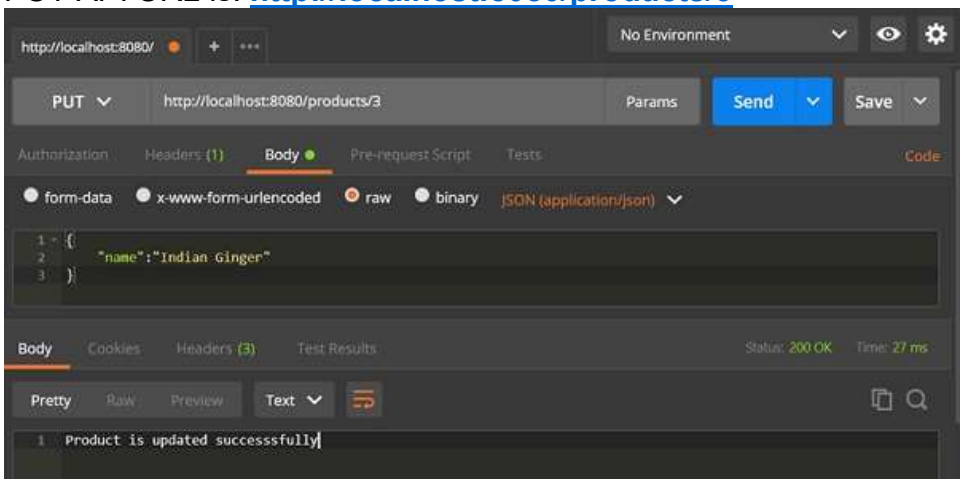




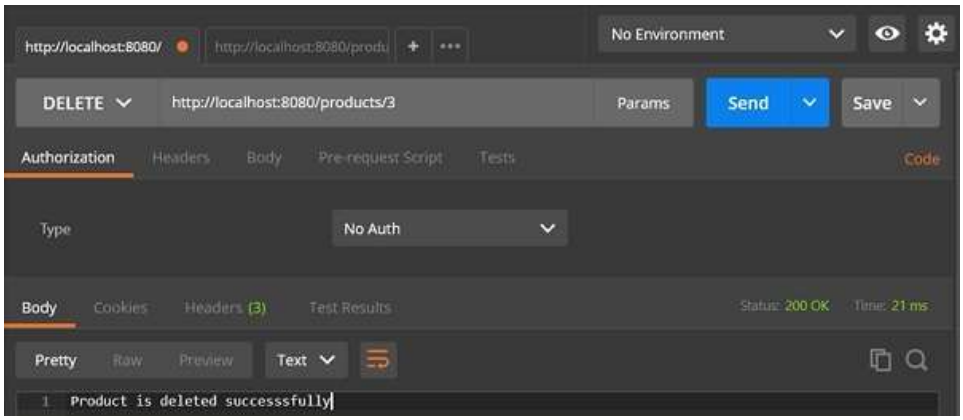
POST API URL is: http://localhost:8080/products



PUT API URL is: http://localhost:8080/products/3

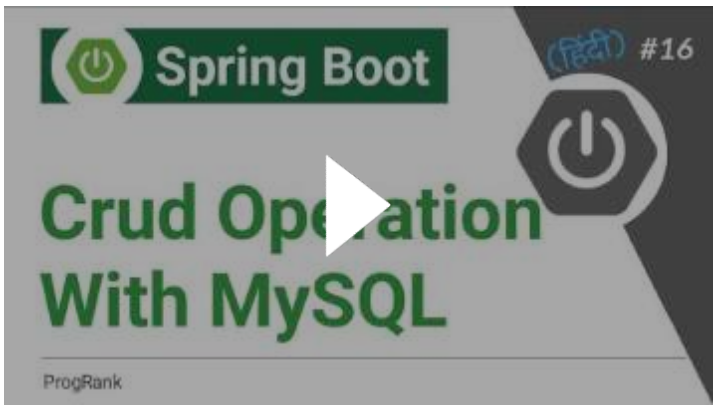


DELETE API URL is: http://localhost:8080/products/3



From <https://www.tutorialspoint.com/spring_boot/spring_boot_building_restful_web_services.htm>

Spring Boot Tutorial [Hindi] | Spring Boot CRUD Example with MySQL | JPA | Rest API | #16



Session 22 & 23:

Testing in Spring

? Testing in Spring

? Unit Testing of Spring MVC Controllers

? Unit Testing of Spring Service Layer

? Integration Testing of Spring MVC Applications: REST API

? Unit Testing Spring MVC Controllers with REST

Testing is a crucial part in the business of software development. It ensures performance and quality of the product. The Java platform supports many testing frameworks. Spring introduces the principle of dependency injection on unit testing and has first-class support for integration testing. This article tries to give a quick introductory idea about the testing support provided by the Spring framework and how it is applied with Spring Boot.

An Overview

Testing is a process that ensures that the quality of the software adheres to the standard laid down by the requirements specification and performs smoothly with the fewest possible glitches. For this, a technique called *unit testing* is the simplest to achieve and can be applied on all applications in their development. Unit testing allows each component of the application to be tested separately. Further, when these individual components are integrated and tested to ensure that multiple components collaborate to work well in the system, it is called *integration testing*.
In a development environment, usually test codes are written separately and are not mixed with the normal code. In fact, testing a program is distinctly a separate project and is stored in a separate directory. The tests constitute test cases that are applied to test the application manually or automatically. Automated test cases are applied repeatedly at different phases of the software development process. This, by the way, is the most recommended process of the Agile framework. And Spring, being a proponent of the framework, readily supports it.
Java is never lacking in support and has numerous testing frameworks—such as JUnit, TestNG, Mockito, and so forth—at its disposal. They also go pretty well with Spring, yet perhaps the distinctive quality is Spring's incorporation of IoC into the game.

Unit Testing

The major advantage of applying dependency injection is that it makes a program code much easier to be tested because the code is much less dependent on the container, as may be the case with other development. The POJOs are individually testable by simple instantiation without adhering to any dependency. Sometimes, the fine line between unit and integration testing overlaps at the best interest of testing and we need to go beyond unit testing and start performing integration testing without deploying the application or connecting to another infrastructure. In such a case, we can mock objects to obtain the value of integration yet test our code in isolation. For example, we can test a service layer or a controller by stubbing or mocking DAO objects, without actually accessing the persistent data.
The Spring framework provides numerous mock objects and support classes. Some of them are as follows.

Mock Objects

The package called *org.springframework.mock.env* contains classes that are useful for creating out-of-container unit tests that depend on environment specific properties:

- *MockEnvironment:* This class is a mock implementation of the *Environment* interface.

- *MockPropertySource:* This class is a mock implementation of the *PropertySource* abstract class.
  The *org.springframework.mock.jndi* package contains classes which can be used to set up a JNDI environment for test suites or standalone applications.
- *ExpectedLookupTemplate:* An extension of *JndiTemplate* and essentially creates a mock object of its type.
- *SimpleNamingContext:* Implementation of JNDI naming context that binds plain objects to String names for a test environment as well as standalone applications.
- *SimpleNamingContextBuilder:* This is an essentially a builder pattern class of the *SimpleNamingContext*.
  There are several classes under *org.springframework.mock.web*. These classes enable us to create a comprehensive mock object for Servlet APIs. These mock objects are particularly useful for testing Web context, controllers, and filters.
  The classes in the *org.springframework.mock.http.server.reactive* package help in creating a mock for *ServerHttpRequest* and *ServerHttpResponse* for WebFlux applications.
  Apart from these, there are quite a number of utility classes in the *org.springframework.test.util* package, such as the *ReflectionTestUtils*, which is a collection for reflection-based utility methods used in both unit and integration testing. The *ModelAndViewAssert* class contained in *org.springframework.test.web* can be used to test Spring MVC *ModelAndView* objects.

Integration Testing

The Spring support for integration testing enables one to perform integration testing without actually deploying the application server or connecting to other dependent infrastructures. This is a huge plus point because it enables us to test the wiring of Spring IoC container contexts. Also, for example, we can test data source accessibility, or SQL statements by using JDBC, an ORM tool. The package *org.springframework.test* container has all the required classes for integration testing with the Spring container.

Spring Boot

Spring Boot provides a set of utilities and annotation to help in the process of unit and integration testing and, of course, makes life easier. The core items for the testing are contained in the modules called spring-boot-test and the configuration is provided by the modules called spring-boot-test-autoconfigure.
We can simply use the spring-boot-starter-test in pom.xml and transitively pull all the required dependencies in a Spring application. The support libraries for testing as pulled by the Maven file are as follows:

- JUnit: It is the standard Java unit testing framework which provides an up-to-date foundation for developer-side testing on the JVM.

- Spring Test: Utilities and Integration support for Spring Boot applications.
- AssertJ: A set of assertions to provide meaningful error messages that leverage readability and easy debugging.
- Hamcrest: Provides a library of matcher object that helps in creating flexible expressions.
- Mockito: Java Mocking framework.
- JSONassert: Helps in writing JSON unit test and testing REST interfaces.
- JsonPath: Xpath for Json.

Spring Boot provides a volley of annotation to designate a test class and test specific parts of an application. For example, we can use *@SpringBootTest* annotation to enable Spring Boot test features. This annotation loads *ApplicationContext* used in a test, through *SpringApplication*. Typically, the *@SpringBootTest* annotation is used in association with *@RunWith(SpringRunner.class)* and does a whole lot of thing behind the scene apart from just loading *ApplicationContext* and having spring beans auto wired to the test instances. The *@SpringBootTest* does not start the server by default. As a result, to test Web endpoints, we can use a mock environment as follows.

Here is a code snippet to illustrate the idea.

```
package org.springboot.app;
import static org.springframework.test.web.servlet
  .request.MockMvcRequestBuilders.*;
import org.junit.*;
import org.junit.runner.RunWith;
import org.springboot.app.controller.*;
import org.springframework.boot.test.autoconfigure.
  web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.junit4.SpringRunner;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet
  .setup.MockMvcBuilders;

@RunWith(SpringRunner.class)
@SpringBootTest
@AutoConfigureMockMvc
public class SpringbootApp1ApplicationTests {
  private MockMvc mock;
  @Before
  public void init() {
    mock = MockMvcBuilders
      .standaloneSetup(new BookController()).build();
  }
  @Test
  public void testBookList() throws Exception {
    mock.perform(get("/")).andExpect(status()
      .isOk()).andExpect(view().name("bookList"))
      .andDo(print());
  }
  // ...
}
```

Conclusion

The Spring test framework integrates well with test frameworks such as Junit, Mockito, and so on. As a result, unit and integration testing becomes easy with a more meaningful outcome. Springs makes it simpler by leveraging annotation-based support. For example, to unit and integration test the Spring *TestContext* framework, we can use *@RunWith*, *@WebAppConfiguration*, and *@ContextConfiguration* to load a Spring configuration and inject *WebApplicationContext* into the MockMvc.
This write-up is just a tip of the iceberg and provides a glimpse of what Spring offers with respect to unit and integration testing. Refer to the following links for a more elaborate explanation on this topic.

From <https://www.developer.com/design/what-is-spring-testing/>

Securing Web Application with Spring Security

? What is Spring Security

Introduction

Spring Security is a framework which provides various security features like: authentication, authorization to create secure Java Enterprise Applications.
It is a sub-project of Spring framework which was started in 2003 by Ben Alex. Later on, in 2004, It was released under the Apache License as Spring Security 2.0.0.
It overcomes all the problems that come during creating non spring security applications and manage new server environment for the application.
This framework targets two major areas of application are authentication and authorization.

Authentication is the process of knowing and identifying the user that wants to access.

Authorization is the process to allow authority to perform actions in the application.

We can apply authorization to authorize web request, methods and access to individual domain.
Technologies that support Spring Security Integration
Spring Security framework supports wide range of authentication models. These models either provided by third parties or framework itself. Spring Security supports integration with all of these technologies.

- HTTP BASIC authentication headers
- HTTP Digest authentication headers
- HTTP X.509 client certificate exchange

- LDAP (Lighweight Directory Access Protocol)
- Form-based authentication
- OpenID authentication
- Automatic remember-me authentication
- Kerberos
- JOSSO (Java Open Source Single Sign-On)
- AppFuse
- AndroMDA
- Mule ESB
- DWR(Direct Web Request)

The beauty of this framework is its flexible authentication nature to integrate with any software solution. Sometimes, developers want to integrate it with a legacy system that does not follow any security standard, there Spring Security works nicely.

Advantages

Spring Security has numerous advantages. Some of that are given below.

- Comprehensive support for authentication and authorization.
- Protection against common tasks
- Servlet API integration
- Integration with Spring MVC
- Portability
- CSRF protection
- Java Configuration support

From <https://www.javatpoint.com/spring-security-introduction>

□ Spring Security with Spring Boot
□ Basic Authentication
□ Authentication with User credentials from Database and Authorization